Slot-Chain: A Preconfirmation Protocol

Audited Design Candidate — v2.25

August 2026

Abstract

Slot-Chain assigns one-second L2 slots to bonded builders. Builders sign parent-linked blocks off chain; any party can prove and submit a batch to L1. Normal candidates compete for a fixed settlement interval. After an objective finality breach, or when a forced L1 message becomes due, anyone can prove an unsigned deterministic escape block. Recovery is an episode of renewable, objectively expiring rounds, so a prover outage does not make its target stale; as with every validity rollup, producing the next proof still requires a reconstructible canonical prestate. This revision separates proved outputs from L1 close metadata, replaces the unauthenticatable queue hash chain with a range-provable Merkle vector, makes forced-message execution Ethereum-valid, and specifies bounded admission, data, reward and migration state. It is an audited architecture candidate, not yet an implementation-ready specification or production authorization: the missing executable profile and the proving, gas, economics and adversarial gates in §14 remain mandatory before production release. The control- plane, migration journals and canonical execution-profile codec in this revision are frozen candidates, but the Settlement–Market mutation wire protocol remains an explicit implementation-readiness blocker.

Contents

1 Status, claims and exclusions

Normative words MUST, MUST NOTand SHOULDhave their usual RFC meaning. Where prose and either executable model disagree, deployment is blocked until the disagreement is resolved; neither source silently overrides the other.

1.1 Claims

Subject to L1 safety/liveness, a sound validity-proof system, availability of a canonical prestate reconstruction package as defined below, and—for kind-1 credits only—safety/finality of the supported source domain and its profile-authenticated, append-only Bridge registry, the protocol claims:

1. **Finalized-state safety.** L1 accepts only a proved state transition extending the stored canonical tuple. Builder signatures never substitute for execution validity.
2. **Bounded protocol recovery.** A builder cartel, an absent aggregator, an empty builder registry, and missing standbys cannot prevent an unsigned escape candidate from advancing the canonical state while the canonical prestate and any unexpired forced bytes remain reconstructible.
3. **Forced transaction inclusion.** A correctly prepaid L1 envelope at the queue head is consumed by the next canonical block after it becomes due; kind 0 is executed if valid bytes are supplied before its bounded validity or consumed-and-discarded otherwise, while durable kind 1 is always applied idempotently.
4. **Permissionless landing and recovery.** No allowlist, builder key, aggregator seat or payment recipient is required to submit a proof or an escape candidate.
5. **Deterministic lookahead.** Every implementation derives the same schedule from an authenticated finalized snapshot and the byte encoding in §4.

1.2 Non-claims

- A preconfirmation is not finality or insurance. Before normal close it may be revoked by builder equivocation, an unprovable skip, a withheld body, or an escape commit. Applications must price this as probabilistic/economic assurance and publish their own exposure limit.
- A bond cannot cap aggregate off-chain promises: the protocol has no reservation ledger and a builder can sign mutually exclusive promises. `L_LEASE` is deterrence and a recovery source, not guaranteed coverage.
- Protocol liveness is a capability claim. Economic liveness additionally assumes at least one actor can fund proof generation and an L1 transaction. A payment failure never invalidates an otherwise valid canonical transition.
- Censorship by L1 for longer than `T_INCLUDE_MAX_SECONDS`, failure of L1 finality, or a broken proof system is outside the model.
- A genesis campaign creates only a finite admission fence, not a persistent checkpoint. During its inclusive block window the legacy proxy rejects new proposals and forced inputs before retaining value, but existing proof-finalization and withdrawal paths remain live. Capped exact scans bind any suffix and forced range that successful migration will abandon, plus all data and time horizons needed for safe resume. Proof verification,

legacy checkpoint/freeze and target adoption then occur in one reverting transaction. If no proof lands by the campaign deadline, admissions reopen automatically and the exact legacy state resumes. A total proving-system outage can delay migration but cannot newly halt a legacy state that was processable before the fence. Resume does not cure a pre-existing invalid proposal or missing archive; such a deployment may need the separate state-migration path.

- A supported source domain finality failure or unauthorized change to its profile-pinned Bridge registry is outside the kind-1 safety claim. It cannot authorize a kind-0 user envelope.
- A source message is not yet an admitted forced envelope. Any preparer must supply its full Message preimage and queue deposit to the same-L1 adapter; source emission alone is not reported as forced-queue acceptance. If nobody enqueues before the record's bounded `enqueueBy`, permissionless source cancellation returns its ETH escrow and permanently forbids later enqueue. Launch V2 is DIRECT-only. ERC20, ERC721, ERC1155 and every official Vault flow remain byte-for-byte V1 and cannot originate a V2 credit. A future asset protocol requires a new kind, domain, release, codec and audit.
- The immutable V2 custody kernel is not self-healing. A defect in its compiled SourceBridge, DestinationBridge, inbox store, accumulator, history router, quota manager, liquidity pool or fixed Merkle verifier cannot be patched for an already-live credit without introducing an authority able to forge delivery/failure and take reserved value. This design chooses fail-closed immutability; production status therefore requires the compiled-code, formal-invariant and differential-test gates in §14, not merely this document.
- A state root is a binding commitment, not an erasure code. If every copy of the canonical state trie, executed bodies, runtime-bytecode preimages and required witness data is destroyed after Ethereum's applicable data-retention horizon, a new prover cannot reconstruct the preimage from L1 roots. Arbitrary-duration recovery after total canonical-data loss is therefore not claimed.

2 Actors and trust model

Builder.
A registered address eligible for scheduled slots. Builders may equivocate, skip parents, withhold bodies, collude, or disappear.

Aggregator.
The current auction seat, expected to collect data, prove and submit normal candidates. It has no exclusive consensus authority.

Prover/lander.
Any address. It supplies a validity proof and names an optional reward beneficiary in the proof statement. Proof validity, not identity, authorizes transition.

Canonical archive.
Any untrusted, replaceable source of canonical L2 bodies, state-trie nodes and runtime-bytecode preimages sufficient to reconstruct the current prestate. Each bytecode object is addressed by its legacy-Keccak `codeHash`; this includes code reached through proxy, beacon or delegation indirection. Content is verified against the L1-canonical block/state roots, so correctness does not require trusting the source; liveness requires at least one complete copy to remain reachable.

Forced-message sender.
Any L1 account that submits a gas-bounded, prepaid L2 transaction envelope.

Bridge source domain.
For kind 1 only at launch, settlement Ethereum, identified by chain ID, genesis hash, fresh immutable non-proxy SourceBridge, non-proxy credit registry, immutable terminal verifier and namespace. The adapter direct-reads its append-only credit record on the same L1. Registry governance is an external safety dependency; a current resolver lookup never substitutes for the emission record.

Protocol governance.
Delayed governance selects each validity verifier, execution profile and Settlement implementation. It is already a safety root: an authorized malicious verifier can finalize an arbitrary state transition. V2 recovery adds no stronger writer. Each immutable Settlement records its own proof-bound canonical history internally; the protocol-lifetime router only registers exact version/code identities and reads those histories. It has no governor, canonical-write callback or caller-supplied-root setter. This root includes the exact authorized creation and runtime bytecode. A malicious or compromised governance that deliberately approves code whose DSV1/TCS2 getters lie, or a validity verifier that accepts false transitions, is outside the protocol safety claim; either capability can already choose an arbitrary state machine. EXTCODEHASH and fixed getters prevent substitution and detect unexpected live state, but are not claimed to prove semantic honesty of root-authorized bytecode. The pinned reproducible-build and constructor- trace verifier below is a mandatory, independently reproducible release gate for honest governance, not a self-attesting on-chain capability. Its complete input bundle, executable hash, report and output hashes are public before the delay starts. Permissionlessness applies to auction participation, proving, landing, recovery and execution of a mature typed operation; it does not mean that arbitrary accounts may select a new verifier or implementation.

Bridge message archive.
Any untrusted, replaceable source of the full Message bytes emitted on L1. Correctness follows from `msgHash`; successful destination execution needs one copy until processing. Loss before `enqueueBy` leads to source cancellation; loss after the L2 pin leads to permissionless destination expiry and failure. Source DIRECT ETH remains recoverable by credit ID from the SourceBridge's pull-refund record, without reconstructing Message bytes.

L1.
Ethereum provides canonical ordering, timestamps, EIP-4788 beacon roots, EIP-2935 historical block hashes, blob commitments and the KZG point-evaluation precompile.

The safety threshold contains no honest-builder assumption. Honest builders improve normal throughput and preconfirmation reliability; the unsigned escape lane provides state and forced-queue progress when all of them collude. The L1-liveness assumption includes transaction inclusion and at least 64 canonical L1 blocks becoming available within `T_DEPTH_MAX=900s`; a longer L1 stall suspends, rather than violates, the L2 recovery clock. Archive operators have no protocol key, seat or exclusive API. Anyone may mirror or serve the same content-addressed data. This is an availability assumption, not an authority assumption; it limits long-horizon cold-start recovery but does not weaken proof soundness or permissionless submission.

3 Clock domains and canonical state

Before activation, Settlement is in `PREACTIVE`; candidate submission, session open/post/seal and every ordinary target-local state mutation are disabled, and slot arithmetic saturates. The sole exception is the immutable Router-only `adoptMigrationCanonicalV1` callback during lifecycle `ACTIVATING` under the exact nonzero activation context in §12. Off-chain data preparation may proceed but cannot create a target session. L2 slot zero is fixed by `GENESIS_TIMESTAMP`, with

```
currentL2Slot=max(0,block.timestamp-GENESIS_TIMESTAMP).
```

L2 slot and every deadline are timestamp seconds. L1 confirmation depth is counted in L1 block numbers. Beacon slot is used only in schedule authentication. Implementations MUST NOTsubstitute one clock for another. `settlementChainId` is the L1 chain executing Settlement and domains every L1 contract commitment; `l2ChainId` is the EVM chain ID enforced for L2 transactions and execution. They are independent immutable launch constants and are both explicit in every cross-domain proof statement and builder header.

The L1 contract stores a proved core and local L1 metadata:

```
CanonicalCore = (l2BlockNumber, tipHash, tipSlot, stateRoot, messageCursor,
                 winningDataCommitment, nextBaseFee, nextExcessBlobGas,
                 terminalRoot, terminalCount)
Canonical = (CanonicalCore core, uint64 canonicalizedAtBlock)
```

The proof binds `baseCanonicalHash`, the hash of both stored objects, but outputs only a new `CanonicalCore`. Settlement compares every base field with storage, verifies the proof, writes the explicit proved core, and only then sets `canonicalizedAtBlock=block.number`. A prover is therefore never asked to predict the L1 block in which its transaction will land. No code attempts to read or store `blockhash(block.number)`, which does not exist during that block.

`l2BlockNumber` is the canonical EVM height, distinct from the one-second slot because slot gaps are legal. A candidate with `blockCount=n` proves `endL2BlockNumber=base.l2BlockNumber+n`; every executed header uses the next consecutive EVM block number. While the deployed `ICheckpointStore` uses `uint48`, launch and every transition additionally require `endL2BlockNumber<=UINT48_MAX`; changing that interface and bound is a protocol-version upgrade. The canonical event always emits `(l2BlockNumber,tipHash,stateRoot,terminalRoot,terminalCount)`. The proof constrains the latter pair to the exact post-state slots of the protocol-lifetime `TerminalAccumulatorV2`; Settlement never accepts them as unverified auxiliary calldata. In the same internal storage transition that writes `Canonical`, Settlement increments a checked `uint64canonicalSequence` and writes the complete sequence-tagged canonical read tuple to its 256-cell history ring. A second ordinary canonical transition requires `block.number>lastCanonicalCommitBlock`; it cannot overwrite two cells in one L1 block. FROZEN instances cannot execute any canonical path. PREACTIVE instances reject every ordinary canonical path; their one activation callback writes sequence zero for GENESIS_IMPORT or checked `sourceCanonicalSequence+1` for VERSION_MIGRATION, the complete history cell, `canonicalizedAtBlock=lastCanonicalCommitBlock=block.number`, and the ordinary canonical event in one transition. Recording this ring is an internal Settlement write, not a router storage write, reward dependency or best-effort publication. `nextBaseFee` and `nextExcessBlobGas` are the exact fork-rule outputs needed to construct the next header without retrieving a prunable historical parent body/header. The proof derives them from the last executed header; Settlement stores them but does not calculate fork arithmetic.

A candidate has one prior-block anchor. Normal activation stores the hash of its earlier arm block using native `blockhash`; candidate submissions hash the supplied execution header to that stored value and use MPT proofs, including the historical forced-queue root/count. EIP-2935 is used later to authenticate that arm hash for equivocation evidence. A recovery round instead stores `(block.number-1,blockhash(block.number-1))`, directly stores the current queue root/count, and requires that anchor to reach depth `F_L1`. The EVM does not expose the parent timestamp. At candidate submission Settlement hashes and parses the supplied RLP parent header, requiring its number/hash to equal the frozen pair; its state root and timestamp become verifier-statement fields but are not separately frozen scalars. Current queue state is never claimed to be part of the parent's historical state root. An L1 reorg rolls the stored canonical state and anchor back together. Initial values and bridge cursors are migration inputs in §12.

Every canonical or ordinary new-work state-mutating entry point begins with `sync()`, except the exact DataSession open/post/seal sidecars, claims, surplus sweep and mode-specific maintenance frozen in §6. If `sync()` changes mode or commits a mature normal candidate, a wrapper that uses this rule `MUST`return `SYNCED` successfully before parsing or validating the caller's candidate. The caller resubmits against the new version. This rule is necessary because a later Solidity revert would otherwise roll back the sync transition in the same transaction.

4 Builder registry and exact lookahead

4.1 Registry lifecycle

A registration contains a nonzero address, a `uint192` bond not exceeding `BOND_MAX`, a monotonic `uint64registrationIndex`, an `effectiveL2Slot`, and separately reserved `L_LEASE[window]` tranches. Address zero is permanently `VACANT`. Duplicate live addresses and reuse while any prior generation remains active or liable are forbidden. Once the old generation's last evidence/reorg deadline has passed, all tranches are terminal, its liability leaf is safely cleared and no evidence can still land, the address may register again with a fresh `registrationIndex`. No permanent address tombstone mapping exists. With `currentWindow=floor(currentL2Slot/384)`, a reservation is accepted only for `currentWindow<=W<=currentWindow+MAX_TRANCHE_AHEAD_WINDOWS`, where the initial ahead bound is 16; an exit request permanently disables new reservations for that generation. Only a present *active* generation with no exit request and no tombstone may execute `FREE->RESERVED`; a replacement or exit atomically closes reservations before moving the old generation into liability. Liability records may only be slashed or released, never extended with a new reservation.

The active table has exactly 64 indexed cells. On accepted registration, `effectiveL2Slot=currentL2Slot+384*ENTRY_DELAY_WINDOWS`; the stored value, not a later recomputation from a window boundary, is authoritative. Thus a registration becomes effective only after `ENTRY_DELAY_WINDOWS=8`. If the table is full it may replace only the smallest-bond effective live entry, breaking ties by greatest registration index, and only with a strictly greater bond. At most `MAX_GENERATION_MOVES_PER_WINDOW=4` replacements or effective exits are processed. Exit requests receive a monotonic sequence and mature at `requestWindow+EXIT_DELAY_WINDOWS`. Every replacement path first processes the oldest matured exits by `(matureWindow,exitSequence)` within the window's remaining movement budget; if any matured exit remains, replacement rejects. Anyone, including the exiting builder, may run this bounded maintenance. Thus transaction ordering cannot starve an eligible exit. A displaced generation is moved, with its tranche root and bond, into the fixed `MAX_LIABILITY_GENERATIONS=1,072` liability ring; an occupied ring cell is never overwritten before its release predicate holds. Registration therefore rejects, rather than losing slashable history, when the replacement budget or ring is full.

The six-level `registryRoot` over the 64 active cells commits each cell's mutable tranche root and therefore changes on reservation transitions. It is used only by schedule sealing. A separate eleven-level `admissionRoot` over 2,048 fixed positions contains stable generation identity for the 64 active and 1,072 liability records plus canonical empty padding; its leaves do *not* contain tranche roots. Only generation admission, movement, tombstoning, slashing or release increments `admissionVersion` and updates `admissionRoot`. A `FREE/RESERVED/LIABLE` tranche transition updates the tranche and registry roots but cannot invalidate signed blocks or proofs by changing the admission pair. The admission root retains displaced scheduled keys. Normal activation and every recovery round freeze the pair `(admissionVersion,admissionRoot)`. Each scheduled signer carries an inclusion proof for a non-tombstoned record effective at that L2 slot; a slash cannot retroactively invalidate an already frozen window. Outside an already-open normal window or live recovery round, a tombstone makes sealed future tickets at or after its effective slot return `VACANT`; inside that frozen candidate context the stored admission pair remains authoritative until close/commit/expiry.

Admission positions are canonical: active cell i is always position i for $0 \leq i < 64$; liability ring slot j is always position $64 + j$ for $0 \leq j < 1,072$. A generation move uses `j=movementSequencemod1072`. The transaction first proves/releases the old occupant, writes the moved generation at that same slot, clears its active cell or writes the replacement, increments `movementSequence`, then updates both affected roots and increments `admissionVersion`; the new admission pair is published atomically. Reservation-only mutations publish only the registry root and leave that pair byte-for-byte unchanged. There is no first-free search, compaction or caller-chosen liability position.

Eligibility is evaluated before ranking. A cell's effective L2 slot must be no later than the source-derived L2 slot, its fully reserved tranche for W must be proven, and its tombstone-effective L2 slot must be later than the source. All 64 registry cells are supplied; only present cells supply tranche-ring proofs at `Wmod512`. A canonical `EMPTY` tranche leaf is valid input and makes the cell ineligible; it need not claim window W. A nonempty leaf with a different stored window is likewise ineligible, not a seal revert. Only an exact `RESERVED` leaf with stored window W and the full `L_LEASE` amount is eligible. The 64 eligible entries with greatest bond, then smallest registration index, form the snapshot set. Supplying proofs only for purported winners is invalid because it cannot prove omission-free ranking.

An exit request becomes eligible for processing only after `EXIT_DELAY_WINDOWS=MAX_LIVE_WINDOWS`; it may wait behind the same bounded FIFO of older matured exits, and the bond remains escrowed until every generation and tranche is releasable. A tranche follows `FREE->RESERVED(W)->LIABLE(W)->RELEASED` or `RESERVED/LIABLE->SLASHED`. Every candidate block slot must be at least the normal window's frozen minimum (or the recovery round start), not merely its tip. Let `windowEndSlot(W)=384*(W+1)-1`. Release requires `windowEndSlot(W)<globalMinReferencedSlot`, no stored best or active round references it, and the absolute evidence and L1-reorg deadlines have expired. At that point maintenance atomically marks the schedule cell `EXPIRED` and may release its tranches; an unexpired sealed record is never overwritten. `evidenceDeadline(W)=GENESIS_TIMESTAMP+384*(W+`

1)+EVIDENCE_DELAY_SECONDS; evidenceReplayDeadline(W)=evidenceDeadline(W)+REORG_MARGIN_SECONDS is both the contract's last admissible evidence time and the tranche's absolute release boundary. Evidence intended to survive a reorg must first be submitted by evidenceDeadline(W). globalMinReferencedSlot is the minimum of the current dynamic floor, an open normal window's frozen floor, an active round's start and the first slot of any stored best (absent values are ignored). Slot indices are never compared with timestamps or window numbers.

The liability-capacity proof is a launch invariant, not an assertion by inspection. A generation moved in window C can have no reservation beyond C+MAX_TRANCHE_AHEAD_WINDOWS. Therefore

```
MAX_LIABILITY_RESIDENCE_WINDOWS =
  MAX_TRANCHE_AHEAD_WINDOWS + 1 +
  ceil((EVIDENCE_DELAY_SECONDS + REORG_MARGIN_SECONDS) / 384) + 2
```

must be strictly less than MAX_LIVE_WINDOWS=268. The two extra windows cover boundary ordering and bounded maintenance. At four moves per window, the ring position reused 268 windows later is consequently releasable. Each generation maintains an exact unreleased-tranche counter, so release and reuse do not scan 512 leaves.

4.2 Authenticated snapshot

For aligned window W, let its start timestamp be GENESIS_TIMESTAMP+384W. The target is the greatest BEACON_GENESIS_TIME+12k not later than eight L1 epochs before that start; L2-genesis alignment is irrelevant. Define sealDeadline(W)=windowStart-H_LOOK. A seal is valid only when its carrier execution block has F_FINAL L1-block depth and block.timestamp<sealDeadline(W). Beacon slots are not used as a surrogate for execution-block depth. At or after the deadline an unsealed window is permanently all-VACANT; a late transaction cannot change it. The contract performs this procedure:

1. The contract itself, not caller data, queries EIP-4788 for $j = 1, \ldots, 64$ at q=targetTimestamp+12j; missing timestamps revert and are skipped. The first successful query identifies the *carrier* execution block. EIP-4788 returns its parent beacon-block root, not the carrier root.
2. For the first successful query the caller supplies the carrier execution header. EIP-2935 authenticates its hash and block number; the header timestamp equals q and its parent_beacon_block_root equals the EIP-4788 result. Fork-specific SSZ branches in that parent authenticate its slot and one execution payload's blockHash, stateRoot, prevRandao and timestamp. The parent slot must be at or before target, its payload timestamp must equal its beacon-slot time, and its payload block hash must equal the carrier header's parent hash. No carrier observed by all 64 contract calls, a malformed header, a later parent, or a fork/gindex mismatch reverts without recording state. Before the deadline only a completely valid seal writes; adversarial query timing or witness bytes can never manufacture a vacant seal. At or after the deadline, consumers deterministically synthesize permanent all-VACANT only from the objective absence of a stored valid seal.
3. Verify Merkle–Patricia storage proofs from that execution state root for the registry header and registryRoot. The caller supplies all 64 canonical serialized cells, including empty cells; recompute their six-level Keccak tree and require equality to registryRoot. Filter eligibility and order the entries, then store the byte-exact 64-leaf entryRoot and seed. Every *present* active cell supplies its tranche leaf and nine-sibling proof at index Wmod512. An

absent registry cell has `trancheRoot=0`, supplies no tranche proof and is synthesized ineligible. For a present cell, canonical `EMPTY`, `FREE`, or a nonempty leaf for a different stored window is valid but ineligible; only exact `RESERVED(W)` with `amount=L_LEASE` is eligible. Proofs occur in ascending cell-index order, and extra/missing proofs reject. The fixed cell count proves completeness with one MPT path rather than 64 independent storage paths. The carrier satisfies `currentExecutionBlock-carrierBlockNumber>=F_FINAL`; the source payload is necessarily older.

`sealWindow` is permissionless. An optional maintenance distributor may pay its event in a later transaction. At or after its deadline a missing stored seal is consumed as `VACANT`; a failing pre-deadline seal attempt never stores that result and cannot give an address an unauthorized ticket. The snapshot age plus 64-slot carrier scan, finality and seal margin fits the eight-epoch geometry and remains far below EIP-4788's 8,191-slot history window. Consensus constants include the Deneb SSZ generalized indices; an L1 fork that changes them requires an explicitly registered verifier, never runtime guessing.

For Deneb, Electra and Fulu, the generalized index from `BeaconBlock` root to `slot` is 8 and to `body.execution_payload` is 201. Within the composed tree, payload `state_root`, `prev_randao`, `timestamp` and `block_hash` are 6,434, 6,437, 6,441 and 6,444 respectively. The seal input names the L1 fork digest; only these configured digests are accepted. The protocol-lifetime ScheduleOracle holds a one-shot append-only `forkDigest->(verifier, codeHash,configHash,gasLimit)` registry controlled only by a delayed ProtocolVersionManager manifest. Seal dispatch uses a bounded `STATICCALL`, exact return length and rechecked `EXTCODEHASH`; failure reverts and leaves the window unsealed; it cannot mutate an old seal. Gloas or any fork that changes the container requires a reviewed verifier, constants, fixtures and delayed registration before its activation. That registration may change witness parsing only: it cannot change the protocol-lifetime seed, eligibility, allocation or placement rules.

The fork registry and verifier wire format are exact. The deployment constructor installs one reviewed current-fork row at `firstWindow=0`. Every later row is appended in strictly increasing `firstWindow` order by the one typed PVM operation:

```
installForkVerifierV1(
 bytes4 forkDigest,uint64 firstWindow,address verifier,bytes32 runtimeHash,
 uint64 beaconSlotGindex,uint64 executionPayloadGindex,
 uint64 stateRootGindex,uint64 prevRandaoGindex,uint64 timestampGindex,
 uint64 blockHashGindex,bytes32 witnessSchemaHash,
 bytes32 configurationHash,bytes4 selector,uint64 gasLimit)
 returns (bytes4 magic,bytes4 returnedForkDigest,uint64 returnedFirstWindow)
forkVerifierRegistrationV1(bytes4 forkDigest) returns
 (bytes4 magic,bytes4 returnedForkDigest,uint64 firstWindow,
  bytes4 successorForkDigest,uint64 successorFirstWindow,address verifier,
  bytes32 runtimeHash,bytes32 configurationHash,bytes4 selector,
  uint64 gasLimit)
scheduleForkVerifierConfigV1() returns
 (bytes4 magic,bytes4 forkDigest,uint64 beaconSlotGindex,
  uint64 executionPayloadGindex,uint64 stateRootGindex,
  uint64 prevRandaoGindex,uint64 timestampGindex,uint64 blockHashGindex,
  bytes32 witnessSchemaHash,bytes32 configurationHash)
```

The selectors are `0xf171816c`, `0xc614591c` and `0x44efa773`; return lengths/magics are

96/FVI1 `0x46564931`, 320/FVR1 `0x46565231` and 320/SFV1 `0x53465631`. Install is PVM-only and receives exactly 500,000 gas. The verifier getter is a 100,000-gas STATICCALL returning the complete row. PVM checks runtime, getter, configuration formula, unused digest and `firstWindow>=currentWindow+MAX_EARLY_SEAL_WINDOWS+1`, and requires `100000<=gasLimit<=5000000`. It sets the previous latest row's formerly zero successor pair once and appends the new row; neither boundary can later change. The getter for the latest row has a zero successor pair. Thus a digest is valid for window W iff `firstWindow<=W` and either successor is zero or `W<successorFirstWindow`; `sealWindow` rejects every caller-selected row outside that unique interval.

The exact configuration layers are

```
forkConstantsHash = H(
 "slot-chain-schedule-fork-constants-v1" || u64(beaconSlotGindex) ||
 u64(executionPayloadGindex) || u64(stateRootGindex) ||
 u64(prevRandaoGindex) || u64(timestampGindex) || u64(blockHashGindex))
scheduleForkOutputSchemaHash = H(
 "ScheduleCarrierOutputV1(bytes32 statementHash,uint64 parentSlot,"
 "uint64 executionBlockNumber,uint64 payloadTimestamp,bytes32 blockHash,"
 "bytes32 stateRoot,bytes32 prevRandao)")
configurationHash = H(
 "slot-chain-schedule-fork-verifier-config-v1" || forkDigest ||
 forkConstantsHash || witnessSchemaHash || scheduleForkOutputSchemaHash ||
 bytes4(0x7e981e0b) || u64(gasLimit))
```

For Deneb, Electra and Fulu the six gindices are exactly 8, 201, 6,434, 6,437, 6,441 and 6,444 in that order. A later fork supplies its reviewed constants and witness schema through a new delayed row; an opaque configuration hash is not accepted.

ScheduleOracle authenticates the caller-supplied beacon root through the exact EIP-4788 path, then STATICCALLs the interval-selected verifier using only

```
verifyScheduleCarrierV1(bytes witness,bytes32 beaconBlockRoot)
 returns (bytes4 magic,bytes32 statementHash,uint64 parentSlot,
          uint64 executionBlockNumber,uint64 payloadTimestamp,
          bytes32 blockHash,bytes32 stateRoot,bytes32 prevRandao)
```

Selector is `0x7e981e0b`, return is exactly 256 bytes and magic is SFC1 `0x53464331`. Calldata has selector, a two-word head with bytes offset `0x40`, one minimal contiguous witness tail, zero padding and no suffix; witness length is nonzero and bounded by the row's reviewed schema and `gasLimit`, with an absolute 131,072-byte protocol cap. Oracle recomputes

```
statementHash = H(
 "slot-chain-schedule-carrier-statement-v1" || u256(settlementChainId) ||
 address20(scheduleOracle) || forkDigest || u64(W) || beaconBlockRoot ||
 u64(parentSlot) || u64(executionBlockNumber) || u64(payloadTimestamp) ||
 blockHash || stateRoot || prevRandao)
```

from the six returned payload fields and requires exact equality before using any leaf. It then repeats the existing carrier age, parent, timestamp and execution-header checks and uses only the returned state root/randao. Revert, OOG, wrong interval/code/config/magic/hash, short/trailing returndata, bad padding or inconsistent header reverts with no write; only a consumer at or after `sealDeadline(W)` interprets the still-unsealed window as VACANT, while a valid pre-deadline retry remains possible.

4.3 Consensus byte encoding and allocation

All hashes use Ethereum legacy Keccak-256. Concatenation below is packed fixed width; strings are the literal ASCII bytes without a length prefix:

$$\begin{aligned} seed_W &= H(\texttt{"slot-chain-seed-v2"} \| u256(settlementChainId) \\ &\quad \| u256(W) \| bytes32(prevRandao)), \\ qTie_i &= H(\texttt{"slot-chain-quota-tie-v1"} \| seed_W \| u64(regIndex_i)), \\ sTie_{p,i} &= H(\texttt{"slot-chain-slot-order-v1"} \| seed_W \| u16(p) \| address20(i)). \end{aligned}$$

Starting from zero, capped D'Hondt awards each of 384 tickets to the unsaturated entry maximizing `bond/(allocation+1)`, with `Q_MAX=76`. Quotients are compared by cross multiplication; a `uint192` bond times `Q_MAX+1` cannot overflow `uint256`. Equal quotients use smallest `qTie`. Remaining tickets are `VACANT`; six addresses are required to fill a window.

Placement processes positions in order. A real address is feasible if it has a ticket remaining, differs from the preceding real address, and removing it preserves `remaining[a] <=otherRemaining+1` for every real address. Choose the feasible value with smallest `sTie`; `VACANT` may repeat. The executable model and its golden vectors are normative test fixtures.

The seed and allocation/placement algorithm are protocol-lifetime scheduling rules, not release-profile rules. In particular, neither a seal nor its seed contains a Settlement protocol version. A sealed future window therefore has identical meaning before and after a Settlement migration, matching the protocol-lifetime ScheduleOracle and retained reservations. An incompatible scheduling change cannot ride an ordinary profile migration: it requires a separately specified schedule-epoch transition that waits until every old-epoch seal and liability is outside the retention horizon. This document defines no such transition.

5 Blocks, promises and proof statement

5.1 Signed header

A builder signs exactly:

```
(settlementChainId, l2ChainId, protocolVersion, verifyingContract,
 slot, parentHash,
blockHash, stateRoot, bodyRoot, anchorNumber, anchorHash,
forceRoot, forceCutoff, messageStart, messageEnd, dataManifestRoot,
coinbase, tier, contextId, admissionVersion, admissionRoot,
 episode, recoveryRevision, recoveryId)
```

The normative EIP-712 type string is:

```
SlotChainBlock(
```

```
uint256 settlementChainId,uint256 l2ChainId,uint256 protocolVersion,
address verifyingContract,
uint64 slot,bytes32 parentHash,bytes32 blockHash,bytes32 stateRoot,
bytes32 bodyRoot,uint64 anchorNumber,bytes32 anchorHash,bytes32 forceRoot,
uint64 forceCutoff,uint64 messageStart,uint64 messageEnd,
bytes32 dataManifestRoot,address coinbase,uint8 tier,
bytes32 contextId,uint64 admissionVersion,bytes32 admissionRoot,
uint64 episode,
uint64 recoveryRevision,
bytes32 recoveryId)
```

The hashed ASCII has one space between each field type and field name and no other whitespace; displayed line breaks, indentation and spaces after commas are typography only. The exact single-line literal and a standalone Keccak implementation are in `commitment-model.py`. `TYPEHASH` is legacy Keccak-256 of those ASCII bytes and equals the concatenation of these two fragments:

```
0xee6a8c8e31e8245cd527869508f6e464
d6084893991203876f734d1855aed87c
```

The EIP-712 domain is `("SlotChain","2",settlementChainId,verifyingContract)`. The explicit `l2ChainId` is the EVM `CHAINID` used by raw L2 transactions. Signatures are exactly 65-byte `r||s||v`; `v` is 27 or 28, s is low, and zero or recovered-zero addresses reject. EIP-2098 and EIP-155 transaction-style `v` values are not accepted. Tier 1 requires

```
contextId = H("slot-chain-normal-context-v1" || baseCanonicalHash ||
              u64(admissionVersion) || admissionRoot ||
              u64(armBlockNumber) || armBlockHash)
```

and its execution anchor is exactly that arm block. `episode=recoveryRevision=0` and `recoveryId=0`; tier 2 requires the exact live round values and `contextId=recoveryId`. The unsigned tier 3 uses that same recovery context. Consequently semantically unused bytes cannot create false equivocation evidence. The execution `blockHash` excludes the off-chain builder signature, avoiding a circular hash definition. The signed `bodyRoot` covers only canonical discretionary transaction bytes; the proof derives the full Ethereum transactions root after adding the profile-defined system and valid forced transactions.

5.2 Three validity tiers

For every tier and every block, the circuit requires the EVM header `timestamp=GENESIS_TIMESTAMP+slot` with checked addition, and derives `blockHash` from that exact header. The signed relative slot can therefore never disagree with EVM time, forced-expiry classification, or the canonical tip clock. Every signed block also carries the exact frozen `admissionRoot`; the circuit checks it together with `admissionVersion`.

1. **Normal signed.** Every block is signed by its scheduled builder under the normal window's frozen `(admissionVersion,admissionRoot)`. Slots strictly increase, each slot is no later than `currentL2Slot+CLOCK_SKEW`, and the first slot strictly exceeds the canonical base slot. Every block is at or above the window's frozen `minAdmissibleSlot`; parent gaps are

at most `G_MAX=3,600`, equal to the objective final-lag trigger and within the 12-window schedule-proof cap.

2. **Recovery signed.** The first block extends the L1-final canonical tuple, binds the active episode and current recovery revision, and may cross an unbounded time gap. Every block still requires its scheduled builder signature under the current recovery round's frozen admission pair. Every slot is at or after that round's `roundStartSlot`.
3. **Escape unsigned.** Exactly one deterministic block extends the L1-final canonical tuple during recovery. It has no builder signature, no beneficiary-controlled coinbase, no discretionary transaction and no blob body. It contains only the protocol anchor and the deterministic maximum prefix of the current round's frozen forced queue. When the queue is empty an SLA-triggered episode may commit an empty anchor-only escape block. Its slot is `max(roundStartSlot+ESCAPE_OFFSET,canonical.tipSlot+1)` and its anchor, queue root/cutoff, block hash and state root are unique functions of the current recovery round and proved execution result.

Every candidate uses exactly one `batchAnchor`; every block repeats that anchor and frozen `forceRoot/forceCutoff`. For a normal candidate the supplied RLP execution header hashes to the EIP-2935 value, its timestamp is no later than the first L2 block timestamp, and MPT proofs under its state root authenticate the historical forced queue and any other contract state. For recovery, the RLP header authenticates the stored parent number/hash and derives its timestamp and state root; the queue pair is authenticated directly by the round fields and is not asserted against that header's state root. Requiring one anchor, not up to 4,096 independent anchors, bounds both circuit and L1 work. Cutoffs therefore do not change within a candidate; the first blocks drain the frozen FIFO prefix and later blocks see an empty prefix.

The circuit verifies parent linkage, strict slot monotonicity, schedules, signatures for tiers 1–2, version binding, anchor/header/MPT authentication, forced-prefix ordering, execution, exact body/data coverage and public outputs. It rejects more than 4,096 blocks, 12 schedule windows, 16 sealed sessions, 2,100 referenced blob records, 256 forced items, 4 MiB forced bytes or 80 million total accounted forced gas per candidate. These candidate-level caps are independent of the per-block caps.

5.3 Verifier statement ABI

The L1 verifier accepts `verify(proof,uint256[2]digestLimbs)`. Settlement constructs the following Solidity struct in exactly this order and computes `statementHash` as legacy Keccak of the ASCII domain `"slot-chain-statement-v2"` followed by `abi.encode(S)`:

```
S = (uint256 settlementChainId, uint256 l2ChainId, uint256 protocolVersion,
     bytes32 executionProfileHash, address verifyingContract,
     uint64 releaseProtocolVersion, bytes32 releaseManifestHash,
     uint8 tier, bytes32 baseCanonicalHash,
     bytes32 candidateCommitment, uint64 blockCount,
     uint64 firstSlot, bytes32 tipHash, uint64 tipSlot,
     uint64 endL2BlockNumber, bytes32 endStateRoot,
     bytes32 endTerminalRoot, uint64 endTerminalCount, uint64 endCursor,
     bytes32 winningDataCommitment, uint64 nextDueAt,
     uint256 nextBaseFee, uint64 nextExcessBlobGas,
     uint64 anchorNumber, bytes32 anchorHash,
```

```
bytes32 anchorStateRoot, uint64 anchorTimestamp,
bytes32 forceRoot, uint64 forceCutoff,
bytes32 forcedDescriptorCommitment,
uint64 startCursor,
uint64 admissionVersion,
bytes32 admissionRoot,
uint64 episode, uint64 recoveryRevision, bytes32 recoveryId,
bytes32 scheduleRootsCommitment, uint8 scheduleWindowCount,
bytes32 sessionRefsCommitment, uint8 sessionCount,
uint16 dataRecordCount, bytes32 executionOutputsCommitment,
address proofBeneficiary)
```

`digestLimbs[0]` and `[1]` are respectively the high and low 128 bits of `statementHash`; neither is reduced modulo the proof field. The circuit recombines them and proves the exact preimage. Settlement independently checks the immutable `protocolVersion->executionProfileHash` mapping, the base canonical state, current/frozen versions and recovery round, recomputes the bounded schedule-root and sealed-session-reference commitments from calldata, recomputes the bounded forced-descriptor commitment from authoritative queue storage, recomputes `winningDataCommitment` from the candidate and sealed-session commitments, requires `endL2BlockNumber=stored.l2BlockNumber+blockCount`, and constructs the new core from the explicit `(endL2BlockNumber,tipHash,tipSlot,endStateRoot, endTerminalRoot,endTerminalCount,endCursor)` fields plus that recomputed value and proved `(nextBaseFee,nextExcessBlobGas)`. It recomputes `executionOutputsCommitment` including the same explicit terminal pair; the circuit proves that pair equals the accumulator's exact post-state slots, so neither calldata nor Settlement can substitute a root. It rejects zero/ill-typed fields, cursor regression, or disagreement with the last proved block, then stamps `canonicalizedAtBlock=block.number` outside the proof. `candidateId=statementHash`. `nextDueAt` is authenticated inside the frozen queue as the boundary leaf's deadline, or `UINT64_MAX` when `endCursor=forceCutoff`. It is only a proof-consistency output. Normal submission additionally verifies the current-root boundary proof described in §7. After every commit and on every activation check, the sole authoritative live head value is `ForcedQueue.dueAt(canonical.messageCursor)`; no recovery-round or canonical-core scalar caches it. For the one proof-carrying launch-activation or migration candidate, Settlement requires `releaseProtocolVersion=targetProtocolVersion` and `releaseManifestHash= targetManifestHash` from the authenticated launch or migration gate; every ordinary candidate requires both fields to be zero. The circuit uses the nonzero pair to require the first block's activation Anchor call and the exact manifest, and requires the exact release/domain seals and registered route to be active in every later block of that candidate. The L1 coordinator verifies this proof before final adoption, legacy FROZEN state or queue-authority change, then adopts its output and consumes the target pair in the same atomic cutover. The genesis campaign's bounded QUIESCENT phase changes neither the legacy canonical core nor Queue authority and automatically resumes ACTIVE at expiry; transaction-local READY cannot survive a revert. No ACTIVE target Settlement can be left waiting on a newly activated verifier to clear a pending release. `proofBeneficiary` is a nonzero proof input, never builder-signed block data or coinbase. It receives the consumed queue-fee interval and any separate proof reward; copying proof bytes cannot redirect either payment.

5.4 Preconfirmation semantics

A builder signature proves authorship and makes same-slot equivocation slashable. It does not prove that the body reached a prover or that the next builder saw it. A later scheduled builder may sign a profitable skip that the protocol cannot distinguish from non-receipt. Accordingly:

- before L1 candidate acceptance, a promise is *observed*;
- after acceptance and before close, it is *provisionally proved*;
- after normal close or recovery commit, it is *canonical*; and
- only the underlying L1 finality policy makes it *L1-final*.

Wallets and applications MUST NOTdisplay the first two states as final. An escape commit explicitly revokes every omitted observed or provisionally proved promise and returns omitted transactions to the mempool.

6 Blob data authentication

Data is posted into publisher-owned, bonded sessions. A session ID is

```
H("slot-chain-session-v1" || u256(settlementChainId) || address20(contract) ||
  address20(owner) || u64(sessionSequence))
```

The contract stores one checked `uint64nextSessionSequence`, initially zero. A successful open uses `s=nextSessionSequence`, requires `s<UINT64_MAX`, derives and returns the ID above, then stores `s+1`. The caller supplies neither ID nor sequence, and a rejected or no-cell open does not consume one. Sequence `UINT64_MAX` is the explicit exhaustion sentinel. The sequence is global rather than per owner: a Sybil cannot create permanent nonce keys, while the owner and Settlement address in the preimage prevent cross-owner and cross-release aliases. A fresh migration target starts at zero; the retired source retains its historical sequence.

DataSession is a bounded storage region of the immutable Settlement, and the contract address in the ID above is that Settlement. Settlement accepts native ETH only through the session open/post surfaces and has no Queue, reward, Market, Bridge or other native liability; each of those custodians is a separate account. Its configuration binds the shared migration gate and immutable top-level `dataRent` sink. Every other payable selector, fallback and receive path rejects, and conformance tests exercise `msg.value>0` against every such selector. This exclusive custody makes the balance invariant and surplus calculation complete without a cross-contract caller-supplied “pinned” or “boundary closed” flag.

The profile/runtime geometry is exact: 1,024 physical cells, at most two LIVE cells per owner, 2,100 records per LIVE cell, at most six blobs per post, eight maintenance inspections and an 86,400-second maximum TTL. Each cell has one exact semantic tag `FREE=0`, `LIVE=1` or `REFUND=2`. Storage is exactly `cells[1024]`, an ID-to-cell-plus-one mapping with at most one key per non-FREE cell, a live-owner-count mapping containing only owners with at least one LIVE cell, checked `liveCount`, `refundCount`, `occupiedCount=liveCount+refundCount<=1024`, and `gcCursor`. A REFUND cell stores only `(sessionId,owner,bondWei,claimDeadline)`; its old MMR words are stale and ignored. Clearing a cell deletes its ID key, and removing an owner’s last LIVE cell deletes its owner key. No refund or historical-owner mapping exists. The three counters and cursor are `uint16`; owner LIVE counts are also `uint16` and never exceed two. PREACTIVE and historical Settlements have zero local LIVE count. OPEN increments local

LIVE and occupied by one; LIVE-to-REFUND decrements local LIVE and increments refund without changing occupied; REFUND-to-FREE decrements refund and occupied. No other transition changes those fields. An eligible owner claim directly from LIVE executes those two logical transitions atomically: its net effect decrements local LIVE and occupied, deletes the owner/ID entries, and reduces LIVE liability and balance by the same bond; refund count and liability have net zero change.

There is deliberately no Router/global LIVE counter and therefore no cross-contract write on OPEN, claim or maintenance. The Router authenticates exactly one current Settlement. Inductively, a PREACTIVE target begins with zero LIVE cells; only the current ACTIVE Settlement may open one; READY requires that source's local LIVE count zero; and cutover freezes that zero-LIVE source before making the zero-LIVE target current. Abort leaves the same source current and never activates the target. The generation-authenticated READY handshake and activation-time static reads below close this induction.

The direct sidecars authenticate that premise through one exact bounded Router read, never through a caller-supplied flag:

```
activeSettlementStateV1() returns
  (bytes4 magic,address activeSettlement,uint64 generation,
   uint64 activeProtocolVersion,uint64 targetProtocolVersion,
   bytes32 targetManifestHash,bytes32 targetRegistrationHash,uint8 phase)
```

The magic is `0x41535231` (ASCII `ASR1`), phases are exactly ACTIVE=0, ARMED=1 and READY=2, and returndata is exactly 256 bytes with canonical ABI padding. OPEN, POST and SEAL read the immutable Router with an exact 50,000-gas static call. They reject revert/OOG/short/trailing or noncanonical data, and require the returned active address to equal this Settlement, the returned active version to equal its immutable protocol version and phase ACTIVE. ACTIVE requires the returned target version, manifest and registration hash to be canonical zero; ARMED and READY require all three nonzero. The finite genesis campaign never adds a public gate phase: the constructor-bound legacy branch remains ACTIVE until the atomic switch. This check and the session mutation share one EVM revert domain.

Payable `openSession(uint16expectedEmptyCell,uint64expiry)` neither calls `sync` nor performs GC. It requires the current non-PREACTIVE component, the shared gate ACTIVE, `expectedEmptyCell<1024`, that exact tag FREE, owner/live/sequence capacity and the exact profile-priced value. It derives and returns the session ID and increments the global sequence only on success. Every failure reverts, returning value and preserving sequence, cursor, cells and liabilities. A cell-hint race therefore costs the winner the full rent and bond but gives no identity authority. Payable `postData` follows the same sidecar rule: it checks current target and ACTIVE directly and reverts on every failure; it does not run a canonical transition while holding caller value.

The native-value split is exact. The symbols below name these exact profile fields:

```
BOND     = refundableBondWei
BASE     = baseRentWei
BYTE     = rentPerPublishedByteWei
BLOB_BPS = blobBaseFeeMultiplierBps
```

The profile requires `BLOB_BPS<=10000`. Opening requires checked `msg.value=BOND+BASE`; only `BOND` increments the LIVE liability and `BASE` is immediately surplus. A post with n manifests requires $1 \le n \le \min(6, 2100 - count)$, nonzero `BLOBHASH(i)` for every $i < n$ and `BLOBHASH(n)` `=0`, so the manifest array is in one-to-one order-preserving correspondence with every blob in that transaction. Define

```
publishedBytes = sum(manifest[i].chunkByteLength)
blobGasUsed    = 131072 * n
blobWeight     = blobGasUsed * BLOB_BPS
blobSurcharge =
  mulDivUp(blobWeight, block.blobbasefee, 10000)
postRent       = publishedBytes * BYTE + blobSurcharge
```

over mathematical nonnegative integers; `mulDivUp(a,b,d)` is exactly $\lceil ab/d \rceil$ using a full 512-bit product and reverts if the result does not fit `uint256`. L1 checks the declared `chunkByteLength<=126972`, but cannot read or decode blob contents. The validity circuit later checks the blob's length prefix, zero padding, decoded bytes and computed chunk root against that declaration. The implementation must produce the exact ceiling without an intermediate-width approximation and then require every result and `postRent` to fit `uint256`; otherwise it reverts. The post requires `msg.value==postRent`; underpayment and overpayment both revert. All post value is immediately surplus and never enters either bond liability. Zero-byte padding still consumes a whole blob and therefore its whole blob-gas surcharge. Any blob/KZG/MMR/value/arithmetic failure rolls back the complete call, including the value, record count, peaks and emitted events.

The externally callable surface is frozen below. `DataPostV1` is the static tuple

```
(bytes32 fullBodyRoot, uint16 blockOrdinal, uint16 chunkIndex,
 uint16 chunkCount, uint32 chunkByteLength, bytes32 chunkRoot, bytes32 y,
 bytes32 commitmentHi, bytes16 commitmentLo,
 bytes32 proofHi, bytes16 proofLo)
```

where concatenating each high/low pair yields exactly 48 bytes; `bytes16` ABI right-padding must be zero. The expanded selectors and returns are:

```
openSession(uint16,uint64) payable returns (bytes32 sessionId)
postData(bytes32,(bytes32,uint16,uint16,uint16,uint32,bytes32,bytes32,
  bytes32,bytes16,bytes32,bytes16)[]) payable
  returns (uint16 firstRecordIndex,uint16 newCount,bytes32 newRoot)
sealSession(bytes32) returns (uint16 count,bytes32 root,uint64 expiry)
maintainDataSessions() returns
  (uint8 status,uint8 inspected,uint8 changed,uint16 nextCursor)
claimSessionBond(bytes32,address) returns (uint256 amount)
sweepSessionSurplus() returns (uint256 amount)
```

Open, post and seal are direct session sidecars: none calls `sync` or moves the maintenance cursor, all recheck current Settlement and ACTIVE, and every invalid input or state reverts. POST and SEAL additionally require `block.timestamp<expiry`; equality is already expired and is eligible for ordinary conversion or a direct claim. Seal also requires a nonempty unsealed LIVE session owned by the caller. Maintenance status is exact: NOOP=0, SYNCED=1 and SCANNED=2. Current ACTIVE first invokes `sync`; if it changes state the call returns SYNCED without a separate scan, otherwise it performs an ordinary exact scan and returns SCANNED.

Current ARMED returns the result of its sync-owned migration work as SYNCED, or NOOP while a boundary remains open. READY/FROZEN and historical components perform only the overdue-REFUND scan; PREACTIVE returns NOOP. SCANNED reports eight inspections, while the other statuses report zero explicit maintenance inspections even if ARMED `sync` performed its separately accounted migration scan. An invalid claim reverts; a zero-surplus sweep returns zero, while sink failure reverts only that sweep. PREACTIVE claim and sweep revert without a CALL, event or write, and PREACTIVE maintenance is the stated NOOP; forced ETH is the only way its balance can change before activation.

For manifest i, the contract derives `recordIndex=oldCount+i`, `versionedHash=BLOBHASH(i)`, `publisher=msg.sender` and `validUntil=session.expiry`; none is caller supplied. It checks `1<=chunkCount<=9`, `chunkIndex<chunkCount`, the declared length bound, field-canonical `y`, the versioned hash/commitment relation and the point-evaluation proof. It derives z and the Appendix leaf separately for each tuple, appends exactly n leaves in array/blob order, and emits exactly n leaf events. One failure at any position reverts all earlier appends, events and value.

The canonical events, with no more than three indexed arguments, are:

```
SessionOpened(bytes32 indexed sessionId,address indexed owner,
  uint16 cell,
  uint64 sequence,uint64 expiry,uint256 bondWei,uint256 baseRentWei)
DataRecordAppended(bytes32 indexed sessionId,uint16 indexed recordIndex,
  bytes32 indexed versionedHash,bytes32 leaf)
SessionSealed(bytes32 indexed sessionId,
  uint16 count,bytes32 root,uint64 expiry)
SessionLiveToRefund(bytes32 indexed sessionId,address indexed owner,
  uint16 cell,
  uint64 claimDeadline,uint64 migrationGeneration)
SessionBondClaimed(bytes32 indexed sessionId,address indexed owner,
  address indexed recipient,uint256 amount)
SessionRefundForfeited(bytes32 indexed sessionId,
  address indexed owner,
  uint16 cell,uint256 amount)
SessionSurplusSwept(address indexed sink,uint256 amount)
DataSessionsMaintained(uint8 mode,
  uint16 startCursor,uint16 nextCursor,
  uint8 inspected,uint8 changed)
```

An ordinary conversion emits generation zero; a migration conversion emits the armed generation. A direct LIVE claim emits only `SessionBondClaimed`; `SessionLiveToRefund` means the cell persistently ended the call as REFUND. Maintenance event mode is ORDINARY=1, MIGRATION=2 or REFUND_ONLY=3. Events are the permanent leaf/discovery oracle, not contract storage.

The exact views are

```
dataSessionCellV1(uint16 cell) returns
  (uint8 tag,bytes32 sessionId,address owner,uint64 sequence,uint64 expiry,
   uint16 count,bool sealed,bytes32 root,bytes32[12] peaks,uint256 bondWei,
   uint64 claimDeadline)
```

```
dataSessionByIdV1(bytes32 sessionId) returns
  (uint16 cellPlusOne,uint8 tag,address owner,uint64 sequence,uint64 expiry,
   uint16 count,bool sealed,bytes32 root,uint256 bondWei,uint64 claimDeadline)
dataSessionAccountingV1() returns
  (bytes4 magic,uint16 liveCount,uint16 refundCount,uint16 occupiedCount,
   uint16 gcCursor,uint64 nextSessionSequence,uint256 liveBondLiability,
   uint256 refundBondLiability,uint64 migrationRefundGeneration,
   uint64 migrationRefundClaimDeadline,bool guardEntered,
   bytes32 dataSessionConfigHash)
```

The accounting magic is `0x44535631` (ASCII `DSV1`); narrow words, booleans, addresses and the fixed array are canonically encoded, and returndata lengths are exactly 704, 320 and 384 bytes respectively. FREE cell/by-ID misses return canonical zero for every output. A REFUND view returns only `sessionId`, owner, bond and deadline nonzero; it masks sequence, expiry, count, sealed, root and peaks to canonical zero without clearing their stale physical words. Runtime code hash binds selector, event, tuple and fee-algorithm semantics. The returned configuration hash is the exact immutable-value commitment

```
H("slot-chain-data-session-config-v1" || u32(340) ||
  u256(settlementChainId) || u64(protocolVersion) || address20(settlement) ||
  address20(activeSettlementRouter) || address20(protocolVersionManager) ||
  address20(dataRent) || executionProfileHash ||
  u256(BOND) || u256(BASE) || u256(BYTE) || u16(BLOB_BPS) ||
  u64(dataSessionMaximumTtlSeconds) ||
  u64(dataSessionRefundClaimWindowSeconds) ||
  u16(1024) || u16(2) || u16(2100) ||
  u8(8) || u8(6) || address20(0x0a) || u32(50000) || u256(BLS_MODULUS) ||
  u32(131072) || u32(126972) || u16(9))
```

Every address/hash is nonzero except that the point-evaluation address is the canonical numeric `0x0a`; the BPS/rate fields may be zero. The delayed profile requires both DataSession time values nonzero and `dataSessionRefundClaimWindowSeconds>=dataSessionMaximumTtlSeconds`; the release fixture sets both to 86400. The delayed target registration must carry this exact value, the target Settlement's wider configuration commitment must include it as a named field, and the Router must compare the accounting return against that registered value. The 232-byte deployment descriptor, target-registration hash and shared activation context close that control-plane binding; the DataSession sub-configuration hash remains distinct from the wider `targetConfigurationHash`.

The union-cell transition writes are normative. FREE-to-LIVE writes the new ID, owner, sequence, expiry and bond and explicitly writes `count=0`, `sealed=false` and the wrapped empty-MMR root. It may leave physical peak words stale, because a peak is read only when its bit in the newly written count is one; the first append at count zero therefore overwrites height zero before using it. LIVE-to-REFUND writes the REFUND tag and claim deadline and preserves only ID, owner and bond semantically. REFUND-to-FREE deletes the ID key and writes the FREE tag while its union words may remain physically stale. FREE and REFUND getters apply the masks above, and no append/root operation may read a stale word hidden by the tag or count. Consequently reuse of a formerly sealed, nonempty cell has exactly the same first-append root as a fresh cell.

Nonpayable maintenance alone owns `gcCursor`. An eligible call includes FREE and unmodified cells in this exact scan and update:

```
cell[j] = (oldCursor + j) mod 1024, for j = 0..7
gcCursor = (oldCursor + 8) mod 1024
```

In ACTIVE, an ordinary LIVE cell becomes REFUND iff `expiry<=block.timestamp` and its ID is not referenced by the stored normal best; its deadline is fixed as saturating `uint64` addition of the claim window to the stored expiry, never to the keeper's scan time. A pre-existing REFUND becomes FREE only when `block.timestamp>claimDeadline`. One scan never both creates and forfeits the same REFUND. Migration and historical modes use the stricter rules below. No call scans all 1,024 cells.

The refundable bond is a LIVE liability and then a REFUND liability; conversion moves the same amount between the two checked scalars without transferring ETH. The owner calls `claimSessionBond(bytes32sessionId,addressrecipient)` through the bounded ID-to-cell lookup. It accepts either that exact REFUND before its deadline, or in current ACTIVE mode an expired LIVE cell not referenced by the normal best before `satAdd64(expiry,claimWindow)`. ARMED LIVE claims are suppressed until its normal/recovery boundary closes, then use that generation's frozen migration deadline. It always requires the exact owner, nonzero recipient and `block.timestamp<=claimDeadline`; checks-effects-interactions clears the mapping/cell/counters/liability before transfer. A failed recipient transfer reverts the whole claim. A shared nonreentrancy guard rejects every nested session or surplus-sweep call without changing state; a recipient may catch that nested rejection and return normally, in which case the outer claim succeeds exactly once. Claims remain unpausable on ACTIVE, ARMED, READY, FROZEN and historical components. After the deadline, maintenance clears the REFUND and its ETH becomes surplus; no owner or sink is called.

The Settlement account enforces `balance>=liveBondLiability+refundBondLiability`. Nonrefundable rent, forfeited bonds and forced ETH are surplus and never inflate a refund. A separate permissionless, non-gating `sweepSessionSurplus()` sends exactly the balance above both liabilities to the immutable `dataRent` sink under one reentrancy guard. Sink failure reverts only that sweep and cannot block GC, claims, READY, cutover or the forced lane. The calibrated rent/bond combination prices the full possible occupancy through TTL and the 86,400-second refund window. A Sybil can still buy all optional DA cells for that bounded period; the forced/escape lane is independent and this is not a protocol-liveness claim.

Each occupied cell owns a fixed `bytes32peaks[12]` and `uint16count<=2100`; it stores no record array. A peak word at height h is meaningful exactly when bit h of count is one; zero-bit words may be stale. Append uses the old count as the leaf index, combines the stored peak on the left while successive count bits are one, writes the carry at the first zero bit and increments count. The wrapped MMR root is recomputed from at most 12 meaningful peaks. The exact bag order and proof grammar are frozen in Appendix A.

Only the owner may append, and a candidate may reference only a session made immutable by `sealSession`. Sealing fixes `(root,count,expiry)`; there is no unseal or append-after-seal operation. A candidate supplies at most 16 sorted unique `(sessionId,count,root)` references. At submission each must equal a live sealed session. Normal acceptance requires `expiry>=normalDeadline+REORG_MARGIN_SECONDS`; recovery requires `expiry>=block.timestamp+REORG_MARGIN_SECONDS`. Opening requires its expiry to cover proving, settlement and the reorg margin:

$$\begin{aligned} expiry &\geq now + P\_PROVE\_MAX + W\_SETTLE\_SECONDS \\ &\quad + REORG\_MARGIN\_SECONDS, \\ expiry &\leq now + DATA\_TTL. \end{aligned}$$

Here `P_PROVE_MAX=recovery.proofTimeMaxSeconds=900` and `W_SETTLE_SECONDS=recovery.settlementWindowSeconds=1200` in the exact execution profile; they are not independent implementation constants. Profile validation separately enforces `DATA_TTL>=P_PROVE_MAX+W_SETTLE_SECONDS+REORG_MARGIN_SECONDS`.

For every blob attached to `postData(session,manifests[])`, the contract:

1. obtains `versionedHash=BLOBHASH(i)` from the same transaction. The exact KZG relation is `versionedHash=bytes1(0x01)||SHA256(commitment48)[1:32]`; the point-evaluation precompile below checks this equality as part of acceptance, so no alternate version byte, digest function or truncation is legal;
2. computes legacy Keccak over this packed fixed-width sequence:

   ```
   "slot-chain-data-fs-v2" || u256(settlementChainId) ||
   u256(protocolVersion) ||
   sessionId || versionedHash || fullBodyRoot || u16(blockOrdinal) ||
   u16(chunkIndex) || u16(chunkCount) || u32(chunkByteLength) ||
   chunkRoot || address20(publisher) || u64(validUntil)
   ```

   It interprets the digest big-endian and reduces it modulo the BLS scalar-field modulus to obtain z;
3. makes a 50,000-gas `STATICCALL` to the EIP-4844 point-evaluation precompile at address `0x0a` with the exact 192 bytes `versionedHash32||z32||y32||commitment48||proof48`. It requires call success and exactly 64 return bytes equal to `u256(4096)||BLS_MODULUS`, where the exact modulus is

   ```
   hex32("73eda753299d7d483339d80809a1d805" ||
         "53bda402fffe5bfeffffffff00000001")
   ```

   short, long, reverting or wrong-constant returns reject; and
4. appends the exact leaf from §A to the session MMR.

The proof receives blob polynomials privately, recomputes $P(z) = y$, and decodes the canonical blob codec. A discretionary body byte string is `u32(txCount)||(u32(txLength)||signedRawTx)*`; its root is `H("slot-chain-body-v1"||u32(byteLength)||bodyBytes)`. Blob payload is `u32(chunkByteLength)||chunkBytes||zeroPadding`, split into 31-byte chunks; each blob field element is `0x00||chunk31`. Exactly 4,096 elements are used, unused payload bytes are zero, and noncanonical encodings reject. A manifest entry binds `(blockOrdinal,chunkIndex,chunkCount,chunkByteLength,fullBodyRoot,chunkRoot,sessionId,recordIndex)`. For each block entries are in strict chunk-index order, cover exactly `0..chunkCount-1`, have `chunkCount<=MAX_CHUNKS_PER_BODY=9`, and concatenate to at most `MAX_BODY_BYTES=1,142,748`. The circuit recomputes every chunk root, the full discretionary body root and then the full Ethereum transactions root. The signed `dataManifestRoot` is *per block*. Its leaves contain only that block's entries, use local positions `0..m-1`, repeat that block's candidate-wide `blockOrdinal`, and are sorted by chunk index. Candidate calldata concatenates these independently rooted lists in increasing block ordinal; no global manifest tree exists. Duplicate, extra, overlapping or uncovered data is invalid. `dataManifestRoot` is the exact per-block tree in Appendix A, not a free-form publisher value.

Data publication has no consensus-coupled reimbursement. A separate service may pay it, but empty payment state cannot revert canonical settlement.

6.1 Canonical reconstruction availability

Blob sessions make an in-flight candidate provable; their TTL and garbage collection are not permanent state archival. On canonical commit, full nodes must retain the canonical raw bodies, receipts, Ethereum trie nodes and every runtime-bytecode preimage needed to execute from `CanonicalCore.stateRoot`. Bytecode is mandatory because an Ethereum account leaf authenticates only `codeHash`, not the corresponding bytes. A reconstruction package is a content-addressed set of those objects plus the canonical block headers; each code object is keyed by `H(runtimeBytecode)` and must cover every account reachable during replay, including implementation code reached through proxy, beacon or delegation indirection. A consumer rejects any object whose transaction/body/header hash, trie path, bytecode hash or final state root disagrees with the L1-canonical commitments, so an archive can be anonymous and malicious without gaining safety authority.

Renewing a recovery round refreshes its L1 anchor, queue cutoff and proof target; it cannot invert a state root or recreate destroyed bytes. Therefore the 2,164-second protocol bound begins only once a prover has the canonical reconstruction package and any required unexpired kind-0 bytes. A warm full node satisfies this condition; a cold-start prover is bounded by data download plus proof time while at least one archive or Ethereum's applicable DA history still serves the package. If no complete copy exists, safety remains intact but progress stops. No governance action may replace the state root, skip the missing prestate, or call that condition a builder failure.

7 Settlement and recovery state machine

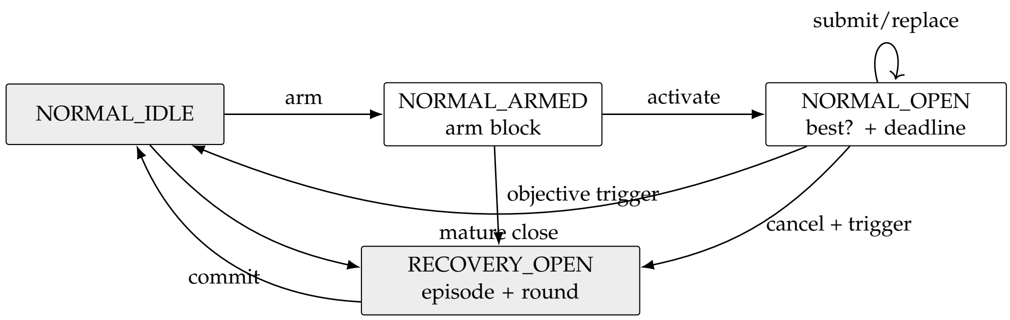


Figure 1: The complete mode machine. Payments are outside every arrow.

7.1 Normal context and candidates

Normal context creation is two-phase. Anyone calls `armNormalContext()` in `NORMAL_IDLE`; the call begins with `sync()`, stores only `armBlockNumber=block.number`, and emits

`NormalContextArmed`. It accepts no caller-supplied anchor or hash. In a later L1 block anyone calls `activateNormalContext()`. It again begins with `sync()` and requires the arm's block-number age to be from 1 through `MAX_ARM_AGE_BLOCKS` (255), reads `armBlockHash=blockhash(armBlockNumber)`, and hashes a supplied RLP header to that value. It then freezes `baseCanonical`, `deadline=block.timestamp+W_SETTLE_SECONDS`, `minAdmissibleSlot=max(0,currentL2Slot-DELTA_TIP)`, the current stable `(admissionVersion,admissionRoot)`, the arm header fields, and `requiredThrough=deadline+T_INCLUDE_MAX_SECONDS`. It derives the sole normal `contextId` and emits `NormalContextActivated`. No candidate is accepted while armed. If age exceeds 255 blocks, any caller may invoke `armNormalContext()` again; after its leading `sync()` it atomically replaces only the stale arm number with the current block, emits `NormalContextRearmed`, and returns `REARMED`. A nonstale arm cannot be replaced. Thus an abandoned arm cannot suppress normal operation.

Builders collect and issue tier-1 signatures only after the activation event. The execution anchor for every candidate is the exact arm block, whose hash and parsed header are already frozen; replacements hash their supplied RLP header to that stored hash and never repeat an EIP-2935 recency lookup. A reorg that removes the arm block necessarily changes the context. If the arm block survives but the activation transaction is reorged, reactivation intentionally recreates the same context: the builder must retain and honor its prior same-context signature. Evidence independently authenticates the arm hash as canonical through EIP-2935 and is accepted only while that history entry is available. The enforced evidence horizon is strictly shorter than 8,191 L1 slots. Thus neither a shallow reorg nor a builder-selected historical anchor can create false or free equivocation.

Activation is permissionless, reserves no builder, and does not block submissions. An empty activated window merely cancels at deadline, so an opener gains no censorship right. Every candidate first proves its frozen-prefix boundary in the circuit. It also supplies a canonical single-leaf Merkle proof at `endCursor` under the queue's current root/count, or proves `endCursor==currentCount`. Settlement verifies the canonical depth-64 singleton proof, which contains exactly 64 sibling hashes, and requires the resulting `currentBoundaryDueAt` to exceed `requiredThrough`. This prevents an old but valid anchor from hiding a post-anchor append. Because due times are monotone, one current boundary leaf proves that every item due through the landing horizon is consumed. Later candidates use the same base, deadline, minimum, admission pair and horizon. Their total order is lexicographic:

$$(\text{block count descending, tip slot descending, tip hash ascending}).$$

Only a strictly better candidate replaces `normalBest`; the stored best contains `endCursor` and the minimum expiry of every referenced data session. Provisional replacement changes no canonical state and pays nobody. At or after the deadline, `sync()` reads `ForcedQueue.dueAt(best.endCursor)` and commits only if that live boundary is greater than `block.timestamp` and `block.timestamp+REORG_MARGIN_SECONDS<=best.minDataExpiry`. Thus a late close cancels rather than committing a block that omits an already-due head, even beyond the assumed inclusion bound, or whose authenticated data has passed its resubmission margin. Data-session GC cannot evict a session referenced by the stored best; its own leading `sync()` clears an expired best first. Because `FORCE_DELAY>=W_SETTLE_SECONDS+T_INCLUDE_MAX_SECONDS`, an append after open cannot violate the frozen acceptance horizon. Settlement then recomputes recovery causes against the new core.

7.2 Activation

While normal, `sync()` opens one persistent recovery episode if either:

- `currentL2Slot>canonical.tipSlot+DELTA_FINAL_LAG`; or
- the forced queue's stored `dueAt[canonical.messageCursor]` is no greater than `block.timestamp`.

A due forced head has precedence over an immature normal window, which is canceled. At a mature deadline, a best that consumes the required due prefix commits before causes are recomputed; an omitting best is canceled. No best is promoted across activation. The contract increments `episode`, records causes and the base-canonical hash, then runs the bounded seat-duty pass in §9: an objectively missed primary duty is allocated, failed over or marked breached as applicable, while a full duty ring fails open to economic vacancy. This path performs no Market call, bond movement or token transfer. Breach enforcement and bond terminalization are later permissionless noncanonical operations against the retained exact receipt; force-only recovery creates no aggregator penalty.

Every round freezes `roundStartSlot`, the preceding L1 block number/hash, current `forceRoot/forceCutoff`, current `(admissionVersion,admissionRoot)`, and `escapeSlot=max(roundStartSlot+ESCAPE_OFFSET,canonical.tipSlot+1)`. Its `expiresAt` is `GENESIS_TIMESTAMP+escapeSlot+DELTA_TIP`. The exact ID is:

```
recoveryId = H("slot-chain-recovery-v2",
               u256(settlementChainId), address20(contract),
               u64(episode), u64(revision), bytes32(baseCanonicalHash),
               u64(roundStartSlot), u64(anchorNumber), bytes32(anchorHash),
               bytes32(forceRoot), u64(forceCutoff), u64(admissionVersion),
               bytes32(admissionRoot), u64(escapeSlot), u8(causes))
```

At `block.timestamp>expiresAt`, and never before, `sync()` replaces the expired target with revision $r+1$ and fresh round fields, then returns `SYNCED`. It does not change episode/base/cause, create another duty, promote, terminalize a bond or pay. The old proof is already inadmissible, so permissionless refresh cannot grief a still-live proof. New queue appends enter the refreshed cutoff. An envelope admitted during recovery receives `dueAt=max(enqueuedAt+FORCE_DELAY,lastDueAt,currentRound.expiresAt+1)`; therefore it cannot become due while a frozen round that omits it remains admissible.

7.3 Recovery acceptance

Recovery is first-valid, not heaviest. A tier-2 or tier-3 candidate MUST:

1. extend the exact current stored canonical state;
2. bind current `episode`, `revision`, `recoveryId`, frozen admission pair, anchor and queue target;
3. land at least `F_L1` L1 blocks after its anchor, have first slot at least `roundStartSlot`, tip no older than `DELTA_TIP`, and no slot later than wall clock plus skew;
4. consume the exact maximum forced-message prefix from the current cursor, bounded by the current round's frozen cutoff; and

5. satisfy its tier-specific signature/data restrictions and validity proof.

The first valid L1 transaction atomically commits the tuple, records the current L1 block *number only*, clears the recovery episode and returns to normal. Later proofs are stale. Candidate rejection cannot undo activation because of the early-return wrapper in §3.

8 Forced queue and unsigned escape

`ForcedEnvelope` is a tagged union. Kind 0 contains `(sender,nonce,l2ChainId,rawTxHash, byteLength,gasLimit,maxFee,validUntil,refundAddress,deposit)`. Admission canonically decodes the EIP-2718 transaction, requires its recovered sender and all duplicated fields to match, and requires `msg.sender==sender`. Launch exposes no relayed kind-0 entry; any future EIP-712 relay requires a new function, profile and codec. The sole launch entry is

```
enqueueForcedTransactionV2(bytes rawTransaction, uint64 validUntil,
                           address refundAddress)
  payable returns (uint8 result, uint64 queueIndex)
```

Its selector is the first four Keccak bytes of `"enqueueForcedTransactionV2(bytes,uint64, address)"`. Word 0 is the canonical bytes offset `0x60`, word 1 the canonically padded validity deadline and word 2 the canonically padded nonzero refund address. The tail is one length word, the nonempty raw EIP-2718 bytes and zero right-padding; total calldata length is exactly `132+ceil32(rawTransaction.length)` bytes. The adapter re-encodes and byte-compares the complete call before state. It recovers the sender and derives nonce, chain ID, raw transaction hash, raw byte length, intrinsic gas, gas limit and maximum fee from those exact bytes; `msg.sender` must equal the recovered sender, the transaction chain must be the configured L2, and `msg.value` must equal the checked shared-schedule deposit. No duplicated caller field or caller hash is authoritative. Kind 1 admission receives one `BridgeAdmissionEnvelope` containing one `uint64liquidityFee` and the complete raw eleven-field `IBridge.Message`:

```
(uint64 id, uint64 fee, uint32 gasLimit, address from,
 uint64 srcChainId, address srcOwner, uint64 destChainId,
 address destOwner, address to, uint256 value, bytes data)
```

Its Solidity name and the envelope field order are exact:

```
struct MessageV1 {
  uint64 id; uint64 fee; uint32 gasLimit; address from;
  uint64 srcChainId; address srcOwner; uint64 destChainId;
  address destOwner; address to; uint256 value; bytes data;
}
struct BridgeAdmissionEnvelopeV2 {
  MessageV1 message; uint64 liquidityFee;
}
sendMessageV2(MessageV1 message, uint64 liquidityFee)
  payable returns (bytes32 msgHash, MessageV1 normalizedMessage)
enqueueBridgeCreditV2(MessageV1 message, uint64 liquidityFee)
  payable returns (uint8 result, uint64 queueIndex)
```

Their selectors are the first four Keccak bytes of respectively `"sendMessageV2((uint64, uint64,uint32,address,uint64,address,uint64,address,address,uint256,bytes) ,uint64)"` and the same expanded arguments under `enqueueBridgeCreditV2`. After either selector, word 0 is the Message offset `0x40` and word 1 is the canonically padded liquidity fee. The Message begins at argument byte `0x40`, has eleven head words in the shown order, and its data offset word is exactly `0x160`; that tail is one length word, exact data and zero right-padding. Thus canonical calldata length is `452+ceil32(data.length)` bytes including selector. Each entry re-encodes the decoded arguments and requires byte-for-byte equality and exact length, so aliases, gaps, overlap, trailing bytes, high integer/address padding or nonzero right-padding reject before state.

The SourceBridge first applies the normalized overwrites, then freezes the legacy V1 hash codec exactly:

```
abi.encode("TAIKO_MESSAGE", normalizedMessage) =
  [0x40, 0x80] || [u256(13), "TAIKO_MESSAGE", zero-pad-to-32] ||
  [the eleven normalized Message ABI head words, dataOffset=0x160] ||
  [u256(data.length), data, zero-pad-to-32]
msgHash = keccak256(that complete 512+ceil32(data.length)-byte preimage)
```

The return from `sendMessageV2` has `msgHash` in word 0, Message offset `0x40` in word 1 and the same canonical Message tuple/tail. Both adapter returns are exactly 64 bytes: result QUEUED=1 carries the new index, SYNCED=2 carries `UINT64_MAX`, and ALREADY_QUEUED=3 is permitted only for a zero-value kind-1 query and carries the stored index. Kind 0 never returns ALREADY_QUEUED. Unknown result, bad padding, or any other sentinel/index combination rejects at its caller. The SourceBridge overwrites `id`, `from` and `srcChainId` with its next ID, the actual caller and immutable chain context before deriving `keccak256(abi.encode("TAIKO_MESSAGE",message))`, data length and `calldataHash=keccak256(message.data)` over exactly the unpadded dynamic bytes. Empty data therefore uses Ethereum's canonical `keccak256("")=c5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470`. For kind 1, `byteLength=uint32(message.data.length)` exactly after canonical ABI decoding; for kind 0 it is the exact EIP-2718 raw-transaction byte length. No caller-supplied length, ABI-padded length, descriptor length or event length is accepted. It requires a nonzero `liquidityFee` and a checked `message.value+message.fee>0`; a zero settlement amount has no ticket, Pool authorization or DONE encoding and is rejected before custody or ID mutation. No hash, length or funding ticket supplied by the caller is authoritative. Only the configured `BridgeInboxAdapter` may enqueue the resulting fixed V11 projection and it has no queue expiry. The source Bridge retains message value plus execution and liquidity fees while recording per-credit pending, user-pull and LP-pull liabilities; every outflow must leave `address(this).balance>=pendingEscrow+userPullLiability+lpPullLiability`. The adapter separately temporarily retains its caller-funded processing fee only across the sync decision; a successful append forwards it entirely to ForcedQueue. Its only forced effect is an idempotent inbox/signal credit keyed by the immutable identifier

```
sourceDomainId = H("slot-chain-source-domain-v4" || u64(srcChainId) ||
  bytes32(sourceGenesisHash) || address20(bridgeCreditRegistry) ||
  address20(terminalSignalVerifier) || address20(srcBridge) ||
  bridgeExecutionHash || bytes32(registryNamespace))
destinationDomainId = H("slot-chain-destination-domain-v7" ||
  u64(destChainId) || bytes32(destinationGenesisHash) ||
  address20(bridgeInboxAdapter) || address20(activeSettlementRouter) ||
  address20(terminalSignalVerifier) || address20(inboxApplyRouterV2) ||
  address20(inboxCreditStoreV2) || address20(protocolReleaseAuthorityV2) ||
  address20(terminalDomainRegistrarV2) ||
  address20(terminalAccumulatorV2) || address20(nativeLiquidityPoolV2) ||
  address20(destinationBridge) ||
```

```
  bytes32(destinationBridgeExecutionHash) ||
  bytes32(destinationInfrastructureHash) ||
  bytes32(destinationNamespace))
creditId = H("slot-chain-bridge-credit-id-v6" || u64(srcChainId) ||
  sourceDomainId || u64(srcEpoch) || address20(srcBridge) ||
  destinationDomainId || msgHash || u64(liquidityFee))
escrowId = H("slot-chain-bridge-escrow-v2" || creditId)
```

Both domain descriptors are append-only and source-approved; no message sender supplies an arbitrary destination trust root. The destination descriptor binds the full stable recovery path: L2 genesis, non-proxy `BridgeInboxAdapter`, immutable `ActiveSettlementRouter`, `TerminalSignalVerifier`, `InboxApplyRouterV2`, `InboxCreditStoreV2`, protocol-lifetime `ProtocolReleaseAuthorityV2`, immutable `TerminalDomainRegistrarV2`, protocol-lifetime `TerminalAccumulatorV2`, immutable `NativeLiquidityPoolV2` and frozen destination Bridge facade. The destination infrastructure hash is the Appendix-defined commitment to the deployed runtimes and constructor immutables of every one of those components; address-only binding is insufficient. The source and destination Bridge addresses are fresh immutable non-proxy lifetime constants for that domain; neither Resolver rotation nor a replacement proxy may change their callback, custody, status or terminal identity. An incompatible implementation requires a separately named endpoint and new domain, while every old endpoint remains callable for its old credits. Arbitrary destination invocation and RETRIABLE processing remain later ordinary bridge operations. The two forced-envelope variants have separate byte-exact leaves.

Launch accepts kind 1 only for the L1-to-slot-chain direction, where:

```
srcChainId = settlementChainId = block.chainid
```

Consequently the adapter omits an EIP-2935/MPT proof of its own L1. It performs a bounded `staticcall` directly to the descriptor's non-proxy `BridgeCreditRegistry`, checks its runtime hash and exact fixed-size return, and compares the immutable `CreditRecord`. A reorg that removes the source record necessarily removes every later enqueue transaction; anyone may retry from the still-durable record on the surviving chain. Cross-L1 sources require a separately reviewed protocol version and are not silently routed through this path.

The kind-0 v2 and kind-1 v11 leaf, descriptor and disposition decoders are protocol-lifetime queue ABIs. Every future execution profile must retain their exact semantics for all entries below `queueCount`; a migration may add a new tagged kind but cannot reinterpret an existing byte. The permanent queue keeps every fixed-width descriptor and kind-1 credit record. For kind 0, a candidate supplies raw EIP-2718 bytes matching `rawTxHash` while unexpired; if no party retains those bytes, the head becomes permissionlessly `EXPIRED_NO_TX` after the bounded seven-day validity horizon, using only the stored descriptor. Thus migration neither assumes an expiring blob for a durable bridge credit nor turns missing user bytes into permanent head-of-line blocking.

The deployed V1 `sendMessage(Message)` selector, counter, event, signal, status and recall semantics remain byte-for-byte V1. No Resolver update or interface probe redirects a V1 caller. Launch V2 is a separate, fresh immutable non-proxy SourceBridge with one additive `sendMessageV2(Message,uint64)` selector, where the second argument is the liquidity fee. It normalizes caller calldata by overwriting `id`, `from` and `srcChainId` with its checked next ID, the actual external caller and the immutable chain context before hashing. It resolves the final destination Bridge and rejects that address as the effective caller. It requires exact `msg.value=message.value+message.fee+liquidityFee`, derives the complete Message hash and exact `keccak256(message.data)` hash/raw length itself, sets checked `enqueueBy=block.timestamp+MAX_BRIDGE_ENQUEUE_DELAY`, and creates only a DIRECT NEW credit. There is no V2 Vault selector, DRAFT state, capsule, restorable token, reverse asset outflow or `getOrDeploy` path. Before incrementing the ID or writing escrow/Registry state it also preserves the V1 safety invariants exactly: `srcOwner`, `destOwner` and `to` are each nonzero; `destChainId` differs from the immutable source chain and equals the selected destination-domain record; the checked sum `message.value+message.fee` is nonzero. If `gasLimit=0`, `fee` must be zero. Otherwise the checked `uint256(gasLimit)-V2_POST_CALL_GAS_RESERVE` must be nonzero; this is the exact destination invocation budget rule. Registry insertion, adapter join, the durable V11 decoder and the validity circuit repeat all those predicates. The corpus sets each owner/target address to zero separately, exercises both gas/fee branches, rejects value and fee both zero, exercises one-below/exactly-at/one-above the reserve, and requires byte-identical nonce, balance, liability, Registry and Queue rollback.

The immutable `BridgeCreditRegistryV2` recomputes `creditId`, accepts writes only from its exact configured SourceBridge capability, and rejects duplicates. Authorization creation, Bridge custody and liability creation are one non-reentrant rollback domain. The Registry stores only this fixed-size authorization and never stores or returns raw Message bytes:

```
(bytes32 sourceDomainId, uint64 srcEpoch, address srcBridge,
 bytes32 bridgeExecutionHash, uint64 emittedAtBlock, bytes32 msgHash,
 bytes32 destinationDomainId, uint64 destChainId,
 uint64 enqueueBy, address sender, address srcOwner, address destOwner,
 uint256 value, uint64 fee, uint64 liquidityFee,
 bytes32 calldataHash, uint32 calldataLength,
 bytes32 escrowId)
```

The SourceBridge alone owns mutable custody:

```
SourceLiability(value, executionFee, liquidityFee, status, queueIndex,
                pullClass, pullBeneficiary, pullAmount)
```

The adapter's source read boundary is exactly two calls:

```
creditAuthorizationV2(bytes32 creditId) view returns
 (bytes32 sourceDomainId, uint64 srcEpoch, address srcBridge,
  bytes32 bridgeExecutionHash, uint64 emittedAtBlock, bytes32 msgHash,
  bytes32 destinationDomainId, uint64 destChainId, uint64 enqueueBy,
  address sender, address srcOwner, address destOwner, uint256 value,
  uint64 fee, uint64 liquidityFee, bytes32 calldataHash,
  uint32 calldataLength, bytes32 escrowId)

creditLiabilityV2(bytes32 creditId) view returns
 (uint256 value, uint64 executionFee, uint64 liquidityFee, uint8 status,
  uint64 queueIndex, uint8 pullClass, address pullBeneficiary,
  uint256 pullAmount, uint256 totalLiveLiability)
```

Their selectors are respectively `0x05ecb6c2` and `0xc978978a`; calldata is exactly 36 bytes, gas is capped at 200,000 for each call, and return length is exactly 576 and 288 bytes. All narrow words have canonical zero high padding. Before either read the adapter checks the descriptor-pinned runtime and exact `componentConfigHashV2()` result. It joins every duplicated amount/identity, requires `status=NEW=1`, `queueIndex=UINT64_MAX`, `pullClass=0`, zero beneficiary and zero pull amount, and checks `totalLiveLiability>=value+fee+liquidityFee` and `BALANCE(sourceBridge)>=totalLiveLiability`. Short/trailing return, malformed padding, substituted field, insolvency or call fault rejects before Queue, Registry, Bridge or adapter mutation. Only the liability moves `NEW->QUEUED->DELIVERED/RECALLED` or `NEW->CANCELLED`. Those two bounded read-only views let the adapter cross-check every immutable field, exact NEW status and aggregate solvency. Queue append, `markQueuedV2`, Registry state, Bridge liability/balance and adapter record are one transaction. Liability remains exactly value plus fee plus liquidity fee through QUEUED. Strict post-`enqueueBy` cancellation and proved FAILED recall reclassify the full amount from pending escrow to the source owner's USER_PULL. Proved DONE authenticates the consumed L2 funding ticket and reclassifies the full amount to its immutable L1 recipient's LP_PULL; it never drops liability or creates unowned SourceBridge surplus. Liability decreases only when the corresponding pull succeeds. Withdrawal is unpausable and CEI-safe, derives the claimant from the transaction frame, accepts only a nonzero claimant-selected recipient, and restores the claim if the recipient call fails or reenters.

The L1 `BridgeDomainRegistry` retains append-only source-domain, destination-domain and frozen-endpoint-support descriptors. Only the delayed `ProtocolVersionManager` may stage a descriptor, atomically with the active immutable release manifest. A staged tuple is unusable until permissionless `confirmDestinationRegistration(protocolVersion, canonicalSequence,proof)` atomically reads the exact sequence-tagged full canonical tuple through `ActiveSettlementRouter`, requires its L1 canonicalization block to be at least `F_L1` deep, and authenticates the exact domain-registration record under that tuple's `stateRoot`. It never accepts a root or tuple from the caller. The active profile pins one immutable, acyclic `RegistrationMptVerifierDescriptorV2`; no default verifier exists. Its exact statement is

```
struct RegistrationStorageStatementV2 {
  uint256 settlementChainId; address activeSettlementRouter;
  address bridgeDomainRegistry; bytes32 routeKey;
  uint256 destinationChainId; uint64 protocolVersion;
  uint64 canonicalSequence; bytes32 stateRoot;
  address terminalDomainRegistrar; bytes32 registrarCodeHash;
  bytes32 storageTrieKey; bytes32 expectedValue;
}
```

`routeKey=keccak256(abi.encode(ROUTE_KEY_TYPEHASH,sourceDomainId,bridgeExecutionHash, destinationDomainId))`. The exact type literal is

```
"RegistrationStorageStatementV2(uint256 settlementChainId,"
"address activeSettlementRouter,address bridgeDomainRegistry,bytes32 routeKey,"
"uint256 destinationChainId,uint64 protocolVersion,uint64 canonicalSequence,"
"bytes32 stateRoot,address terminalDomainRegistrar,bytes32 registrarCodeHash,"
"bytes32 storageTrieKey,bytes32 expectedValue)"
```

with the displayed fragments concatenated without whitespace or newline. `REGISTRATION_STATEMENT_TYPEHASH` and `publicInputSchemaHash` are both Keccak of those bytes, and `statementHash=keccak256(abi.encode(TYPEHASH,fields...))`. Registry derives all twelve fields from its staged route, immutable chain/contract identities and the Router-returned canonical history; proof calldata supplies none of them.

The packed proof schema is committed by

```
REGISTRATION_MPT_PROOF_SCHEMA_LITERAL = concat(
  "MptProofV1=be16(accountNodeCount)||be16(storageNodeCount)||",
  "(be16(nodeLength)||canonicalRlpNode)*accountNodeCount||",
  "(be16(nodeLength)||canonicalRlpNode)*storageNodeCount;",
  "rootToLeaf;EthereumKeccak;canonicalHexPrefix;canonicalRlp;",
  "absenceRejected;valueRequired")
proofSchemaHash = keccak256(REGISTRATION_MPT_PROOF_SCHEMA_LITERAL)
```

The two hash layers are

```
REGISTRATION_MPT_VERIFIER_CONFIG_TYPE_LITERAL = concat(
  "RegistrationMptVerifierConfigV2(bytes32 publicInputSchemaHash,",
  "bytes32 proofSchemaHash,bytes4 selector,uint16 maximumNodesPerPath,",
  "uint16 maximumTotalNodes,uint16 maximumNodeBytes,",
  "uint32 maximumProofBytes,uint64 verificationGasLimit)")
registrationMptVerifierConfigurationHash = keccak256(abi.encode(
  keccak256(REGISTRATION_MPT_VERIFIER_CONFIG_TYPE_LITERAL),
  publicInputSchemaHash, proofSchemaHash, selector, 66, 132, 600, 80000, 8000000))

REGISTRATION_MPT_VERIFIER_DESCRIPTOR_TYPE_LITERAL = concat(
  "RegistrationMptVerifierDescriptorV2(address verifier,bytes32 runtimeHash,",
  "bytes32 configurationHash,bytes32 publicInputSchemaHash,",
  "bytes32 proofSchemaHash,bytes4 selector,uint16 maximumNodesPerPath,",
  "uint16 maximumTotalNodes,uint16 maximumNodeBytes,",
  "uint32 maximumProofBytes,uint64 verificationGasLimit)")
registrationMptVerifierDescriptorHash = keccak256(abi.encode(
  keccak256(REGISTRATION_MPT_VERIFIER_DESCRIPTOR_TYPE_LITERAL),
  verifier, runtimeHash, registrationMptVerifierConfigurationHash,
  publicInputSchemaHash, proofSchemaHash, selector, 66, 132, 600, 80000, 8000000))
```

The configuration layer deliberately excludes verifier, runtime hash and itself; the descriptor repeats every configuration field and rejects unless its `configurationHash` equals the independently recomputed first layer. The immutable verifier exposes only

```
registrationMptVerifierConfigHashV2() external view returns (bytes32)
REGISTRATION_MPT_VERIFIER_CONFIG_READ_GAS = 50000
```

Registry first checks `EXTCODEHASH`, then makes a zero-value exact-four-byte `STATICCALL` with that gas stipend and accepts exactly one 32-byte word equal to the recomputed configuration hash. Empty, short, trailing, faulting or mismatched returns reject before proof dispatch.

The verifier interface is exactly

```
verifyRegistration(
  RegistrationStorageStatementV2 calldata statement, bytes calldata proof)
    external view returns (bytes32 statementHash)
```

with selector from `"verifyRegistration((uint256,address,address,bytes32,uint256, uint64,uint64,bytes32,address,bytes32,bytes32,bytes32),bytes)"`. Its ABI head is the twelve statement words followed by the proof offset word `0x1a0`; the tail is canonical bytes length/data/zero-padding with no trailing calldata. Its total calldata length is exactly `452+ceil32(proof.length)` bytes; the 80,000-byte cap measures the packed inner `proof.length`, not the outer ABI envelope. BridgeDomainRegistry makes one zero-value `STATICCALL` forwarding exactly 8,000,000 gas and accepts exactly 32 return bytes equal to its locally recomputed statement hash. Revert, OOG, malformed return, substituted statement or a Boolean result rejects before support state changes.

`proof` is `MptProofV1`: big-endian `u16(accountNodeCount)||u16(storageNodeCount)` followed by each root-to-leaf canonical-RLP node as `u16(nodeLength)||nodeBytes`. Each path has at most 66 nodes, each node at most 600 bytes, the whole proof at most 80,000 bytes, and any noncanonical RLP, wrong inline/hash reference, surplus node/byte or gas use above 8,000,000 rejects. The account proof fixes the registrar address and code hash; the storage proof fixes one nonzero `registrationCommitment[protocolVersion]` word. No Solidity struct or adjacent slot is inferred from that one path. The registrar exposes `registrationCommitmentV2(uint64)viewreturns(bytes32)` with exact 32-byte returndata, and stores only:

```
registrationCommitment =
  H("slot-chain-destination-registration-v1" ||
    u64(protocolVersion) || releaseManifestHash ||
    u256(destinationChainId) || destinationNamespace ||
    destinationDomainId || address20(destinationBridge) ||
    destinationInfrastructureHash || executionProfileHash)

baseSlot = H("slot-chain-terminal-domain-registrar.registrationCommitment.v1")
elementSlot = H(bytes32(uint256(protocolVersion)) || baseSlot)
storageTrieKey = H(elementSlot)
```

The account trie key is `keccak256(address20(terminalDomainRegistrar))`. The root reference is exactly `stateRoot`; a child reference shorter than 32 bytes must equal the complete inline child RLP and a 32-byte reference must equal Keccak of the next complete node. Other reference lengths reject. Canonical RLP uses the shortest length form, forbids leading length zeros and uses long form only for payloads of at least 56 bytes. A branch has exactly 17 items; an extension/leaf has exactly two. Hex-prefix flags are only 0–3, the even form has a zero low nibble, an extension path is nonempty, and a leaf is accepted only at exact key exhaustion. The account leaf is exactly `[nonce,balance,storageRoot,codeHash]`; nonce and balance are minimal unsigned integers of at most 32 bytes and both roots are exact 32-byte strings. The storage proof starts at that authenticated `storageRoot`, consumes the exact `storageTrieKey`, and returns the canonical minimal-RLP unsigned value equal to `expectedValue`. Missing, surplus or out-of-order nodes, trailing container bytes and an alternative yet value-equivalent RLP all reject. The storage-trie leaf value is canonical RLP of the minimal big-endian unsigned integer represented by the nonzero 32-byte commitment (and therefore preserves leading-zero semantics through the expected value). L1 recomputes the full commitment from its staged release manifest and accepts only exact equality; substitution of any field, mapping key, code hash, account or root rejects. The confirmation block, not manifest staging, starts the 214-block support delay. Each release adds at most 64 entries; there is no global cap, deletion, redirect or reinterpretation. A conflicting duplicate or inactive manifest rejects. Every credit creation performs an O(1) check that its exact `(sourceDomainId,bridgeExecutionHash,destinationDomainId)` support entry has been active for 214 blocks; adding D2 never makes it usable with an old source endpoint before D2 itself is final.

V1 and V2 never share a Bridge account. The existing V1 proxy, Resolver, balance, nonce, status, pause/lock, QuotaManager and every Vault flow remain unchanged. V2-aware applications explicitly accept a fresh Bridge address and ContextV2; applications hard-coding V1 remain V1-only.

The SourceBridgeV2 bundle is deployed permissionlessly through a manifest-pinned immutable CREATE2 factory. Its descriptor binds factory address/runtime/config, bundle salt/init-code hash and the derived bundle-deployer runtime, the legacy V1 Bridge address that must not be reused, resulting SourceBridge address/runtime/config/layout, the Bridge-unique Registry and V2-only QuotaManager, support Registry/registrar/ epoch, immutable terminal verifier and Router binding, pauser and SignalService. The bundle-deployer address must equal the final 20 bytes of

```
keccak256(0xff || factory || salt || keccak256(init_code)).
```

The bundle init-code hash commits every acyclic child code/configuration primitive and excludes only the three addresses that its constructor derives from its own address. The exact Appendix formulas derive the bundle via CREATE2 and the Bridge, Registry and QuotaManager via its constructor's CREATE nonces 1–3 in one atomic transaction. A front-run is accepted only when the complete current bundle, code and configuration are exact; otherwise activation fails closed. No child address is embedded in the bundle CREATE2 preimage, so the graph is acyclic; the outer descriptor separately binds every derived address and current runtime/configuration.

Router retains one protocol-lifetime BridgeDomainRegistry, immutable factories, source bundles by descriptor ID and an append-only version-to-descriptor map. Current pointers are aliases only. Every incompatible successor has a fresh domain and fresh SourceBridge/Registry/Quota accounts; historical adapters keep their original bundle for terminal release and refunds. Each bundle owns its private nonce, status, pull and aggregate-liability mappings. It has no owner, upgrade, reinitializer, delegatecall or public mutation helper. The source generation is a fixed nonzero registration number and every credit binds its exact Bridge execution descriptor. At launch all source components share settlement Ethereum; cross-L1 support requires a new reviewed kind/domain. Admission also requires `0<srcChainId<=UINT64_MAX`, nonzero `sourceDomainId`, `0<srcEpoch<=UINT64_MAX`, `0<emittedAtBlock<=UINT64_MAX`, nonzero `msgHash`, `destChainId=l2ChainId<=UINT64_MAX`, a registered nonzero `destinationDomainId` and a nonzero `srcBridge`; no narrowing cast or destination-chain inference is permitted.

`BridgeInboxAdapter` is a non-proxy immutable contract. Its profile-bound runtime and constructor values fix the credit registry, source Bridge, immutable settlement router and ProtocolVersionManager. Its sole mutable configuration is a zero-to-nonzero `destinationDomainId` slot sealed once by that manager during manifest activation; the setter deletes its authority bit and cannot run twice. Enqueue rejects while the slot is zero. It has no upgrade, delegatecall, mutable target, arbitrary-call or caller-supplied authenticated value path. A different destination uses a distinct adapter/domain; an old adapter remains executable for its existing records.

The adapter is the exactly-once machine `NONE->QUEUED(index)`, keyed by `creditId`. Any caller supplies the separate queue processing fee and the full message preimage. The payable adapter initially retains `msg.value`, derives both domains and the ID, direct-reads the immutable record and live source liability, and validates all fixed bounds including `block.timestamp<=enqueueBy`. It first calls the router's nonpayable

```
syncIngressV2() returns
  (uint8 result, uint64 activeProtocolVersion, uint64 routerGeneration)
```

as a registered immutable adapter. Its selector is the first four Keccak bytes of `"syncIngressV2()"`; calldata is exactly four bytes and returndata is exactly 96 bytes. STAMP=1 requires both narrow words nonzero; SYNCED=2 requires both zero. Unknown result, noncanonical high padding, short/trailing return or fault rejects and rolls the adapter back. If that call changes mode, the adapter records no credit or queue leaf, adds the still-local full value to `claimable[msg.sender]`, and returns `SYNCED`; a CEI pull withdrawal is callable independently of protocol mode and the caller retries in a later transaction. If unchanged, the same adapter immediately calls the router's payable

```
appendFromAdapterV2(uint64 activeProtocolVersion, uint64 routerGeneration,
                    uint8 kind, bytes descriptorBytes)
  payable returns (uint8 result, uint64 queueIndex)
```

with the exact STAMP words, its immutable kind and canonical fixed descriptor. Its selector is the first four Keccak bytes of `"appendFromAdapterV2(uint64,uint64,uint8,bytes)"`. The four-word head is version, generation, canonically padded kind and offset `0x80`; the tail is the exact length, 220-byte kind-0 or 541-byte kind-1 descriptor, and zero right-padding. Total calldata is respectively 388 or 708 bytes, with no other length valid. The exact 64-byte return is QUEUED=1 plus the new non-sentinel index; every other result, padding, length or index rejects. This second entry point does not call `sync()` again: it requires the registered caller, ACTIVE phase, and exact current version and generation stamp, then forwards all value with the exact descriptor to the queue. The immutable adapter has no entry point that can split, defer or redirect this pair and makes the second call before any other external call in the same top-level transaction. EVM call execution supplies no interleaving between them. A stale stamp after any migration transition rejects and returns the value by call-level rollback; it cannot preserve a second state transition inside a reverting payable call. A successful router retains zero wei. The queue checks `queueCount<UINT64_MAX`, appends at the current count and timestamps the deadline; then the source Bridge changes NEW to QUEUED at that exact index and the adapter records the returned index, all in the same revert domain. No reservation, source proof, PENDING state or finalization call exists. A repeated zero-value query returns the stored index; a duplicate with funds reverts. The source event alone never means queued.

Kind 0 requires a nonexpired `validUntil`, `validUntil<=enqueuedAt+MAX_FORCE_VALIDITY_SECONDS`, and `accountedGas=max(gasLimit,intrinsicGas,MIN_FORCE_ACCOUNTED_GAS)`. The initial validity horizon is seven days. Kind 1 uses the permanent conservative bound `accountedGas=MAX_FORCE_MESSAGE_GAS=5,000,000` for its encoded credit and routed pin. Kind 0 requires `0<accountedGas<=5,000,000` and `0<byteLength<=131,072`; kind 1 permits an empty Message payload and requires `byteLength<=131,072`. The active profile fixes one shared five-word fee schedule

```
(fixedIngressWei, executionWeiPerAccountedGas,
 proofWeiPerAccountedGas, permanentWeiPerByte,
 maximumAcceptedFeeWei).
```

```
requiredDeposit = fixedIngressWei
  + accountedGas * (executionWeiPerAccountedGas
                    + proofWeiPerAccountedGas)
  + byteLength * permanentWeiPerByte.
```

All additions and multiplications are checked `uint256`; every coefficient and the cap is nonzero, `requiredDeposit<=maximumAcceptedFeeWei`, and the release is invalid unless the maximum gas/byte geometry and `UINT64_MAX*maximumAcceptedFeeWei` are representable. Each payable adapter requires `msg.value=descriptor.deposit=requiredDeposit` exactly; overpayment and underpayment both revert before retained state. `fixedIngressWei` is the release-calibrated upper bound for all admission costs independent of `accountedGas` and `byteLength`, including the fixed descriptor, event, cold-account and fixed storage-write geometry. `executionWeiPerAccountedGas`, `proofWeiPerAccountedGas` and `permanentWeiPerByte` are respectively upper bounds, with the release's explicit safety margin, for execution gas, proving cost and permanent publication/storage of one raw input byte. The profile records the L1 base-fee, blob-fee and L2/prover price caps and margin used to derive all four amounts; a release whose benchmark corpus exceeds any bound rejects. The five words are identical in every authorization of one profile and are componentwise nondecreasing across successor profiles, so an old exact adapter never receives a cheaper active schedule after migration. This deposit covers execution, proof credit and permanent calldata/state costs. It is distinct from the SourceBridge's already escrowed `value+fee+liquidityFee`. The adapter verifies kind-1 source escrow and all static bounds before calling the router; failure leaves neither deposit, queue leaf nor partial credit. The mandatory codec corpus pins kind-0 raw-transaction lengths and kind-1 `Message.data.length` at 0, 31, 32, 33 and 131,072 bytes (with kind 0 rejecting zero), and rejects a one-byte mismatch between the normalized Message, descriptor, fee and per-block byte accounting.

Only `ActiveSettlementRouter` may append, and it accepts these two entry points only from an exact typed `authorizedIngress[caller]` entry. The target release/profile precommits the complete authorization root and deployed adapter objects. Cutover atomically binds Kind 0 and stages the exact Bridge support route. After the canonical destination-registration proof and the 214-block support delay, anyone may one-shot bind that release's exact historical Bridge adapter; no post-cutover governance action is required. Address, object, domain or kind-role conflicts reject, while old exact adapters remain usable for status, cancellation and refund of their historical records. Only the exact adapter deployment named by the active authorization may append; after cutover an old adapter cannot enqueue a still-NEW predecessor credit. That source credit remains permissionlessly cancelable after its strict `enqueueBy`. Every adapter fixes this router and queue in constructor/configuration. The router calls the active Settlement's `sync()` only in nonpayable `syncIngress` under a non-reentrant guard and reports `SYNCED` without accepting value if any transition occurred. Its adapter caller preserves that transition, creates no admission record and keeps the entire fee locally claimable by the original caller. Otherwise it computes `enqueuedAt=block.timestamp` and `dueAt=max(enqueuedAt+FORCE_DELAY,lastDueAt,recoveryExpiry+1)` (the last term is zero outside recovery), and appends atomically. Neither a sync-result Boolean, enqueue timestamp nor due time is accepted in router calldata; payable append rechecks the shared gate is ACTIVE and the exact stamp before forwarding value, without a second sync decision. Router, queue and adapter are nonreentrant, and failure of any later append/Bridge/adapter write reverts the value transfer and all state. No public caller can invoke the forwarding path. `ForcedQueue` stores one authoritative `QueueDescriptor` per index. It is the complete fixed-width kind-0 or kind-1 leaf preimage in Appendix A, excluding only kind-0 `rawTx` bytes themselves but including their hash. Thus sender, nonce, chain IDs, byte/gas fields, fee, validity, refund address, enqueue/due times, deposit and every bridge-credit field remain durable. Kind 1 requires DIRECT and zero Vault/capsule fields. The Merkle leaf is always recomputed from this stored descriptor; admission atomically writes the descriptor, deposit/escrow accounting, append frontier and root or writes nothing. The fee is deliberately nonrefundable after append: it buys FIFO storage, proving and processing even when an expired or invalid kind-0 item has no Ethereum transaction. The queue stores `depositPrefix[i+1]=depositPrefix[i]+deposit[i]` with checked addition. It is also the sole owner of `cursor`; neither Settlement, Protocol nor either ingress adapter has a second descriptor array, root, count, cursor or due-time cache. Settlement reads the queue's `dueAt` directly for the canonical head and a stored best's live boundary; there is no callback, cached deadline or duplicated scalar. Missing indices return `UINT64_MAX`. Thus `dueAt` is nondecreasing and force-due/coverage checks are one authoritative O(1) read. The queue exposes exactly two cursor-authority transitions:

```
advanceCursor(uint64 expectedStart, uint64 end,
              address proofBeneficiary)              // active Settlement only
setActiveSettlement(address expectedOld, address new) // router only
```

`advanceCursor` requires `cursor=expectedStart<=end<=count` and writes the cursor and, in O(1), moves `depositPrefix[end]-depositPrefix[expectedStart]` from unconsumed escrow to `claimable[proofBeneficiary]`. The beneficiary is the exact nonzero public input authenticated by the accepted proof; it is not chosen by the queue caller. The queue remains the ETH custodian and enforces the solvency lower bound `address(this).balance>=unconsumedEscrow+totalClaimable`; every claim-ledger mutation updates `totalClaimable` by the same checked amount, so no global sum or scan is performed. ETH forced into the contract without an append is surplus, never enters either liability and cannot make an append, advance or withdrawal revert. It exposes an unpausable CEI pull withdrawal. There is no per-item callback and no "unused" refund. `setActiveSettlement` requires the stored old address and is called only inside bootstrap or an atomic version switch. The ingress router never advances consumption, and a frozen Settlement cannot do so.

The queue is a protocol-lifetime, depth-64 append-only Merkle vector with fixed capacity `UINT64_MAX` items: the final leaf is deliberately unused. It stores root, `uint64` count/cursor, cumulative deposit prefix and a 64-node append frontier. Admission rejects only when `count=UINT64_MAX`; no version migration rotates or resizes the queue. A canonical DFS range multiproof authenticates the consumed leaves and, when present, the first excluded boundary leaf under the frozen `(forceRoot,forceCutoff)`. It supplies no fully covered subtree and no unused hash; at most 257 sibling hashes authenticate a 65-leaf block range. Skipping, reordering, changing the start index or claiming an early boundary changes the reconstructed root. Kind-0 raw bytes are enqueue calldata and their hash is in the leaf; blobs are forbidden on this availability path. The deliberately unused final tree leaf avoids a nonexistent height-64 frontier slot when the old index has all 64 bits set. At one billion appends per second the usable space lasts more than five centuries, so exhaustion is outside any credible Ethereum lifetime while remaining an explicit hard stop.

Each proved block consumes the unique maximum FIFO prefix satisfying all three bounds simultaneously. `forceGasBudget` is 13,000,000 exactly for the first release-activation block and 20,000,000 for every steady block:

$$count \le 64, \qquad \sum byteLength \le 1{,}048{,}576, \qquad \sum accountedGas \le forceGasBudget.$$

Every admitted item individually fits all three. The circuit exposes (`start,end,count,totalBytes,totalAccountedGas,forceRoot,forceCutoff,nextDueAt,forceGasBudget`) and verifies the range multiproof. Maximality means either `end=forceCutoff`, or the authenticated next leaf would exceed at least one remaining per-block cap; an early stop with a fitting leaf rejects. Candidate totals are independently capped as stated in §5. Launch enforces the separate activation and steady Anchor/force/margin inequalities in §12, plus the analogous witness-byte bounds. Because an item is at most 5,000,000 accounted gas, even the activation cap always admits the due FIFO head; a 20m backlog is split maximally under 13m in that one block and 20m thereafter rather than making activation invalid.

For the candidate's exact consumed interval, Settlement reads at most 256 descriptors plus the first excluded boundary descriptor when one exists under the frozen cutoff. Internal per-block boundaries are already the next consumed descriptor. It computes

```
H("slot-chain-force-descriptor-list-v2" || u64(start) ||
  u16(consumedCount) || u8(hasBoundary) ||
  (u64(index)||u8(kind)||u16(descriptorByteLength)||descriptorBytes)*)
```

as `forcedDescriptorCommitment`. The number of rows is the consumed count plus the zero-or-one boundary and is at most 257. `descriptorBytes` is the exact kind-specific packed sequence used by the leaf after its ASCII domain and index; its length is the unique constant for that kind. The circuit reconstructs every leaf, range proof, per-block maximum, candidate total and processing-fee transfer from these L1-authenticated descriptors. It additionally requires raw bytes for an unexpired kind 0 and the durable credit record for kind 1. An expired kind 0 needs neither historical calldata nor any unstored preimage.

Forced envelopes are auxiliary protocol inputs, not blindly copied into the Ethereum transaction list. Each block's transaction order is: (1) the exact profile-defined AnchorV4 transaction, (2) one deterministic `InboxApplyRouterV2` system transaction, (3) upfront-valid kind-0 raw transactions in FIFO order, then (4) discretionary transactions. Sequential pre-state classification puts expired, wrong-nonce, underfunded or underpriced kind-0 items only in `InboxApplyRouterV2`'s disposition vector; they have no Ethereum transaction, nonce or receipt. Upfront-valid kind 0 appears as an ordinary Ethereum transaction, whose success or revert has ordinary nonce/gas/receipt semantics. Kind 1 always produces its idempotent inbox credit inside `InboxApplyRouterV2` and never calls the eventual destination. The system calldata commits (`forceStart,forceEnd,(queueIndex,disposition,txIndex,resultHash)*`); the circuit derives the transaction and receipt roots from this exact order. The full consumed fee interval is pull-credited to the proof beneficiary by the same L1 canonical commit. `refundAddress` remains part of the signed kind-0 descriptor for transaction-level compatibility but creates no queue-fee entitlement in this version; changing consumption or payment requires changing the descriptor and proof statement.

Disposition bytes are fixed: `0=EXPIRED_NO_TX`, `1=NONCE_NO_TX`, `2=FUNDS_NO_TX`, `3=FEE_NO_TX`, `4=INCLUDED_TX` and `5=BRIDGE_CREDIT`. Classification tests in that order; the first applicable no-transaction reason wins. Codes 0–3 require `txIndex=UINT32_MAX` and `resultHash=0`. Code 4 requires the exact Ethereum transaction index; `resultHash` is legacy Keccak-256 of the raw signed EIP-2718 transaction bytes. This ordinary Ethereum transaction hash is known before execution. It is deliberately not a receipt hash: putting a later receipt hash in the earlier InboxApply calldata would create a fixed point through cumulative gas. Code 5 requires `txIndex=UINT32_MAX` and this exact durable credit hash:

```
H("slot-chain-bridge-credit-result-v11" || u64(queueIndex) || creditId ||
  msgHash || u256(srcChainId) || sourceDomainId ||
  u64(srcEpoch) || address20(srcBridge) ||
  bridgeExecutionHash || u64(emittedAtBlock) || destinationDomainId ||
  u256(destChainId) || u64(enqueueBy) || address20(sender) ||
  address20(srcOwner) || address20(destOwner) || u256(value) ||
  u64(fee) || u64(liquidityFee) || calldataHash ||
  u8(refundMode) || address20(refundVault) ||
  refundCapsuleHash || escrowId)
```

InboxApplyRouterV2 recomputes it from the authenticated descriptor before routing the exact tuple and `resultHash` to `InboxCreditStoreV2`. That immutable store, neither Bridge nor legacy SignalService, owns the persistent destination pin. Resolver rotation and SignalService upgrades cannot retarget, fabricate or orphan an existing credit. Unknown codes or noncanonical auxiliary fields reject.

AnchorV4 and InboxApplyRouterV2 use a profile-defined, unsigned custom L2 system transaction type whose sender is injected by the post-cutover fork, not recovered from ECDSA. That type is invalid under every legacy fork, and the two reserved sender addresses are chosen with no known signing key. The circuit requires their exact profile transactions once each at indices 0 and 1 and rejects any ordinary, forced or discretionary transaction recovered from either sender. These reserved senders and the router/store pin ABI are permanent across every V2 profile; a later fork must still dispatch old routes through this router. InboxApplyRouterV2 stores the last applied L2 block number and rejects a second application in that block. An ordinary caller therefore cannot invoke the privileged path before or after cutover.

The launch fork deploys one protocol-lifetime non-proxy immutable `InboxApplyRouterV2` before every endpoint-specific immutable `InboxCreditStoreV2`. Their permanent operations are:

```
struct InboxCreditV2 {
  uint64 queueIndex; uint64 srcChainId; bytes32 sourceDomainId;
  uint64 srcEpoch; address srcBridge; bytes32 destinationDomainId;
  bytes32 msgHash; bytes32 resultHash; bytes32 sourceContextHash; uint256 value;
  uint64 executionFee; uint64 liquidityFee;
}
struct InboxRowV2 {
  uint64 queueIndex; uint8 disposition; uint32 txIndex;
  bytes32 resultHash; bytes kind1Descriptor;
}
InboxApplyRouterV2.apply(uint64 forceStart, InboxRowV2[] calldata rows)
markReceivedFromInboxBatch(InboxCreditV2[] calldata rows)
    returns (bytes4 magic)
routeConfigHashV2() view returns (bytes32)
verifyInboxCredit(uint64 srcChainId, bytes32 sourceDomainId,
                  uint64 srcEpoch, address srcBridge,
                  bytes32 destinationDomainId, bytes32 msgHash)
    view returns (bytes32 creditId, uint64 processBy, uint256 value,
                  uint64 executionFee, uint64 liquidityFee,
                  bytes32 sourceContextHash)
```

```
getInboxCreditSlot(bytes32 sourceDomainId, address srcBridge,
                   bytes32 destinationDomainId,
                   bytes32 creditId) pure returns (bytes32)
liquidityQuoteV2(bytes32 creditId) view returns
  (bytes32 resultHash, uint256 value, uint64 executionFee,
   uint64 liquidityFee, uint64 processBy)
DestinationBridge.liquidityFundingStateV2(bytes32 creditId) view returns
  (bytes32 destinationDomainId, address destBridge, address inboxCreditStore,
   uint8 status, uint64 terminalIndexPlusOne)
```

Every endpoint store constructor authorizes the same InboxApplyRouterV2; only that router may call the store batch operation, and only after that endpoint's one-shot activation. Router state includes checked `nextQueueIndex`, which the circuit requires to equal `CanonicalCore.messageCursor`. Every block calls the router once with its complete contiguous zero-to-64 forced-row interval, including kind-0 dispositions, in global queue order. It first validates the whole vector, exact result hashes, no duplicate `(domain,creditId)`, and each one-shot route `destinationDomainId->(store,Bridge,routeConfigHash,codeHash)`. Only the registrar may append a route; domain, store and Bridge are each unique and no route can be deleted or redirected. The router rechecks code/config/domain and Bridge on every use, partitions only pin side effects into maximal contiguous same-domain runs, then makes at most 64 fixed-selector, zero-value calls and requires the exact success magic. It advances `nextQueueIndex` only after all calls succeed. A missing route, conflict or later call failure reverts the cursor and every earlier store write. There is no sorting and no loop over historical routes. The batch selector is the canonical Solidity selector for the signature above. Precisely, it is the first four Keccak bytes of `"apply(uint64,(uint64,uint8,uint32,bytes32,bytes)[])"`. Calldata after the selector has `forceStart` in word 0 and the canonical rows offset `0x40` in word 1. At byte `0x40` is the row count $n \leq 64$, followed by n element offsets relative to the beginning of the element-head area (byte `0x60` of the arguments). Each element encoding is five head words in the displayed field order; its bytes offset is exactly `0xa0`, followed by the bytes length, data and zero right-padding. A kind-0 row requires zero descriptor length. A kind-1 row requires exactly the Appendix 541-byte durable descriptor body, padded to 544 bytes. Element offsets must be contiguous and minimal; aliases, gaps, overlap, trailing calldata, nonzero padding or a length inconsistent with the row kind rejects. Consequently the largest 64-row calldata is 49,252 bytes including selector. The bound statement's `inboxSystemCalldataHash` is Keccak of this complete byte string, while `inboxSystemTxHash` is Keccak of the complete type-`0x7f` envelope that contains it. The canonical Inbox rows in the L1 activation call are the data-availability preimage for both hashes; a proof-only hash is insufficient. Success returndata is exactly one 32-byte ABI word equal to `0x49425632` (ASCII "IBV2") followed by 28 zero bytes; empty, short, long, malformed or different returndata rejects even if the call itself succeeds. The getter likewise must return exactly 32 bytes, and its value is

```
H("slot-chain-inbox-route-config-v1" ||
  address20(inboxApplyRouterV2) || address20(destinationBridge) ||
  address20(terminalDomainRegistrarV2) ||
  destinationDomainId)
```

The router pins that hash and the store runtime hash at registration and rechecks both by bounded `STATICCALL/EXTCODEHASH` before any batch call. Runtime hash, not a caller assertion, pins the getter semantics. A successful no-op implementation cannot advance the cursor because the same transaction's post-state circuit must find every fresh pin at the Appendix-defined slot; the conformance corpus includes a magic-returning no-op. That pin stores the exact authenticated result hash, source-context hash, value, execution fee, liquidity fee and `processBy`; none is inferred from a later Message. `liquidityQuoteV2` returns exactly five canonical ABI words and is callable only through a registrar-routed Store identity. The Pool checks the Store runtime/configuration and route before reading it; short/trailing/malformed return or a zero result hash rejects without opening an authorization. The validity circuit proves those stored words equal the consumed V11 descriptor/result hash, so an under- or over-debit cannot be created by substituting a quote. `liquidityFundingStateV2` likewise returns exactly five canonical ABI words under the manifest-pinned Bridge code/configuration. It preserves the existing IBridge enum encoding NEW=0, RETRIABLE=1, DONE=2, FAILED=3 and RECALLED=4. NEW and RETRIABLE are the only fundable statuses and require `terminalIndexPlusOne=0`; every terminal status has the checked terminal index plus one. Padding, short/trailing data or an inconsistent Store/domain/ Bridge tuple rejects. The canonical empty vector requires `forceStart=nextQueueIndex`, makes no store call and leaves the cursor unchanged, but still records the exact L2 block number; a second apply in that block rejects.

`kind1Descriptor` is empty for dispositions 0–4 and exactly the Appendix-defined 541-byte kind-1 descriptor for disposition 5; all other lengths reject. The router decodes that descriptor, recomputes `creditId` and the complete `bridge-credit-result-v11` hash, and derives the destination route. The circuit separately proves the same row/descriptor against the consumed L1 queue leaf. No absent owner, value, execution-hash, capsule or escrow field is treated as authoritative calldata.

Every registered store is nonproxy, unpausable, callback-free and permanently implements this exact all-or-revert pin ABI. Each run is completely validated before its first write. Activation is blocked unless the profile's system gas margin covers the maximum *reachable* cold path under the same count and accounted-gas caps proved by the circuit. In a steady block the route-call maximum is `min(64,floor(20,000,000/5,000,000))=4`, hence at most four route calls and sixteen fresh pin writes. In the first activation block the analogous 13,000,000-gas cap admits at most two kind-1 rows and eight fresh pin writes. The mandatory benchmark set includes all-cold four-kind-1 steady execution, the 64-row steady mix of 61 minimum-gas kind-0 rows plus three kind-1 rows, and the 64-row activation mix of 62 minimum-gas kind-0 rows plus two kind-1 rows. It also proves that the next non-fitting FIFO leaf is excluded. An unreachable 64-kind-1/256-write trace is not an activation requirement. Each kind-1 row conservatively carries `accountedGas=MAX_FORCE_MESSAGE_GAS=5,000,000`; opcode repricing may reduce throughput but cannot leave an old head underfunded. The 20,000,000 per-block forced-gas budget therefore admits at most four kind-1 rows per block. The store validates widths and nonzero fields and performs all writes atomically. The slot is exactly

```
H("slot-chain-inbox-credit-slot-v4" || sourceDomainId ||
  address20(srcBridge) || destinationDomainId || creditId)
```

Each pin occupies exactly four mapping values under the profile-pinned storage layout: `inboxResultHash[slot]`, `inboxSourceContextHash[slot]`, `inboxValue[slot]` and `inboxPackedTerms[slot]`. Packed-term bits 0–63 are `processBy`, 64–127 `executionFee`, 128–191 `liquidityFee`, and 192–255 are zero. A nonzero result hash is the received bit. An absent result stores those four words, with `processBy=block.timestamp+BRIDGE_PROCESS_TTL_SECONDS`. The initial TTL is 2,592,000 seconds. The contract exposes no proxy, upgrade, delegatecall, mutable writer/reader/domain target, arbitrary storage primitive or pause gate. A repeated row succeeds without writes only when the stored result, context, amounts and deadline match and never extends `processBy`; any conflict, ordering error or excess row reverts the whole batch before a write. `verifyInboxCredit` recomputes `creditId` and the slot, requires the received bit and nonzero result, and requires `msg.sender` to be the constructor-fixed destination Bridge for that exact local domain before returning all six exact words. Runtime hash, constructor router/Bridge/gate values, the one-shot sealed local-domain slot and initial empty mappings are migration-proved; the noncircular construction descriptor deliberately excludes the derived domain value. This is a new bytes32-domain ABI; it never overloads legacy `getSignalSlot(uint64,address,bytes32)`.

BridgeV2 introduces these exact static context tuples:

```
struct SourceContextV2 {
  uint64 protocolVersion; uint8 kind; bytes32 creditId; bytes32 msgHash;
  bytes32 sourceDomainId; uint64 sourceRegistrationEpoch;
  address sourceBridge; bytes32 sourceBridgeExecutionHash;
  uint64 emittedAtBlock; uint64 queueIndex;
}
struct DestinationContextV2 {
  uint256 destinationChainId; bytes32 destinationDomainId;
  address destinationBridge; bytes32 releaseManifestHash;
  bytes32 executionProfileHash;
}
```

Here `protocolVersion`, manifest and profile are the immutable destination release registered for that domain; `kind=1`. The remaining words are recomputed from the durable pin and V11 descriptor. Address, `uint8` and `uint64` ABI padding is canonical. The Bridge rejects any caller-supplied word mismatch before status, Pool authorization, value or external effects. The typed context commitment is

```
SOURCE_CONTEXT_TYPE_LITERAL =
  "SourceContextV2(uint64 protocolVersion,uint8 kind,bytes32 creditId,"
  "bytes32 msgHash,bytes32 sourceDomainId,uint64 sourceRegistrationEpoch,"
  "address sourceBridge,bytes32 sourceBridgeExecutionHash,"
  "uint64 emittedAtBlock,uint64 queueIndex)"
SOURCE_CONTEXT_TYPEHASH = keccak256(concat(the four quoted fragments))
sourceContextHash = keccak256(abi.encode(SOURCE_CONTEXT_TYPEHASH,
  all SourceContextV2 fields in the displayed order))
DESTINATION_CONTEXT_TYPE_LITERAL =
  "DestinationContextV2(uint256 destinationChainId,bytes32 destinationDomainId,"
  "address destinationBridge,bytes32 releaseManifestHash,"
  "bytes32 executionProfileHash)"
DESTINATION_CONTEXT_TYPEHASH = keccak256(concat(the three quoted fragments))
destinationContextHash = keccak256(abi.encode(DESTINATION_CONTEXT_TYPEHASH,
  all DestinationContextV2 fields in the displayed order))
```

Each literal is the byte concatenation of its displayed quoted fragments with no inserted whitespace or newline. InboxApplyRouterV2 derives it only from the V11 descriptor, queue index and registrar-bound release version, and the Store pins it with the credit. `verifyInboxCredit` returns exactly six canonical ABI words including that hash. Destination Bridge recomputes it from the supplied tuple and requires equality; queue index, emission block and source execution hash are therefore not caller assertions hidden behind `creditId`. The existing send ABI remains V1; the complete additive V2/recovery ABI is:

```
struct LiquiditySettlementV1 {
  bytes32 ticketId; address l1Recipient; uint256 settlementAmount;
}
sendMessageV2(Message, uint64 liquidityFee) -> (bytes32 msgHash, Message)
messageContextV2(bytes32 msgHash)
  -> (bytes32 creditId, SourceContextV2, DestinationContextV2)
markQueuedV2(bytes32 creditId, uint64 queueIndex)
cancelUnqueuedMessageV2(bytes32 creditId)
finalizeDoneV2(bytes32 creditId, uint64 protocolVersion,
               uint64 canonicalSequence, uint64 leafIndex,
               LiquiditySettlementV1 settlement,
               bytes32[64] siblings)
processMessageV2(Message, SourceContextV2, DestinationContextV2)
  -> (Status, StatusReason)
retryMessageV2(Message, SourceContextV2, DestinationContextV2,
               bool isLastAttempt)
failMessageV2(Message, SourceContextV2, DestinationContextV2)
expireMessageV2(bytes32 creditId)
recallMessageV2(bytes32 creditId, uint64 protocolVersion,
                uint64 canonicalSequence, uint64 leafIndex,
                bytes32[64] siblings)
withdrawSourceCreditV2(address payable recipient)
withdrawLiquidityClaimV2(address payable recipient)
messageStatusV2(bytes32 creditId) -> Status
invocationContextV2() view returns (SourceContextV2, DestinationContextV2)
ActiveSettlementRouter.canonicalAt(uint64 protocolVersion,
                                   uint64 canonicalSequence)
  -> CanonicalHistoryV2
TerminalSignalVerifier.verifyTerminalSignal(
  bytes32 destinationDomainId, address destBridge,
  bytes32 creditId, uint8 terminal, bytes32 liquiditySettlementHash,
  uint64 protocolVersion,
  uint64 canonicalSequence, uint64 leafIndex,
bytes32[64] siblings) -> bytes32
```

Immediately around an allowed recipient CALL, Bridge installs those complete tuples in transient storage. `invocationContextV2` succeeds only while that exact outer call frame is live; it reverts before installation, after the finally-style clear, and during any failed/reverted attempt. The shared non-reentrant guard forbids a nested Bridge attempt from replacing the context. An implementation that lacks EIP-1153 support for the launch fork is invalid; persistent scratch storage is not an alternative.

Recipient invocation is a frozen release policy, not an application allowlist. The profile publishes a list of at most 64 addresses, sorted by unsigned 160-bit value and free of duplicates, denied addresses and

```
INVOCATION_POLICY_TYPEHASH = keccak256(
  "InvocationPolicyV2(uint16 count,bytes32 addressesHash,bytes4 hookSelector)")
addressesHash = keccak256(address20(denied[0]) || ... || address20(denied[n-1]))
invocationPolicyHash = keccak256(abi.encode(
  INVOCATION_POLICY_TYPEHASH, uint16(n), addressesHash,
  IMessageInvocable.onMessageInvocation.selector))
```

The destination Bridge configuration, source support record and release profile all bind that hash and exact list; historical lists cannot be changed. The list must contain the zero address, destination Bridge, both SignalServices, native QuotaManager, manifest pauser, every V1 Vault named by the release, any DelegateController, InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2, NativeLiquidityPoolV2, TerminalAccumulatorV2, TerminalSignalVerifier, every context/helper endpoint and every other system or Bridge-trusting endpoint enumerated by the profile. SourceBridge rejects a denied `message.to` before any ID, escrow or Registry write. Destination Bridge repeats the check; a historical or corrupted omission transitions to FAILED with zero target, quota, fee and Pool consumption, enabling the ordinary source refund proof. When discovered inside a Pool-funded call, the authorization remains OPEN and the ticket remains byte-identical; the exact FAILED wrapper result is joined by the Pool before it clears the authorization. The corpus includes a target denied only at destination after source authorization, proving no LP principal is consumed. ProtocolVersionManager materializes the source-side list once at finalized domain-support confirmation. BridgeDomainRegistry exposes only

```
invocationPolicyV2(bytes32 destinationDomainId, address target) view returns
  (bytes32 invocationPolicyHash, bool denied, bytes4 magic)
```

with exactly 96 bytes, canonical Boolean/padding and magic `0x49505632` (`IPV2`). SourceBridge pins Registry code/configuration and a gas limit, requires the returned hash to equal the supported domain and rejects on any call/length/value fault. Destination Bridge uses its deployment-proved local mapping with the same hash/list; neither side accepts a Merkle proof, caller Boolean or mutable application registry.

All other addresses are permissionless and may acquire code after send through CREATE or CREATE2; code presence is decided only at execution. No-code accounts are treated as EOAs. Empty data with zero value succeeds without CALL. Empty data with positive value and data of length one to three use an ordinary CALL and therefore the recipient fallback/receive path. For data length at least four, a contract CALL is permitted only when the first four bytes equal `IMessageInvocable.onMessageInvocation(bytes).selector` and the complete calldata is the canonical ABI encoding of that one bytes argument; malformed or another selector is invocation-prohibited. Invocation-prohibited processing makes no target CALL: it creates the destination owner's value pull and the successful processor's execution-fee pull, consumes quota and atomic Pool funding, and appends DONE atomically. EOA delivery uses ordinary CALL semantics and never interprets a selector. The profile corpus covers zero/1/3/4-byte boundaries, unknown and post-CREATE2 accounts, each denyset member, malformed hook ABI and a target attempting context/reentrancy substitution. This permission does not let a later release turn a previously unprivileged counterfactual address into an old-Bridge-trusting system endpoint. Every future privileged endpoint that accepts a Bridge/context call must constructor-pin exactly one fresh `destinationDomainId` and its fresh DestinationBridge, and must revalidate the complete transient `DestinationContextV2` inside its hook. It must reject every historical Bridge and any registry-wide "recognized Bridge" predicate. Each successor profile, deployment commitment and verifier enforce that rule for every newly privileged endpoint before publishing its address. The conformance corpus pins the counterfactual sequence send-to-X, deploy-new-privileged-X, call-from-old- Bridge and requires X to reject without effect. An application that chooses to trust historical Bridges is ordinary application risk and cannot be named a protocol/Vault/system endpoint by a release. `CanonicalHistoryV2` is the fixed tuple

```
(uint64 protocolVersion, bytes32 executionProfileHash,
 uint64 canonicalSequence, uint48 l2BlockNumber, bytes32 blockHash,
 uint64 tipSlot, bytes32 stateRoot, uint64 messageCursor,
 bytes32 winningDataCommitment, uint256 nextBaseFee,
 uint64 nextExcessBlobGas, bytes32 terminalRoot,
 uint64 terminalCount, uint64 canonicalizedAtBlock)
```

and the only caller-supplied proof payload is

```
(uint64 protocolVersion, uint64 canonicalSequence,
 uint64 leafIndex, bytes32[64] siblings)
```

The verifier reads the exact sequence-tagged ring entry from Router, requires `leafIndex<terminalCount`, derives the exact domain-separated leaf from index, domain, Bridge, credit ID, terminal and liquidity-settlement hash, and folds exactly 64 siblings to the stored root. There is no caller-supplied Canonical tuple, dynamic array, RLP node, current EVM state proof or trailing element. `terminal` is exactly 1 for DONE or 2 for FAILED; all other values reject. FAILED requires `liquiditySettlementHash=bytes32(0)`. DONE requires

```
liquiditySettlementHash = H("slot-chain-liquidity-settlement-v1" ||
  ticketId || address20(l1Recipient) || u256(settlementAmount))
settlementAmount = message.value + message.fee.
```

For DONE, `ticketId`, `l1Recipient` and the resulting `liquiditySettlementHash` are each nonzero; checked addition rejects overflow. A Pool deposit whose derived ticket ID is zero reverts before creating a row or accepting value. SourceBridge recomputes that hash from the caller-supplied fixed settlement tuple and the immutable credit, so a proof copier cannot redirect the LP pull. The verifier constructor fixes only `ActiveSettlementRouter` and depth 64; it is generic across every append-only destination endpoint. Its explicit domain and Bridge arguments are low-level leaf inputs, never source authority. Each historical source Bridge terminal transition loads `destinationDomainId` from its immutable `CreditAuthorization`, bounded-static-reads the corresponding permanent `destBridge` from `BridgeDomainRegistry`, and passes those values. It rejects if the descriptor is absent or differs from the proof; no caller or proof chooses either value. A D1 source endpoint can therefore finalize D1 and a later registered D2 without changing its verifier. Here `Message` is the existing IBridge tuple and `msgHash=hashMessage(Message)`. Destination processing first requires the context's destination Bridge to equal its own address and the context's domain to equal its one-shot sealed local destination domain. It then recomputes that domain from the pinned chain ID, genesis, BridgeInboxAdapter, active settlement router, terminal verifier, InboxApplyRouterV2, InboxCreditStoreV2, protocol-release authority, terminal-domain registrar, terminal accumulator, Bridge address/frozen execution hash, infrastructure hash and namespace. Only then do they recompute `creditId` from the Message hash and context before any status, value or external effect. No caller-supplied foreign domain may query a pin through a different funded

Bridge. Source terminal functions load the immutable authorization and current permanent liability by credit ID, while destination expiration loads the permanent inbox pin; neither needs the Message preimage. Queue marking is authorized only to the exact adapter in the recorded destination domain. The read-only status view accepts the derived key. The SourceBridge records both contexts. Its profile-pinned V2 event carries the credit ID, message hash, both contexts and the normalized Message; destination changes emit the credit ID and new status.

Native delivery uses one protocol-lifetime immutable `NativeLiquidityPoolV2`, not a finite reserve, native minting or a long-lived reservation. Any address may create or top up its own ticket and bind the L1 reimbursement recipient:

```
depositLiquidityV2(address l1Recipient, bytes32 salt) payable
  returns (bytes32 ticketId, uint256 availableWei)
withdrawLiquidityV2(bytes32 ticketId, address payable recipient,
                    uint256 amount)
  returns (uint256 remainingAvailableWei)
processWithLiquidityV2(bytes32 ticketId, address destinationBridge,
  MessageV1 message, SourceContextV2 sourceContext,
  DestinationContextV2 destinationContext)
  returns (uint8 resultingStatus, uint8 statusReason)
retryWithLiquidityV2(bytes32 ticketId, address destinationBridge,
  MessageV1 message, SourceContextV2 sourceContext,
  DestinationContextV2 destinationContext, bool isLastAttempt)
  returns (uint8 resultingStatus, uint8 statusReason)
ticketAccountingV2(bytes32 ticketId) view returns
  (address depositor, address l1Recipient, uint256 availableWei)
poolAccountingV2() view returns
  (bytes4 magic, bytes32 configurationHash, bool active, bool entered,
   bytes32 activeAuthorizationHash, uint256 totalAvailableWei,
   uint256 balanceWei, uint256 surplusWei)
```

The exact identifier is

```
ticketId = H("slot-chain-liquidity-ticket-v2" || u256(chainId) ||
  address20(pool) || address20(depositor) || address20(l1Recipient) || salt).
```

Deposit requires nonzero value and recipient and rejects the cryptographically exceptional zero ID. An absent row creates the committed tuple; a present row may only checked-add when both stored identities equal the same preimage. Withdrawing or consuming the final wei captures the tuple before deleting the row. Recreating the exact tuple and salt intentionally recreates the same ID; the same salt under another depositor or recipient derives another ID. There is no global nonce, approval, operator, transfer, salt registry, tombstone, lifetime counter or mapping enumeration. Old DONE claims never consult the current Pool row: each terminal leaf independently binds its credit, ticket, captured L1 recipient and exact amount.

Every withdrawal and funded attempt requires `msg.sender=ticket.depositor`. Withdrawal is CEI-first under the Pool guard, calls only the nonzero chosen recipient, and restores the exact row and aggregate on revert or reentrancy. A Pool-funded attempt is also initiated at the Pool, not at the Bridge: this prevents any current or future registered Bridge from claiming to be an LP and debiting an arbitrary lifetime ticket. The Pool bounded-static-reads the registrar-routed Store quote and Bridge state with exactly `POOL_EXTERNAL_READ_GAS=50000` each. It requires exact five-word canonical returns, the pinned result hash, domain, Store and Bridge, status NEW or RETRIABLE, zero terminal index, nonexpired `processBy`, and checked `amount=value+executionFee>0`. It never trusts caller-supplied amount or fee fields.

The Pool then opens one transient storage authorization:

```
authorizationHash = H("slot-chain-native-liquidity-attempt-v1" ||
  u256(destinationChainId) || destinationDomainId || address20(pool) ||
  address20(destinationBridge) || address20(inboxCreditStore) || creditId ||
  ticketId || address20(depositor) || resultHash || u256(amount) ||
  sourceContextHash || destinationContextHash || u8(operation) ||
  u8(isLastAttempt ? 1 : 0))
```

where PROCESS=1 and RETRY=2; PROCESS requires `isLastAttempt=false`, while a last attempt is valid only for RETRY. The Pool stores the same exact tuple plus state OPEN under its entered guard. It calls only the registrar-bound nonretired Bridge with no value and canonical calldata. For positive Message gas the requested amount is exactly `V2_POOL_BRIDGE_PRE_CALL_GAS_BOUND+V2_ATTEMPT_PRE_CALL_GAS_BOUND+message.gasLimit;` owner-only zero-gas mode requests only the gas remaining above `POOL_AUTH_CLEANUP_GAS=50000`. Before CALL, after fixed memory and warm account costs, Pool enforces both the exact EIP-150 forwarding inequality and the 50,000-gas worst-case cleanup reserve, and copies at most 192 return bytes. That Bridge accepts the call only from the immutable Pool, recomputes the credit, contexts, quote, authorization and effective `processor=depositor`, then invokes its external only-self child attempt. Its exact entry and return are:

```
attemptFromLiquidityPoolV2(bytes32 ticketId, address depositor,
  MessageV1 message, SourceContextV2 sourceContext,
  DestinationContextV2 destinationContext, uint8 operation,
  bool isLastAttempt, bytes32 authorizationHash)
  returns (bytes4 magic, bytes32 creditId, uint8 resultingStatus,
           uint8 statusReason, uint64 terminalIndexPlusOne,
           bytes32 liquiditySettlementHash)
```

Its selector is the first four Keccak bytes of the exact fully expanded tuple signature in the publication corpus. Calldata is the canonical `1124`-byte vector for the 33-byte Message fixture, rejects every dynamic offset/padding/trailing-byte alias, and binds the authorization as its final head word. The Pool-to-Bridge wrapper returns exactly six canonical words (192 bytes):

```
(bytes4 magic, bytes32 creditId, uint8 resultingStatus, uint8 statusReason,
 uint64 terminalIndexPlusOne, bytes32 liquiditySettlementHash)
POOL_BRIDGE_RESULT_MAGIC = 0x4c415632 // ASCII LAV2
```

It never normalizes a low-level revert, OOG or malformed return into an accepted status. No permit or delegated-funder mode exists in this revision.

Inside that child frame only, the Bridge calls

```
consumeAuthorizedLiquidityV2(bytes32 ticketId, bytes32 creditId,
  address depositor, uint256 amount, bytes32 resultHash)
  returns (bytes32 returnedTicketId, address l1Recipient,
           uint256 consumedAmount)
```

The Bridge requests exactly the profile-pinned `V2_POOL_CONSUME_CALL_GAS_BOUND` for this call after enforcing the same EIP-150 relation and retaining its measured child suffix; that bound includes the Pool's value-CALL overhead and 100,000-gas callback. The Pool permits this one reentry only from the authorization's exact Bridge, for the OPEN authorization, matching active child attempt, caller, tuple and amount. It CEI-debits or deletes the ticket and aggregate, marks the authorization CONSUMED, transfers exactly the captured amount through the narrow callback below, and returns exactly 96 canonical bytes. Every other Pool entry, duplicate consume, direct Bridge call, target reentry and tuple substitution rejects while `entered=true`.

The native-value callback is:

```
DestinationBridge.acceptLiquidityValueV2(
  bytes32 creditId, bytes32 ticketId, uint256 amount)
  payable returns (bytes4 magic)
POOL_VALUE_CALLBACK_GAS = 100000
POOL_VALUE_MAGIC = 0x4e4c5632 // ASCII NLV2
```

Before consume, the child writes the exact active frame and

```
H("slot-chain-native-liquidity-acceptance-v3" ||
  u256(destinationChainId) || destinationDomainId ||
  address20(destinationBridge) || address20(nativeLiquidityPool) || creditId ||
  ticketId || address20(depositor) || resultHash || u256(amount) || attemptDigest)
```

as its sole expected callback. The Pool uses canonical 100-byte calldata, exact value and the 100,000-gas stipend and accepts only the exact 32-byte `NLV2` word. The callback requires the immutable Pool, same active child frame, exact tuple/value/commitment and no prior receipt; it only clears the expectation, stores the receipt and returns magic. Plain receive/fallback, duplicate or direct calls, wrong tuple/value, revert/OOG and short/trailing/bad magic fail. Bridge next requires the exact 96-byte Pool tuple, cleared expectation, exact receipt and raw-balance delta before any target call.

Target execution, quota debit, pull creation, DONE status and terminal append all remain in that same child frame. A target, quota, liability, callback, return-data or terminal fault reverts the child and therefore restores the ticket, Pool aggregate/value transfer, target, Bridge and authorization to OPEN byte-for-byte. The Bridge's outer catch may then write only the already-defined NEW, RETRIABLE or owner-authorized FAILED result. `isLastAttempt=true` still requires the effective processor to equal `destinationOwner`; an unrelated LP may only make a non-last retry. Manual failure and strict expiry never call the Pool.

On return to the Pool, it requires the exact magic and credit and joins the wrapper result to its authorization state. DONE requires CONSUMED, nonzero `terminalIndexPlusOne`, and a settlement hash recomputed from the captured ticket/recipient/amount. FAILED requires OPEN, nonzero terminal index and zero settlement hash. NEW or RETRIABLE requires OPEN and both terminal fields zero. RECALLED and unknown statuses are impossible, and every status accepts only its normative reason-code set. Any other status, low-level failure, malformed return or state combination reverts the complete top-level call. The Pool clears the authorization before returning only the status/reason pair to its caller. Thus LP principal moves iff the same journal commits DONE; a failing target has no option on LP capital and cannot leave a lease or orphaned row.

The hard Pool invariant is

```
address(pool).balance >= totalAvailableWei
sum(live ticket.availableWei) = totalAvailableWei
surplusWei = address(pool).balance - totalAvailableWei.
```

The sum is a model invariant, never a production scan. Forced ETH is surplus and creates no ticket. Mapping size is bounded by currently funded rows; dust rows pay their own storage and no protocol path enumerates them. Deposit is disabled until the first registrar activation, while later releases preserve all tickets and the same Pool. Ticket and Pool views return exactly 96 and 256 bytes; Pool magic is `0x504c4132` (`PLA2`), Boolean/high padding is canonical, and an absent ticket is all-zero. Invalid mutating calls revert.

The exact event surface is:

```
event LiquidityDepositedV2(bytes32 indexed ticketId,
  address indexed depositor, address indexed l1Recipient,
  uint256 amount, uint256 availableWei)
event LiquidityConsumedV2(bytes32 indexed creditId, bytes32 indexed ticketId,
  bytes32 indexed destinationDomainId, address destinationBridge,
  address l1Recipient, address processor, uint256 amount)
event LiquidityWithdrawnV2(bytes32 indexed ticketId,
  address indexed depositor, address indexed recipient,
  uint256 amount, uint256 remainingAvailableWei)
```

The byte-exact corpus fixes all selectors, topics, calldata/returndata lengths, padding, authorization and callback commitments and negative substitutions.

There is deliberately no claim of backlog-independent delivery. One signed Pool transaction can be put through the forced path without locking principal while it waits, but it succeeds only if the credit and forced envelope remain within their deadlines and the owner has not withdrawn or spent the ticket. Paid FIFO saturation can outlast those deadlines. Permissionlessness means anyone may deposit and compete with their own capital; the first successful canonical transaction wins the source reimbursement, while a front-runner cannot spend the victim's ticket. The success-only liquidity fee compensates only voluntary successful execution, not failed gas or an objective delivery guarantee. Because delegation is deliberately absent, third-party LP processing rejects owner-only zero-gas and last-retry modes; those succeed only when the destination owner funds and calls through its own ticket. Direct manual fail and permissionless expiry remain outside the Pool. The protocol does not claim that passive third-party liquidity can exercise destination-owner authority.

The child rollback boundary is one exact external self-call, never an ordinary internal call and never a second public reentrancy frame:

```
executeAttemptV2(
  MessageV1 message, SourceContextV2 sourceContext,
  DestinationContextV2 destinationContext, address processor,
  uint8 expectedEntryStatus, bool isLastAttempt,
  bytes32 ticketId, bytes32 authorizationHash)
  external returns (uint8 resultingStatus, uint8 statusReason)
finalizeFailedAttemptV2(bytes32 creditId)
  external returns (uint8 resultingStatus, uint8 statusReason)
error TargetCallFailedV2(bytes32 attemptDigest)
```

Its selector is the first four Keccak bytes of the exact expanded signature

```
executeAttemptV2((uint64,uint64,uint32,address,uint64,address,uint64,address,
address,uint256,bytes),(uint64,uint8,bytes32,bytes32,bytes32,uint64,address,
bytes32,uint64,uint64),(uint256,bytes32,address,bytes32,bytes32),address,uint8,
bool,bytes32,bytes32)
```

The failure-finalizer selector is the first four Keccak bytes of "`finalizeFailedAttemptV2(bytes32)`"; the custom-error selector is the first four Keccak bytes of "`TargetCallFailedV2(bytes32)`". The immutable implementation records `SELF=address(this)` at construction; both external entries require `msg.sender==SELF` and `address(this)==SELF`, so a direct caller, proxy or delegatecall context rejects before a read or write. The Pool-authenticated Bridge wrapper already holds the shared guard, authenticates the Message/pin/contexts/authorization, derives the effective processor from the Pool tuple and checks entry status, owner and last-attempt policy before making the self-call with zero value. The child defensively recomputes every field, consumes only that authorization and owns the complete Pool–target–quota–pull–terminal journal. The failure finalizer rechecks RETRIABLE, the same pin and absent terminal and owns only FAILED plus terminal; it never calls the Pool. No other entry may invoke one of those steps.

The wrapper ABI-encodes only those derived values, reserves the profile-pinned `V2_OUTER_CATCH_GAS_RESERVE`, and makes a bounded external call to `SELF`; its success return must be exactly two canonical ABI words (64 bytes). A failed low-level target CALL never returns normally and never bubbles or hashes target returndata. Instead the Bridge computes

```
attemptDigest = H("slot-chain-destination-attempt-v2" || creditId ||
  sourceContextHash || destinationContextHash || address20(processor) ||
  u8(expectedEntryStatus) || u8(isLastAttempt ? 1 : 0) ||
  ticketId || authorizationHash)
```

and reverts the complete inner frame with the exact 36-byte canonical custom error `TargetCallFailedV2(attemptDigest)`. This restores the consumed ticket, Pool transfer, target effects and Bridge state to their entry values. The outer wrapper locally recomputes the digest and recognizes only that selector, exact length and exact word. Target returndata is discarded before the Bridge creates this error, so a target-crafted identical error cannot spoof the catch.

After an authenticated target-failure catch, non-owner initial failure and non-last retry return the unchanged NEW or RETRIABLE state without writes; an owner initial failure writes RETRIABLE in the outer frame; and an owner-last retry makes the zero-value `finalizeFailedAttemptV2(creditId)` self-call. That second entry writes FAILED and appends its zero-settlement terminal leaf atomically after the first child has restored the ticket. If it reverts or returns anything but the exact canonical 64-byte FAILED result, the outer call reverts, leaving the entry status and ticket unchanged. Any first-frame revert other than the authenticated target-failure error, and any short/long return, noncanonical enum or unexpected status, discards revert bytes and maps only to `(expectedEntryStatus,ATTEMPT_ROLLED_BACK)` without an outer write. The release profile freezes both selectors, tuple grammars, custom-error selector/domain, valid status/reason matrix, self-call gas caps and catch reserve. Direct-inner, delegatecall, callback reentry, nested process/retry, target-crafted error, target failure with ticket then retry, owner-last rollback-before-FAILED, malformed return, and post-target quota/pull/terminal fault vectors prove the separation and byte-identical rollback.

The destination quota is exact: its only V2 key is

```
nativeQuotaKey = H("slot-chain-native-quota-v2" ||
                   destinationDomainId || "NATIVE_ETH")
destinationQuotaDebit = checked(message.value + message.fee).
```

Both a successful real CALL and invocation-prohibited DONE debit that amount in the same nested terminal journal. Target failure, UNFUNDED, NEW, RETRIABLE, manual failure and expiry debit zero. A missing key, overflow, insufficient quota or quota-call fault rolls back the target call, Pool transfer/ticket debit, Bridge status/pulls and terminal append together. Refund, source recall, withdrawal and terminal proof paths never read or debit quota. The fixed corpus covers value zero/fee positive, value positive/fee zero, both-zero rejection, addition overflow, quota exactly equal/one wei short and a post-target quota miss with byte-identical rollback.

On a proved DONE leaf, SourceBridge reclassifies exactly `value+fee+liquidityFee` from pending escrow to the authenticated `l1Recipient`'s LP pull. On FAILED or pre-enqueue cancellation, it reclassifies the same total to the source owner's user pull. Its exact conservation invariant is

```
totalLiveLiability = pendingEscrow + userPullLiability + lpPullLiability
address(sourceBridge).balance >= totalLiveLiability.
```

DONE never turns locked L1 ETH into ownerless surplus. Source liability falls only after the matching pull transfer succeeds. This removes deterministic lifetime-reserve exhaustion while leaving existing V1 native exits untouched. It does not make liquidity free: delivery liveness is explicitly economic and requires at least one LP willing to atomically fund execution from pre-existing L2 ETH.

The destination domain's fresh immutable non-proxy Bridge owns V2 status, retry state, ETH custody and terminal index for the domain's entire lifetime. Its address, runtime and storage never rotate and it never delegates execution. This prevents another implementation or address from seeing an empty status and processing a pinned credit twice, and keeps RETRIABLE liquidity in the same account. The sole V2 admission is `verifyInboxCredit`; there is no caller-supplied nonempty source-proof path on L2. `processMessageV2` is NEW-only. A non-owner invocation failure remains NEW; only a destination-owner failed initial invocation becomes RETRIABLE. `retryMessageV2` is RETRIABLE-only. Invocation-prohibited handling precedes owner checks and permissionlessly creates the destination owner's value pull. For a real invocation, zero gas or `isLastAttempt=true` is owner-only; a non-last failure is side-effect free, while an owner last failure becomes FAILED. Manual failure is owner-only and pausable. Strictly after `processBy`, `expireMessageV2(creditId)` is raw-preimage-free, permissionless and unpausable from NEW or RETRIABLE. DONE, FAILED and RECALLED are terminal.

The V2 fee is a success-only bounty. No NEW, RETRIABLE, FAILED, manual-fail or expiry path pays it. A non-owner successful real invocation receives all fee; owner self-processing also receives all fee. For a real CALL, an insufficient caller-owned ticket returns UNFUNDED before the attempt. The nested attempt then consumes only the exact OPEN Pool authorization; a transfer or raw-balance failure reverts that frame rather than authorizing from a free-balance view. A successful target is followed by exact quota consumption, execution-fee pull-credit creation, terminal append and the hard `balance>=aggregateV2Liability` assertion in the same journal. Any quota, liability-floor, authorization or terminal failure rolls back Pool, target and Bridge, including an owner-last success. Deterministic no-CALL success paths may precheck their exact capacity. Every entry and withdrawal shares one top-level non-reentrant frame. Send/process/retry/manual-fail may be paused; cancellation, expiry, terminal append, DONE finalization, FAILED recall and pull withdrawal are outside every pause and ordinary quota gate. On transition to DONE or FAILED, the immutable destination Bridge makes exactly one call to the protocol-lifetime `TerminalAccumulatorV2`. Every process, retry, fail and expire entry shares one non-reentrant guard, and arbitrary recipient execution finishes before a terminal status is installed. The Bridge then writes DONE or FAILED and `terminalIndex=UINT64_MAX`, calls only `appendTerminalV2(creditId)`, and replaces the sentinel with the returned index before returning. The accumulator accepts no terminal or domain argument. The permanent ABI is:

```
TerminalAccumulatorV2.appendTerminalV2(bytes32 creditId)
    returns (uint64 terminalIndex)
DestinationBridge.terminalCommitmentV2(bytes32 creditId)
    view returns (bytes32 destinationDomainId, address destBridge,
                  uint8 terminal, uint64 terminalIndex,
                  bytes32 liquiditySettlementHash)
TerminalAccumulatorV2.terminalStateV2()
    view returns (uint64 count, bytes32 root)
event TerminalAppended(uint64 indexed index,
  bytes32 indexed destinationDomainId, address indexed destBridge,
  bytes32 creditId, uint8 terminal, bytes32 liquiditySettlementHash,
  bytes32 leaf,
  uint64 count, bytes32 root)
```

All selectors are canonical Solidity selectors of those signatures. Append returndata is exactly 32 bytes with a zero-padded `uint64`; commitment returndata is exactly five ABI words (160 bytes), with canonical high padding, `terminal=1` for DONE or 2 for FAILED and `terminalIndex=UINT64_MAX`. State returndata is exactly two ABI words. Short, trailing, noncanonical, overflow, revert or out-of-gas returndata rejects atomically. The accumulator uses exactly 50,000 gas for the commitment `STATICCALL`; the release gas benchmark must show 30% margin or revise this normative constant and the profile before launch. With that fixed gas stipend and exact return length it `STATICCALL`s the registered caller's `terminalCommitmentV2(creditId)`, requires the sealed domain, caller address, DONE/FAILED status, sentinel and the terminal-specific settlement-hash rule, then appends:

```
leaf = H("slot-chain-terminal-leaf-v2" || u64(terminalIndex) ||
         destinationDomainId || address20(destBridge) ||
         creditId || u8(terminal) || liquiditySettlementHash)
```

Any call or return failure reverts status, sentinel, leaf and count together. During a V1 recipient callback there is no concurrent sentinel, so a message targeting the accumulator cannot forge a leaf even though `msg.sender` is the Bridge. V2 additionally rejects the complete manifest denyset, including the accumulator, before any CALL.

The accumulator stores no per-credit "already appended" set. The exact immutable registered Bridge is the sole writer for its domain, and its permanent terminal status/index plus non-reentrant APPENDING sentinel make a second or concurrent append for one credit unreachable. The append and Bridge transition share one EVM revert domain, so a failed suffix restores frontier/count/root and the Bridge sentinel/status/index together.

The accumulator stores exactly a checked `uint64count`, wrapped root and 64-word frontier. At old index i it starts with the new leaf, combines `frontier[h]` on the left while bit h of i is one, writes the carry only to the first zero-bit frontier slot, increments count, then folds all 64 heights with canonical empty nodes to obtain the tree root and wrapped count/root. Stale frontier values at zero bits are ignored. Append rejects old `count=UINT64_MAX`; the final leaf is unused.

The complete leaf preimage, leaf, new count and root are emitted in `TerminalAppended`. Proofs are reconstructed from canonical L2 receipts by an open-source deterministic prover and at least two independently administered, replaceable log archives. Anyone verifies a supplied 64-sibling proof on L1; archives have no signing key or correctness authority. Loss of every leaf log copy can stop future proof generation and therefore terminal release, just as loss of every canonical state preimage stops recovery. Archive-independent proof generation is not claimed; it would require the rejected unbounded completed-subtree store. Production must benchmark carry depths 0–63 and the complete cold post-call suffix before fixing `V2_POST_CALL_GAS_RESERVE`. The same harness must measure and freeze nonzero profile fields `V2_ATTEMPT_PRE_CALL_GAS_BOUND` and `V2_OUTER_CATCH_GAS_RESERVE`, plus `V2_POOL_BRIDGE_PRE_CALL_GAS_BOUND` and `V2_POOL_CONSUME_CALL_GAS_BOUND`. For positive Message gas, the wrapper requests exactly `V2_ATTEMPT_PRE_CALL_GAS_BOUND+message.gasLimit` for the self-call and first applies the exact EIP-150 sufficiency relation while retaining the catch reserve; the inner target receives exactly `message.gasLimit-V2_POST_CALL_GAS_RESERVE`. For owner-only zero-gas execution, it forwards all gas above the outer reserve and the inner retains the post-call reserve. Production vectors cover first/cold Pool and quota slots, target OOG, custom-error construction/decoding, the second failure-finalizer self-call and outer guard cleanup. Until all five measured values, the fixed Pool read/callback paths and a 30-percent margin are published, the execution profile and document remain non-implementation-ready.

Registration is append-only. Two protocol-lifetime non-proxy contracts are:

```
ProtocolReleaseAuthorityV2
TerminalDomainRegistrarV2
```

Only the registrar may register an accumulator writer. Release authority is bound to the fork-reserved system sender and manifest namespace, not a particular Anchor version or endpoint. Activation passes the value, never merely its hash:

```
struct ComponentDescriptorV2 {
  address component; bytes32 runtimeHash; bytes32 configHash;
}
struct DestinationBridgeDescriptorV2 {
  address bridge; bytes32 runtimeHash; bytes32 configurationHash;
  bytes32 storageLayoutHash;
  bytes32 bridgeKernelProfileHash; address inboxCreditStore;
  address terminalAccumulator; address terminalDomainRegistrar;
  address quotaManager; address nativeLiquidityPool;
```

```
}
struct ReleaseManifestV2 {
  uint64 protocolVersion;
  uint256 settlementChainId; uint256 destinationChainId;
  bytes32 destinationGenesisHash; bytes32 executionProfileHash;
  bytes32 manifestNamespace; bytes32 destinationNamespace;
  address anchorV4; bytes32 anchorRuntimeHash;
  bytes32 destinationDomainId; address destinationBridge;
  bytes32 destinationBridgeExecutionHash;
  DestinationBridgeDescriptorV2 destinationBridgeDescriptor;
  bytes32 destinationInfrastructureHash;
  bytes32 migrationVerifierDescriptorHash;
  bytes32 ingressAuthorizationRoot;
  address nativeLiquidityPool; bytes32 poolRuntimeHash;
  bytes32 poolConfigurationHash;
  ComponentDescriptorV2[10] components;
}
AnchorV4.activateRelease(ReleaseManifestV2 calldata manifest,
                         uint64 retirementQueueCount)
ProtocolReleaseAuthorityV2.activateRelease(
    ReleaseManifestV2 calldata manifest, uint64 retirementQueueCount)
TerminalDomainRegistrarV2.activateDomain(
    ReleaseManifestV2 calldata manifest, uint64 retirementQueueCount)
```

The release manifest is a fully static Solidity tuple: no string, bytes or dynamic array occurs in it. Every address/integer has canonical ABI padding and the ten components have fixed order. Its ABI tuple contains exactly 58 words (1,856 bytes). In the hash preimage, word 0 is `RELEASE_MANIFEST_TYPEHASH`, words 1–12 are the outer fields through `destinationBridgeExecutionHash`, words 13–22 are the ten destination descriptor fields, words 23–28 are the remaining outer fields through `poolConfigurationHash`, and words 29–58 are the ten three-word components. The typehash is Keccak of this exact single ASCII string (shown wrapped only for typesetting):

```
ReleaseManifestV2(uint64 protocolVersion,uint256 settlementChainId,
uint256 destinationChainId,bytes32 destinationGenesisHash,
bytes32 executionProfileHash,bytes32 manifestNamespace,
bytes32 destinationNamespace,address anchorV4,bytes32 anchorRuntimeHash,
bytes32 destinationDomainId,address destinationBridge,
bytes32 destinationBridgeExecutionHash,
DestinationBridgeDescriptorV2 destinationBridgeDescriptor,
bytes32 destinationInfrastructureHash,bytes32 migrationVerifierDescriptorHash,
bytes32 ingressAuthorizationRoot,address nativeLiquidityPool,
bytes32 poolRuntimeHash,bytes32 poolConfigurationHash,
ComponentDescriptorV2[10] components)
ComponentDescriptorV2(address component,bytes32 runtimeHash,bytes32 configHash)
DestinationBridgeDescriptorV2(address bridge,bytes32 runtimeHash,
bytes32 configurationHash,bytes32 storageLayoutHash,
bytes32 bridgeKernelProfileHash,address inboxCreditStore,
address terminalAccumulator,address terminalDomainRegistrar,
```

```
address quotaManager,address nativeLiquidityPool)
```

The byte string contains no whitespace or newline between the three displayed type declarations. Dependencies are in ascending type-name order. Its commitment is the type-hashed `keccak256(abi.encode(...))` below. The function selector is Keccak's first four bytes over the canonical expanded ABI signature

```
activateRelease((uint64,uint256,uint256,bytes32,bytes32,bytes32,bytes32,
address,bytes32,bytes32,address,bytes32,
(address,bytes32,bytes32,bytes32,bytes32,address,address,address,address,address),
bytes32,bytes32,bytes32,address,bytes32,bytes32,
(address,bytes32,bytes32)[10]),uint64)
```

again with the displayed newlines removed. Tx0 calldata is that four-byte selector followed by the 58 manifest words and one canonical padded `uint64retirementQueueCount` word: exactly 1,892 bytes. Appendix golden vectors pin every boundary word and the final selector/hash. The dependency order is profile and primitive descriptors, Bridge bundle init/config, addresses, release manifest, then registration commitment; neither Bridge init/config nor the profile may depend on the manifest or derived domain. It is

```
releaseManifestHash = H(abi.encode(RELEASE_MANIFEST_TYPEHASH,
                                   canonicalReleaseManifestV2))
```

Zero values, duplicate component addresses, a Bridge different from component 10, a component count/order mismatch, or any named/hash field inconsistent with the Appendix grammars rejects.

The first AnchorV4 custom system transaction calls exactly `activateRelease(manifest, retirementQueueCount)`. Its canonical uint64 word must equal the migration statement's authenticated L1 ForcedQueue count; there is no optional/default watermark or Inbox-cursor substitute. The raw tx0 bytes contain the fixed manifest words immediately followed by that count and contain no MPT proof, statement hash, candidate digest or tx0 hash. This avoids a hash cycle. The validity circuit authenticates the already published L1 release manifest and exact L1 origin as separate statement inputs, requires this tx0 at index zero, and constrains its raw EIP-2718 hash and decoded values. Anchor recomputes the manifest commitment, stores the active release once, calls the release authority and registrar with the identical value/count, and rolls all three back on any failure.

The authority recomputes the commitment, requires `msg.sender` and its `EXTCODEHASH` to equal the manifest Anchor, requires `tx.origin` to be the reserved Anchor-system sender, and requires Anchor's one-shot active manifest/count to match. It permanently records the release and exact retirement watermark. A direct call by that reserved sender, an EOA, Bridge or an Anchor from another manifest rejects. Before either side hashes or stores the manifest, L1 requires `settlementChainId=block.chainid<=UINT64_MAX`; L2 independently requires `destinationChainId=block.chainid<=UINT64_MAX`. The registrar constructor fixes only the protocol-lifetime release authority, InboxApplyRouterV2, TerminalAccumulatorV2 and NativeLiquidityPoolV2. Its non-reentrant activation recomputes the same manifest hash, requires the authority's exact stored version/watermark, and compares every protocol-lifetime row plus the fresh release-owned Store, QuotaManager and DestinationBridge against the manifest. Components on L1 were already checked by ProtocolVersionManager; the L2 registrar never pretends to inspect cross-chain code.

Every destination release is fresh-only. Tx0 authenticates exact non-proxy runtime/configuration/layout for Store, QuotaManager and Bridge; arbitrary pre-activation Bridge surplus; zero nonce, liability and private status/pull/ terminal mappings; full initial nonzero V2-only quota; and `v2Active=false`. It authenticates the lifetime Pool code/configuration and its unchanged aggregate solvency, then one-shot registers the fresh domain/Bridge/Store as an append-only atomic-funding authority, installs the inbox route and accumulator writer, and seals Store plus `v2Active=true` in one rollback domain. Tx0 moves no native value, creates no LP ticket and mints nothing. There is no activation-gate account, legacy proxy branch, exact-reuse branch, owner, implementation slot or delegate target. A historical domain or Bridge can never be reactivated even if code is identical. Any partial state, duplicate identity, Pool insolvency, conflicting route or late failure reverts tx0 entirely.

InboxApplyRouterV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2, TerminalAccumulatorV2 and NativeLiquidityPoolV2 are protocol-lifetime objects repeated byte-exactly by every successor manifest. Their cursor, old routes/releases/registrations, Pool tickets and accumulator frontier/root/count remain intact; only the absent release, route, Registrar funding route and one-to-one writer are appended. The Pool itself stores no domain row. The registrar stores the single Appendix-defined `registrationCommitment[protocolVersion]` only after the full seal. The accumulator has no public registrar or root setter and calls only the registered Bridge's bounded terminal getter during append. Status, sentinel, frontier/root/count update and terminal index are atomic. The application-level leaf sequence, not a versioned EVM storage proof, remains the source release statement across later Settlement profiles.

Release rotation and old-endpoint reclamation are bounded and explicit. Successful tx0 replaces the registrar's one `activeDestinationBridge` with the fresh Bridge and records for the predecessor `retirementQueueWatermark[oldBridge]=retirementQueueCount`; neither may be rewritten. The count is the migration statement's exact L1 Queue count. Thus the retired adapter cannot append after the watermark, while `InboxApplyRouterV2.nextQueueIndex>=watermark` proves that every earlier queued predecessor descriptor has been applied.

The same L2 tx0 journal appends a local successor receipt; an L2 Bridge never reads the L1 Router receipt. Its ID and permanent views are:

```
destinationReceiptId = H("slot-chain-destination-activation-receipt-v2" ||
  u256(destinationChainId) || address20(terminalDomainRegistrarV2) ||
  u64(successorIndex) || u64(oldProtocolVersion) ||
  u64(newProtocolVersion) || oldManifestHash || newManifestHash ||
  oldDestinationDomainId || newDestinationDomainId ||
  address20(oldDestinationBridge) || address20(newDestinationBridge) ||
  u64(retirementQueueCount) || u64(activatedAtBlock))
destinationActivationReceiptV2(bytes32 destinationReceiptId) view returns
  (bytes4 magic, bytes32 returnedReceiptId, uint64 successorIndex,
   uint64 oldProtocolVersion, uint64 newProtocolVersion,
   bytes32 oldManifestHash, bytes32 newManifestHash,
   bytes32 oldDestinationDomainId, bytes32 newDestinationDomainId,
   address oldDestinationBridge, address newDestinationBridge,
   uint64 retirementQueueCount, uint64 activatedAtBlock, uint8 sealed)
destinationSuccessorReceiptV2(bytes32 oldDestinationDomainId,
                              address oldDestinationBridge) view returns
  (bytes32 destinationReceiptId, uint64 successorIndex, bytes4 magic)
```

The full return is exactly 448 bytes with magic `0x44525632` (`DRV2`), `sealed=1` and canonical padding; the index return is 96 bytes with magic `0x44535632` (`DSV2`). Registrar derives all fields from its current active release and the proof-bound identical manifest/count, increments a checked `uint64successorIndex`, and writes the immutable receipt, predecessor index, new active tuple and all route/Pool/writer seals in one rollback domain. Genesis uses canonical-zero predecessor fields and writes no predecessor index. A successor requires nonzero old fields, distinct new domain/Bridge, exact stored old manifest, absent predecessor index and exact watermark; partial or duplicate sealing rejects.

Each release-owned `InboxCreditStoreV2` maintains a checked `uint64pinnedCount`, incremented only when its authorized InboxApply route writes an absent credit pin. The accumulator maintains a checked per-domain `uint64terminalizedPinnedCount`. It increments only in the same journal as a terminal append when the caller is that registrar-bound domain Bridge, the route and Store identities match, the pin exists, status is DONE or FAILED, and the `(domainId,creditId)` terminal guard was previously absent. A duplicate, unpinned credit, foreign Store/Bridge or failed append changes no counter, leaf, guard or status.

Anyone may call the old Bridge's bounded `reclaimSurplus()` only after a distinct successor has the Registrar-local sealed destination activation receipt above, InboxApply has reached the immutable watermark, `terminalizedPinnedCount[oldDomain]=oldStore.pinnedCount`, and the old Bridge's aggregate pull liability is zero. The call reauthenticates the old Bridge's aggregate execution-fee pull liability is zero. Atomic funding leaves no Pool state attached to a domain across transactions; unrelated ticket funds remain withdrawable by their LP. The call bounded-static-reads both Registrar views, checks exact code/config/ length/magic and old/new version/manifest/domain/Bridge/watermark/seal, then reauthenticates the old manifest and complete lifetime Router/authority/registrar/accumulator/Pool graph, requires the old route and domain writer unchanged, and transfers the entire remaining raw Bridge surplus to the profile's typed NATIVE_ETH `bridgeSurplus` sink; it never enumerates a mapping or consumes a Pool ticket. Its result is

```
enum ReclaimResult { REJECTED, RECLAIMED_ZERO, RECLAIMED_VALUE }
returns (ReclaimResult result, uint256 amount)
```

so sink rejection cannot masquerade as a valid zero-balance reclaim. Only after an accepted full transfer does it set the one-shot `retired` bit; partial transfer, callback/reentrancy, later processing, double reclaim and any state mismatch revert or return REJECTED without mutation. Historical status, terminal index/events and proof verification remain readable forever. Historical SourceBridges remain callable until every USER_PULL and LP_PULL is withdrawn; a DONE reimbursement is never reclaimable as surplus. Repeated release rotation changes no ticket owner, source liability or terminal proof.

Every execution proof additionally authenticates the accumulator account and proves that the candidate's `terminalRoot` and `terminalCount` equal its exact post-state root/count slots. These fields are part of `CanonicalCore` and `executionOutputsCommitment`; tier-2 and tier-3 use the same public-output constraint. A caller cannot supply an unconstrained root; no checkpoint publisher exists in this revision. The same proof authenticates the protocol-lifetime InboxApplyRouterV2 account and, for each block, requires its pre-state `nextQueueIndex` to equal that block's `messageStart` and its post-state value to equal `messageEnd`. Before proof acceptance, the L1 base invariant is `baseCanonical.messageCursor=ForcedQueue.cursor`, while the proof's first L2 pre-state cursor equals the same start. The final proved L2 post-state cursor and new `CanonicalCore.messageCursor` both equal the candidate end. After proof verification and before its history write, the active Settlement calls `ForcedQueue.advanceCursor(start,end,proofBeneficiary)`. That call moves the exact cumulative processing-fee interval to the beneficiary's pull claim. A failed call reverts the entire canonical commit. L1 never writes the L2 router; the end value is authenticated L2 poststate adopted by the proof. Launch bootstrap proves all three base values equal in the imported L2 state. A later migration occurs only at a canonical boundary, checks the two L1 values directly, and inherits L2 equality inductively from the last accepted proof; the target profile must preserve that same cursor slot and transition. No activation transaction may repair, overwrite or select among divergent cursors.

The vector root is

```
H("slot-chain-terminal-root-v2" || u64(terminalCount) || treeRoot)
```

where the Appendix fixes empty nodes, node hashing and bit order.

Every versioned Settlement is a permanent non-proxy contract with no `SELFDESTRUCT`, delegate target, history pause or code upgrade. Its internally written history cell contains the exact sequence plus the full `CanonicalCore` and `canonicalizedAtBlock`; a read requires exact tag equality. The protocol-lifetime non-proxy `ActiveSettlementRouter` keeps an append-only mapping from protocol version to exact Settlement address, `EXTCODEHASH` and execution-profile hash. Historical entries cannot be deleted, redirected or reactivated. One constructor-bound, tagged `LEGACY_BOOTSTRAP` descriptor is the sole exception: it binds the deployed legacy Inbox proxy, expected proxy/runtime/configuration identity and the `legacyDeploymentHash`, but is never inserted into the version mapping or canonical history. While `genesisConsumed=false`, the active-state getter returns that immutable proxy as the source. A failed or orphaned genesis activation preserves this branch; successful GENESIS_IMPORT sets `genesisConsumed=true` in the same atomic switch, after which the branch is permanently tombstoned and cannot be reused. Registration, `canonicalAt` and every later migration reject a legacy-tagged row. Its only switch is:

```
struct CanonicalCoreV2 {
  uint48 l2BlockNumber; bytes32 tipHash; uint64 tipSlot; bytes32 stateRoot;
  uint64 messageCursor; bytes32 winningDataCommitment; uint256 nextBaseFee;
  uint64 nextExcessBlobGas; bytes32 terminalRoot; uint64 terminalCount;
}
struct MigrationActivationFixedV2 {
  uint8 transitionKind; uint64 migrationGeneration;
  uint64 sourceProtocolVersion; uint64 targetProtocolVersion;
  uint64 sourceCanonicalSequence; bytes32 candidateDigest;
  CanonicalCoreV2 outputCore; address proofBeneficiary;
  uint256 anchorNumber; bytes32 anchorHash; uint64 forceCutoff;
  uint64 preInboxLastAppliedPlusOne;
}
activateVersionWithMigration(
  MigrationActivationFixedV2 calldata fixedInput,
  ReleaseManifestV2 calldata manifest,
  InboxRowV2[] calldata rows,
  bytes calldata importedHeaderRlp,
  bytes calldata proof)
```

The selector is the first four Keccak bytes of this exact canonical expanded signature, with no whitespace:

```
activateVersionWithMigration((uint8,uint64,uint64,uint64,uint64,bytes32,
(uint48,bytes32,uint64,bytes32,uint64,bytes32,uint256,uint64,bytes32,uint64),
address,uint256,bytes32,uint64,uint64),
(uint64,uint256,uint256,bytes32,bytes32,bytes32,bytes32,address,bytes32,
bytes32,address,bytes32,
(address,bytes32,bytes32,bytes32,bytes32,address,address,address,address,address),
bytes32,bytes32,bytes32,address,bytes32,bytes32,
(address,bytes32,bytes32)[10]),
(uint64,uint8,uint32,bytes32,bytes)[],bytes,bytes)
```

The first two tuples are static and occupy 21 and 58 words. Therefore the top-level ABI head is exactly 82 words: words 0–20 are `fixedInput`, 21–78 are `manifest`, and words 79–81 are the rows, imported-header and proof offsets. The rows offset is exactly `0x0a40`. The other offsets are the immediately following canonical tails: rows encode as one length word, n element offsets and n contiguous five-word element heads plus their bytes tails; then the header and proof each encode one length word, bytes and zero right-padding. Aliases, gaps, overlap, non-minimal offsets, nonzero padding or trailing calldata reject. There are at most 64 rows; kind-0 descriptor bytes are empty and kind-1 bytes are exactly 541. GENESIS_IMPORT requires a nonempty canonical RLP header of at most 2,048 bytes; VERSION_MIGRATION requires zero header bytes. Proof length is nonzero, no greater than the descriptor's `maximumProofBytes`, and that bound is at most 131,072 bytes. Later VERSION_MIGRATION activation is permissionless only while the Router's exact public migration gate is READY. GENESIS_IMPORT instead uses one separately delayed, finite campaign while that gate remains ACTIVE on the constructor-bound legacy branch. Its exact target and review commitment are final before the admission-fence cutoff; during the inclusive campaign window any caller may land the proof-carrying atomic cutover. No caller, including governance, has a privileged landing path or post-cutoff approval. A separate Router execution lifecycle is the global reentrancy journal:

```
RouterLifecycle = IDLE=0 | ARMING=1 | READYING=2 |
                  ABORTING=3 | ACTIVATING=4 | REGISTERING=5 |
                  PUBLISHING=6
```

Every Router and ProtocolVersionManager mutator begins only in IDLE. The later-migration arm, readiness and abort set their distinct lifecycle before the first external interaction. GENESIS_IMPORT enters ACTIVATING before its first external read or bounded STATICCALL; a failing proof or callback reverts that write with the whole transaction. Every successful control-plane call restores IDLE only as its final write. During ACTIVATING, and only then, the Router stores a nonzero `activeActivationContextHash`; outside that lifecycle every activation-only context field is canonical zero. The public ACTIVE/ARMED/READY gate does not change during a callback. Genesis moves directly from legacy ACTIVE to target ACTIVE inside the same transaction; local READY exists only between LGAR and LGFN in that transaction and is never a canonical stopping state. This lifecycle is not a governance pause and has no public setter.

8.1 Immutable delayed protocol authority

No proxy, owner-controlled executor or generic-call timelock is a protocol authority. The release uses exactly one non-proxy, ownerless `ProtocolChangeTimelockV1` and one non-proxy, ownerless `ProtocolVersionManagerV2`. Their runtime hashes and constructor-fixed configuration are release/profile fields. Neither runtime contains `DELEGATECALL`, an upgrade path, a target setter, an arbitrary `call(address,bytes)` path or a self-call that can reach one. The timelock's only proposer is the immutable DAO proposer; execution is permissionless. This deliberately retains governance as the validity/release safety root, but makes every protocol change publicly visible for at least `PROTOCOL_CHANGE_DELAY_SECONDS=604800` seconds and removes every bypass around that delay.

The timelock accepts only operation kinds

```
REGISTER_RELEASE=1 | REGISTER_FORK_VERIFIER=2 |
PUBLISH_GENESIS_CAMPAIGN=3 | PUBLISH_MIGRATION_ARM=4
```

and rejects every other value. Its exact identity is

```
protocolChangeOperationId = H(
 "slot-chain-protocol-change-operation-v1" ||
 u256(settlementChainId) || address20(protocolChangeTimelock) ||
 address20(protocolVersionManager) || u64(operationNonce) ||
 u8(operationKind) || u32(payloadBytes) || H(payload))
```

where `payloadBytes` is nonzero and at most 131,072. The sole proposer calls `queueProtocolChangeV1(uint8, bytes)`; the timelock assigns the next checked nonzero `uint64` nonce, records the exact hash/length and `executeAfter=checked(uint64(block.timestamp)+604800)`, and emits the complete payload. Any caller later supplies the same nonce, kind and bytes to `executeProtocolChangeV1(uint64,uint8,bytes)`. Execution requires `block.timestamp>=executeAfter`, moves QUEUED to EXECUTING before the only external call, invokes only the immutable manager's `applyProtocolChangeV1(uint64,uint8,bytes)`, requires the sole exact `0x50415031` (PAP1) return word, and writes EXECUTED last. Failure reverts the whole operation to QUEUED. Only the DAO proposer may call `cancelProtocolChangeV1(uint64, uint8,bytes)`, and only while QUEUED; it authenticates the complete preimage and writes CANCELED without an external call. NONE=0, QUEUED=1, EXECUTING=2, CANCELED=3 and EXECUTED=4 are the only states. CANCELED and EXECUTED retain the tuple forever, nonces are never reused, and the EXECUTING guard makes same-block execute/reenter/cancel order atomic.

The four mutating ABIs are exact:

```
queueProtocolChangeV1(uint8 kind,bytes payload)
 returns (bytes4 magic,bytes32 operationId,uint64 nonce,uint64 executeAfter)
cancelProtocolChangeV1(uint64 nonce,uint8 kind,bytes payload)
 returns (bytes4 magic)
executeProtocolChangeV1(uint64 nonce,uint8 kind,bytes payload)
 returns (bytes4 magic,bytes32 operationId)
applyProtocolChangeV1(uint64 nonce,uint8 kind,bytes payload)
 returns (bytes4 magic)
```

Their selectors are `0xbd5c80a8`, `0x5701c308`, `0x31d81ea2` and `0xaf3927f6`; return lengths/magics are respectively 128/PCQ1 `0x50435131`, 32/PCC1 `0x50434331`, 64/PCE1 `0x50434531` and 32/PAP1 `0x50415031`. Queue calldata has a two-word head and exact bytes offset `0x40`; the other three have a three-word head and exact offset `0x60`. Each has one length word, contiguous bytes, zero right-padding and no suffix. Narrow arguments use canonical padding. Apply is Timelock-only; the other three have the caller rules above.

The payload is canonical ABI for exactly one fixed schema:

1. REGISTER_RELEASE is exactly `abi.encode(uint64expectedPredecessorProtocolVersion, ReleaseManifestV2,SettlementDeploymentDescriptorV1,bytesprofileBytes)`. Its head is the canonically padded expected-predecessor word, 58 manifest words, then the eight descriptor words (`addressfactory,bytes32factoryRuntimeHash,bytes32factoryConfiguration` `bytes32salt,bytes32initCodeHash,addresstargetSettlement,bytes32targetRuntimeHash, bytes32targetConfigurationHash`) followed by the sole dynamic offset, exactly `0x0880`. Its tail is one length word, 1–65,536 canonical profile bytes and zero right-padding, making total payload length `2208+ceil32(profileBytes.length)`. Alias, gap, non-minimal offset, nonzero padding or trailing bytes reject. The manager strictly decodes and byte-for-byte re-encodes the strict canonical ABI profile grammar below, requires `executionProfileHash=H("slot-chain-execution-profile-v2"||u32(profileBytes.length)||profileBytes)`, and checks every manifest profile-derived field, ingress row, verifier, component and Market read tuple against that decoded profile. It checks CREATE2 derivation, the live root factory identity and target code/configuration, requires the expected predecessor to equal Router's current active protocol version and `0<=expectedPredecessorProtocolVersion<protocolVersion` (zero is the GENESIS_IMPORT sentinel), recomputes the descriptor's 232-byte packed hash, then derives

   ```
   targetRegistrationHash = H(
    "slot-chain-target-registration-v2" || u64(protocolVersion) ||
    address20(targetSettlement) || targetRuntimeHash ||
    targetConfigurationHash || settlementDeploymentDescriptorHash ||
    executionProfileHash || migrationActivationProfileRecordHash ||
    releaseManifestHash ||
    u64(expectedPredecessorProtocolVersion) || u64(settlementChainId) ||
    ingressAuthorizationRoot || migrationVerifierDescriptorHash)
   ```

   from the same manifest/profile and current Router predecessor. It may only append that exact version/deployment/registration, its profile-derived ingress rows and its Market authorization to the named immutable registries. A different predecessor, target, manifest or registration requires a new delayed operation and cannot reuse this record.

2. REGISTER_FORK_VERIFIER is `abi.encode(bytes4forkDigest,uint64firstWindow, addressverifier,bytes32runtimeHash,uint64beaconSlotGindex,uint64executionPayloadGindex, uint64stateRootGindex,uint64prevRandaoGindex,uint64timestampGindex,uint64blockHashGindex bytes32witnessSchemaHash,bytes32configurationHash,bytes4selector,uint64gasLimit)`. These are exactly 14 static words. Every field is nonzero, the selector is exactly `0x7e981e0b`, the window is strictly future, and `configurationHash` is recomputed from the displayed constants under the ScheduleOracle grammar below. The runtime/configuration getter is checked, and `forkDigest` occupies the high four word bytes with 28 zero trailing bytes; it is the one-shot ScheduleOracle registry key used by `sealWindow`.

3. PUBLISH_GENESIS_CAMPAIGN is `abi.encode(uint64forceCutoffBlock,uint64proposalCutoffBloc`

uint64quiesceNotBeforeBlock,uint64resumeByBlock,uint64resumeByTimestamp, uint64reviewFinalizedByBlock,addresstargetSettlement,uint64targetProtocolVersion, bytes32targetManifestHash,bytes32targetRegistrationHash). Nonce, generation, review commitment and campaign ID are derived, never supplied.

4. PUBLISH_MIGRATION_ARM is abi.encode(uint64expectedSourceProtocolVersion, uint64targetProtocolVersion,bytes32targetManifestHash,bytes32targetRegistrationHash). Generation, arm time and hard abort time are derived.

Canonical ABI means the displayed number of 32-byte words, zero high padding for every unsigned/address value, fixed bytes left-aligned with zero trailing padding, and no suffix. The manager decodes by kind, re-encodes byte-for-byte, exact-reads the EXECUTING timelock row, records the operation ID consumed before its first downstream call, and dispatches only the kind's named contracts and selectors. It has no raw target or calldata branch. Every manager mutation requires its private lifecycle IDLE. Applying an operation writes APPLYING and the consumed ID before the first external call; lease expiry writes ABORTING before its first Router call; each path restores IDLE only as its final write, and any failure reverts the entire frame. A downstream callback therefore cannot execute another operation, register another release or interleave a lease abort. REGISTER_RELEASE appends Router registration, profile-derived registry rows and Market authorization in this one Timelock–PVM rollback domain and exact-checks every fixed return before PAP1; partial publication is impossible. REGISTER_RELEASE is also the only way to install AggregatorSeatMarket's append-only Settlement authorization: the manager derives it solely from the same manifest and target registration checked by Router. There is no separate Release Manager.

REGISTER_RELEASE uses this one fixed cross-contract journal:

```
ActiveSettlementRouter.registerTargetReleaseV2(
 uint64 expectedPredecessorProtocolVersion,
 ReleaseManifestV2 manifest,
 SettlementDeploymentDescriptorV1 deployment,
 bytes profileBytes)
 returns (bytes4 magic,bytes32 manifestHash,bytes32 targetRegistrationHash)
ActiveSettlementRouter.targetReleaseRegistrationV2(uint64 protocolVersion)
 returns (bytes4 magic,uint64 returnedProtocolVersion,
          uint64 expectedPredecessorProtocolVersion,address targetSettlement,
          bytes32 targetRuntimeHash,bytes32 targetConfigurationHash,
          bytes32 settlementDeploymentDescriptorHash,
          bytes32 executionProfileHash,
          bytes32 migrationActivationProfileRecordHash,
          bytes32 dataSessionConfigurationHash,
          bytes32 releaseManifestHash,
          bytes32 targetRegistrationHash)
ActiveSettlementRouter.migrationActivationProfileV2(uint64 protocolVersion)
 returns (bytes4 magic,uint64 returnedProtocolVersion,
          bytes32 executionProfileHash,bytes32 activationProfileRecordHash,
          address verifier,bytes32 verifierRuntimeHash,
          bytes32 verifierConfigurationHash,bytes32 verifyingKeyHash,
          bytes32 proofSystemId,bytes32 publicInputSchemaHash,
          bytes4 verifierSelector,uint32 maximumProofBytes,
```

```
          uint64 verificationGasLimit,uint64 supportedL1BlockGasLimit,
          uint64 worstCaseActivationAdoptionGas,uint64 sourceFreezeGasLimit,
          uint64 targetAdoptionGasLimit,uint64 queueMigrationGasLimit,
          uint64 activationContextReadGasLimit,uint64 postStateReadGasLimit,
          uint64 legacyStateReadGasLimit,uint64 legacyArmGasLimit,
          uint64 legacyFinalizeGasLimit,uint64 postCallbackReserveGas)
ProtocolVersionManagerV2.profileIngressRootV2(uint64 protocolVersion)
 returns (bytes4 magic,uint64 returnedProtocolVersion,
          uint16 count,bytes32 ingressAuthorizationRoot)
ProtocolVersionManagerV2.profileIngressAuthorizationV2(bytes32 authorizationId)
 returns (bytes4 magic,bytes32 returnedAuthorizationId,
          ProfileIngressAuthorizationV2 row)
```

The selectors are respectively `0x9aa71eff`, `0xf588fec3`, `0xc65ff64e`, `0x2d2bbe23` and `0x2181b974`. Router registration calldata is the four-byte selector followed by the exact REGISTER_RELEASE payload; its selector is the Keccak selector of the fully expanded 58-word manifest tuple printed in the activation ABI and the eight-word deployment tuple above. Returns are 96/RTR2, 384/RTR2, 768/MPR2, 128/PIR2 and 800/PIA2 bytes; magics are `0x52545232`, `0x4d505232`, `0x50495232` and `0x50494132`. The row return is magic, returned ID and the 23 static struct words in the published field order. Unknown version/ID, short/trailing data or noncanonical padding rejects rather than returning a zero row.

The fixed MPR2 words are decoded from canonical profile bytes and committed as

```
migrationActivationProfileRecordHash = H(
 "slot-chain-migration-activation-profile-v2" || u64(protocolVersion) ||
 executionProfileHash || address20(verifier) || verifierRuntimeHash ||
 verifierConfigurationHash || verifyingKeyHash || proofSystemId ||
 publicInputSchemaHash || bytes4(verifierSelector) ||
 u32(maximumProofBytes) || u64(verificationGasLimit) ||
 u64(supportedL1BlockGasLimit) || u64(worstCaseActivationAdoptionGas) ||
 u64(sourceFreezeGasLimit) || u64(targetAdoptionGasLimit) ||
 u64(queueMigrationGasLimit) || u64(activationContextReadGasLimit) ||
 u64(postStateReadGasLimit) || u64(legacyStateReadGasLimit) ||
 u64(legacyArmGasLimit) || u64(legacyFinalizeGasLimit) ||
 u64(postCallbackReserveGas))
```

Router stores that immutable record with the registration; PVM verifies its verifier descriptor hash against the manifest. Genesis and later activation recover every verifier/gas word only from MPR2, recheck its record/profile/ registration hashes and target code/configuration, and reject a caller-supplied profile byte, descriptor or gas value. The transient full ABI preimage is therefore needed only to validate and register the compact activation record and ingress rows; activation never attempts to invert `executionProfileHash`.

Before any protocol-change write, PVM checks EXTCODEHASH of the actual Timelock caller and makes a 100,000-gas exact PCT1 STATICCALL back to that caller. It recomputes the governance-delay descriptor from the actual chain, caller address/code, returned DAO/PVM/delay/domain and requires equality to its immutable descriptor. REGISTER_RELEASE additionally requires profile words 16–19 to equal those live observations. A Timelock with a substituted DAO or caller-dependent PCT1 response therefore cannot use an otherwise authentic queued payload.

PVM then derives all expected values from the raw strict profile and joins the complete control tuple to the actual root deployment. It checks its own EXTCODEHASH against word 21 and exact-reads its own 1,056-byte PVM1 view; Router independently checks `msg.sender` EXTCODEHASH and makes the same 100,000-gas, four-byte PVM1 STATICCALL before its first write. Each caller strictly decodes PVM1, recomputes the complete PVM configuration hash against word 22, checks the six non-Market control accounts in this order: Router, ForcedQueue, BuilderRegistry, ScheduleOracle, BridgeDomainRegistry and BridgeCreditRegistry; it then checks Market EXTCODEHASH and its fixed `componentConfigHashV2()` against the PVM1 root and independently recomputes the Market configuration from the actual five constructor values. It next checks the protocol-root ERC-2470 address and EXTCODEHASH; ERC-2470 has no protocol-specific configuration getter. Finally it checks target EXTCODEHASH, `componentConfigHashV2()` (selector `0xf6c0f7d2`), DSV1 and TCS2 in that order. PVM's postread repeats the five target observations. Short/trailing, bad padding, revert/OOG or caller-dependent PCT1/PVM1/control getter behavior rejects in the common outer revert domain. This exact order is part of the cold-state gas certificate and MUST NOT be reordered. The ten destination component rows remain profile/manifest commitments at this L1 step; their live identities are proved by the L2 migration/activation gate, never by impossible cross-chain STATICCALLs. After complete validation but before any registration write, PVM writes APPLYING/consumed. It then calls Router registration with exactly `releaseRouterRegistrationGas`. The historical model value is not a release authority: the production PVM configuration instead contains nonzero immutable `releaseRouterRegistrationGas`, `releaseMarketInstallationGas`, `releasePostreadGas` and `releasePostCallbackReserveGas`. Each is bound by the PVM configuration hash and the release gas certificate; the 3,000,000/500,000/100,000 figures in model fixtures are lower-bound examples only. Before each nested call PVM checks EIP-150-forwardable gas plus all remaining stipends, bounded copy cost and the post-callback reserve. The maximum 65,536-byte cold-state registration benchmark must succeed at the published values and fail at one less. Router requires the manager caller in PVM APPLYING while Router is IDLE, independently re-derives expected values from the same raw profile and repeats the PVM1, control, Market, factory and target live reads as its own caller before writing REGISTERING. It then appends the version row and restores Router IDLE last. Caller-dependent getter behavior, short/trailing returns, revert/OOG or any late mismatch reverts the common outer journal. Because these are STATICCALLs, the target is nonproxy/selfdestruct-free and Ethereum code cannot change during the transaction; after Router returns PVM nevertheless repeats target EXTCODEHASH/config/DSV1/TCS2 and exact-checks all Router stores, closing mutable return substitution rather than relying on a second read by the same caller. PVM exact-checks RTR2/hash pair, stores the sorted ingress rows and root internally, calls Market installation with exactly `releaseMarketInstallationGas`, and then makes `releasePostreadGas`-bounded exact STATICCALL postreads of the Router registration, MPR2 activation profile, PIR2 root, every PIA2 row and Market SAT1 row. Market installation itself direct-reads the same Router registration with exactly 100,000 gas before its write. The call target is the Market constructor's immutable `activeSettlementRouter`; it is never an install argument or a caller-supplied expected row. Only after every postread agrees does PVM return PAP1; Timelock then writes EXECUTED. This order gives Market an already existing authenticated Router row, while the outer revert domain removes every Router/PVM/Market write if any later call or decode fails. A registration may be armed only when its stored predecessor still equals Router's active version; registration does not itself close admissions.

The exact authority descriptors are

```
marketAuthorityConfigurationHash = H(
 "slot-chain-pvm-derived-market-authority-config-v1" || u16(124) ||
 u256(marketChainId) || address20(aggregatorSeatMarket) ||
 u256(settlementChainId) || address20(protocolVersionManager) ||
 address20(activeSettlementRouter))
governanceDelayAuthorityDescriptorHash = H(
 "slot-chain-protocol-change-timelock-v1" ||
 u256(settlementChainId) || address20(protocolChangeTimelock) ||
 protocolChangeTimelockRuntimeHash || address20(daoProposer) ||
 address20(protocolVersionManager) || u64(604800) ||
 H("slot-chain-protocol-change-operation-v1"))
protocolVersionManagerConfigurationHash = H(
 "slot-chain-protocol-version-manager-config-v1" ||
 u256(settlementChainId) || address20(protocolChangeTimelock) ||
 governanceDelayAuthorityDescriptorHash ||
 address20(activeSettlementRouter) || activeSettlementRouterRuntimeHash ||
 activeSettlementRouterConfigurationHash ||
 address20(forcedQueue) || forcedQueueRuntimeHash ||
 forcedQueueConfigurationHash ||
 address20(builderRegistry) || builderRegistryRuntimeHash ||
 builderRegistryConfigurationHash ||
 address20(scheduleOracle) || scheduleOracleRuntimeHash ||
 scheduleOracleConfigurationHash ||
 address20(aggregatorSeatMarket) || aggregatorSeatMarketRuntimeHash ||
 marketAuthorityConfigurationHash ||
 address20(bridgeDomainRegistry) || bridgeDomainRegistryRuntimeHash ||
 bridgeDomainRegistryConfigurationHash ||
 address20(bridgeCreditRegistry) || bridgeCreditRegistryRuntimeHash ||
 bridgeCreditRegistryConfigurationHash || manifestNamespace ||
 u64(604800) || u64(604800) || u16(64) ||
 u64(releaseRouterRegistrationGas) ||
 u64(releaseMarketInstallationGas) || u64(releasePostreadGas) ||
 u64(releasePostCallbackReserveGas))
protocolVersionManagerDescriptorHash = H(
 "slot-chain-protocol-version-manager-v1" ||
 address20(protocolVersionManager) || protocolVersionManagerRuntimeHash ||
 protocolVersionManagerConfigurationHash)
```

For this protocol root `marketChainId=settlementChainId`. Market constructor immutables are exactly those five preimage values, its fixed `componentConfigHashV2()` return is `marketAuthorityConfigurationHash`, and profile word 37 MUST equal that hash recomputed from profile words 2, 35, 20 and 23. All three addresses are distinct. PVM and Router additionally compare profile words 20/23/26/29/32/35/38/41 to the actual PVM identity and complete PVM1 root. PVM1 binds the runtime/configuration pairs in words 24–25, 27–28, 30–31, 33–34, 36–37, 39–40 and 42–43. Each caller independently checks EXTCODEHASH and the fixed-gas live configuration getter for Router, ForcedQueue, BuilderRegistry, ScheduleOracle, Market, BridgeDomainRegistry and BridgeCreditRegistry. The Market configuration is also independently recomputed from the actual five constructor values. Thus a publisher cannot make a self-consistent alternate control graph. Substituting any lookalike control component makes registration fail before a Router, PVM or Market write. Words 18 and 37 are fully recomputable from other profile words and the strict profile decoder rejects either mismatch. Word 22 is deliberately checked at the live-root join, not by the context-free ABI decoder: its preimage includes the four measured protocol-root release gas caps, which are immutable PVM1 values but are not duplicated as per-release profile fields. PVM and Router therefore exact-read PVM1 and recompute word 22 before their respective first writes; a nonzero publisher value alone never authorizes registration. The final seven configuration values are minimum governance delay, maximum live VERSION_MIGRATION duration, review-finality blocks, `releaseRouterRegistrationGas`, `releaseMarketInstallationGas`, `releasePostreadGas` and `releasePostCallbackReserveGas`. The last four are protocol-root immutable MAX_PROFILE caps measured before the PVM is deployed against the 65,536-byte/252-field cold worst case, not per-release mutable values or guessed constants. Every later release certificate must measure at or below these caps; exceeding one requires the independent new-root migration described above. The executable model's 15,000,000/1,000,000/500,000/2,000,000 values are fixtures only and MUST NOT populate a production deployment without the cold max-profile benchmark. All addresses, runtime hashes, namespace and descriptor hashes are nonzero; the two contracts, DAO proposer and every downstream component are pairwise distinct where their roles differ.

Configuration reads use exactly 100,000 gas and these canonical ABIs:

```
protocolChangeTimelockConfigV1() // selector 0xb80095ca; 160 bytes
 returns (bytes4 magic,address daoProposer,address protocolVersionManager,
          uint64 minimumDelaySeconds,bytes32 operationDomain) // PCT1
protocolVersionManagerConfigV1() // selector 0x4deb7821; 1056 bytes
 returns (bytes4 magic,uint256 settlementChainId,address timelock,
          address activeSettlementRouter,address forcedQueue,
          address builderRegistry,address scheduleOracle,
          address aggregatorSeatMarket,bytes32 aggregatorSeatMarketRuntimeHash,
          bytes32 marketAuthorityConfigurationHash,address bridgeDomainRegistry,
          address bridgeCreditRegistry,bytes32 timelockDescriptorHash,
          bytes32 manifestNamespace,uint64 minimumDelaySeconds,
          uint64 maximumLiveMigrationSeconds,uint16 reviewFinalityBlocks,
          uint64 releaseRouterRegistrationGas,
          uint64 releaseMarketInstallationGas,uint64 releasePostreadGas,
          uint64 releasePostCallbackReserveGas,
          bytes32 activeSettlementRouterRuntimeHash,
          bytes32 activeSettlementRouterConfigurationHash,
          bytes32 forcedQueueRuntimeHash,bytes32 forcedQueueConfigurationHash,
          bytes32 builderRegistryRuntimeHash,
          bytes32 builderRegistryConfigurationHash,
          bytes32 scheduleOracleRuntimeHash,
          bytes32 scheduleOracleConfigurationHash,
          bytes32 bridgeDomainRegistryRuntimeHash,
          bytes32 bridgeDomainRegistryConfigurationHash,
          bytes32 bridgeCreditRegistryRuntimeHash,
          bytes32 bridgeCreditRegistryConfigurationHash) // PVM1
protocolChangeOperationV1(bytes32) // selector 0x4b80fe68; 256 bytes
```

```
 returns (bytes4 magic,uint8 state,uint64 nonce,uint8 operationKind,
          uint32 payloadBytes,bytes32 payloadHash,
          uint64 queuedAt,uint64 executeAfter) // PCO1
```

The magics are PCT1 `0x50435431`, PVM1 `0x50564d31` and PCO1 `0x50434f31`; `operationDomain` is exactly `H("slot-chain-protocol-change-operation-v1")`. Revert, OOG, short/trailing return, bad magic, bad padding, wrong code hash, wrong constant or descriptor mismatch rejects publication before a protocol component changes.

A later migration arm appends one non-extendable manager arm row:

```
abortAfterTimestamp = checked(uint64(block.timestamp) + 604800)
versionMigrationArmId = H(
 "slot-chain-version-migration-arm-v2" || u256(settlementChainId) ||
 address20(protocolVersionManager) || u64(generation) ||
 u64(sourceProtocolVersion) || u64(targetProtocolVersion) ||
 targetManifestHash || targetRegistrationHash || u64(block.timestamp) ||
 u64(abortAfterTimestamp) || protocolChangeOperationId)
liveVersionMigrationLeaseV1() // selector 0xaaac4c97; 320 bytes
 returns (bytes4 magic,uint8 state,uint64 generation,
          uint64 sourceProtocolVersion,uint64 targetProtocolVersion,
          bytes32 targetManifestHash,bytes32 targetRegistrationHash,
          bytes32 armId,uint64 armedAtTimestamp,uint64 abortAfterTimestamp) // VML1
```

The magic is `0x564d4c31`; NONE=0 and LIVE=1 are the only live-view states, and NONE requires every other word zero. Arm rows are append-only and keyed by `armId`; they are historical evidence, never a singleton liveness lock. On every call the getter exact-reads Router's active settlement state and migration gate. It returns the unique matching row only while the gate is ARMED/READY and otherwise returns canonical NONE, including immediately after successful activation. REGISTER_RELEASE and PUBLISH_MIGRATION_ARM likewise derive the current predecessor only from Router's exact ASR1 ACTIVE read; PVM has no shadow active-version authority. A new arm requires Router IDLE/ACTIVE, the current Router version and the registered target predecessor, so a CONSUMED/ABORTED historical row cannot block the next generation. Router activation exact-reads this row, requires the complete live tuple and `block.timestamp<abortAfterTimestamp` before verifier dispatch, and never accepts a caller-supplied lease. At or after the bound, `permissionlessAbortExpiredMigrationV1()` (selector `0xea3e96c2`) is callable by anyone and executes the Router ABORTING journal. If activation already won an earlier transaction, the derived live getter is already NONE and the call is a no-op; otherwise it aborts ARMED or READY and restores source ACTIVE. At the exact expiry timestamp activation rejects, so abort wins independent of ordering. No operation can refresh, replace or extend a LIVE lease. A proving outage, lost governance key or abandoned target therefore cannot leave admissions closed forever.

The remaining PVM–Router mutation boundaries are

```
publishLegacyGenesisCampaignV1(
 bytes32 operationId,uint64 executeAfter,
 uint64 forceCutoffBlock,uint64 proposalCutoffBlock,
 uint64 quiesceNotBeforeBlock,uint64 resumeByBlock,
 uint64 resumeByTimestamp,uint64 reviewFinalizedByBlock,
```

```
 address targetSettlement,uint64 targetProtocolVersion,
 bytes32 targetManifestHash,bytes32 targetRegistrationHash)
 returns (bytes4 magic,uint64 nonce,uint64 generation,
          bytes32 reviewCommitment,bytes32 campaignId)
armVersionMigrationV1(
 bytes32 operationId,bytes32 armId,uint64 armedAtTimestamp,
 uint64 abortAfterTimestamp,uint64 expectedSourceProtocolVersion,
 uint64 targetProtocolVersion,bytes32 targetManifestHash,
 bytes32 targetRegistrationHash)
 returns (bytes4 magic,uint64 generation,bytes32 returnedArmId)
abortExpiredVersionMigrationV1(bytes32 armId)
 returns (bytes4 magic,bytes32 returnedArmId,uint64 generation)
```

Their selectors are `0x5f0ed7f5`, `0xe3bcfcb4` and `0xc4eee12d`; exact returns are 160/LGP1 `0x4c475031`, 96/VMA1 `0x564d4131` and 96/VMB1 `0x564d4231`. Each call receives exactly 8,000,000 gas, is nonpayable and PVM-only, checks canonical calldata and starts from Router lifecycle IDLE. Publication writes PUBLISHING before its first external profile/legacy read, independently recomputes the complete EXECUTING operation ID/timing/registration/review/campaign and restores IDLE last. PVM requires the LGP1 fields and then exact-postreads LGC1.

For arm, PVM first writes APPLYING/consumed and the derived LIVE VML1 lease, then calls Router. Router writes ARMING, exact-reads that lease and the stored RTR2/MPR2 target, performs the source callback, writes ARMED and restores IDLE last. PVM checks VMA1 and exact-postreads ASR1 plus VML1 before PAP1. Expiry abort writes PVM ABORTING, calls the Router abort selector while VML1 is still LIVE and expired, exact-checks VMB1/ASR1, then clears VML1 to canonical NONE and restores manager IDLE. If a sealed successor receipt proves activation already won, PVM skips the Router mutation, terminalizes only the matching stale lease and still exact-checks the receipt/ASR1. Any callback, return, postread or final write failure reverts both contracts and the source callback. No raw Router selector or second order is legal.

The Router derives the target Settlement, manifest, registration, execution profile and immutable verifier solely from the stored delayed gate or genesis campaign and rejects a caller-supplied address or authority. For a later migration it first performs every source, target, Queue, receipt and index preflight, reconstructs the complete typed statement and invokes the profile-pinned verifier by bounded STATICCALL while lifecycle is IDLE and the public gate is READY. For genesis it first validates the exact live campaign, enters ACTIVATING, exact-reads the campaign-bound QUIESCENT LGS1 boundary while the public Router gate remains ACTIVE on legacy, reconstructs the statement and invokes the verifier by bounded STATICCALL before LGAR. Launch header/MPT/SignalService authentication is part of that same verifier and statement, not an opaque Boolean. A later switch checks on L1 that the old `CanonicalCore.messageCursor=ForcedQueue.cursor`; it does not pretend to read the live L2 Inbox cursor. The preflight reads the old canonical core, sequence and last-commit block; Queue root/count/frontier/cursor/liabilities, balance and last due time; the exact fresh target predicate; and all code, configuration and object identities. The target proof executes the release Anchor and validates L2 component rows 4–10. A missing component, collision, bad verifier or proving outage therefore leaves the source and Queue unchanged.

After verifier success, Router computes the one shared join key:

```
activationContextHash =
 H("slot-chain-l1-activation-context-v1" ||
   u256(settlementChainId) || address20(router) || u8(transitionKind) ||
   u64(generation) || u64(sourceProtocolVersion) ||
   u64(targetProtocolVersion) || sourceManifestHash || targetManifestHash ||
   targetRegistrationHash || address20(sourceSettlement) ||
   address20(targetSettlement) || u64(sourceCanonicalSequence) ||
   baseCanonicalHash || statementHash || candidateDigest ||
   canonicalCoreV2Hash(outputCore) || u64(targetCanonicalSequence) ||
   address20(forcedQueue) || queueRoot || u64(queueCount) ||
   u64(startCursor) || u64(endCursor) || address20(proofBeneficiary) ||
   u64(block.number))
```

Here `sourceCanonicalSequence=0` and `targetCanonicalSequence=0` are mandatory sentinels for GENESIS_IMPORT; the Router rejects any other genesis source sequence before calling the verifier. The target sequence is checked `sourceCanonicalSequence+1` for VERSION_MIGRATION. For a later migration, Router writes ACTIVATING and this complete context only after verifier success. For genesis it writes ACTIVATING first as the reentrancy journal, then derives the context, campaign-bound arm ID and launch ID from the exact QUIESCENT LGS1 boundary and stored campaign before the proof STATICCALL. The compatibility implementation's campaign fence prevents that boundary from moving once clean. Each component authenticates it through the bounded STATICCALL

```
migrationActivationContextV1() returns
 (bytes4 magic,uint8 lifecycle,bytes32 activationContextHash,
  address sourceSettlement,address targetSettlement,uint64 generation,
  uint64 sourceProtocolVersion,uint64 targetProtocolVersion,
  bytes32 targetManifestHash,bytes32 targetRegistrationHash)
```

whose selector is `0x7cf70319`, magic is `0x4d414354` (MACT), and returndata is exactly 320 bytes. Wrong lifecycle, target, tuple, length, magic, padding, revert or OOG rejects. Outside ACTIVATING, MACT remains callable as the sole lifecycle view: it returns the current ARMING/READYING/ABORTING/IDLE value and canonical zero for every activation-context field. Later-migration arm, readiness and abort callbacks authenticate that lifecycle and obtain their target tuple from `activeSettlementStateV1()`; genesis LGAR authenticates ACTIVATING and the exact live campaign ID, generation, target and activation context. At every transaction and external-call boundary the context is nonzero exactly in ACTIVATING; the adjacent lifecycle/context stores have no intervening external interaction.

For VERSION_MIGRATION, the exact source callback is

```
freezeMigrationSourceV1(bytes32 activationContextHash) returns
 (bytes4 magic,bytes32 returnedActivationContextHash,
  bytes32 sourcePostStateCommitment)
```

with selector `0x45a80913`, 36-byte calldata, 96-byte return and magic `0x4d46525a` (MFRZ). It is nonpayable and accepts only the immutable Router, exact MACT tuple and local MIGRATION_READY state. It rechecks its canonical core/sequence, arm receipt, closed boundary and zero-live readiness, then writes only MIGRATION_READY to FROZEN plus the context and frozen block. All canonical history, claims, refunds, rewards, reservations, liabilities and balances remain byte-for-byte live on the permanent source. It makes no external interaction after the MACT read and returns

```
H("slot-chain-source-freeze-poststate-v1" || activationContextHash ||
  address20(sourceSettlement) || u8(FROZEN=4) || u64(generation) ||
  u64(sourceProtocolVersion) || u64(sourceCanonicalSequence) ||
  baseCanonicalHash || u64(frozenAtBlock)) .
```

The target adoption callback is

```
adoptMigrationCanonicalV1(
 uint8 transitionKind,uint64 generation,uint64 sourceProtocolVersion,
 uint64 targetProtocolVersion,uint64 sourceCanonicalSequence,
 bytes32 targetManifestHash,bytes32 candidateDigest,
 CanonicalCoreV2 outputCore)
returns (bytes4 magic,uint64 targetCanonicalSequence,
         bytes32 targetPostStateCommitment)
```

Its exact expanded selector is `0x557c4e13`; calldata is 548 bytes, returndata is 96 bytes and magic is `0x4d43414e` (MCAN). It is the only PREACTIVE mutation entry and accepts only Router plus the exact MACT target, version, manifest and registration. It requires untouched canonical/history, mutation and DataSession regions, then writes the output core, target sequence, history tag/cell, both L1 block fields, the ordinary canonical event, stored context and locally prepared ACTIVE state. For a later migration Router's public gate remains READY; for genesis the legacy branch remains unconsumed and the Router remains ACTIVATING. In both cases the target remains PREACTIVE to ordinary callers. After its initial MACT read it makes no external call. Its returned commitment is

```
H("slot-chain-l1-adoption-v1" || u256(settlementChainId) ||
  address20(router) || address20(targetSettlement) || activationContextHash ||
  u8(transitionKind) || u64(generation) || u64(sourceProtocolVersion) ||
  u64(targetProtocolVersion) || u64(sourceCanonicalSequence) ||
  targetManifestHash || candidateDigest || canonicalCoreV2Hash(outputCore) ||
  u64(targetCanonicalSequence) || u64(canonicalizedAtBlock)) .
```

Ordinary commits continue to use `advanceCursor`. Activation instead uses the sole Router-only combined Queue transition:

```
migrateAuthorityForActivationV1(
 bytes32 activationContextHash,address sourceSettlement,
 address targetSettlement,bytes32 expectedRoot,uint64 expectedCount,
 uint64 startCursor,uint64 endCursor,address proofBeneficiary)
returns (bytes4 magic,bytes32 returnedActivationContextHash,
         uint256 creditedWei,bytes32 queuePostStateCommitment)
```

with selector `0x9461f698`, 260-byte calldata, 128-byte return and magic `0x514d4947` (QMIG). It authenticates Router and MACT, old authority, root/count/current cursor and `start<=end<=count`; checked-adds the exact prefix fee interval to the beneficiary's pull claim, advances the cursor and changes authority to the target atomically. It calls no source, target, beneficiary or arbitrary sink, and changes no leaf, deadline, root or count. Its poststate commitment is

```
H("slot-chain-queue-migration-poststate-v1" || activationContextHash ||
  address20(forcedQueue) || address20(targetSettlement) || queueRoot ||
  u64(queueCount) || u64(endCursor) || address20(proofBeneficiary) ||
  u256(creditedWei) || u256(postAccountedLiabilityWei) ||
  u256(postTotalPullClaimableWei)) .
```

Source, target and Queue each implement the common view

```
migrationActivationPostStateV1(bytes32 activationContextHash) returns
 (bytes4 magic,uint8 role,bytes32 returnedActivationContextHash,
  bytes32 postStateCommitment)
```

with selector `0x66e664cb`, 36-byte calldata, 128-byte return, magic `0x4d415053` (MAPS), and roles SOURCE=1, TARGET=2, QUEUE=3. Router performs exactly these three bounded STATICCALLs after QMIG and independently recomputes all commitments. The source reproduces MFRZ, target reproduces MCAN, and Queue reproduces QMIG. No mutating external call is legal after QMIG.

The exact VERSION_MIGRATION journal is verifier STATICCALL; write ACTIVATING/context; MFRZ; MCAN; QMIG; the three MAPS reads; internal target- registration consumption, receipt and successor-index writes; the final current- target/public-ACTIVE word; clear context; and write IDLE last. Every callback, post-read, return decoder and later write is in the outer transaction's single revert domain. A fault at any boundary restores READY/IDLE, the old source and Queue authority, cursor/payment state, empty target, unused registration and absent receipt, so permissionless retry or mature abort remains possible.

"Empty PREACTIVE target" is an exact predicate, not a claimed constructor default. The target Settlement's internal DataSession region requires all 1,024 cell tags FREE; empty ID-to-cell and live-owner maps; `liveCount=refundCount=occupiedCount=0`; `nextSessionSequence=0`, `gcCursor=0`; `migrationRefundGeneration=0`, `migrationRefundClaimDeadline=0`, the shared session/sweep guard in its canonical NOT_ENTERED state; and zero live- and refund-bond liability. The actual target Settlement ETH balance is deliberately not required to be zero: CREATE2 prefunding or forced ETH is surplus and cannot veto activation. Deployment may occur while the source has live sessions, because the protocol-lifetime gate is shared; cutover separately requires that gate to be READY after authenticating the source-local live-session count zero. Exact fresh creation code plus the absence of every PREACTIVE mutation surface proves the fixed cells and hidden maps empty. Logs are not storage and are not part of this predicate. Before cutover the Router uses the exact `dataSessionAccountingV1()` ABI to recheck every observable scalar, guard word and configuration hash; it does not pretend to enumerate a mapping. It also repeats the source-readiness handshake defined below. Both reads must report the same generation/version/manifest, closed boundary, zero LIVE count, NOT_ENTERED guard and registered DataSession configuration immediately before any authority, queue or canonical write.

Settlement constructor provenance is authenticated by the delayed registration's exact 232-byte `SettlementDeploymentDescriptorV1`:

```
(address20(factory), bytes32(factoryRuntimeHash),
 bytes32(factoryConfigurationHash), bytes32(salt), bytes32(initCodeHash),
 address20(targetSettlement), bytes32(targetRuntimeHash),
 bytes32(targetConfigurationHash))
```

Its hash is `H("slot-chain-settlement-deployment-v1"||u32(232)||packedDescriptor)`. Every field is nonzero and `targetSettlement=low20(keccak256(0xff||address20(factory)||salt||initCodeHash))`. Registration must publish the complete init-code artifact and immutably bind this descriptor inside `targetRegistrationHash`; arm, activation, cancellation and the activation receipt must bind that same hash. The Router checks the root factory and target `EXTCODEHASH`, then applies the exact four-byte `componentConfigHashV2()` call with 50,000 gas and exactly 32 return bytes to the target, and rechecks the derivation at arm and activation. Hidden mapping emptiness follows only from that authenticated constructor and the lack of PREACTIVE mutation entry points; it is never inferred from runtime/config getters alone. The same authenticated Settlement init code and its target configuration commitment bind the internal DataSession storage layout, shared gate, `dataRent` sink and the rule that every non-session payable selector, fallback and receive path rejects; there is no second custody contract or cross-contract session authority. The delayed target registration, arm/cancel tuple, statement, deployment commitment, MACT context and sealed receipt all carry the same nonzero `targetRegistrationHash`; every mismatch rejects before ACTIVATING.

The source callback freezes only new work; its permanent history and all mature reward, refund, bond and Queue claims remain claim-only and do not gate cutover. The imported base remains only in the old version's history; MCAN writes the target's proof-authenticated output cell. QMIG is the only activation operation that moves Queue cursor/payment/authority, and it is followed only by the fixed poststate STATICCALLs. Failure of any step reverts freeze, adoption, payment, authority, receipt, indexes and the final switch together.

The router's stable `syncIngress` status/refund preflight remains callable by every historically registered immutable BridgeInboxAdapter. The payable `appendFromAdapter` path is narrower: it requires the caller to equal the current active authorization's exact adapter deployment and authorization ID. Its nonpayable half resolves the active version, calls that Settlement's bounded `sync()`, preserves a SYNCED transition without accepting value, and otherwise returns the exact version/generation stamp. Its payable half makes no second sync decision and forwards the exact value and descriptor only after that stamp still matches the singleton ForcedQueue's ACTIVE authority and the same active adapter/authorization binding; a predecessor adapter cannot append after its retirement watermark. Historical adapter status, cancellation and pull-refund functions remain callable without an append capability. No Settlement calls the ingress/version router during an ordinary canonical commit; the activation-only MACT read and Router's MAPS postreads are the fixed exceptions. Ordinary commits call only the queue's non-callback `advanceCursor`; activation uses QMIG. Router `canonicalAt(protocolVersion,canonicalSequence)` performs a bounded fixed-selector `STATICCALL` to the permanently registered Settlement, requires the exact return length, code hash, version/profile and cell tag, and returns no caller fields. This gives target replacement the same delayed trust root as the validity verifier without adding a canonical-path dependency.

The non-proxy, unpausable `TerminalSignalVerifier` accepts only the fixed 64-sibling proof against one exact routed tuple and the exact domain/Bridge/credit/terminal leaf. Source finalize/recall verifies and mutates the source credit in one transaction; there is no publication stage to front-run. Sparse 4,096-block L2 jumps cannot choose a cell, and at most one canonical write per L1 block gives a live version's 256 cells at least 3,072 seconds of retention. Frozen versions retain their last 256 cells forever. The Bridge stores `terminalIndex[creditId]`. Any archive reconstructs the siblings from the canonical complete `TerminalAppended` event sequence, and L1 verifies them against the selected history cell. No archive signature or privileged indexer is accepted. Source terminal functions never call legacy SignalService. No V2 failure, recall, delivery finalization or view falls back to `msgHash`. The release profile pins every selector, event, status, liability and accumulator slot plus positive and negative lifecycle vectors.

V2 launch contains no asset codec or Vault flow. ERC20, ERC721, ERC1155 and all official Vault calls remain V1, with their existing V1 status, quota and recall semantics. A future asset protocol must define bidirectional round-trip and delivered-backing accounting, immutable asset policy and canonical/deployment identity, mapping-rotation history, destination release-versus-mint supply conservation and an unpausable exit. It requires a new kind, domain, release, profile, codec and audit and cannot reinterpret DIRECT.

The fresh L2 Bridge's sealed `v2Active` state enables processing only after the proof-authenticated tx0 atomically seals its Store and Bridge. Source V2 send remains disabled until that canonical registration is proven to L1 and its 214-block support delay completes. Every old `sendMessage` call and all L2-to-L1 sends retain V1 process/retry/fail/recall. InboxApplyRouterV2 makes only the bounded manifest-routed store calls above, and neither router nor store invokes a message recipient. Legacy proof-based `proveSignalReceived` remains unchanged for V1 and is never a V2 terminal dependency. AnchorV4 and InboxApplyRouterV2 are profile-defined system transactions: their custom type, reserved senders, system nonces, zero beneficiary tip and fee treatment are consensus fork rules, not ordinary user accounts. Their gas and the batch loop still count against the block limit. Deployment and migration require vector-tested InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2 and TerminalAccumulatorV2 bytecode. Legacy SignalService remains V1-only and, after a completed launch or outside an active genesis campaign, may be upgraded without entering a V2 trust path. From campaign publication through local resume/freeze, however, its exact checkpoint implementation and upgrade authority are covered by the genesis campaign fence above. Authorizing an unreviewed caller is invalid.

An unexpired kind-0 item must reveal raw bytes hashing to its stored `rawTxHash`; the circuit decodes them and matches every duplicated descriptor field before execution. Once `validUntil` is strictly before the L2 block timestamp, the circuit emits the unique `EXPIRED_NO_TX` disposition from the authenticated durable descriptor without needing those bytes. Consequently a disappeared sender or pruned history can delay its own item only until the bounded validity horizon, never wedge the FIFO permanently. A user that wants execution remains an available source for its own signed bytes. The 2,164-second recovery target is conditional on unexpired head bytes being retrievable; the unconditional protocol bound adds at most `MAX_FORCE_VALIDITY_SECONDS`.

For a normal signed block, `forceRoot/forceCutoff` are authenticated by its L1 anchor. For recovery they equal the current round's directly stored pair. The circuit consumes only up to that cutoff, so a later append cannot invalidate an already signed block or in-flight proof. All blocks in one candidate use the same cutoff. A due current head cannot be bypassed: `sync()` cancels an immature window, or closes a mature best only when its live boundary is not due, before opening recovery.

The 1,500-second force delay applies to the queue head, not to an arbitrary position behind existing prepaid work. An admitted sender can compute its worst-case number of escape blocks from the gas ahead at admission. As with L1 itself, the protocol does not promise backlog-independent latency against an adversary willing to buy all available capacity; it promises that builders cannot selectively veto the FIFO head.

An escape block is deterministic under a governance-frozen `executionProfileHash`. That profile commits the L2 fork rules; complete header-field derivation (gas limit, base fee, extra data, withdrawals/requests roots and randomness); the deployed AnchorV4, InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2 and TerminalAccumulatorV2 bytecode/constructor hashes; reserved senders, nonces and gas; AnchorV4 checkpoint calldata and `ancestorsHash` transition; and the transaction/receipt derivation above. There is no protocol signature or account nonce. The permanent unsigned typed system envelope is exactly

```
0x7f || rlp([chainId, systemKind, systemNonce, gasLimit, to, value, data])
```

with minimal RLP integers, a 20-byte `to`, zero value, no signature fields and `systemNonce=block.number-firstV2BlockNumber` independently for each kind. `firstV2BlockNumber` is the protocol-lifetime value `launchImportedCanonical.l2BlockNumber+1`; launch installs it in the cutover commitment and execution profile, every later profile must reproduce it exactly, and checked subtraction rejects a pre-cutover block. Kind 0 is Anchor at transaction index 0 and kind 1 is InboxApply at index 1; the fork injects their distinct reserved senders. Ordinary accounts cannot submit type `0x7f`. System gas counts against the block limit, but the envelope has zero tip and is exempt from account balance/base-fee charging. Its receipt is exactly `0x7f||rlp([status,cumulativeGasUsed,bloom,logs])`; both transactions contribute normally to transaction/receipt roots and cumulative gas.

The type's state transition is fully separate from EIP-1559 signing. Decoding requires `chainId=block.chainid=destinationChainId<=UINT64_MAX`; a foreign, zero or nonminimal chain ID rejects before execution. The profile maps each kind to exactly one reserved sender and target. Both `CALLER` and `ORIGIN` are that reserved sender. Sender and target start warm in addition to the fork's ordinary precompile warm set. No sender account nonce is read, compared or incremented; no sender balance is read, debited or credited; `GASPRICE` returns zero while `BASEFEE` retains the block's ordinary value. There is no access list, creation target, blob field or value transfer. Intrinsic gas is exactly `21000+4*zeroCalldataBytes+16*nonzeroCalldataBytes`; execution starts with `gasLimit-intrinsicGas`. A malformed envelope or intrinsic gas above the exact kind gas limit makes the block invalid.

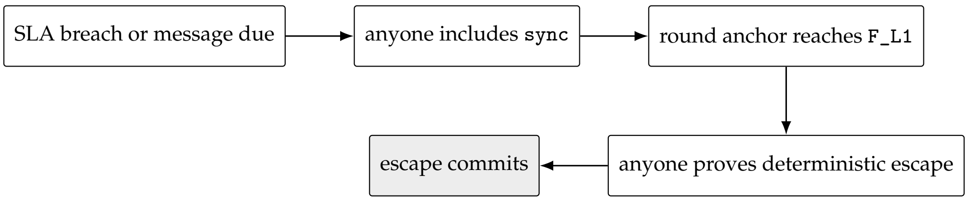
The transaction hash is legacy Keccak-256 of the complete typed envelope above, and that same byte string is the transaction-trie value. After execution let `spentBeforeRefund=gasLimit-gasRemaining` (therefore including intrinsic gas). Under the launch fork's London refund quotient 5, `refundGas=min(refundCounter,floor(spentBeforeRefund/5))` and `gasUsed=spentBeforeRefund-refundGas`; unused or refunded gas creates no account credit. REVERT or exceptional halt produces status zero and reverts state/logs under ordinary EVM rules while retaining that gas accounting; success produces status one. The typed receipt bytes above are the receipt-trie value, and `cumulativeGasUsed` is the ordinary running sum in exact transaction order. The block rejects any mismatch in envelope bytes, injected environment, warm set, state change, transaction hash/root, receipt bytes/root, gas used or cumulative gas.

The steady kind-0 call has gas limit 1,000,000, selector `0x523e6854` and ABI tuple

```
(uint48(anchorNumber), anchorHash, anchorStateRoot)
```

from the candidate's one batch anchor. The first pending-release activation kind-0 call has gas limit 12,000,000 and the activation selector/calldata defined above. Its kind-1 forced budget is 13,000,000 and that block permits no discretionary gas. Kind 1 always targets the protocol-lifetime InboxApplyRouterV2, has gas limit 5,000,000, selector `0x6b326168` for `apply(uint64,` `(uint64,uint8,uint32,bytes32,bytes)[])`, and uses the exact ABI rows derived in §8; the empty vector is still a canonical call. Activation uses 12m Anchor + 13m forced + 5m margin, exactly the 30m block limit. Steady blocks use 1m Anchor + 20m forced + 5m margin, at most 30m. Its timestamp is `GENESIS_TIMESTAMP+escapeSlot`, coinbase is the protocol sink and there is no discretionary body. A repeated batch anchor still performs the exact per-block ancestors transition. Changing the execution profile requires a delayed protocol-version upgrade and new full-block golden vectors; runtime guessing is forbidden.

For a cartel with no usable builder block, the recovery sequence is:



With the initial bounds, activation inclusion (120 s), the frozen `ESCAPE_OFFSET=1,900s` (covering `T_DEPTH_MAX=900s`, proving at 900 s and 100 s margin), submission inclusion (120 s), and clock margin (24 s) total 2,164 seconds, below `DELTA_FINAL_LAG=3,600s`. Tip admissibility is separate: the escape slot is at the end of the offset, so only submission and skew—144 seconds—must fit `DELTA_TIP=1,200s` when proof generation meets its bound.

9 Perpetual aggregator seat market

The aggregator market is an optional service and payment layer around the permissionless proof path. It has four installed ranks—one primary and three standbys—and a continuous reverse auction with four shared waiting cells. It has no auction epoch or fixed close window. A healthy funded primary persists until competitive handover, voluntary exit, its one objective duty, funding expiry or migration. No seat, operator action, premium balance or Market call is a precondition for proof submission, canonical commit, forced recovery, recovery renewal or migration readiness.

9.1 Authority, identity and bounded call graph

Settlement alone owns the installed roster, immutable seat terms and service intervals, one staged-handover commitment, duties, successor selection, failover, breach receipts, migration state and retained duty history. Every canonical transition is bounded, scans at most four seat/duty cells and performs no external call. If optional seat state is absent, stale, economically unmaintained or locally full, Settlement closes the affected economics to vacancy and continues canonical recovery.

The protocol-lifetime non-proxy `AggregatorSeatMarket` alone owns pending offers, native-ETH SLA bonds, premium custody, reserves and pull credits. It has no canonical writer, migration coordinator, `armMigration` or `abortMigration` entry point. It stores append-only exact Settlement authorizations installed only by the immutable `ProtocolVersionManagerV2`'s typed REGISTER_RELEASE operation. The manager derives the authorization ID, target, runtime/configuration, selectors, gas bounds, version and chain from the exact release manifest and target registration; neither the DAO proposer nor an execute caller supplies an authorization row. Duplicate or conflicting installation rejects, and no delete or redirect exists. A Market call may read Settlement only through a separately specified bounded `STATICCALL`; it checks exact chain, version, address, runtime/configuration hash, selector, return length and magic. No caller-supplied authority, phase, generation, operator, close time or validity Boolean is accepted.

The sole installation and audit views are

```
installSettlementAuthorizationV1(
 uint64 protocolVersion,address target,bytes32 runtimeHash,
 bytes32 configurationHash,bytes4 expectedMagic,
 bytes32 targetRegistrationHash)
 returns (bytes4 magic,bytes32 authorizationId)
settlementAuthorizationV1(bytes32 authorizationId) returns
 (bytes4 magic,uint64 protocolVersion,address target,bytes32 runtimeHash,
  bytes32 configurationHash,bytes4 expectedMagic,
  bytes32 targetRegistrationHash)
```

Their selectors are `0x72a3e937` and `0x1693ae01`; returns are exactly 64 and 224 bytes; install magic is `0x53414931` (SAI1) and getter magic is `0x53415431` (SAT1). Install is nonpayable, ProtocolVersionManagerV2-only and legal only inside its consumed REGISTER_RELEASE frame. It recomputes the V1 authorization ID below from the Market's immutable chain/address plus the supplied manifest-derived fields, then makes exactly one 100,000-gas STATICCALL to Router's `targetReleaseRegistrationV2(protocolVersion)`. It requires the exact 384-byte RTR2 return and compares its version, target, runtime hash, configuration hash and target-registration hash with the supplied fields before writing an unused ID once. Market does not call the target during installation: PVM and Router already performed independent target live reads, and PVM repeats them after Router returns and before invoking Market. After the Market callback PVM exact-reads Router and Market stores in the same common rollback domain. The manager exact-checks the full return; a callback cannot install a second row because its operation/frame is already consumed. The three target read selectors are fixed to `0xcf52185b`, `0x76d5ecd4` and `0x9a649489` for state, term and duty respectively, each with exactly 100,000 gas. They are protocol constants, not installer fields; every configuration hash commits the implementations of those exact ABIs.

The sole offer-book state read is the no-argument selector `seatMarketTargetStateV1()` with canonical ABI return

```
(address target, uint256 settlementChainId, uint64 protocolVersion,
 bytes32 runtimeHash, bytes32 configurationHash, bytes4 magic,
 uint8 phase, uint64 seatGeneration)
```

where `target=address(this)`, `runtimeHash=address(this).codehash`, `magic=0x53454154` (ASCII `SEAT`), and phase is exactly ACTIVE=1, ARMED=2, READY=3 or FROZEN=4. Return length is exactly 256 bytes; address, `uint64`, `bytes4` and `uint8` padding must be canonical. Market checks the target's pinned `EXTCODEHASH` before a descriptor-gas-bounded `STATICCALL` and compares every word with the append-only authorization. Revert, OOG, short/trailing data or any mismatch fails closed. `AggregatorSeatMarket.syncSeatGeneration()` returns exactly `(uint8result,uint8purgedCount,uint64generation)` as three ABI words, where result is UNCHANGED=0 or SYNCED=1 and `purgedCount<=4`; it reads the current target, terminalizes every stale waiting tranche in one rollback domain and commits the generation cache last. It is the sole permissionless *offer-book* entry point that reads Settlement. Insertion, requote, pending exit/refund and pull withdrawal perform no target read and require the already initialized cached generation. Stage/apply authenticate the same generation only inside their enclosing atomic Settlement call. Premium, installed-release, breach and duty-reclamation paths may make only the two separately bounded historical reads below:

```
seatMarketRecordV1(bytes32 seatTermId) view returns
  (bytes4 magic, bytes32 authorizationId, uint64 seatGeneration,
   bytes32 returnedSeatTermId, bytes32 trancheId, address operator,
   uint64 responsibilityStart, uint64 premiumCap, uint64 serviceCloseAt,
   uint64 termRemovedAt, uint64 lastLiabilityAt, uint16 liveDutyCount,
   bytes32 latestDutyId, uint8 latestDutyDisposition,
   uint64 latestDutyDispositionAt, bytes32 breachReceiptId,
   uint64 breachRecordedAt)
seatDutyRecordV1(bytes32 dutyId) view returns
  (bytes4 magic, bytes32 authorizationId, uint64 seatGeneration,
   bytes32 returnedDutyId, bytes32 seatTermId, bytes32 trancheId,
   uint8 disposition, uint64 dispositionAt, uint64 lastLiabilityAt,
   bytes32 breachReceiptId, uint64 breachRecordedAt)
```

Both magic words are `0x53485231` (ASCII `SHR1`); exact return lengths are 544 and 352 bytes. Every narrow word and address has canonical padding. The append-only target authorization pins both selectors, separate gas limits, runtime/configuration, version and chain. Market checks code hash, makes one bounded STATICCALL, and compares authorization, generation and returned ID before using a word. Revert, OOG, short/trailing/noncanonical data, unknown record, a live-duty count above four or an invalid disposition rejects. Historical records are permanent after close and cannot be redirected by a successor. These are the only post-install Settlement reads made by Market; the activation-only MACT/MFRZ/MCAN/MAPS protocol is separate. No caller-supplied close cap, breach receipt, duty binding, magic or Boolean is authoritative.

The non-proxy `ActiveSettlementRouter` is the protocol-lifetime root of the Settlement graph and immutably owns the shared migration gate, `ForcedQueue` and InboxApply deployment descriptor. There is no replaceable `L1HeaderOracle`: every Settlement authenticates historical L1 hashes by direct EIP-2935 system-contract reads plus strict canonical RLP-header decoding under the profile-pinned fork rules, and fresh hashes by native `BLOCKHASH`. A deployment without those primitives requires a different reviewed execution profile rather than an authority object. Every live, PREACTIVE and historical Settlement must identity-alias those exact objects. The Router neither reads nor writes a live L2 object. ProtocolVersionManagerV2 keeps the append-only target authorizations under its manifest-derived Market-only branch, while Router is the sole *L1* activation-receipt and successor-index store. The separately domained L2 Registrar receipt exists only for destination-Bridge reclamation; its counter, ID and magic are never accepted on L1, and the L1 receipt is never accepted on L2. Historical targets expose only premium accrual/claim, reserve reconciliation, breach enforcement, installed release, bond-credit claim and duty-history reclamation; they cannot accept new offers, stages, sessions or ingress.

Every successful switch appends one exact sealed receipt. Its identity is

```
receiptId = H("TAIKO_ACTIVATION_RECEIPT_V1" ||
```

```
  u256(settlementChainId) || address20(router) || u64(routerGeneration) ||
  u64(successorIndex) || u8(transitionKind) ||
  u64(sourceProtocolVersion) || u64(targetProtocolVersion) ||
  sourceManifestHash || targetManifestHash ||
  sourceAuthorizationId || targetAuthorizationId ||
  targetRegistrationHash ||
  address20(sourceSettlement) || address20(targetSettlement) ||
  oldDestinationDomainId || newDestinationDomainId ||
  address20(oldDestinationBridge) || address20(newDestinationBridge) ||
  u64(queueWatermark) || candidateDigest || outputCanonicalHash ||
  u64(outputCanonicalSequence) || activationContextHash ||
  transitionAuxiliaryHash || sourcePostStateCommitment || adoptionCommitment ||
  queuePostStateCommitment || u64(activatedAtBlock))
```

The Router exposes only these fixed views:

```
activationReceiptV1(bytes32 receiptId) view returns
  (bytes4 magic, bytes32 returnedReceiptId, uint256 settlementChainId,
   address router, uint64 routerGeneration, uint64 successorIndex,
   uint8 transitionKind, uint64 sourceProtocolVersion,
   uint64 targetProtocolVersion, bytes32 sourceManifestHash,
   bytes32 targetManifestHash, bytes32 sourceAuthorizationId,
   bytes32 targetAuthorizationId, bytes32 targetRegistrationHash,
   address sourceSettlement,
   address targetSettlement, bytes32 oldDestinationDomainId,
   bytes32 newDestinationDomainId, address oldDestinationBridge,
   address newDestinationBridge, uint64 queueWatermark,
   bytes32 candidateDigest, bytes32 outputCanonicalHash,
   uint64 outputCanonicalSequence, bytes32 activationContextHash,
   bytes32 transitionAuxiliaryHash, bytes32 sourcePostStateCommitment,
   bytes32 adoptionCommitment,
   bytes32 queuePostStateCommitment, uint64 activatedAtBlock, uint8 sealed)
seatSuccessorReceiptV1(bytes32 oldAuthorizationId) view returns
  (bytes32 receiptId, uint64 successorIndex, bytes4 magic)
bridgeSuccessorReceiptV1(bytes32 oldDestinationDomainId,
                         address oldDestinationBridge) view returns
  (bytes32 receiptId, uint64 successorIndex, bytes4 magic)
```

The receipt return is exactly 992 bytes with magic `0x41525631` (`ARV1`), `sealed=1` and canonical padding; each index return is exactly 96 bytes with magic `0x41535631` (`ASV1`). The Router checks and increments one global `uint64successorIndex`, derives every field from the verified activation/current registrations, and writes receipt plus both nonzero predecessor indexes in the same journal before ACTIVE. A GENESIS_IMPORT has canonical-zero predecessor authorization/domain/Bridge and writes no predecessor index; its `sourceManifestHash` is the authenticated `legacyDeploymentHash`, not zero, and its `transitionAuxiliaryHash` is the exact nonzero legacy-abandonment receipt hash defined below. VERSION_MIGRATION requires every predecessor field nonzero, a distinct target, absent indexes, exact queue watermark, `targetRegistrationHash`, `outputCanonicalSequence`, shared context and all three independently recomputed poststate commitments, and requires `transitionAuxiliaryHash=0`;

no key can be consumed twice. The full receipt is immutable and a zero, partial, mismatched or unsealed write cannot survive rollback.

Market rotation accepts a successor only after bounded code/config/length checks on both Router views and exact old/new authorization, versions, manifests, target registration, Settlements, generation, candidate/output, activation context, the canonical-zero transition auxiliary, all three poststate commitments and seal. Old-Bridge reclamation does not use these L1 views; it uses the proof-bound Registrar-local destination receipt specified in the release lifecycle below. A caller-supplied receipt, index, successor address or Boolean is never authority on either layer.

9.2 Orthogonal lifecycle and conservation

Lifecycle dimensions are deliberately independent:

```
OfferLocation  = NONE | PENDING | STAGED
TrancheUsage   = OFFER | STAGED | INSTALLED | CLOSED_UNINSTALLED
BondDisposition = NONE | OWNER_CREDITED | PENALTY_CREDITED
ReserveLifecycle = ABSENT | UNSTARTED | OPEN | CLOSED_TAIL
```

Only PENDING occupies a waiting cell and only STAGED occupies the single stage. Installation sets location to NONE but preserves the immutable `SeatTerm.offerId`. The stage retains the capacity of its selected waiting cell, so at all times

```
pendingCount + stagedCount <= 4
stagedCount <= 1.
```

Staging changes the pair by $(-1, +1)$; ordinary or lineup expiry restores the same offer by $(+1, -1)$ and can never overwrite or refund a fifth entry.

Every cross-contract record uses one fixed-width legacy-Keccak identity; no ABI dynamic encoding, string enum or caller-selected ID is accepted:

```
authorizationId = H("TAIKO_SEAT_TARGET_AUTHORIZATION_V1" ||
  u256(marketChainId) || address20(market) || u256(settlementChainId) ||
  u64(protocolVersion) || address20(target) || runtimeHash ||
  configurationHash || bytes4(expectedMagic))
trancheId = H("TAIKO_SEAT_TRANCHE_V1" || authorizationId ||
  u64(seatGeneration) || address20(operator) || u256(bondWei) ||
  u256(creationSequence))
offerId = H("TAIKO_SEAT_OFFER_V1" || authorizationId ||
  u64(seatGeneration) || trancheId || address20(payout) ||
  u256(askWeiPerSecond) || u256(eligibleAtTimestamp) ||
  u256(eligibleAtBlock) || u256(quoteSequence))
stageId = H("TAIKO_SEAT_STAGE_V1" || authorizationId ||
  u64(seatGeneration) || lineupCommitment || offerId ||
  u256(selectedRank) || u256(handoverAt) || u256(expiresAt))
seatTermId = H("TAIKO_SEAT_TERM_V1" || authorizationId ||
  u64(seatGeneration) || offerId || trancheId ||
  u64(installedAt) || u64(installRevision))
lineupCommitment = H("TAIKO_SEAT_LINEUP_V1" || u64(lineupRevision) ||
```

```
  primaryTermId || standby0TermId || standby1TermId || standby2TermId)
dutyId = H("TAIKO_SEAT_DUTY_V1" || seatTermId || u64(dutySequence) ||
  u64(baseCanonicalSequence) || u64(baseTipSlot))
selectionId = H("TAIKO_SEAT_SELECTION_V1" || seatTermId || trancheId ||
  offerId || u64(selectedCanonicalSequence) || u64(selectedAt) ||
  u64(targetTip) || u8(selectionSource) || predecessorDutyIdOrZero)
breachReceiptId = H("TAIKO_SEAT_BREACH_V1" || dutyId || seatTermId ||
  trancheId || u64(recordedAt))
```

Absent lineup terms and predecessor duty are `bytes32(0)`; selection source is DUTY_FAILOVER=1 or HEALTHY_EXPIRY=2. All widths are checked before hashing, all counters are monotone, and a hash collision reverts rather than overwriting an existing record. Market and Settlement recompute the same ID at every handoff; the release bundle contains cross-language golden and one-field- substitution vectors for each domain.

Each tranche stores immutable operator, bond, creation sequence, offer and term bindings, one-shot release timestamp and terminal disposition. The exact terminal credit is

```
creditId = H("TAIKO_SEAT_BOND_CREDIT_V1" || u256(marketChainId) ||
           address20(market) || trancheId || u8(OWNER|PENALTY))
```

and stores immutable beneficiary and amount plus one claimed bit. A terminal transition requires `disposition=NONE`, moves the exact bond from escrow to exactly one owner or penalty credit, and writes the disposition atomically. Claim marks the exact credit before transfer and never erases history.

Event	**Offer location**	**Tranche usage**	**Bond disposition**
Accept	NONE → PENDING	create OFFER	NONE
Displace/purge	PENDING → NONE	OFFER → CLOSED_UNINSTALLED	OWNER_CREDITED
Request pending exit	PENDING → NONE	OFFER → CLOSED_UNINSTALLED	NONE until delay
Finalize pending exit	unchanged	unchanged	OWNER_CREDITED
Stage	PENDING → STAGED	OFFER → STAGED	unchanged
Ordinary/lineup expiry	STAGED → PENDING	STAGED → OFFER	unchanged
Migration cancellation	STAGED → NONE	STAGED → CLOSED_UNINSTALLED	OWNER_CREDITED
Install	STAGED → NONE	STAGED → INSTALLED	unchanged
Healthy release	unchanged	remains INSTALLED	OWNER_CREDITED
Breach enforcement	unchanged	remains INSTALLED	PENALTY_CREDITED

A pending exit immediately leaves ranking/capacity and sets `pendingRefundAt=exitRequestedAt+EXIT_DELAY_SECONDS`; anyone finalizes at or after equality. A staged offer cannot exit. Never-installed tranches never use installed release, and installed tranches never use pending refund. An earlier installed release request cannot defeat later valid breach enforcement.

Market accounting obeys

```
accountedMarketBalance = bondEscrow + outstandingOwnerBondCredits
  + outstandingPenaltyCredits + freePremium + reservedPremium
  + outstandingPremiumClaims
actualMarketBalance >= accountedMarketBalance
reservedPremium = sum(live PremiumReserve.reservedWei).
```

Forced ETH above the accounted amount is surplus and creates no claim.

Staging moves exactly `askWeiPerSecond*SEAT_RUNWAY_SECONDS` from `freePremium` into the exact stage reserve. Apply rekeys it to the exact seat term with no balance delta. An installed standby is UNSTARTED and earns nothing; after canonical promotion, the first noncanonical economic call derives the true start from immutable Settlement service state and may not trust a Market sentinel.

For a started service,

```
claimMaturedThrough = max(0, now - PREMIUM_CLAIM_DELAY_SECONDS)
accrueTo = max(lastAccruedAt,
  min(claimMaturedThrough, Settlement.previewPremiumCap(seatTermId),
      premiumFundedUntil))
earnedWei = askWeiPerSecond * (accrueTo - lastAccruedAt).
```

Accrual moves the checked earned value from that reserve to the immutable payout pull credit before withdrawal. The `Settlement.previewPremiumCap` notation means the authenticated `premiumCap` word returned by `seatMarketRecordV1`; there is no third selector. That word is the earliest applicable immutable close, satisfaction, objective failover, unfunded eligibility end, or ring-full recovery cap; omitted maintenance can never increase it.

For an atomic healthy close let $a = lastAccruedAt$, $f = premiumFundedUntil$, $c = \min(authenticatedCloseAt, f)$ and $m = \max(a, \min(c, now - PREMIUM\_CLAIM\_DELAY\_SECONDS))$, with saturating subtraction and $a \leq c \leq f$. It allocates

```
maturedWei  = ask * (m-a)
tailWei     = ask * (c-m)
unearnedWei = ask * (f-c).
```

The transaction credits matured value, returns only unearned value and retains the tail as CLOSED_TAIL until `c+PREMIUM_CLAIM_DELAY_SECONDS`. Canonical/asynchronous closes make no Market call; later permissionless reconciliation authenticates the permanent Settlement cap, credits earned value and returns only proven unearned value. Bond terminalization and duty-cell reclamation are forbidden while a reserve remains.

9.3 Continuous reverse auction and service clocks

Pending offers sort by ascending `askWeiPerSecond`, `eligibleAtTimestamp`, `eligibleAtBlock`, `quoteSequence`, then operator address. A fifth offer must strictly beat the worst under this full order or revert without retaining funds; displacement creates the exact owner bond credit atomically. Stale target/generation offers cannot rank or displace.

Requote is pending-only and requires the immutable operator, exact live offer/tranche, no exit, exact active authorization/generation, nonzero payout and an ask within the immutable maximum. It may lower the ask, or retain the ask while changing payout; increases and no-ops revert. It keeps the tranche and bond, derives a checked fresh quote sequence and offer ID, and copies both original `eligibleAtTimestamp` and `eligibleAtBlock` exactly. Quote maturity is the one-time admission delay of the bonded tranche, not a delay which a current holder may push forward: no successor quote may postpone either dimension. Every revision voluntarily takes the fresh sequence's tie-break position. Before inserting the successor, Market deletes the superseded pending quote; the old ID is immediately stale and repeated payout changes cannot create unbounded live or historical contract storage. Requote performs no ETH movement or Settlement call. Consequently four zero-ask offers cannot freeze all four cells by changing payout before maturity: at the original timestamp/block equality every current successor is stageable by any caller.

Direct and promoted primary service share:

```
SEAT_RUNWAY_SECONDS >= MIN_PRIMARY_TENURE_SECONDS
  + HANDOVER_EXECUTION_BUFFER_SECONDS + SLA_TAIL_SECONDS
HANDOVER_EXECUTION_BUFFER_SECONDS >= HANDOVER_DELAY_SECONDS
  + STAGE_GRACE_SECONDS + T_INCLUDE_MAX_SECONDS
SLA_TAIL_SECONDS = DELTA_SLASH_LAG - DELTA_RECOVERY_LAG
minimumTenureUntil = responsibilityStart + MIN_PRIMARY_TENURE_SECONDS
premiumFundedUntil = responsibilityStart + SEAT_RUNWAY_SECONDS
serviceEligibleUntil = premiumFundedUntil - SLA_TAIL_SECONDS.
```

All operations are checked-width. Standby tenure remains separately bounded by `MIN_STANDBY_TENURE_SECONDS`. Primary tenure gates only competitive replacement and voluntary primary exit; duty completion, failover, funding expiry, ring-full vacancy and migration override it.

Anyone may call `stageBest()`. Settlement first performs its bounded leading sync. If sync changes canonical/recovery state it returns SYNCED with no Market call. Otherwise Market scans at most four offers in total order and selects the first mature, current and structurally feasible transition: direct primary into vacancy, strictly cheaper primary replacement preserving all standbys, or deterministic standby insertion when fewer than four terms exist. The first structurally feasible candidate must be fully funded from the same gross reserve snapshot; staging cannot anticipate release of the outgoing primary reserve. Candidate selection, reserve debit, PENDING-to-STAGED and Settlement stage recording are one revert domain.

Replacement handover is `max(active.minimumTenureUntil,now+HANDOVER_DELAY_SECONDS)`; vacancy fill or standby insertion uses `now+HANDOVER_DELAY_SECONDS`. Expiry is handover plus `STAGE_GRACE_SECONDS`. The stage binds target, generation, selected rank, outgoing primary or zero, offer, reserve and

```
lineupCommitment = H("TAIKO_SEAT_LINEUP_V1" || u64(lineupRevision) ||
  primarySeatTermId || standbySeatTermIds[0] ||
  standbySeatTermIds[1] || standbySeatTermIds[2]).
```

Absent ranks are zero. It also requires `stageExpiresAt+T_INCLUDE_MAX_SECONDS<=serviceEligibleUntil` for a live primary. No later quote replaces a stage or moves its deadlines.

Permissionless apply repeats leading sync and requires exact unchanged stage, lineup, authorization, generation, health, maturity and headroom. Opening recovery tombstones the stage before returning SYNCED. Apply derives the term only at its actual timestamp:

```
installRevision = checked(lineupRevision + 1)
seatTermId = H("TAIKO_SEAT_TERM_V1" || authorizationCommitment ||
  u64(seatGeneration) || offerId || trancheId ||
  u64(installedAt) || u64(installRevision)).
```

Market rekeys the reserve and binds the tranche in the same transaction in which Settlement changes the roster and revision.

Installed exit request is one-shot and operator-only. Primary exit matures at the exact stored `primaryExitAt=max(exitRequestedAt+EXIT_DELAY_SECONDS,minimumTenureUntil)`; standby exit matures at `standbyExitAt=max(exitRequestedAt+EXIT_DELAY_SECONDS, minimumStandbyTenureUntil)`. The immutable one-shot `exitRequestedAt` can be written only from zero, and a request requires `seatTermId!=selectedSuccessorTermId`; the latter is the exact term ID in the current immutable selection record, or zero when no successor is selected. Finalization is permissionless, begins with sync and removes the term only on a fresh retry if sync changed state. Until removal the bond and every attached duty remain live. Funding expiry may close service earlier; later sponsorship cannot extend an installed term or postpone its exit.

9.4 Duty, failover, release and reclamation

Service start allocates no duty. It stores one prospective canonical base and objective recovery/failover/slash thresholds. A qualifying ordinary commit at or before recovery refreshes that prospective base. After strict `now>recoveryAt`, Settlement's one four-cell pass allocates at most one duty; equality remains curable. If no duty cell is locally reusable, or the checked sequence is exhausted, the same sync closes optional seat economics to vacancy at objective `recoveryAt` and immediately opens SLA recovery. It creates no duty, calls no Market and never waits for `DELTA_FINAL_LAG`.

The pass applies objective failover/slash to existing duties before cure and then performs prospective/selection work. A commit after strict failover may still satisfy the failed-over predecessor but starts a selected successor only when its exact selection record names that duty. A commit after strict slash may advance canonical state but cannot cure BREACHED. Recovery candidates use one O(1) preflight followed by that same pass. Failed history adoption restores all state.

Selection records are immutable and bind unique selection ID, term/tranche/ offer, revision/time and source DUTY_FAILOVER or HEALTHY_EXPIRY. A healthy expiry allocates no fake duty. Every roster removal records immutable `termRemovedAt`. A selected-but-unstarted successor counts as seatless for the existing `DELTA_FINAL_LAG/G_MAX` recovery floor; a pointer can never suppress recovery or backdate liability. A usable later commit/recovery starts it with fresh thresholds, while an unusable revision vacates the lineup.

Installed release is permitted only for exact closed Settlement history, an INSTALLED tranche with no disposition, a refundable terminal duty and no live reserve. Anyone may create its one-shot request. Define

```
dispositionStableAt = dispositionAt + REORG_STABILITY_SECONDS (or 0)
evidenceSafeAt = lastLiabilityAt + EVIDENCE_DELAY_SECONDS
                 + REORG_STABILITY_SECONDS
finalizeReleaseAt = max(releaseRequestedAt + RELEASE_CHALLENGE_SECONDS,
                        dispositionStableAt, evidenceSafeAt)
```

```
reserveMatureAt = settlementCap + PREMIUM_CLAIM_DELAY_SECONDS (or 0)
ownerTerminalAt = max(finalizeReleaseAt, reserveMatureAt).
```

Permissionless finalization uses the exact authenticated `seatMarketRecordV1`, requires zero live duties and the matching term/ tranche/operator/times, reconciles and deletes the reserve, then credits only the immutable operator. For breach, `penaltyTerminalAt=max(recordedAt+REORG_STABILITY_SECONDS,reserveMatureAt);` permissionless enforcement authenticates the exact receipt and duty binding from both historical getters, reconciles the reserve and credits only the immutable penalty sink. Neither path can overwrite a terminal disposition.

Market's monotone `isDutyHistorySafe(dutyId,seatTermId,trancheId)` requires the exact three-way binding, terminal bond disposition, no live offer/stage/roster/reserve, all timing horizons, and an identity that can never be reused. Immutable closed term and binding history remain queryable forever. Settlement's `reclaimDutyCell` begins with leading sync; if sync changes state it returns SYNCED with no Market call. Otherwise it authenticates the exact predicate and caches only a local reusable bit. Credit withdrawal is irrelevant, so a beneficiary cannot pin history.

9.5 Migration interaction

ProtocolVersionManager alone consumes the delayed typed arm operation, which creates the non-extendable LIVE lease. In one revert domain it requires Router lifecycle IDLE, writes ARMING before any external interaction, writes Router ACTIVE-to-ARMED, then calls the old Settlement's manager-only completion callback. The callback rereads the complete router tuple, runs leading sync internally without a generic SYNCED early return and with READY transition explicitly disabled, recomputes its inputs, increments `seatGeneration`, closes services non-fault, excuses unresolved duties and selected successors, tombstones the stage and returns exact fixed-length `SEAT_ARMED_MAGIC` binding generation, versions, manifest, target registration and resulting seat generation. Any mismatch, short/trailing return or local fault reverts the global arm. After all local and Router writes succeed, IDLE is the transaction's final store.

The two manager-only selectors are the canonical ABI selectors for

```
completeSeatMigrationArmV1(uint64,uint64,uint64,bytes32,bytes32)
    returns (bytes response)
completeSeatMigrationAbortV1(uint64,uint64,uint64,bytes32,bytes32)
    returns (bytes response)
```

Their selectors are respectively `0x91d657cf` and `0xc69d5579`. Their five calldata words are, in order, Router generation, active version, target version, target manifest hash and target registration hash; every `uint64` word has zero high padding and trailing calldata rejects. The returned `bytes` payload is exactly 100 bytes:

```
bytes4 magic || u64(routerGeneration) || u64(activeProtocolVersion) ||
u64(targetProtocolVersion) || bytes32(targetManifestHash) ||
bytes32(targetRegistrationHash) ||
u64(seatGeneration)
```

with `magic=0x5341524d` (ASCII `SARM`) for arm and `0x53414254` (ASCII `SABT`) for abort. Full ABI returndata is exactly 192 bytes: offset `0x20`, length `0x64`, the payload and 28 zero pad bytes.

PVM validates the complete encoding and every field; raw 100-byte, short, long, nonzero-padding or substituted responses revert the Router word, Settlement callback and all local cleanup together.

Abort is symmetric: the manager authenticates the expired immutable LIVE lease, writes lifecycle ABORTING while the public gate remains ARMED or READY, calls the manager-only abort callback, and requires `SEAT_ABORTED_MAGIC` bound to the canceled tuple, target registration and retained seat generation. It then records cancellation, clears the target tuple, restores the old-version ACTIVE word and writes IDLE last. It performs no post-abort callback or same-transaction sync and accepts no fresh deadline. The next ordinary entry may sync under ACTIVE. Abort never resurrects a term, quote, duty, successor or stage. Market is not called by either canonical callback. Later Market rotation authenticates Router's immutable successor receipt, cancels the exact old stage reserve if present, purges at most four pending offers, disables the old authorization and advances one receipt per call. Historical economic claim/reconciliation surfaces remain available; only the exact ACTIVE tip can resynchronize generation and accept new offers.

10 Economics, slashing and payments

Event	Objective evidence	Effect
Same-context equivocation	Two distinct protocol-domain signatures for one key/slot/context	Slash that schedule window's tranche once; tombstone key; emit reporter entitlement.
Aggregator SLA breach	Exact duty passes its strict slash threshold uncured	Record BREACHED in Settlement; later permissionless enforcement reconciles the reserve and terminalizes that tranche once to the penalty credit.
Force-only activation	Due queue head	No aggregator penalty; open recovery.
Skip or absence	None that distinguishes fault from non-receipt	Not slashable; promise may be revoked.
Invalid proof/data	Verifier failure	Reject; no canonical or payment effect.

The release commits one canonical `taiko.slot-chain.economic-profile.v2` JSON artifact and its measurement commit. Units are not interchangeable: native amount is `wei`, native rate is `wei/second`, and builder amount is atomic units of the one profile-bound builder lease token. That token binds chain ID, nonzero address, `EXTCODEHASH` and decimals. Its fields are exactly `leasePerWindowAtomic`, `maximumBondAtomic` and `reporterRewardCapAtomic`; no generic untyped "bond" scalar is decoded. For one builder slash,

```
reporterRewardAtomic = min(reporterRewardCapAtomic, builderSlashAtomic)
builderPenaltyAtomic = builderSlashAtomic - reporterRewardAtomic.
```

Both operations are checked in that token's atomic units.

There is one top-level immutable sink table. Each entry is the typed pair `(assetId,address)`. `builderPenalty` must use BUILDER_LEASE; `dataRent`, `seatPenalty`, `forcedExpiry` and `bridgeSurplus` must use NATIVE_ETH. All sink addresses are nonzero and a nested or component-local sink is invalid; in particular DataSession cannot substitute a rent sink. Reporter reward goes only to the evidence submitter and the remainder only to the typed builder penalty sink. Seat SLA bond, premiums and ingress deposits are native ETH and never enter builder-token accounting.

A production profile must have `status=CALIBRATED`, contain every known key and no unknown key, and make every required identity/measurement nonnull. It rejects wrong unit strings, malformed decimal strings, asset-chain mismatch, zero identity, unchecked arithmetic or any failed relation, including:

```
dataSession geometry = (86400,86400,1024,2,2100,8,6)
1024 * refundableBondWei <= UINT256_MAX
refundableBondWei + baseRentWei <= UINT256_MAX
2100 * 126972 * rentPerPublishedByteWei <= UINT256_MAX
DATA_TTL >= P_PROVE_MAX + W_SETTLE_SECONDS + REORG_MARGIN_SECONDS
refundClaimWindowSeconds >= T_INCLUDE_MAX_SECONDS + REORG_MARGIN_SECONDS

PREMIUM_CLAIM_DELAY_SECONDS >= REORG_STABILITY_SECONDS

SEAT_RUNWAY_SECONDS >= MIN_PRIMARY_TENURE_SECONDS
  + HANDOVER_EXECUTION_BUFFER_SECONDS + SLA_TAIL_SECONDS

HANDOVER_EXECUTION_BUFFER_SECONDS >= HANDOVER_DELAY_SECONDS
  + STAGE_GRACE_SECONDS + T_INCLUDE_MAX_SECONDS

SLA_BOND_WEI >= MAX_ASK_WEI_PER_SECOND
                  * PREMIUM_CLAIM_DELAY_SECONDS

SLA_BOND_WEI >= MAX_ASK_WEI_PER_SECOND
                  * MIN_PRIMARY_TENURE_SECONDS
  + MAX_AVOIDED_SERVICE_COST_WEI + COLLUSION_SAFETY_MARGIN_WEI

maximumBondAtomic <= 2^192-1
maximumBondAtomic * (Q_MAX+1) < 2^256.
```

The first bond inequality prevents cheap closed-tail capital grief; the second bounds the direct subsidy when one actor fails a predecessor to promote its own maximum-ask standby. Neither claims to insure unbounded external application loss. Every other geometry, retention, proof-size, queue-solvency and fee relation in the parameter table is validated by the same checked profile decoder. The committed example remains UNCALIBRATED and therefore cannot be used to deploy or activate a release.

Equivocation evidence is idempotent per `(builder,window)`; separate windows slash separate tranches. `L_LEASE` is calibrated against at most 76 assigned slots but is not insurance for unbounded signed value. Tombstoning is atomic with the first valid slash; future snapshots exclude the key.

Evidence does not depend on retaining an obsolete `admissionRoot`. It contains two distinct low-s EIP-712 headers with the same recovered nonzero key, slot, tier, context ID, admission version, admission root, episode, revision and recovery ID. For tier 1 it recomputes the normal context from the same base, admission pair and arm-block anchor carried by both headers and authenticates that arm hash as canonical through EIP-2935; tier 2 recomputes the recovery context. The contract enforces at launch that the last tier-1 evidence/replay instant precedes expiry of that 8,191-block history entry. It supplies one proof under the signed admission root and one under the *current* admission root for the same address and `registrationIndex`; the latter must be that still-retained active/liability generation. Its current stored tranche root separately authenticates the window leaf, which is `RESERVED(W)` or `LIABLE(W)`. The two complete struct hashes must differ. Signing two conflicting protocol headers in one such context is slashable even if the key was not ultimately selected at that schedule position; builders must not sign off-ticket headers. The evidence transaction derives W from the slot, requires the tranche leaf's stored window to equal W, and lands by `evidenceReplayDeadline(W)=evidenceDeadline(W)+REORG_MARGIN_SECONDS`. Evidence available by `evidenceDeadline(W)` therefore has the full margin for reorg detection and re-inclusion; first publication delayed into the replay tail has no replay guarantee. Address reuse is forbidden while that retained generation exists, so the current proof cannot select a newer incarnation. This rule keeps evidence verification fixed-depth while the liability-capacity invariant retains every slashable generation through the replay deadline.

Canonical settlement performs no reward calculation, token balance write or external call. It always emits a candidate-committed event containing the ID, beneficiary and reward class. Best-effort protocol claims use a fixed `MAX_REWARD_RECEIPTS=256` ring, selecting `uint8(uint256(candidateId))`: if that cell is empty or its prior receipt is both claimed/expired and past the reorg margin, Settlement records the three fields, commit block and `claimed=false`; otherwise it only emits the event. Ring occupancy can therefore forfeit a reward but can never revert or delay canonical state. A separate distributor verifies this stored receipt, sets `claimed=true` before transfer, and pays at most the funded class cap before `REWARD_CLAIM_WINDOW=86,400s`. Logs alone are explicitly not on-chain-verifiable entitlements. Burns are never recycled into the same episode's bounty. Normal service/data compensation likewise cannot affect candidate validity or canonical commit.

Seat premium and bond accounting follows the conservation and maturity rules in §9. Canonical progress never pushes ETH, waits for Market or infers a penalty from mere non-receipt. A breach requires the objective retained duty receipt. Each seat bond has one terminal disposition, while every premium withdrawal is backed by an exact reserve debit and delayed through `PREMIUM_CLAIM_DELAY_SECONDS`. The example economic profile remains UNCALIBRATED and is invalid for deployment.

11 Security and liveness analysis

Adversary/failure	Protocol result	Bound or residual
Invalid execution or forged state	Proof rejects.	Finalized-state safety reduces to proof-system and L1 safety.
Aggregator offline/byzantine	Objective duty recovery/failover proceeds without Market; any prover uses tier 2 or 3.	At most the configured recovery trigger plus 2,164 s under stated inclusion/proof and head-data bounds; economic enforcement is asynchronous.
No seat or no standby	State remains valid; escape has no seat dependency.	Same protocol bound; reward may be zero.

All builders collude/absent	Unsigned escape advances anchor and forced queue.	Meaningful discretionary throughput falls to forced envelopes; no builder censorship veto.
Builder equivocation	Competing signed branches; one may win normal order.	Direct per-window slash; pre-close promises remain revocable.
Builder skips/withholds body	Honest tail may be unprovable or omitted.	Not objectively slashable; eventual escape restores state progress but may revoke promises.
Data-session spam	Exact rent/bond buys at most 1,024 bounded LIVE/REFUND cells.	A funded Sybil may censor the optional session market through TTL plus refund grace; escape uses no blob session.
Missing/pruned kind-0 bytes	Unexpired head waits; after validity it is consumed as EXPIRED_NO_TX from its durable descriptor.	Adds at most seven days; a kind-1 descriptor is durable, while successful destination invocation still needs Message bytes.
V1 or V2 endpoint upgrade exploit	V1 remains untouched; every V2 source/destination Bridge is a fresh immutable non-proxy account.	A new endpoint/domain cannot redirect an old writer; every old endpoint retains its terminal/refund path.
Unauthorized source-support fill	Version manager rejects entries absent from the active delayed manifest.	At most 64 audited additions per release; no attacker-consumable lifetime cap.
Message bytes disappear	Enqueue expires to source cancellation; a pinned credit expires to destination FAILED.	The full DIRECT ETH value, execution fee and liquidity fee refund remains permissionless by credit ID; successful destination invocation is lost.
Source escrow drain	The fresh SourceBridge checks every payout against pending, user-pull and LP-pull aggregate liabilities.	DONE reclassifies the full escrow to the proof-bound LP recipient; cancel/FAILED reclassifies it to the user, and liability falls only on a successful pull.
No willing destination LP	The credit remains UNFUNDED without consuming Pool capital or blocking unrelated credits, then may expire to FAILED.	Delivery liveness is economic: permissionless tickets remove deterministic finite-reserve exhaustion but cannot compel capital at an unattractive `liquidityFee`.
Target or post-call tail fails	The only-self attempt frame reverts Pool consume, transfer, target and Bridge effects; target-failure policy runs only after the authenticated catch.	The caller's ticket remains available; owner-last failure appends FAILED only after that rollback.

Guardian pause or zero quota	Risky sends/process/retries stop; expiry, terminal verification and reserved refunds continue.	Frozen V2 proof/refund paths have no transitive pause or ordinary quota dependency.
Foreign destination replay	Local domain and `destBridge==address(this)` checks reject before status or value effects.	The immutable pin store and accumulator both bind one exact domain/Bridge pair.
Terminal proof-format change	Canonical state carries a stable application-level depth-64 vector root/count.	Source verification does not parse the destination EVM state format; later roots preserve old leaves.
Retired destination reclaim	Reclaim waits for successor receipt, Queue watermark, pin/terminal equality and zero pull liability.	Atomic Pool funding leaves no domain-scoped capital; only unaccounted Bridge surplus reaches the pinned sink, and tickets are never enumerated or swept.
Legacy call to installed V2 code	The fresh release-owned Bridge and Store remain inactive and the custom system transaction type is invalid.	Proved tx0 atomically seals Store and Bridge once; legacy V1 addresses are never redirected.
All canonical prestate copies lost	No party can construct the next EVM witness from a root alone; proof generation stops without changing canonical state.	Explicit information-theoretic availability limit; restore any root-verifiable archive copy.
Transparent-mempool front-run	First valid recovery transaction wins.	Proof statement binds beneficiary, episode, revision and outputs; copying cannot redirect payment or state.
Payment-vault exhaustion	Claim is zero or partial.	Never blocks safety or canonical liveness; economic liveness is conditional.
L1 reorg within policy	Contract state, burns and claims roll back together.	Clients replay canonical logs and proofs against new version/hash.
L1 censorship/outage	Neither activation nor proof can land.	Outside model after `T_INCLUDE_MAX_SECONDS`; no L2 protocol can force its L1 contract to execute.
Genesis campaign reaches QUIESCENT but no valid target proof lands	Canonical state and Queue authority do not move; before either hard expiry a valid proof may retry, and at expiry anyone or the next ordinary call restores legacy ACTIVE.	Exact scan and resume-profile checks ensure the bounded pause does not expire recoverable data or change a validity/custody deadline. A corrected target needs a higher-nonce delayed campaign.

The escape lane establishes strong transaction-entry permissionlessness even when the capital-ranked builder set is captured. It does not make the normal builder market egalitarian: top-

64 ranking and per-window capital remain a meaningful entry barrier. The product must distinguish these two claims.

12 Implementation blueprint

12.1 L1 modules and bounded storage

- `BuilderRegistry`: protocol-lifetime non-proxy, with 64 active cells, 1,072 liability generations, 512-leaf tranche roots, tombstones, replacement counters and the versioned `admissionRoot`. Every transition is fixed-depth or constant-size.
- `ScheduleOracle`: protocol-lifetime non-proxy sealed window root/seed under the version-independent `slot-chain-seed-v2` and frozen allocation/ placement rules, plus the one-shot delayed fork-verifier registry, in a ring covering `MAX_LIVE_WINDOWS=268`. `MAX_EARLY_SEAL_WINDOWS=8` covers the earliest snapshot geometry. In window units the capacity is at least:

  ```
  MAX_EARLY_SEAL_WINDOWS + MAX_CANDIDATE_WINDOWS +
  ceil((W_SETTLE_SECONDS + DELTA_FINAL_LAG + DELTA_TIP +
        REORG_MARGIN_SECONDS + EVIDENCE_DELAY_SECONDS) / 384) + 2
  ```

  An entry is overwritten only after it is `EXPIRED`; the same transaction checks the release predicate and no stored normal best/round reference.
- Settlement's internal DataSession region: exclusive session ETH custody inside the immutable Settlement and a 1,024-cell FREE/LIVE/REFUND ring, at most two LIVE cells per owner, one global checked session sequence, exact live/refund/occupied counters and a live-owner-only mapping. Each LIVE cell has a derived owner/sequence ID, `bytes32peaks[12]`, checked count at most 2,100, sealed root/count, expiry and bond; REFUND reuses that cell for one bounded owner/bond/deadline claim. There is no on-chain leaf array, permanent per-owner nonce or claim mapping. Only nonpayable ordinary/migration/refund maintenance advances the cursor by exactly eight cells; payable open selects one caller-hinted FREE cell and never runs maintenance. Expiry and migration perform no callback, and surplus-sweep failure cannot gate any protocol transition.
- `ForcedQueue`: protocol-lifetime immutable non-proxy depth-64 append-only Merkle root/count/frontier, last due time, complete fixed-width per-index descriptors, deposit prefix, unconsumed escrow and pull claims. An expired kind-0 descriptor reconstructs its leaf and prefix accounting without historical calldata, while kind-1 credits remain durable. Only the router appends and only the active Settlement advances the cursor/payment interval.
- `BridgeDomainRegistry`: non-proxy append-only source, destination and frozen-endpoint-support mappings. Only bounded entries from the manifest being atomically activated by ProtocolVersionManager may be added, and destination support additionally requires a finalized canonical L2 proof of the exact registrar record; there is no delete, redirect or age-proportional iteration.
- `BridgeInboxAdapter`: exactly-once NONE/QUEUED records and caller-funded queue processing fees. It direct-reads same-L1 CreditRecord and Bridge liability state, rejects while PREACTIVE and, after router SYNCED, commits that transition, creates no record and pull-credits the still-local full caller fee before a nonreverting return. A successful append forwards the whole fee to ForcedQueue and atomically marks the permanent source

record QUEUED. It is non-proxy and immutable; its runtime and constructor-fixed registry, Bridge, router and version manager plus its one-shot sealed destination-domain slot are profile/domain-bound, with no mutable target, delegatecall or arbitrary-call selector.

- `InboxApplyRouterV2` and `InboxCreditStoreV2`: the router is a protocol-lifetime non-proxy dispatcher with one global cursor and append-only registrar-authenticated domain routes; every endpoint store fixes that same router and its own Bridge/domain. Full-vector validation, at most 64 contiguous-run calls and EVM rollback make mixed-domain application atomic. Neither has an upgrade, resolver, callback or caller-selected target.
- `ProtocolReleaseAuthorityV2`: protocol-lifetime non-proxy release manifest authenticator for the manifest-derived Anchor caller, permanent reserved system-transaction origin and namespace. Its version records are one-shot and have no endpoint target.
- `TerminalDomainRegistrarV2`: non-proxy immutable version-activation coordinator bound only to the release authority and accumulator. It derives endpoint targets from the authenticated manifest and atomically seals the fresh Store, Bridge `v2Active` bit, inbox route and accumulator writer. Records are permanent, one-shot and one-to-one.
- `TerminalAccumulatorV2`: protocol-lifetime non-proxy depth-64 append-only vector with checked count/wrapped root, a 64-word frontier and the complete canonical leaf event. It has one-to-one domain/Bridge writer registration and no root setter. Its sentinel/static-read handshake derives the terminal from the Bridge; each transition stores the returned index atomically.
- Fresh immutable non-proxy SourceBridgeV2 and DestinationBridgeV2 bundles retain V2 custody, aggregate/per-credit liabilities, status, pull credits and ContextV2 identity. V1 accounts and Vaults remain untouched. Every successor uses fresh domain/Bridge/Registry/Quota accounts; none has an upgrade, delegatecall, owner or reinitializer.
- `Settlement`: permanent non-proxy state machine with `PREACTIVE`, `ACTIVE`, `MIGRATION_ARMED`, `MIGRATION_READY` and `FROZEN` states; canonical core, one normal best, recovery episode/round, seat pointer, 256-cell sequence-tagged full-core history and a 256-cell best-effort receipt ring. A delayed cutover stops opening new work and reaches MIGRATION_READY after the current transition settles, so an adversary cannot veto migration by continuously creating a best or recovery round. Rewards and burn disposal remain separate; no loop is proportional to chain age.
- `ActiveSettlementRouter`: non-proxy, no generic target setter. Its append-only version registry permanently binds Settlement address/code/profile; proof-carrying activation binds the exact old core, same ForcedQueue/registry/schedule objects and migration-ready state, then atomically installs the proved target poststate. Delayed exact-target abort restores the unchanged old authority before cutover. Its stable stamped two-call sync/append facade preserves old adapters, while `canonicalAt(version,sequence)` only reads exact historical cells.
- Terminal verifier: immutable, read-only and unpausable; it checks one fixed 64-sibling vector proof against an exact router-routed version and sequence cell. The leaf separately binds index, domain, Bridge, credit and terminal status.

The non-authoritative `CanonicalArchive` is independently mirrored off-chain canonical bodies, receipts, state-trie nodes and runtime bytecode indexed by canonical block, state root and code hash. Every object is commitment-verifiable; no archive address, signature or availability vote appears in Settlement or the verifier statement.

All setters enforce parameter inequalities atomically and are disabled after the launch freeze except through the existing delayed governance path. Every counter has an explicit width and checked increment; no sentinel shares a legal value.

12.2 Historical authentication

Native `blockhash` plus a supplied matching RLP header authenticates a fresh normal arm block; EIP-2935 authenticates later normal-context evidence and schedule carrier headers. A recovery round captures its immediately preceding block hash with `blockhash(block.number-1)` and later hashes a supplied RLP header to that stored value; it never tries to read an unavailable parent timestamp, queries an expired history entry or mutates the meaning of a past header. Its queue pair is round storage, not an MPT claim. EIP-4788 plus SSZ branches authenticates the schedule's parent beacon/execution payload; its state root plus MPT proofs authenticates the registry. Deployments lacking either primitive are unsupported by this profile and require a new exact profile and separate review.

The EIP-2935 read is byte exact and has no Solidity selector. The execution profile pins `HISTORY_STORAGE_ADDRESS=0x0000F90827F1C53a10cb7A02335B175320002935`, its fork-specific runtime hash, activation block, `HISTORY_SERVE_WINDOW=8191` and `HISTORY_READ_GAS=50000`. Settlement first checks that code hash, then STATICCALLs with zero value and exactly `u256(requestedBlockNumber)` as 32-byte big-endian calldata. The call must return exactly one nonzero 32-byte hash and the requested number must be in `[block.number-8191, block.number-1]`; revert, OOG, short/trailing/zero return, a pre-activation empty cell or a supplied RLP header whose Keccak/block number differs rejects. No wrapper address, cached Boolean or caller-supplied history root is accepted. This is the EIP-2935 `get` wire format; the SYSTEM_ADDRESS-only `set` path is never called by the protocol.

12.3 Executable execution profile

`executionProfileHash` is not a free-form governance label. Each `protocolVersion` maps immutably to exactly one strict canonical Solidity ABI value

```
profileBytes = abi.encode(ExecutionProfileV2)
executionProfileHash = H("slot-chain-execution-profile-v2" ||
                         u32(byteLength) || profileBytes)
```

where `1<=byteLength<=65536`. V2 is protocol-lifetime frozen: the immutable V2 manager accepts only this schema. An incompatible extension is outside every authority path specified here and requires an independently reviewed new L1 protocol root/genesis migration; this manager cannot install or reinterpret it. Stored V2 hashes are never reinterpreted. There is no CBOR or legacy fallback.

Exact ABI grammar. `ExecutionProfileV2` is one tuple of 252 fixed value fields followed by the sole dynamic `bytestargetCodeArtifact`. Its outer root word is exactly `u256(0x20)`. Value field i below is the ABI word at tuple offset $32i$; word 252 is the sole offset and equals `u256(8096)=u256(0x1fa0)`. The total fixed head is therefore 253 words. The tail is the ordinary ABI bytes encoding `u256(artifactLength)||artifact||zeroPadding`, with no gap, alias, trailing word or nonzero padding. The complete field order is:

```
 0.. 15 core:
   uint64 schemaVersion, uint64 protocolVersion,
   uint256 settlementChainId, uint256 l2ChainId,
   bytes32 settlementGenesisHash, bytes32 destinationGenesisHash,
   bytes4 forkDigest, uint64 firstV2BlockNumber, uint64 genesisTimestamp,
   bytes32 manifestNamespace, bytes32 destinationNamespace,
   bytes32 routerNamespace, bytes32 poolNamespace,
   address anchorV4, bytes32 anchorRuntimeHash,
   bytes32 anchorConfigurationHash
16.. 43 control objects, in this order:
   (address protocolChangeTimelock, bytes32 runtimeHash, bytes32 configHash),
   address daoProposer,
   (address protocolVersionManager, bytes32 runtimeHash, bytes32 configHash),
   (address activeSettlementRouter, bytes32 runtimeHash, bytes32 configHash),
   (address forcedQueue, bytes32 runtimeHash, bytes32 configHash),
   (address builderRegistry, bytes32 runtimeHash, bytes32 configHash),
   (address scheduleOracle, bytes32 runtimeHash, bytes32 configHash),
   (address aggregatorSeatMarket, bytes32 runtimeHash, bytes32 configHash),
   (address bridgeDomainRegistry, bytes32 runtimeHash, bytes32 configHash),
   (address bridgeCreditRegistry, bytes32 runtimeHash, bytes32 configHash)
44.. 56 target artifact:
   address settlementFactory, bytes32 settlementFactoryRuntimeHash,
   bytes32 settlementFactoryConfigurationHash, bytes32 settlementSalt,
   bytes32 targetRuntimeHash, bytes32 targetCreationCodeHash,
   bytes32 targetAbiHash, bytes32 targetStorageLayoutHash,
   bytes32 targetConstructorSchemaHash, bytes32 targetCompilerBuildHash,
   bytes32 targetCompileTimeRulesHash,
   bytes32 targetImmutableReferencesHash, bytes32 targetLinkReferencesHash
57.. 62 target verification infrastructure:
   (address settlementL1HistoryStorageAddress, bytes32 runtimeHash,
    bytes32 readConfigurationHash),
   (address registrationMptVerifier, bytes32 runtimeHash, bytes32 configHash)
63.. 71 sinks:
   address builderLeaseToken, bytes32 builderLeaseTokenRuntimeHash,
   uint8 builderLeaseTokenDecimals, address builderPenaltySink,
   address dataRentSink, address seatPenaltySink, address forcedExpirySink,
   address bridgeSurplusSink, address protocolCoinbaseSink
72.. 85 recovery uint64 values:
   settlementWindowSeconds, includeMaxSeconds, finalLagSeconds,
   tipLagSeconds, proveMaxSeconds, l1FinalityBlocks, depthMaxSeconds,
   clockSkewSeconds, escapeOffsetSeconds, forceDelaySeconds,
   maximumParentGapSlots, maximumForceValiditySeconds,
   evidenceDelaySeconds, reorgMarginSeconds
86..100 seat values:
   uint64 seatRunwaySeconds, minimumPrimaryTenureSeconds,
   minimumStandbyTenureSeconds, handoverDelaySeconds, stageGraceSeconds,
   exitDelaySeconds, recoveryLagSeconds, slashLagSeconds,
   premiumClaimDelaySeconds, reorgStabilitySeconds, releaseChallengeSeconds;
```

```
    uint256 maximumAskWeiPerSecond, seatSlaBondWei,
    maximumAvoidedServiceCostWei, collusionSafetyMarginWei
101..106 DataSession values:
    uint256 dataSessionBondWei, uint256 dataSessionBaseRentWei,
    uint256 dataSessionRentPerPublishedByteWei,
    uint16 dataSessionBlobBaseFeeMultiplierBps,
    uint64 dataSessionMaximumTtlSeconds,
    uint64 dataSessionRefundClaimWindowSeconds
107..117 target gas uint64 values:
    activeSettlementStateReadGas, seatMarketStateReadGas,
    seatTermRecordReadGas, seatDutyRecordReadGas,
    componentConfigurationReadGas, registrationVerifierConfigReadGas,
    registrationVerifierCallGas, migrationActivationContextReadGas,
    migrationPostStateReadGas, targetAdoptionCallGas,
    targetPostCallbackReserveGas
118..137 compile-time rules:
    uint8 slotSeconds; uint16 scheduleWindowSlots; uint8 seatCount;
    uint8 dutyRingCapacity; uint8 forceTreeDepth; uint8 dataMmrDepth;
    uint16 dataSessionCellCount; uint16 maximumDataSessionsPerOwner;
    uint16 maximumDataRecordsPerSession; uint8 maximumGcSteps;
    uint8 maximumBlobsPerPost; uint16 canonicalHistoryCapacity;
    uint8 destinationComponentCount; uint8 ingressAuthorizationCount;
    address pointEvaluationPrecompile; uint32 pointEvaluationGas;
    uint256 blsModulus; uint32 blobGasUsed;
    uint32 maximumBlobPayloadBytes; uint16 maximumBlobChunkCount
138..157 MPR2 source:
    address migrationVerifier; bytes32 migrationVerifierRuntimeHash;
    bytes32 migrationVerifierConfigurationHash;
    bytes32 migrationVerifyingKeyHash; bytes32 migrationProofSystemId;
    bytes32 migrationPublicInputSchemaHash; bytes4 migrationVerifierSelector;
    uint32 migrationMaximumProofBytes; then uint64
    migrationVerificationGas, supportedL1BlockGasLimit,
    worstCaseActivationAdoptionGas, sourceFreezeGas, targetAdoptionGas,
    queueMigrationGas, activationContextReadGas, postStateReadGas,
    legacyStateReadGas, legacyArmGas, legacyFinalizeGas,
    postCallbackReserveGas
158..201 destination:
    address inboxSystemSender, address anchorSystemSender;
    for i=1..10, expanded in increasing i:
      (address destinationComponent_i, bytes32 runtimeHash, bytes32 configHash);
    bytes32 destinationBridgeStorageLayoutHash,
    bytes32 bridgeKernelProfileHash, bytes32 bridgeKernelAbiHash,
    bytes32 bridgeStatusLayoutHash, bytes32 bridgeCustodyLayoutHash,
    address destinationQuotaManager, bytes32 destinationQuotaManagerRuntimeHash,
    bytes32 destinationQuotaManagerConfigurationHash,
    uint256 destinationInitialNativeQuota, uint64 poolReadGas,
    uint64 poolAuthorizationCleanupGas, uint64 poolValueCallbackGas
202..227 source:
```

```
    address sourceBundleFactory, bytes32 sourceBundleFactoryRuntimeHash,
    bytes32 sourceBundleFactoryConfigurationHash, bytes32 sourceBundleSalt,
    bytes32 sourceBundleInitCodeHash, address sourceBundleDeployer,
    bytes32 sourceBundleDeployerRuntimeHash, address legacyV1SourceBridge,
    address sourceBridge, bytes32 sourceBridgeRuntimeHash,
    bytes32 sourceBridgeConfigurationHash, bytes32 sourceBridgeStorageLayoutHash,
    address sourceCreditRegistry, bytes32 sourceCreditRegistryRuntimeHash,
    bytes32 sourceCreditRegistryConfigurationHash, address sourceQuotaManager,
    bytes32 sourceQuotaManagerRuntimeHash,
    bytes32 sourceQuotaManagerConfigurationHash, address sourceSupportRegistry,
    bytes32 sourceSupportRegistryRuntimeHash,
    bytes32 sourceSupportRegistryConfigurationHash, address sourcePauser,
    address sourceSignalService, uint64 sourceRegistrationEpoch,
    bytes32 sourceNamespace, bytes32 sourceGenesisHash
228..234 kind-0 ingress:
    address kind0IngressAdapter, bytes32 kind0IngressAdapterRuntimeHash,
    uint256 fixedIngressWei, uint256 executionWeiPerAccountedGas,
    uint256 proofWeiPerAccountedGas, uint256 permanentWeiPerByte,
    uint256 maximumAcceptedFeeWei
235..251 execution:
    uint64 l2BlockGasLimit, uint64 baseFeeElasticity,
    uint64 baseFeeChangeDenominator, uint64 blobTarget,
    uint64 blobUpdateFraction, bytes32 emptyOmmersHash,
    bytes32 emptyTransactionsRoot, bytes32 emptyWithdrawalsRoot,
    bytes32 emptyRequestsHash, uint64 l1HistoryFirstSupportedBlock,
    bytes32 l2HistoryStorageRuntimeHash,
    uint64 l2HistoryStorageActivationBlock, bytes32 headerRulesHash,
    bytes32 systemTransactionRulesHash, bytes32 forcedInputRulesHash,
    bytes32 stateTransitionAbiHash, bytes32 legacyExecutionRulesHash
252 bytes targetCodeArtifact offset (=8096)
```

Every unused high byte of `uint8/16/32/64`, every unused high byte of an address, and every unused low byte of `bytes4` is zero. Every field is nonzero except the three explicitly zero-permitted economic words 102–104. `schemaVersion=2`; the fixed geometry is `(1,384,4,4,64, 12,1024,2,2100,8,6,256,10,2)`, followed by the canonical point-evaluation address `0x0a`, gas 50000, BLS modulus, blob gas 131072, maximum payload 126972 and chunk cap 9. Words 114/115/116/117 must equal words 152/153/150/157 respectively; disagreement is rejected, so there is one gas authority. Word 54 is derived, not a publisher-selected label:

```
compileTimeRulesBytes = concat(profileWord[118],...,profileWord[137])
targetCompileTimeRulesHash = H(
  "slot-chain-target-compile-time-rules-v2" || u16(640) ||
  compileTimeRulesBytes)
```

The concatenation is exactly the 20 canonical ABI words in the displayed order. A bounded word-table/assembly decoder is permitted and preferred; compiler-generated decoding of a 252-field tuple is not an implementation assumption. Fixed tuple arity means unknown, missing, duplicated or reordered fields cannot be represented canonically.

Word 57 is fixed to the L1 EIP-2935 system address `0x0000F90827F1C53a10cb7A02335B175320002935`;

word 58 and the independent L2 runtime word 245 are each fixed to the Keccak-256 hash of the normative 83-byte runtime, `0x6e49e66782037c0555897870e29fa5e552daf4719552131a0abce779daec0a5d`; mere equality between publisher-selected hashes is insufficient. Word 59 is exactly

```
H("slot-chain-eip2935-read-config-v1" ||
  address20(0x0000F90827F1C53a10cb7A02335B175320002935) ||
  0x6e49e66782037c0555897870e29fa5e552daf4719552131a0abce779daec0a5d ||
  u64(l1HistoryFirstSupportedBlock) || u64(8191) || u64(50000)) .
```

Word 244 is that nonzero canonically padded uint64 L1 first-supported block; it is not an address and changing it without recomputing word 59 rejects. The L2 history address is a schema constant equal to the same EIP-2935 system address, not another profile authority. Word 246 is only the nonzero release-bound L2 activation block and need not equal the L1 boundary; word 7 MUST be at least word 246. The protocol-root Router has one immutable `l1HistoryFirstSupportedBlock`; its configuration commitment includes that uint64 and the complete word-59 read configuration. Before its first REGISTER_RELEASE write the Router compares that immutable uint64 with decoded word 244 and rejects disagreement. The target constructor stores word 244, and every target-side history consumer loads that stored value as its lower bound; it has no second default or replaceable adapter boundary. Thus a later release may name a different boundary only after an independent protocol-root migration installs a Router with the same value. Both Router and target reject a zero history cell or any request below that bound or outside the 8191-block window.

Authenticated code artifact and constructor. The inner dynamic bytes have exactly

```
u32(creationLength) || creationCode || u32(runtimeLength) || runtimeCode
```

with no suffix. Both lengths are positive, `H(creationCode)=targetCreationCodeHash`, `H(runtimeCode)=targetRuntimeHash`, `runtimeLength<=24576`, and the final init code below is at most 49152 bytes. The two metadata commitments are exactly

```
targetImmutableReferencesHash =
  H("slot-chain-solc-immutable-references-v1" || u32(0))
targetLinkReferencesHash =
  H("slot-chain-solc-link-references-v1" || u32(0))
```

and the pinned build manifest must independently report empty Solidity `immutableReferences`, empty `linkReferences`, no library placeholder and byte-for-byte deployed runtime equal to `runtimeCode`. Empty compiler metadata alone is insufficient: the release verifier proves for all accepted constructor arguments that the only constructor RETURN path emits those exact runtime bytes; handwritten assembly/Yul runtime patching is forbidden.

`TargetParametersV2` is exactly the 252 fixed value words above, in the same order and canonical padding, excluding only word 252 and the dynamic tail. The constructor tail is exactly

```
abi.encode(TargetParametersV2) || executionProfileHash ||
migrationActivationProfileRecordHash || targetRuntimeHash || targetArtifactHash
```

or 256 words/8192 bytes. Consequently `creationLength<=40960` and `initCode=creationCode||constructorTail`. The constructor reads this fixed suffix, requires zero value, `msg.`

`sender==settlementFactory`, and `block.chainid==settlementChainId`; it recomputes every hash and stores the exact initial inventory below. Let `P=keccak256("slot-chain.target.parameters.storage.v2")`; it writes profile fixed word i to slot `P+i` for every $0 <= i < 252$. It then writes `executionProfileHash`, `migrationActivationProfileRecordHash`, `targetRuntimeHash`, `targetArtifactHash`, `targetParametersHash`, `dataSessionConfigurationHash` and `targetConfigurationHash` to the seven ERC-7201-style slots obtained by Keccak of those exact lower-camel names prefixed with `"slot-chain.target.v2."`. It performs no other SSTORE and no TSTORE; all mappings, guards, counters, queues, sessions and migration fields therefore remain canonical zero. The constructor-poststate commitment is

```
H("slot-chain-target-constructor-poststate-v2" || u16(252) ||
  abi.encode(TargetParametersV2) || executionProfileHash || MPR2Hash ||
  targetRuntimeHash || targetArtifactHash || targetParametersHash ||
  dataSessionConfigurationHash || targetConfigurationHash)
```

and the build/storage proof checks exactly that inventory and non-collision of all 259 slots against the published storage layout.

The constructor environment-opcode allowlist is exact: CALLER is used only for the factory equality, CALLVALUE only for zero, CHAINID only for chain equality, and ADDRESS only in the two committed configuration derivations. All other environment, foreign-account, transient-state or external-effect observations are forbidden, including ORIGIN, GASPRICE, BALANCE, SELFBALANCE, EXTCODESIZE, EXTCODECOPY, EXTCODEHASH, BLOCKHASH, COINBASE, TIMESTAMP, NUMBER, PREVRANDAO, GASLIMIT, CHAINID outside the equality, BASEFEE, BLOBHASH, BLOBBASEFEE, GAS, CALLCODE, CALL, STATICCALL, DELEGATECALL, CREATE, CREATE2, SELFDESTRUCT, TLOAD and TSTORE. The build verifier proves the reachable init-code opcode trace obeys this allowlist and that its sole RETURN emits `runtimeCode`; a blacklist claim or source-code inspection is insufficient. Its returned runtime is argument- and environment-independent.

The target exposes the exact no-argument view

```
targetConstructorStateV2() returns
 (bytes4 magic,bytes32 constructorPoststateCommitment,
  bytes32 targetConfigurationHash)
```

with selector `0x654f7fce`, exactly four calldata bytes, zero value, the MPR2 `migrationPostStateReadGas` stipend, exactly 96 return bytes and magic `0x54435332` (TCS2) in canonically padded word zero. It recomputes the 259-word constructor inventory commitment from its own stored fixed inventory; it accepts no profile, witness or slot list and returns no cached caller-supplied commitment. PVM and Router independently derive the expected inventory and configuration from the strict raw profile. The authenticated constructor trace proves exactly those 259 SSTORE operations, no other SSTORE/TSTORE, and the authenticated runtime has no PRE-ACTIVE mutation surface; hidden-map emptiness therefore follows from creation provenance rather than an impossible mapping enumeration.

Immediately after CREATE2, at registration, and after MCAN but before Router writes ACTIVE, the caller checks EXTCODEHASH and the already normative fixed-gas `componentConfigHashV2()` exact 32-byte getter against the independently derived `targetConfigurationHash`. It also exact-reads the existing 384-byte `dataSessionAccountingV1()` and requires its configuration word to equal the profile derivation and every counter, liability, generation, deadline and guard word to be canonical zero. Word 111 componentConfigurationReadGas is required to equal the protocol constant 50,000, so activation need not recover the full profile; the accounting stipend is MPR2's migrationPostStateReadGas. Short/trailing/bad-padding, revert or mismatch rolls the whole journal back.

The artifact descriptor is nine 32-byte words in this order: creation-code, runtime, ABI, storage-layout, constructor-schema, compiler-build, compile-time-rules, immutable-references and link-references hashes, and

```
targetArtifactHash = H("slot-chain-settlement-artifact-v2" || u16(288) ||
                       those nine words)
targetParametersHash = H("slot-chain-target-parameters-v2" || u16(8064) ||
                         abi.encode(TargetParametersV2))
```

The four target build-metadata hashes are not opaque labels. The release bundle publishes their four preimage files and hashes exact bytes:

```
targetAbiHash = H("slot-chain-target-abi-v2" || u32(abiBytes.length) || abiBytes)
targetStorageLayoutHash = H("slot-chain-target-storage-layout-v2" ||
                            u32(layoutBytes.length) || layoutBytes)
targetConstructorSchemaHash = H("slot-chain-target-constructor-schema-v2" ||
                                u32(schemaBytes.length) || schemaBytes)
targetCompilerBuildHash = H("slot-chain-target-compiler-build-v2" ||
                            u32(buildBytes.length) || buildBytes)
```

abiBytes is the RFC 8785 JCS UTF-8 encoding (no BOM/newline) of the complete solc ABI JSON array; layoutBytes is the same encoding of the complete solc storageLayout object. schemaBytes is UTF-8, with no BOM/newline, of the exact ASCII literal TargetConstructorV2(ExecutionPro ,bytes32,bytes32,bytes32,bytes32). buildBytes is RFC 8785 JCS of exactly the object keys solcVersion, solcLongVersion, evmVersion, optimizerEnabled, optimizerRuns, viaIR, metadataBytecodeHash, metadataAppendCBOR, solcBinaryHash, sourceTreeHash and standardJsonInputHash, targetContractSourcePath and targetContractName; unknown or missing keys reject. JSON strings/numbers/booleans have RFC 8785 semantics and each file is at most 1,048,576 bytes. The pinned build verifier regenerates the four target byte strings from the standard-json input and compares every hash before registration. The version/EVM/metadata values, all named hashes, the target source path and target contract name are JSON strings; optimizerEnabled, viaIR and metadataAppendCBOR are JSON booleans; optimizerRuns is an integer in $[0, 2^{32} - 1]$. The three inner hashes are lowercase 0x-prefixed 32-byte hex and are exactly

```
solcBinaryHash = H(solc executable bytes)
standardJsonInputHash = H("slot-chain-solc-standard-json-v1" ||
                          u32(inputBytes.length) || inputBytes)
sourceTreeHash = H("slot-chain-source-tree-v1" || u32(entryBytes.length) ||
                   entryBytes)
entryBytes = concat, in unsigned UTF-8 bytewise path order, of
             u16(path.length) || path || u32(content.length) || content
```

inputBytes is the no-BOM/no-newline RFC 8785 JCS encoding of the complete standard-json input. Each source path is a nonempty relative POSIX UTF-8 path in Unicode NFC with neither ., .., empty segment, backslash nor leading slash; duplicates reject. Content is the exact source bytes named by the standard-json input (no newline or Unicode normalization), each path is at most 65535 bytes and the concatenation at most 16,777,216 bytes. Every standard-json `sources[path]` object contains exactly one `content` string and no `urls`; every import resolves to another entry in that same source map. The verifier invokes the pinned solc binary in an empty directory with only `--standard-json`, no base/include/allow paths, no import callback, no filesystem fallback and no network. The target source path obeys the same path grammar and exists in `sources`; the target contract name is a nonempty ASCII Solidity identifier. Creation code, runtime code, ABI, storage layout, immutable/link-reference tables and their hashes must all come from the single output object `contracts[targetContractSourcePath][targetContractName]`. Mixing fields from different source or contract entries, selecting by a suffix or using a different compiler invocation rejects. ERC-2470 is a protocol-root dependency, not a publisher-built artifact and therefore does not appear in this target compiler manifest. The mandatory off-chain conformance tool emits a canonical binary report in addition to its human-readable diagnostics:

```
buildConformanceReportHash = H(
 "slot-chain-build-conformance-report-v1" || u16(481) || u8(1) ||
 verifierExecutableHash || standardJsonInputHash || sourceTreeHash ||
 executionProfileHash || migrationActivationProfileRecordHash ||
 targetCreationCodeHash || targetRuntimeHash || targetArtifactHash ||
 targetParametersHash || targetConfigurationHash ||
 constructorPoststateCommitment || targetAbiHash ||
 targetStorageLayoutHash || targetConstructorSchemaHash ||
 targetCompilerBuildHash)
```

All 15 hashes are nonzero. The release repository publishes the verifier executable bytes, complete input bundle, binary report and diagnostics; the DAO proposal metadata names this report hash before queueing. Two independently built verifier binaries must reproduce the same report and all profile outputs. The report is an auditable governance-release artifact, not an on-chain proof: REGISTER_RELEASE remains secure against untrusted executors and bytecode/state substitution, while deliberate authorization of malicious bytecode is the governance-root failure stated in Section 2. The deployment/configuration graph is then exactly

```
initCodeHash = H(creationCode || constructorTail)
target = last20(H(0xff || address20(settlementFactory) ||
                  settlementSalt || initCodeHash))
dataSessionConfigurationHash = the 340-byte formula in Section 3
targetConfigurationHash = H("slot-chain-settlement-config-v2" || u16(212) ||
  address20(target) || targetRuntimeHash || executionProfileHash ||
  migrationActivationProfileRecordHash || targetParametersHash ||
  targetArtifactHash || dataSessionConfigurationHash)
```

The target, profile, MPR2, PIA2 rows/root, configuration, deployment descriptor, manifest, registration and predecessor are derived outputs and MUST NOT occur inside the profile. Registration recomputes this DAG from profile bytes and compares all eight deployment words, the manifest chain/version/core/component joins, exactly two independently reconstructed PIA2 rows and their root. It also recomputes every source CREATE2/config/domain and destination component, infrastructure/Bridge/domain relation; a getter or just-written Router row is not an independent expected value.

The Settlement deployer is the protocol-root ERC-2470 Singleton Factory, not a release-publisher contract. The root fixes this complete identity:

```
factoryAddress = 0xce0042B868300000d44A59004Da54A005ffdcf9f
singleUseDeployer = 0xBb6e024b9cFFACB947A71991E386681B1Cd1477D
factoryCreationCodeBytes = 340
factoryCreationCodeHash =
  0x122b6b28aeddfd05fa3ce4348e93d357b3ce50d9ab7dda4e8ee524a5b9a6ab3b
factoryRuntimeBytes = 308
factoryRuntimeHash =
  0xc4d5542b53a8b779595a20a8ddd60e58a6c49d3c3decc2df83ced1c69c8ca807
factoryDeploymentTransactionHash =
  0x803351deb6d745e91545a6a3e1c0ea3e9a6a02a1a4193b70edfcd2f40f71a01c
factoryCreationCode =
  hex("608060405234801561001057600080fd5b50610134806100206000396000f3fe")
  || factoryRuntimeCode
factoryRuntimeCode = hex(concat(
 "6080604052348015600f57600080fd5b506004361060285760003560e01c8063",
 "4af63f0214602d575b600080fd5b60cf60048036036040811015604157600080",
 "fd5b810190602081018135640100000000811115605b57600080fd5b82018360",
 "2082011115606c57600080fd5b80359060200191846001830284011164010000",
 "000083111715608d57600080fd5b91908080601f016020809104026020016040",
 "5190810160405280939291908181526020018383808284376000920191909152",
 "50929550509135925060eb915050565b604080516001600160a01b03909216825251",
 "9081900360200190f35b6000818351602085016000f5939250505056fea26469",
 "706673582212206b44f8a82cb6b156bfcc3dc6aadd6df4eefd204bc928a4397",
 "fd15dacf6d5320564736f6c63430006020033"))
```

The root release bundle also publishes the exact raw non-EIP-155 deployment transaction below; its bytes, recovered sender, creation code, resulting address and transaction hash MUST all match the tuple above before a chain is supported:

```
hex(concat(
 "f9016c8085174876e8008303c4d88080b9015460806040523480156100105760",
 "0080fd5b50610134806100206000396000f3fe6080604052348015600f576000",
 "80fd5b506004361060285760003560e01c80634af63f0214602d575b600080fd",
 "5b60cf60048036036040811015604157600080fd5b8101906020810181356401",
 "00000000811115605b57600080fd5b820183602082011115606c57600080fd5b",
 "80359060200191846001830284011164010000000083111715608d57600080fd",
 "5b91908080601f01602080910402602001604051908101604052809392919081",
 "8152602001838380828437600092019190915250929550509135925060eb91505056",
 "5b604080516001600160a01b039092168252519081900360200190f35b600081",
 "8351602085016000f5939250505056fea26469706673582212206b44f8a82cb6",
 "b156bfcc3dc6aadd6df4eefd204bc928a4397fd15dacf6d5320564736f6c6343",
 "00060200331b83247000822470"))
```

The only deployment wire used by a release is

```
deploy(bytes initCode,bytes32 salt) returns (address target)
```

```
selector = 0x4af63f02
calldata = selector || u256(0x40) || salt || u256(initCode.length) ||
           initCode || zeroPaddingToWord
returndata = bytes12(0) || address20(target) // exactly 32 bytes
```

Calldata has no suffix and value is zero. The factory passes the supplied bytes unchanged to `CREATE2(0,initCode,salt)`. The permissionless release workflow first recomputes target from the root factory, salt and init-code hash. If target has no code, any account may call the factory and MUST require the exact 32-byte returned address, then exact-check target EXTCODEHASH, configuration, TCS2 and DSV1. If target already has code, the workflow skips the factory call and performs the same exact postreads. A CREATE2 collision returns zero under this runtime; zero/wrong returndata or any postread mismatch fails deployment. Idempotence therefore belongs to the external workflow, not to a fictitious factory replay branch. This external transaction is not a nested protocol call; the release publishes its measured whole-transaction gas ceiling.

Profile words 44 and 45 MUST equal the root address and runtime hash. Word 46 is independently recomputed as

```
settlementFactoryConfigurationHash = H(
 "slot-chain-erc2470-factory-config-v1" || u16(147) ||
 address20(factoryAddress) || factoryRuntimeHash ||
 factoryCreationCodeHash || factoryDeploymentTransactionHash ||
 address20(singleUseDeployer) || bytes4(0x4af63f02) ||
 u32(49152) || u16(32) || u8(1))
```

where the final three fields commit the maximum accepted init code, exact return width and zero-value-only rule. PVM and Router compile and enforce this triple; neither trusts a self-reporting factory getter or publisher-selected behavior cases. REGISTER_RELEASE cannot rotate it. Changing any root tuple member requires the independent protocol-root migration. Registration safety then depends on the independently derived CREATE2 address and exact target code, configuration, constructor-state and zero-accounting reads; an absent or wrong factory makes a release unavailable, never accepted.

Release conformance rejects the former `a1617601` and A4 `a4000101400241000380` CBOR fixtures even if a caller rehashes them. It also gates deployed PVM, Router and target runtime at no more than 24576 bytes, target creation at no more than 40960 bytes, final initcode at no more than 49152 bytes, and publishes cold-state maximum-profile deployment and nested Timelock–PVM–Router registration gas measurements. Every fixed call budget includes EIP-150 forwardability and a named post-call reserve; no unmeasured 65536-byte profile is implementation ready.

Every profile contains exactly one immutable `MigrationTransitionVerifierDescriptorV2`:

```
struct MigrationTransitionVerifierDescriptorV2 {
  address verifier; bytes32 runtimeHash; bytes32 configurationHash;
  bytes32 verifyingKeyHash; bytes32 proofSystemId;
  bytes32 publicInputSchemaHash; bytes4 selector;
  uint32 maximumProofBytes; uint64 verificationGasLimit;
}
```

All fields are nonzero, `selector` equals `bytes4(keccak256("verifyMigrationTransition(bytes, uint256[2])"))`, and the two hash layers are acyclic and byte exact. First, `configurationHash` is the canonical verifier-instance configuration:

```
MIGRATION_VERIFIER_CONFIG_TYPE_LITERAL =
  "MigrationTransitionVerifierConfigV2(bytes32 verifyingKeyHash,"
  "bytes32 proofSystemId,bytes32 publicInputSchemaHash,bytes4 selector,"
  "uint32 maximumProofBytes,uint64 verificationGasLimit)"
MIGRATION_VERIFIER_CONFIG_TYPEHASH = keccak256(
  concat(the three quoted fragments))
configurationHash = keccak256(abi.encode(
  MIGRATION_VERIFIER_CONFIG_TYPEHASH, verifyingKeyHash, proofSystemId,
  publicInputSchemaHash, selector, maximumProofBytes, verificationGasLimit))
```

It deliberately excludes `verifier`, `runtimeHash` and itself; the immutable verifier getter is exactly

```
migrationVerifierConfigHashV2() external view returns (bytes32)
MIGRATION_VERIFIER_CONFIG_READ_GAS = 50000
```

Its selector is the first four Keccak bytes of `"migrationVerifierConfigHashV2()"`. PVM and Router first require `EXTCODEHASH(verifier)==runtimeHash`, then issue a zero-value `STATICCALL` with exactly those four calldata bytes, no suffix and the fixed 50,000-gas stipend. Success returns exactly one 32-byte word equal to `configurationHash`; empty, short, trailing, faulting or mismatched returns reject before any state write. Second,

```
MIGRATION_VERIFIER_DESCRIPTOR_TYPE_LITERAL =
  "MigrationTransitionVerifierDescriptorV2(address verifier,"
  "bytes32 runtimeHash,bytes32 configurationHash,bytes32 verifyingKeyHash,"
  "bytes32 proofSystemId,bytes32 publicInputSchemaHash,bytes4 selector,"
  "uint32 maximumProofBytes,uint64 verificationGasLimit)"
MIGRATION_VERIFIER_DESCRIPTOR_TYPEHASH = keccak256(
  concat(the four quoted fragments))
migrationVerifierDescriptorHash = keccak256(abi.encode(
  MIGRATION_VERIFIER_DESCRIPTOR_TYPEHASH, verifier, runtimeHash,
  configurationHash, verifyingKeyHash, proofSystemId,
  publicInputSchemaHash, selector, maximumProofBytes,
  verificationGasLimit))
```

Each type literal is the byte concatenation of its displayed fragments with no inserted whitespace or newline. These are ordinary canonical ABI words: address occupies the low 20 bytes; `bytes4` occupies the high four bytes; unsigned integers occupy the low bytes; every unused byte is zero. The profile decoder recomputes both layers and rejects noncanonical padding or any mismatch. PVM independently recomputes them while validating the complete manifest and requires its `migrationVerifierDescriptorHash` to equal the second layer. The Router repeats those checks immediately before bounded verifier dispatch and uses only the descriptor recovered from the pinned profile. The profile hash, release manifest and append-only historical registration therefore commit one identical descriptor; no constructor default, caller-supplied descriptor or global verifier exists. The verifier's separate implementation configuration hash is not a synonym for the descriptor hash. A successor may rotate the descriptor only by activating a new profile and release. Golden/reject vectors replace each field in turn and cover zero values, short/long getter returns, caller-substituted configuration, nonzero narrow-field padding and the former self-referential fixed-point construction; all must reject except the one exact canonical vector. The same profile records nonzero `supportedL1BlockGasLimit`, `worstCaseActivationAdoptionGas`, `sourceFreezeGasLimit`, `targetAdoptionGasLimit`, `queueMigrationGasLimit`, `activationContextReadGasLimit`, `postStateReadGasLimit`, `legacyStateReadGasLimit`, `legacyArmGasLimit`, `legacyFinalizeGasLimit` and `postCallbackReserveGas`. These values are part of the exact executable profile bytes/hash and cannot be supplied by a caller or changed in-place. The Router copies at most the one exact ABI return length for each call and discards arbitrary revert data; no callee can force unbounded memory expansion. `worstCaseActivationAdoptionGas` is the measured cold-state upper bound for the *complete* L1 Router execution after transaction intrinsic gas: canonical ABI decode/re-encode and allocation, every cold Router/Queue/history/profile/manifest read, statement and transaction hashing, `EXTCODEHASH`, the configuration getter and its call overhead, verifier dispatch consuming the full `verificationGasLimit` including EIP-150 and memory/call overhead, and every post-verifier callback/write in the atomic Router/Queue/old-Settlement/new-Settlement/seat/receipt suffix. It is not a post-verifier-only estimate or a governance number detached from the release bytecode. A benchmark that omits any prefix, call overhead or suffix cannot populate this field.

Before each fixed-gas call, Router requires enough gas for the requested stipend under EIP-150 plus the sum of all remaining call stipends, bounded return-copy and `postCallbackReserveGas`. In particular the call-local available gas after memory and CALL/STATICCALL charges is at least `ceil(64*stipend/63)`, so a syntactically requested full stipend is actually forwardable. The reserve covers all three MAPS reads, receipt/index and target- registration writes, final ACTIVE/context clearing and IDLE. The release harness pins cold/warm access and compiler/fork call overhead, succeeds at each exact threshold, fails at one gas less, and injects full-stipend consumption and late OOG at every MFRZ/LGFN, MCAN, QMIG, MACT and MAPS boundary. A compiler, fork or bytecode change invalidates the measurements until republished.

The public-input schema is the exact static Solidity tuple below. The two core hashes are the Appendix canonical hashes of the complete supplied base and output `CanonicalCore`; neither hides a caller-selected partial tuple.

```
struct MigrationTransitionStatementV2 {
  uint256 settlementChainId;
  address activeSettlementRouter;
  bytes32 routerRuntimeHash; bytes32 routerConfigurationHash;
  uint8 transitionKind; uint64 migrationGeneration; uint64 sourceProtocolVersion;
  uint64 targetProtocolVersion; uint64 sourceCanonicalSequence;
  bytes32 executionProfileHash; bytes32 targetManifestHash;
  bytes32 targetRegistrationHash;
  bytes32 candidateDigest; bytes32 baseCanonicalHash;
  bytes32 outputCanonicalHash;
  address forcedQueue; bytes32 queueRuntimeHash;
  bytes32 queueConfigurationHash; bytes32 queueRoot;
  uint64 queueCount; uint64 startCursor; uint64 endCursor;
  bytes32 forcedDescriptorCommitment; address proofBeneficiary;
  uint256 anchorNumber; bytes32 anchorHash;
```

```
  bytes32 forceRoot; uint64 forceCutoff;
  bytes32 sourceDomainId; uint64 sourceRegistrationEpoch;
  bytes32 sourceBridgeExecutionHash;
  bytes32 releaseSystemCalldataHash; bytes32 inboxSystemCalldataHash;
  bytes32 releaseSystemTxHash; bytes32 inboxSystemTxHash;
  uint8 releaseSystemTxPosition; uint8 inboxSystemTxPosition;
  bytes32 importedHeaderHash; bytes32 importedStateRoot;
  bytes32 legacySignalCheckpointHash;
  bytes32 legacyDeploymentHash; bytes32 legacyArmId;
  bytes32 legacyLaunchId;
  bytes32 deploymentCommitment;
  uint64 preInboxLastAppliedPlusOne;
  uint64 postInboxLastAppliedPlusOne;
}
```

The typehash is Keccak of the exact canonical Solidity signature with the field names and order shown, and `statementHash=keccak256(abi.encode(TYPEHASH,fields...))`. `publicInputSchemaHash` is Keccak of that same signature. The plus-one cursor words use zero only for an uninitialized lifetime InboxApply router and otherwise encode the checked block number plus one. L1 derives every L1-authoritative field from current Router/Queue/history state or exact bounded calldata. `transitionKind` is exactly GENESIS_IMPORT=1 or VERSION_MIGRATION=2. For GENESIS_IMPORT, the imported header hash must equal the legacy Inbox's stable finalized-block hash authenticated by the exact QUIESCENT LGS1 and transaction-local LGAR; its state root equals `importedStateRoot`. The checkpoint hash is exactly `H("slot-chain-legacy-checkpoint-v1"||u48(header.number)||importedHeaderHash||importedStateRoot)` and the circuit proves the legacy SignalService record at that height has those values. The same genesis statement binds the registered compatibility proxy's nonzero `legacyDeploymentHash`, campaign/scan-bound `legacyArmId` and target-derived `legacyLaunchId`. For VERSION_MIGRATION, `importedHeaderHash`, `importedStateRoot` and `legacySignalCheckpointHash`, `legacyDeploymentHash`, `legacyArmId` and `legacyLaunchId` are all canonical zero; the base core and state root are derived exclusively from Router's exact sequence-tagged current Canonical. Mixing a legacy witness into a later migration, or zeroing one for launch, rejects. The sole deployment public input is derived as follows:

```
componentsHash = keccak256(abi.encode(manifest.components))
prestatePolicyHash = H("slot-chain-deployment-prestate-policy-v2" ||
                       u8(transitionKind))
poststatePolicyHash = H("slot-chain-deployment-poststate-policy-v2" ||
                        u8(transitionKind))
DEPLOYMENT_COMMITMENT_TYPEHASH = keccak256(
  concat(
   "DeploymentCommitmentV2(uint8 transitionKind,",
   "uint64 targetProtocolVersion,bytes32 targetManifestHash,",
   "bytes32 targetRegistrationHash,",
   "bytes32 destinationInfrastructureHash,bytes32 componentsHash,",
   "bytes32 poolConfigurationHash,uint64 retirementQueueCount,",
   "bytes32 prestatePolicyHash,bytes32 poststatePolicyHash)"))
deploymentCommitment = keccak256(abi.encode(
```

```
  DEPLOYMENT_COMMITMENT_TYPEHASH, transitionKind, targetProtocolVersion,
  targetManifestHash, targetRegistrationHash,
  manifest.destinationInfrastructureHash,
  componentsHash, manifest.poolConfigurationHash, queueCount,
  prestatePolicyHash, poststatePolicyHash))
```

`componentsHash` encodes the ten three-word rows in manifest order with no dynamic offset. Router derives every field and supplies the same one hash as both statement input and verifier expectation; there is no caller-selected or "observed" hash. The type literal is the single concatenation shown, with no display newline or extra whitespace.

For GENESIS_IMPORT, the prestate-policy tag means exact lifetime code/config, empty Router routes/cursor, empty release/domain/writer records, accumulator empty root/count/frontier, Pool zero ticket/accounted total, fresh Store/Quota/Bridge private state and arbitrary forced ETH surplus. For VERSION_MIGRATION it means exact lifetime code/config and byte-for-byte preservation of every old route/release/domain/writer, Inbox cursor, accumulator frontier/root/count and Pool ticket/accounting word, with only the new release/domain/writer/route absent and the new Store/Quota/Bridge fresh. Both poststate tags mean the exact tx0 manifest/count is sealed, only those new append-only entries and endpoint activation are added, all old lifetime state is preserved, no Pool ticket is created and no ETH is moved or minted. The circuit proves each enumerated account/code/storage and balance relation directly under authenticated pre/post roots and proves the complete output core. It also proves the deployment policy encoded by the single `deploymentCommitment` holds, `queueCount=retirementQueueCount`, positions are zero and one, and the two transaction hashes are legacy Keccak of the exact type-`0x7f` RLP envelopes reconstructed from the separately bound calldata hashes, fixed targets, kind, nonce, gas, zero value and destination chain ID.

Verifier calldata is exactly

```
verifyMigrationTransition(bytes proof, uint256[2] digestLimbs)
    external view returns (bytes32 statementHash)
```

with ABI head `[0x60,high128(statementHash),low128(statementHash)]`, then the canonical bytes length/data/padding tail. Proof length is at most the descriptor bound. ActiveSettlementRouter makes one descriptor-gas-bounded `STATICCALL`, after checking `EXTCODEHASH` and configuration, and accepts exactly 32 return bytes equal to its reconstructed `statementHash`. The circuit recombines both unreduced 128-bit limbs and proves this exact typed preimage. Revert, OOG, short/trailing return, wrong hash, substituted code/key/schema or a Boolean-shaped result rejects before any state write. The target Settlement never receives a verifier address, result capability or callback; its authority begins only after Router validates and atomically adopts the exact result.

The commitment dependency graph is acyclic and normative:

```
kernel/component configs/runtime descriptors
  -> Bridge execution/infrastructure/domain
  -> canonical profile bytes -> executionProfileHash
  -> ReleaseManifestV2 -> releaseManifestHash
  -> destination registration commitment
```

The profile may contain the manifest schema version, manager/authority/registrar addresses, namespaces, ABI and activation/validation rules. It MUST NOT contain the release-manifest hash, registration commitment, destination-registration commitment, or any field derived from them; otherwise decoding rejects rather than attempting a hash fixed point. In particular it contains one nonzero `poolNamespace`, distinct from the router, manifest and destination namespaces. That primitive profile value is an input to component-9 `poolConfigurationHash`; it is not derived from the Pool address, manifest or destination domain. The Pool address/configuration then participates in the infrastructure and domain hashes in the direction shown above, so either side can independently reconstruct the graph without a fixed point.

- settlement/L2 chain IDs, both genesis hashes, fork digest, EIP-2935 address and protocol-lifetime `firstV2BlockNumber`; BuilderRegistry, ScheduleOracle, its protocol-lifetime schedule-rules hash and every fork-verifier address/code/config/gas tuple, domain-registry and version-manager addresses, runtime hashes and exact immutable/configuration commitments; the legacy resume root, non-proxy fixed-key RISC0/SP1 adapters, their four immutable key-policy values, recursively closed remote-verifier dispatch graphs, and the direct legacy SignalService proxy/implementation/layout/authority fence (with ForkRouter rejected); every append-only source, destination and support descriptor and manifest schema/validation binding; each nonzero domain namespace, fresh immutable SourceBridge/Registry/Quota and DestinationBridge/Store/Quota bundle runtime/config/layout, CREATE2 factory/salt/init-code derivation, and the ban on every V2 proxy, owner, upgrade and delegate target, terminal verifier, immutable settlement router, its append-only ingress authorization rules, immutable ForcedQueue runtime and exact configuration hash, the kind-0 ingress adapter and Bridge inbox adapter, InboxApply router, InboxCredit Store, release authority, terminal registrar and accumulator runtime plus every constructor immutable, infrastructure hash, ABI, CreditRecord/liability/status/index layouts and direct-call gas limit;
- gas limit, EIP-1559 elasticity/change denominator, blob target/update fraction, empty ommers/withdrawals/requests constants and exact derivation of base fee, blob fields, logs bloom, randomness, parent beacon root, requests hash and every post-fork header field;
- the Anchor, InboxApplyRouterV2, InboxCreditStoreV2, the release authority, terminal registrar and accumulator, and both legacy SignalService contracts, fresh source/destination Bridge bundles, Bridge-unique quota managers and NativeLiquidityPoolV2 address/runtime/configurati every DIRECT-only V2 selector and ABI tuple; the unsigned custom system transaction type and unforgeable reserved senders; system nonce rules, fee exception, chain-ID equality, caller/origin injection, warm set, intrinsic gas, refund rule, zero GASPRICE, account non-transition, exact typed trie values, one-shot Store/Bridge seal and gas ceilings;
- the exact AnchorV4 checkpoint/ancestors transition, forced disposition rules, InboxApplyRouterV2 global cursor, route and batch transition, immutable store inbox slot, result and deadline storage, Bridge V2 aggregate and per-credit liability, both endpoint statuses, terminal indices, vector count, root, 64-word terminal frontier and full leaf event, pull credits, cancellation, expiry, delivery and recall transitions and the sole inbox-pin path; and
- at least six complete vectors, each covering parent, prestate, input, transaction, receipt, header and poststate: empty escape, valid kind 0, expired kind 0 without bytes, kind 1 under two separately registered immutable source generations, source and destination domain replacement, direct-record absence and mismatch, enqueue and cancel ordering, router-SYNCED refund and retry, capacity race, delayed append, PREACTIVE kind 0 and

kind 1 rejection, legacy attempts to invoke inbound V2, first post-cutover release activation, pin survival across arbitrary legacy SignalService upgrades, endpoint-support manifest authorization/finality and finalized L2 registration proof, squatting/unauthorized/out-of-order registration, mixed D1/D2/D1 routing, old-domain enqueue after new-domain activation, missing-route atomic rollback, multi-credit batch ordering/conflict, Message loss before enqueue and after pin, deadline expiration, DONE/FAILED terminal replay and a cap-limited multi-block prefix, plus extra-reserved-sender, unauthorized clone, mismatched/local-foreign domains, pooled-balance drain attempts, zero ordinary quota during V2 refund, legacy SignalService pause during V2 terminal proof, forbidden upgrade/delegatecall, exact legacy-send compatibility, DIRECT-only V2 and rejected Vault callers, old-domain accumulator append after new-domain activation, canonical-sequence cell collisions, accumulator-target caller confusion, sentinel/wrong-credit/duplicate-append, terminal leaf/index/root substitution, historic-count proof reconstruction, unauthorized/fake/full-core-discontinuous router switches, same-block history writes, migration-ready anti-griefing, conflicting result/terminal hash and every V1/V2 boundary.

For escape, parent hash/height/state and next fee/blob scalars come from the stored canonical core; timestamp/slot, L1 origin and forced inputs come from the round. Every remaining header field and both system transactions are a pure profile function of those values and the execution result. The circuit loads the profile selected by the public protocol version and proves its hash equals Settlement's immutable mapping. Launch tooling rejects a version lacking the ABI artifact, verifier-key binding or complete vectors. Thus a profile change is a new protocol version and review, not runtime interpretation.

12.4 Reorg rule

The canonical L1 log is the journal. A reorg truncates and replays, in order: Router lifecycle/context and public gate; source freeze or legacy cutover phase; target canonical/history adoption; ForcedQueue cursor, payment claim and authority; all three activation poststate commitments; sealed receipt/indexes; then ordinary canonical tuple, mode/version, normal arm/activation/best/deadline/ frozen admission, episode/round, seat duties/services/selections, retained breach receipts, Market credits/reserves, data-session root/count and reward eligibility. These activation writes are one L1 transaction and therefore truncate together; no indexer treats an intermediate callback event as a committed switch. Off-chain workers index by `(settlementChainId,contract,blockHash,logIndex)` and discard orphaned jobs. Withdrawal/bridge execution waits for the configured L1 finality policy.

12.5 Genesis and migration

Deployment occurs at least `D_SNAP+F_FINAL+sealMargin` before activation, so first live sources are post-deployment and sealable. New Settlement begins `PREACTIVE`; registry and scheduling preparation plus off-chain data preparation may run, but target session open/post/seal, every candidate and every kind-0/kind-1 forced-ingress or bridge enqueue call rejects before creating target-local state, a deposit, adapter record or queue leaf.

The legacy Inbox does not share this design's queue or expose a safe sidecar freeze. A sidecar cannot intercept direct calls to the deployed proxy's old proposal and forced-inclusion selectors. GENESIS_IMPORT therefore requires the delayed legacy governance path to install one final `LegacyGenesisCutoverInboxV1` implementation in that same proxy. It has byte-identical legacy storage layout/configuration, consumes only a published collision-free gap slot, pins the non-proxy Router immutable, preserves every legacy event and bond-withdrawal path, and exposes no delegate target. Direct implementation calls reject `onlyProxy`. Its upgrade authorization is legal only in local ACTIVE and exact-reads LGC1 before authorizing. Any SCHEDULED campaign before either expiry bound blocks every proxy implementation upgrade, even before the admission cutoffs. A published campaign is immutable and noncancelable; the campaign-wide 600-block/7,200-second bound ends this fence through expiry and local resume. EXPIRED and a time-expired retained SCHEDULED tuple permit the ordinary delayed legacy upgrade path only after all local scan/campaign words are cleared through the independent permissionless resume described below. DRAINING, QUIESCENT, READY and FROZEN can never be bypassed by installing another implementation. A Router read failure, wrong tuple or implementation/profile mismatch fails closed rather than authorizing the upgrade. Router authenticates proxy/runtime, implementation address/runtime and the exact 17-word configuration during activation preflight and again at LGAR, LGS1 and LGFN. If an existing deployment cannot install this exact compatibility implementation, sound in-place GENESIS_IMPORT for that deployment is impossible; a separately initialized chain/state migration is required.

ProtocolVersionManager publishes at most one live delayed genesis campaign. The Router exposes its exact fail-closed view:

```
legacyGenesisCampaignV1() returns
 (bytes4 magic,uint8 status,uint64 nonce,uint64 generation,
  uint64 forceCutoffBlock,uint64 proposalCutoffBlock,
  uint64 quiesceNotBeforeBlock,uint64 resumeByBlock,
  uint64 resumeByTimestamp,uint64 reviewFinalizedByBlock,
  address targetSettlement,uint64 targetProtocolVersion,
  bytes32 targetManifestHash,bytes32 targetRegistrationHash,
  bytes32 reviewCommitment,bytes32 campaignId)
```

Its selector is `0x718b2ac7`, magic is `0x4c474331` (LGC1), and returndata is exactly 512 bytes. Status is NONE=0, SCHEDULED=1, CONSUMED=3 or EXPIRED=4; value 2 is reserved and rejects if observed. All high padding is zero. NONE requires every other word zero; every non-NONE status retains the same complete nonzero immutable tuple and campaign ID so proxy resume and reorg replay never depend on a cleared target. No behavioral or caller-supplied substitute is accepted. Let `publicationBlock=block.number`, `publicationTimestamp=block.timestamp` and `executeAfter` be the retained PCO1 value in the one EXECUTING PUBLISH_GENESIS_CAMPAIGN operation. Publication atomically enforces all of these checked inequalities:

```
publicationBlock <= reviewFinalizedByBlock
reviewFinalizedByBlock + 64 <= forceCutoffBlock
forceCutoffBlock + 64 <= proposalCutoffBlock
proposalCutoffBlock + 128 + 64 <= quiesceNotBeforeBlock
quiesceNotBeforeBlock < resumeByBlock
resumeByBlock - publicationBlock <= 600
executeAfter <= publicationTimestamp < resumeByTimestamp
resumeByTimestamp - publicationTimestamp <= 7200
```

```
resumeByTimestamp - executeAfter <= 7200
```

The operation payload carries every right-hand campaign bound, and PVM derives the two publication values from the executing transaction; no caller supplies or later refreshes them. Thus a queued operation that is executed too late rejects, and SCHEDULED cannot freeze upgrades or proposer/configuration authority for an arbitrarily distant cutoff. The `reviewFinalizedByBlock` is at least `F_FINAL_BLOCKS=64` blocks before `forceCutoffBlock`. Once the campaign publication transaction succeeds, neither governance nor a copied proof can change or cancel the target or campaign. Expiry tombstones that campaign ID; every later canonical attempt increments `nonce`, even when the target and generation are unchanged. A reorg instead removes the orphaned campaign and all of its scan/quiescence state together; proofs from it reject unless the identical campaign is canonically republished.

Before publishing any campaign, including a higher-nonce retry, ProtocolVersionManager additionally exact-staticreads LGS1 and LGSS from the compatibility proxy. LGS1 must be the canonical all-cutover-words-zero ACTIVE row; LGSS must be EMPTY with every campaign, endpoint, cursor, count, byte, root, value, expiry and profile word zero. Thus Router cannot replace the LGC1 tuple while an expired attempt still has a partial/complete scan or a QUIESCENT snapshot whose old campaign ID is needed for resume. Anyone first calls the old campaign's permissionless resume; a bad, short or stale local view blocks publication, not legacy operation.

The compatibility implementation exact-staticreads LGC1 before every selector that can change the five checkpoint boundary fields. At and after `forceCutoffBlock`, payable forced ingress rejects before retaining ETH or writing the tail. At and after `proposalCutoffBlock`, proposal admission rejects before consuming a forced record, transferring its fee, incrementing a proposal ID or storing a hash. Existing proof/finalization remains open until QUIESCENT, and historical bond paths never close. An LGC1 revert, OOG, wrong code, magic, length, padding, nonce, campaign ID or stage fails closed. Once either `block.number>=resumeByBlock` or `block.timestamp>=resumeByTimestamp`, checkpoint/activation rejects and admissions resume even if nobody first calls Router expiry. These comparisons make same-block ordering irrelevant: an ingress transaction in a cutoff block cannot precede the fence.

A SCHEDULED campaign also applies hard preparation caps immediately when its publication transaction completes. Publication rejects unless the unfinalized proposal count and pending forced count are each at most 1,024. Until their respective cutoffs, proposal/forced admission rejects before mutation if it would exceed either count or the codec-derived 4,194,304-byte total scan bound. A canonical row consumes at most its profile-pinned maximum encoded bytes, so this is checked without trusting a later caller. Scan batches contain exactly `min(16,end-cursor)` consecutive rows; a non-final short batch is invalid. Hence every successful call makes maximum progress and at most 128 successful scan calls cover both 1,024-row maximum ranges, even when an adversary front-runs a keeper. The immutable stage inequalities above provide `F_FINAL_BLOCKS` blocks from force cutoff to proposal cutoff and `128+F_FINAL_BLOCKS` blocks from proposal cutoff to `quiesceNotBeforeBlock`. Entering QUIESCENT additionally requires at least 128 L1 blocks and 1,800 seconds remaining; the campaign-wide interval is already hard-capped at 600 L1 blocks and 7,200 seconds. Thus one included scan batch per block completes before quiescence under the same L1 inclusion assumption as proof landing; governance cannot authorize an unbounded maintenance window.

After proposal cutoff, anyone scans the fixed unfinalized proposal interval and pending forced interval in bounded calls. Each proposal preimage must hash to its still-addressable ring cell; each source and direct forced row supplies its canonical blob timestamp/hash slice. Streaming domain-separated accumulators bind every index, stored hash and derived expiry in order. The scan starts from the then-current `lastFinalizedProposalId+1` and `forcedHead`, ends at the cutoff-frozen `nextProposalId` and `forcedTail`, and may include a proposal later finalized before quiescence; that harmless superset prevents a moving-start race. The deployment profile fixes `LEGACY_BLOB_RETENTION_SECONDS=1,572,864`, the conservative 4,096-epoch mainnet minimum at 32 slots per epoch and 12 seconds per slot. For each blob-bearing source the compatibility implementation derives, rather than accepts, `dataExpiry=checked(uint64(blobSlice.timestamp)+LEGACY_BLOB_RETENTION_SECONDS)`. A proposal row's expiry is the minimum of all of its blob-bearing source expiries; a forced row has exactly its stored source expiry. A row containing no blob-bearing source has expiry `UINT64_MAX`. Zero timestamp, arithmetic overflow, malformed source shape or a caller-supplied expiry rejects. Every blob row in the scanned unfinalized/pending superset contributes to `minDataExpiry`, including a row already outside that exact retention horizon. Before entering QUIESCENT the contract rechecks the fixed endpoints, complete counts/roots and requires `minDataExpiry>=resumeByTimestamp+LEGACY_RESUME_PROOF_GENERATION_MAX_SECONDS+REORG_MARGIN_SECONDS+T_INCLUDE_MAX_SECONDS`, where the reviewed release value of `LEGACY_RESUME_PROOF_GENERATION_MAX_SECONDS` is 900. The two ZK proofs may be generated in parallel, but each complete route proof must meet that bound in the shadow-fork release gate. Therefore a failed campaign cannot itself age any legacy data out of availability or resume into a suffix that the deployed no-skip protocol cannot process. A stale row rejects quiescence even if a successful migration would have abandoned it. Operators must first finalize or otherwise repair that legacy state; if the exact deployed rules provide no safe repair, in-place GENESIS_IMPORT is unsupported and a separately initialized state migration is required.

Blob availability is not the sole resume predicate. The campaign's `reviewCommitment` binds a golden `legacyResumeProfileHash` over the proxy/implementation/configuration, proof verifier and every composed sub-verifier runtime/configuration, the legacy SignalService checkpoint path, proposer checker, prover permission state, bond parameters and all time-dependent legacy branches. The design does *not* assume that the same proof bytes remain valid: deployed SGX instances expire after 90 days. A supported profile instead pins one canonical sufficient verifier route whose members ignore both `proposalAge` and `block.timestamp`. Its root is the new non-proxy, ownerless `LegacyResumeZkPairVerifierV1`; generic ComposeVerifier roots are unsupported. It accepts exactly the ascending pair `[RISC0_RETH=5,SP1_RETH=6]` and no SGX-containing or one-proof route, so fresh proof bytes may be generated after resume. The route uses reviewed non-proxy, ownerless fixed-key adapters: the RISC0 adapter accepts exactly one block image and one aggregation image, and the SP1 adapter accepts exactly one block program key and one aggregation program key. Those four keys are immutable code/configuration, not entries in the deployed mutable trust mappings. Each adapter recursively binds the exact remote verifier dispatch graph and rejects every other key. A generic owner-controlled RISC0/SP1 trust-map wrapper is not a supported resume-route member, because its mapping is not enumerable and an on-chain profile check cannot prove that no additional key was authorized.

For a supported profile, `minBond=livenessBond=0`, proving is public, there is no contest/skip deadline, a later forced-due time only strengthens the next proposal's inclusion requirement, and historical withdrawals may only mature. Therefore advancing by the bounded QUIES-

CENT duration cannot make the bound state transition unprovable solely because time advanced; the data-expiry check covers its per-record availability horizon. Router and proxy recheck that exact profile at campaign publication, quiescence, activation and resume. A root verifier that requires SGX on every sufficient route (including the legacy `MainnetVerifier`), a route member that consumes proposal age or wall clock, nonzero liveness-bond deadline, challenge window, expiring claim, mutable runtime, or any other non-pause-aware validity/custody branch makes in-place GENESIS_IMPORT unsupported. The pre-campaign compatibility upgrade must point the Inbox to the exact fixed-pair root; otherwise a separately initialized state migration is required.

The checkpoint write is a separate consensus dependency. Legacy `Inbox.prove` calls `SignalService.saveCheckpoint` in the same transaction before storing its finalized core state. The bound `signalServiceCheckpointDescriptorHash` therefore commits to the exact proxy runtime, direct implementation address and runtime hash, `VERSION=1` checkpoint storage layout, implementation-embedded authorized syncer equal to the legacy Inbox, remote SignalService and pauser immutables, and the upgrade-authority/fence descriptor. A ForkRouter or any implementation with a reachable delegate target beyond the one bound UUPS proxy-to-implementation hop is unsupported; governance must first install the reviewed final direct SignalService implementation before campaign publication. This avoids making checkpoint liveness depend on an incompletely enumerated dispatch graph. Pausing is deliberately not a profile bit because the reviewed V1 `saveCheckpoint` path is not pause-gated; any replacement that changes that property has a different runtime hash. Publication, quiescence, activation and resume recheck this descriptor. Its implementation/configuration authority must be burned or share the same exact campaign fence as the Inbox and verifier graph. If the live checkpoint graph cannot be fenced, in-place migration is unsupported. For the pinned layout, `inner=H(u256(1)||u256(254))` and `record=H(u256(blockNumber)||inner)` under Solidity's mapping-key encoding; `blockHash` is storage word `record` and `stateRoot` is checked `record+1`. The layout hash below commits to that VERSION, base slot, field order and widths.

The byte-exact profile commitment is

```
legacyCampaignFenceDescriptorHash = H(
 "slot-chain-legacy-genesis-campaign-fence-v1" ||
 address20(router) || address20(legacyProxy) ||
 bytes4(0x718b2ac7) || bytes4(0x4c474331) || u16(512) || u32(100000) ||
 bytes4(0x9b698000) || bytes4(0x4c475331) || u16(512) || u32(100000) ||
 bytes4(0xef3cdce0) || bytes4(0x4c475353) || u16(608) || u32(100000))
risc0ResumeKeyPolicyHash = H(
 "slot-chain-legacy-genesis-risc0-key-policy-v1" ||
 risc0BlockImageId || risc0AggregationImageId)
sp1ResumeKeyPolicyHash = H(
 "slot-chain-legacy-genesis-sp1-key-policy-v1" ||
 sp1BlockProgramVKey || sp1AggregationProgramVKey)
risc0RethVerifierDescriptorHash = H(
 "slot-chain-legacy-genesis-risc0-verifier-v1" ||
 address20(risc0Adapter) || risc0AdapterRuntimeHash || u64(l2ChainId) ||
 address20(risc0RemoteVerifier) || risc0RemoteVerifierRuntimeHash ||
 risc0ResumeKeyPolicyHash)
sp1RethVerifierDescriptorHash = H(
 "slot-chain-legacy-genesis-sp1-verifier-v1" ||
```

```
 address20(sp1Adapter) || sp1AdapterRuntimeHash || u64(l2ChainId) ||
 address20(sp1RemoteVerifier) || sp1RemoteVerifierRuntimeHash ||
 sp1ResumeKeyPolicyHash)
proofVerifierGraphHash = H(
 "slot-chain-legacy-genesis-proof-verifier-graph-v1" ||
 address20(resumePairRoot) || resumePairRootRuntimeHash || u8(2) ||
 u8(5) || address20(risc0Adapter) || risc0RethVerifierDescriptorHash ||
 u8(6) || address20(sp1Adapter) || sp1RethVerifierDescriptorHash)
legacyResumeVerifierRouteHash = H(
 "slot-chain-legacy-genesis-resume-verifier-route-v1" ||
 proofVerifierGraphHash || u8(2) ||
 u8(5) || risc0RethVerifierDescriptorHash || risc0ResumeKeyPolicyHash ||
 u8(6) || sp1RethVerifierDescriptorHash || sp1ResumeKeyPolicyHash)
proposerCheckerDescriptorHash = H(
 "slot-chain-legacy-genesis-proposer-checker-v1" ||
 address20(proposerCheckerProxy) || proposerCheckerProxyRuntimeHash ||
 address20(proposerCheckerImplementation) ||
 proposerCheckerImplementationRuntimeHash || u16(64) ||
 legacyCampaignFenceDescriptorHash)
proverWhitelistDescriptorHash = H(
 "slot-chain-legacy-genesis-public-proving-v1" || address20(0) || u8(1))
checkpointStorageLayoutHash = H(
 "slot-chain-legacy-genesis-checkpoint-layout-v1" ||
 u256(1) || u16(254) ||
 H("CheckpointRecord(bytes32 blockHash,bytes32 stateRoot)"))
signalServiceCheckpointDescriptorHash = H(
 "slot-chain-legacy-genesis-signal-service-checkpoint-v1" ||
 address20(signalServiceProxy) || signalServiceProxyRuntimeHash ||
 address20(signalServiceImplementation) ||
 signalServiceImplementationRuntimeHash || u256(signalServiceVersion) ||
 address20(signalServiceAuthorizedSyncer) ||
 address20(remoteSignalService) || address20(signalServicePauser) ||
 checkpointStorageLayoutHash || legacyCampaignFenceDescriptorHash)
legacyResumeTimePolicyHash = H(
 "LegacyGenesisResumeTimePolicyV2(sameProofPreserved=false,"
 "ageIndependentRouteRequired=true,publicProvingRequired=true,minBond=0,"
 "livenessBond=0,noContest=true,forcedDueOnlyStrengthens=true,"
 "withdrawalOnlyMatures=true)")
legacyResumeProfileHash = H(
 "slot-chain-legacy-genesis-resume-profile-v2" ||
 legacyDeploymentHash || legacyCampaignFenceDescriptorHash ||
 proofVerifierGraphHash ||
 legacyResumeVerifierRouteHash || signalServiceCheckpointDescriptorHash ||
 proposerCheckerDescriptorHash || proverWhitelistDescriptorHash ||
 u64(minBond) || u64(livenessBond) || u64(withdrawalDelay) ||
 u64(provingWindow) || u64(permissionlessProvingDelay) ||
 u64(maxProofSubmissionDelay) || u48(ringBufferSize) ||
 u16(forcedInclusionDelay) || u64(forcedInclusionFeeInGwei) ||
```

```
 u64(forcedInclusionFeeDoubleThreshold) ||
 u16(permissionlessInclusionMultiplier) ||
 u16(maxForcedInclusionsPerProposal) ||
 u16(maxNormalBlobHashesPerProposal) ||
 u64(legacyBlobRetentionSeconds) ||
 u64(legacyResumeProofGenerationMaxSeconds) || legacyResumeTimePolicyHash)
```

Configuration reads use exactly 100,000 gas each and these canonical ABIs:

```
legacyResumeVerifierConfigV1() // selector 0x7e52cacd; 160 bytes
 returns (bytes4 magic,address risc0Adapter,address sp1Adapter,
          bytes32 risc0KeyPolicyHash,bytes32 sp1KeyPolicyHash) // LRV1
legacyResumeRisc0ConfigV1()    // selector 0xe516c43e; 192 bytes
 returns (bytes4 magic,uint64 l2ChainId,address remoteVerifier,
          bytes32 remoteRuntimeHash,bytes32 blockImageId,
          bytes32 aggregationImageId)                        // LR01
legacyResumeSp1ConfigV1()      // selector 0xe2b5d958; 192 bytes
 returns (bytes4 magic,uint64 l2ChainId,address remoteVerifier,
          bytes32 remoteRuntimeHash,bytes32 blockProgramVKey,
          bytes32 aggregationProgramVKey)                   // LSP1
legacyCheckpointConfigV1()     // selector 0x68011023; 192 bytes
 returns (bytes4 magic,address authorizedSyncer,
          address remoteSignalService,address pauser,
          bytes32 checkpointStorageLayoutHash,
          bytes32 campaignFenceDescriptorHash)              // LCK1
```

The four magics are respectively `0x4c525631`, `0x4c523031`, `0x4c535031` and `0x4c434b31`; every narrow/address/magic word has canonical left padding. The proposer checker uses exact 32-byte reads `impl()` `0x8abf6077`, `operatorCount()` `0x7c6f3158`, `getOperatorForCurrentEpoch()` `0x343f0a68` and `getOperatorForNextEpoch()` `0x72a8a551`, each with the same stipend. The SignalService proxy also returns its implementation from `impl()` and `VERSION()` `0xffa1ad74` as exact 32-byte words. Every returned field must reproduce the descriptor preimages above; revert, OOG, short/trailing data, wrong magic, padding, code hash, address or key rejects before publication or state change.

These are the complete descriptor grammars; no caller may substitute an opaque component hash. The root and both adapters are direct non-proxies. Its proof bytes are the canonical ABI encoding of exactly two `SubProof(uint8verifierId,bytesproof)` rows in IDs 5,6 order; minimal offsets, zero padding and no trailing bytes are required. The fixed RISC0 row is canonical `abi.encode(bytesseal,bytes32blockImageId,bytes32aggregationImageId)`. The fixed SP1 row is exactly `aggregationProgramVKey||blockProgramVKey||opaqueProof`, with a nonempty opaque suffix. Both adapters recompute the existing Taiko public input using their own bound address and chain ID before the remote STATICCALL. Each remote verifier is also direct, stateless, ownerless and non-delegating, contains no reachable `SSTORE/SELFDESTRUCT` or configuration mutator, and has the exact runtime hash in its leaf descriptor; any proxy, gateway with mutable dispatch or unlisted child makes the profile ineligible. The Inbox's `proverWhitelist` address must be exactly zero—a mutable zero-count whitelist is no longer accepted. The proposer checker and legacy SignalService are the only descriptor leaves allowed one UUPS hop, in addition to the source Inbox proxy itself. Their reviewed implementations apply the common fence to every operator/ejecter/configuration/ownership-affecting mutator and upgrade; at each profile checkpoint, `operatorCount` must be in 1–64 and both current- and next-epoch operator getters must return nonzero addresses.

The common fence makes exact 100,000-gas STATICCALLs to LGC1, LGS1 and LGSS and accepts only their specified lengths/magics/canonical padding. Every fenced mutation rejects while the Router retains an unexpired SCHEDULED campaign or while local LGS1 is not clean ACTIVE with EMPTY LGSS and zero cutover words. EXPIRED Router state alone is insufficient until permissionless local cleanup completes. Revert, OOG, short/trailing return or inconsistent Router/ local campaign fails closed. The compatibility Inbox, proposer checker and direct SignalService implementation use this identical descriptor; their runtime hashes prove that every authority-bearing mutator invokes it. Thus a separately worded or self-reported fence hash is never accepted.

The fixed-key route adapters and remote leaves have no mutation authority; the three campaign-aware UUPS implementations may change only outside the common fence. A proxy implementation, key, child pointer or configuration that an independent administrator can change during the fence is unsupported. The supported deployment profile additionally requires the constructor/profile values 0, 0, 604,800, 14,400, 432,000, 180, 21,600, 576, 1,000,000, 50, 160, 10, 21, 1,572,864 and 900 in the field order above. The intervening IInbox.Config-only `basefeeSharingPctg` is exactly 75; its five address words equal the descriptor leaves named above, including the zero prover whitelist, and its bond token is the reviewed deployed token. Thus the exact 17-word getter, not a list of assumed constructor inputs, determines `legacyInboxConfigurationHash`. The two source-shape bounds pin the deployed Inbox's ten-forced-source constant and the current Fulu mainnet maximum of 21 normal blob hashes. The retention value is a conservative protocol dependency, not an assertion that every archival provider retains data that long; publication must be disabled before a canonical L1 fork shortens that minimum or raises the per-block blob maximum above 21. Any other profile receives a different reviewed commitment and must independently prove resume safety.

The review artifact is byte-bound rather than referenced by prose:

```
reviewCommitment = H("slot-chain-legacy-genesis-review-v1" ||
  legacyDeploymentHash || legacyResumeProfileHash ||
  u64(targetProtocolVersion) || targetManifestHash ||
  targetRegistrationHash)
```

ProtocolVersionManager recomputes this exact value from live proxy/component descriptors and the delayed target tuple in the publication transaction; a caller-supplied review hash is never authoritative. The review is reusable across timing-equivalent campaign retries, while the separately computed `campaignId` binds nonce, generation and every campaign deadline.

The proposal preimages required by the scan are canonical `abi.encode(Proposal)` values. The `Proposed` log supplies ID, proposer, parent proposal hash, submission-window end, basefee share and every source; its containing canonical L1 header supplies the proposal timestamp, and that header's canonical parent supplies `originBlockNumber=eventBlockNumber-1` and `originBlockHash`. The stored ring hash authenticates the reconstructed whole, so the compatibility contract need not trust a caller's archive or use the 256-block `BLOCKHASH` window. Forced rows are read from proxy storage. No builder or proposer secret is required. Their availability is the same canonical-L1- log/header archive assumption already required to reconstruct legacy proofs. A missing canonical log/header copy stops the campaign before QUIESCENT; it cannot strand the proxy. The final scan and activation receipt separately bind proposal scan start, actual abandoned proposal start `lastFinalizedProposalId+1`, proposal end, both counts, forced start/end/count, both ordered roots, aggregate abandoned native wei, zero abandoned bond liability, minimum data expiry and campaign ID. A proposal finalized after scan begin remains in the conservative scan root but lies before the receipt's actual abandoned start and is not reported lost. Each aggregate is revalidated when QUIESCENT is entered and again before LGAR. Exceeding any hard count, byte, call or time cap rejects the campaign rather than silently weakening the permissionlessness claim. `abandonedNativeWei` sums only the exact stored fees of pending forced rows. Raw proxy balance and forced ETH are not a campaign precondition: the review runbook records an informational balance observation, but no raw-balance word is accepted by or bound into activation, and all unaccounted value remains locked. Therefore an unsolicited `SELFDESTRUCT` transfer cannot veto migration or invalidate a prepared proof. There is intentionally no protocol cap on the accounted forced fees that governance may knowingly abandon; the fixed forced-row count bounds computation, while the final review/runbook must disclose the exact amount before quiescence.

The exact campaign and scan commitments are

```
campaignId = H("slot-chain-legacy-genesis-campaign-v1" ||
  legacyDeploymentHash || u64(nonce) || u64(generation) ||
  u64(forceCutoffBlock) || u64(proposalCutoffBlock) ||
  u64(quiesceNotBeforeBlock) || u64(resumeByBlock) ||
  u64(resumeByTimestamp) || u64(reviewFinalizedByBlock) ||
  address20(targetSettlement) || u64(targetProtocolVersion) ||
  targetManifestHash || targetRegistrationHash || reviewCommitment)
scanCommitment = H("slot-chain-legacy-genesis-scan-v1" || campaignId ||
  u48(proposalScanStart) || u48(nextProposalId) || u16(proposalCount) ||
  u32(proposalBytes) || proposalRowsRoot || u48(forcedHead) ||
  u48(forcedTail) || u16(forcedCount) || u32(forcedBytes) ||
  forcedRowsRoot || u256(abandonedNativeWei) ||
  u64(minDataExpiry) || legacyResumeProfileHash)
proposalRowsRoot[0] = H("slot-chain-legacy-genesis-proposal-rows-empty-v1")
proposalRowsRoot[i+1] = H("slot-chain-legacy-genesis-proposal-row-v1" ||
  proposalRowsRoot[i] || u48(proposalId) || u32(encodedBytes) ||
  storedProposalHash || u64(dataExpiry) || u256(0))
forcedRowsRoot[0] = H("slot-chain-legacy-genesis-forced-rows-empty-v1")
forcedRowHash = H("slot-chain-legacy-genesis-forced-record-v1" ||
                  canonicalAbiEncodedForcedInclusion)
forcedRowsRoot[i+1] = H("slot-chain-legacy-genesis-forced-row-v1" ||
  forcedRowsRoot[i] || u48(forcedIndex) || u32(encodedBytes) ||
  forcedRowHash || u64(dataExpiry) || u256(feeInGwei * 1 gwei))
```

The supplied proposal preimage is canonically re-encoded and its existing legacy hash must equal `storedProposalHash`; the forced record is read from storage and canonically ABI-encoded. Checked additions derive row counts, encoded byte totals and abandoned native value. The row domains and empty roots are published as golden vectors; implementations must reproduce them from the exact legacy proposal and forced-record codecs.

Those codecs are the deployed Solidity structs, not opaque bytes:

```
BlobSlice = (bytes32[] blobHashes,uint24 offset,uint48 timestamp)
DerivationSource = (bool isForcedInclusion,BlobSlice blobSlice)
```

```
Proposal = (uint48 id,uint48 timestamp,
 uint48 endOfSubmissionWindowTimestamp,address proposer,
 bytes32 parentProposalHash,uint48 originBlockNumber,
 bytes32 originBlockHash,uint8 basefeeSharingPctg,
 DerivationSource[] sources)
ForcedInclusion = (uint64 feeInGwei,BlobSlice blobSlice)
proposalPreimage = abi.encode(Proposal)
forcedPreimage = abi.encode(ForcedInclusion)
```

Because each top-level struct is dynamic, the first ABI word is the canonical `0x20` tuple offset. Every nested offset is minimal, 32-byte aligned and relative to its ABI-defined container; tails are monotone and contiguous, all narrow words/bools/addresses have canonical padding, and trailing bytes reject. A deployed proposal has 1–11 sources: zero to ten leading forced sources, each with `isForcedInclusion=true` and exactly one nonzero blob hash, followed by exactly one normal source with `isForcedInclusion=false` and 1–21 nonzero hashes. A stored ForcedInclusion has exactly one nonzero blob hash. Each `BlobSlice.offset` is uint24-canonical and every timestamp is nonzero uint48. The compatibility implementation decodes, validates this shape, re-encodes and only then hashes; `encodedBytes` is the length of that unique encoding. The profile maxima make the publication-time row/byte bound valid; another legacy shape or L1 blob schedule requires a separately reviewed profile.

The resulting profile bounds are exact:

```
MAX_PROPOSAL_ROW_BYTES = 3,808
MAX_FORCED_ROW_BYTES = 256
MAX_PROPOSAL_BATCH_ROWS = 16
MAX_PROPOSAL_BATCH_ROW_BYTES = 60,928
MAX_PROPOSAL_SCAN_CALLDATA_BYTES = 62,084
MAX_PROFILE_TOTAL_SCAN_BYTES = 1,024 * (3,808 + 256) = 4,161,536
SCAN_BYTES_HARD_CAP = 4,194,304
GENESIS_SCAN_BATCH_GAS_RELEASE_CAP = 12,000,000
```

The calldata maximum includes selector, the three-word argument head, `bytes[]` length/offsets and each element's length word. Publication relies only on rows created by the reviewed deployed Inbox while the canonical L1 maximum was at most 21; from campaign publication, compatibility admission also rejects any source count/hash count or encoded proposal length above this profile before mutation. A decoded historical row outside the profile rejects its scan and therefore cannot enter QUIESCENT; it is evidence that this in-place profile is unsupported, not permission to use a smaller batch. The production Foundry suite must execute a 16-row maximum-shaped proposal batch from cold and warm storage, including exact LGC1/profile reads, strict decode/re-encode, roots, expiry and LGSS writes, below the 12,000,000-gas release cap. Failure blocks this profile before any campaign is published. Thus deterministic maximum-progress batches cannot concentrate the aggregate 4 MiB allowance into an unmineable transaction.

The GENESIS_IMPORT receipt's nonzero transition auxiliary is

```
legacyGenesisAbandonmentReceiptHash =
 H("slot-chain-legacy-genesis-abandonment-receipt-v1" ||
   campaignId || scanCommitment ||
```

```
   u48(proposalScanStart) || u48(abandonedProposalStart) ||
   u48(proposalEnd) || u16(scannedProposalCount) ||
   u16(abandonedProposalCount) || u32(proposalBytes) ||
   proposalRowsRoot || u48(forcedStart) || u48(forcedEnd) ||
   u16(forcedCount) || u32(forcedBytes) || forcedRowsRoot ||
   u256(abandonedNativeWei) || u256(abandonedBondLiabilityWei) ||
   u64(minDataExpiry) || legacyResumeProfileHash)
```

Here `proposalEnd` is the scan's fixed proposal endpoint. The activation transaction samples the abandonment start and derives both counts as follows:

```
abandonedProposalStart = lastFinalizedProposalId + 1
scannedProposalCount = proposalEnd - proposalScanStart
abandonedProposalCount = proposalEnd - abandonedProposalStart
```

Every addition and subtraction is checked. The Router emits the following exact event in the same rollback domain as the sealed receipt:

```
event LegacyGenesisAbandonmentSealed(
 bytes32 indexed receiptId, bytes32 indexed transitionAuxiliaryHash,
 bytes32 campaignId, bytes32 scanCommitment,
 uint48 proposalScanStart, uint48 abandonedProposalStart,
 uint48 proposalEnd, uint16 scannedProposalCount,
 uint16 abandonedProposalCount, uint32 proposalBytes,
 bytes32 proposalRowsRoot, uint48 forcedStart, uint48 forcedEnd,
 uint16 forcedCount, uint32 forcedBytes, bytes32 forcedRowsRoot,
 uint256 abandonedNativeWei, uint256 abandonedBondLiabilityWei,
 uint64 minDataExpiry, bytes32 legacyResumeProfileHash);
```

The event fields must recompute the auxiliary hash embedded in the immutable ARV1 receipt; the zero bond-liability word is checked, not conventional. The raw proxy balance is deliberately absent because anyone can change it without calling the proxy. A missing/mismatched event or auxiliary write reverts before ACTIVE, and a VERSION_MIGRATION neither emits this event nor accepts a nonzero auxiliary.

At or after `quiesceNotBeforeBlock`, before either expiry bound, any caller may enter QUIESCENT only after these checks. No target proof is required for that reversible step.

Campaign publication first uses the exact read-only preflight

```
legacyGenesisPreparationV1() returns
 (bytes4 magic,uint48 nextProposalId,uint48 lastFinalizedProposalId,
  uint48 forcedHead,uint48 forcedTail,
  uint16 unfinalizedProposalCount,uint16 pendingForcedCount,
  uint32 maximumScanBytes,bytes32 legacyResumeProfileHash)
```

Its selector is `0x6880cd05`, magic is LGPR `0x4c475052`, and return length is exactly 288 bytes. Counts are checked differences of the preceding endpoints, `maximumScanBytes` is their checked product/sum under the exact profile row maxima, and the profile hash is recomputed from live code/config; none is a cached or caller-supplied readiness assertion.

The permissionless scan/quiescence surface is exactly

```
beginLegacyGenesisScanV1(uint64 generation,bytes32 campaignId)
 returns (bytes4 magic,uint48 proposalStart,uint48 proposalEnd,
          uint48 forcedStart,uint48 forcedEnd)
scanLegacyGenesisProposalsV1(
 uint64 generation,bytes32 campaignId,bytes[] proposalPreimages)
 returns (bytes4 magic,uint48 nextCursor,bytes32 rowsRoot,
          uint32 bytesScanned,uint64 minDataExpiry)
scanLegacyGenesisForcedV1(
 uint64 generation,bytes32 campaignId,uint16 maxRows)
 returns (bytes4 magic,uint48 nextCursor,bytes32 rowsRoot,
          uint32 bytesScanned,uint256 abandonedNativeWei,
          uint64 minDataExpiry)
legacyGenesisScanStateV1() returns
 (bytes4 magic,uint64 generation,bytes32 campaignId,
  uint48 proposalStart,uint48 proposalCursor,uint48 proposalEnd,
  uint16 proposalCount,uint32 proposalBytes,bytes32 proposalRoot,
  uint48 forcedStart,uint48 forcedCursor,uint48 forcedEnd,
  uint16 forcedCount,uint32 forcedBytes,bytes32 forcedRoot,
  uint256 abandonedNativeWei,uint64 minDataExpiry,
  bytes32 legacyResumeProfileHash,uint8 scanPhase)
enterLegacyGenesisQuiescenceV1(uint64 generation,bytes32 campaignId)
 returns (bytes4 magic,uint64 generation,bytes32 scanCommitment)
resumeLegacyGenesisV1(uint64 generation,bytes32 campaignId)
 returns (bytes4 magic,uint64 generation,bytes32 campaignId)
```

Their selectors are respectively `0xe9d1a07f`, `0x7da66460`, `0x032ae99e`, `0xef3cdce0`, `0x4c0ae8da` and `0x2bf6b656`. Magics are LGBS `0x4c474253`, LGSP `0x4c475350`, LGSF `0x4c475346`, LGSS `0x4c475353`, LGQS `0x4c475153` and LGRS `0x4c475253`. Fixed returns are exactly 160, 160, 192, 608, 96 and 96 bytes in that order. Scan phase is EMPTY=0, SCANNING=1 or COMPLETE=2. Begin is one-shot per campaign and snapshots the post-proposal-cutoff endpoints. Proposal input contains exactly `min(16,proposalEnd-proposalCursor)` consecutive canonical `abi.encode(Proposal)` preimages; zero remaining rows rejects. The dynamic array uses minimal monotone offsets, contiguous tails, zero right-padding and no trailing bytes. Forced scan requires `maxRows=min(16,forcedEnd-forcedCursor)>0` and reads, rather than accepts, each stored row. Thus a front-run may invalidate a stale transaction but must advance the shared cursor by the same deterministic batch size. The returned cursor/root/count/byte/value/expiry fields must equal LGSS after the call. COMPLETE requires both cursors equal their ends and exact checked count differences; no caller supplies an aggregate.

All six calls are nonpayable, only-proxy and non-reentrant but otherwise permissionless. Direct implementation calls reject. Each begins with a 100,000-gas exact LGC1 STATICCALL; profile/configuration reads at begin, quiescence and resume use their separately published fixed stipends. A wrong stage, endpoint, row index, preimage hash, offset, length, padding, campaign, profile, expiry or return value reverts the entire batch. No external call follows a scan-state write. Explicit resume accepts the exact retained campaign in SCHEDULED after either clock bound or in EXPIRED. It accepts either QUIESCENT with that campaign or local ACTIVE with that campaign's nonempty SCANNING/COMPLETE scan state, then clears every scan/campaign/cutover word and returns or retains ACTIVE without executing a proposal, proof or other business rule. ACTIVE with no matching local campaign state is already clear; a mismatched campaign, DRAINING, READY or FROZEN rejects. Resume does not call or mutate Router. An ordinary legacy mutator performs the same local clear before continuing, then still exact-reads LGC1 under ACTIVE. Separately, anyone may call Router `expireLegacyGenesisCampaignV1(uint64,bytes32)` (selector `0xa4a37936`) after either bound; it returns the sole exact LGEX word `0x4c474558` and marks the Router campaign expired/tombstoned. Local resume and Router expiry commute, so neither is a reentrant callback and neither is required to precede the other.

The exact compatibility interface is

```
legacyGenesisStateV1() returns
 (bytes4 magic,uint8 phase,uint64 generation,bytes32 campaignId,
  bytes32 scanCommitment,uint64 targetProtocolVersion,
  bytes32 targetManifestHash,bytes32 targetRegistrationHash,bytes32 armId,
  bytes32 launchId,uint48 nextProposalId,uint48 lastFinalizedProposalId,
  bytes32 lastFinalizedBlockHash,uint48 forcedHead,uint48 forcedTail,
  bytes32 poststateCommitment)
checkpointLegacyGenesisV1(uint64 generation,bytes32 campaignId)
 returns (bytes4 magic,uint64 generation,bytes32 campaignId,bytes32 armId)
finalizeLegacyGenesisV1(uint64 generation,uint64 targetProtocolVersion,
  bytes32 targetManifestHash,bytes32 targetRegistrationHash,
  bytes32 candidateDigest,bytes32 outputCoreHash)
 returns (bytes4 magic,bytes32 launchId,bytes32 poststateCommitment)
```

The selectors are respectively `0x9b698000`, `0x8781a058` and `0xc2de6417`. State return is exactly 512 bytes; LGAR returns exactly 128 bytes and LGFN exactly 96 bytes. Magics are LGS1 `0x4c475331`, LGAR `0x4c474152` and LGFN `0x4c47464e`. Calls are nonpayable, Router-only, only-proxy, non-reentrant and fixed-gas. All narrow/address/magic padding is canonical; short, trailing, malformed or arbitrary bubbled returndata rejects.

The local phase numbers remain ACTIVE=0, DRAINING=1, QUIESCENT=2, READY=3 and FROZEN=4 for storage/ABI compatibility. DRAINING is unreachable and fails closed. QUIESCENT is a bounded, reversible campaign fence; READY exists only inside a successful activation transaction; FROZEN is permanent. The default-zero cutover region needs no initializer. Entering QUIESCENT records the exact monotone generation, campaign ID, completed scan commitment and an O(1) snapshot of `nextProposalId`, `lastFinalizedProposalId`, `lastFinalizedBlockHash`, `forcedHead` and `forcedTail`; every target, arm, launch and poststate word remains zero. While QUIESCENT all five boundary-changing selectors reject. At or after either campaign expiry bound, any caller may resume ACTIVE, and the first ordinary mutator performs that same resume before its normal checks. The resume tombstones the attempt, clears every cutover and scan word, changes no legacy core, proposal hash, queue cursor, bond or claim, and cannot run after a successful freeze.

During atomic activation LGAR authenticates Router ACTIVATING/MACT and the exact live campaign, moves QUIESCENT to READY, retains the scan and boundary, computes the campaign-bound arm ID, and returns it. Router exact-staticreads LGS1 before calling LGFN in the same transaction. LGFN writes the exact target, launch ID, FROZEN and poststate. A fault in LGAR, LGS1, LGFN or any later adoption write reverts to the pre-existing QUIESCENT snapshot; no canonical transaction can leave READY. Expiry then still restores ACTIVE if no later retry succeeds.

The snapshot may contain unfinalized proposal IDs and a nonempty native forced range. Successful atomic activation intentionally imports only the last finalized header/checkpoint. The proposal suffix `[lastFinalizedProposalId+1,nextProposalId)` and every pending or already consumed forced input reachable only from that unfinalized suffix are non-canonical and permanently abandoned when the proxy enters FROZEN. The legacy head/tail and proposal slots are not rewritten, no refund owner is invented, forced fees and raw Inbox ETH remain forever in the legacy proxy, and no owner or governance sweep exists. This loss is an explicit genesis migration trade-off forced by the deployed record format, which stores neither a refund owner nor durable blob bytes. A deployment requiring lossless treatment of those records cannot use in-place GENESIS_IMPORT and must use a separately initialized state migration.

FROZEN permits only permanent reads and the existing legacy bond request/cancel/withdrawal paths. QUIESCENT admits reads, those same historical bond paths, campaign resume and the Router's LGAR; transaction-local READY adds only Router LGFN. Every still-escrowed legacy bond becomes requestable under its existing withdrawal delay without a runtime minimum-bond floor and without slash or relabeling. Phase 1, any change to the QUIESCENT/READY snapshot, or any stale/nonzero field outside the phase matrix rejects.

The fresh depth-64 ForcedQueue is deployed empty with root/count/cursor golden values, `dueAt(0)=UINT64_MAX`, zero liability and this exact legacy proxy as its initial active authority. Genesis QMIG therefore has a real expected old authority and atomically changes it to the target Settlement with `start=end=count=creditedWei=0`; no zero-address authority convention is accepted. The deployment configuration is not an opaque hash. Router/PVM call `getConfig()` with selector `0xc3f909d4` and exactly 100,000 gas. The return is exactly 544 bytes—the 17 static IInbox.Config words in this order—with canonical high padding:

```
(address proofVerifier,address proposerChecker,address proverWhitelist,
 address signalService,address bondToken,uint64 minBond,uint64 livenessBond,
 uint48 withdrawalDelay,uint48 provingWindow,
 uint48 permissionlessProvingDelay,uint48 maxProofSubmissionDelay,
 uint48 ringBufferSize,uint8 basefeeSharingPctg,
 uint16 forcedInclusionDelay,uint64 forcedInclusionFeeInGwei,
 uint64 forcedInclusionFeeDoubleThreshold,
 uint8 permissionlessInclusionMultiplier)
legacyInboxConfigurationHash = H(
 "slot-chain-legacy-genesis-inbox-config-v1" ||
 address20(proofVerifier) || address20(proposerChecker) ||
 address20(proverWhitelist) || address20(signalService) ||
 address20(bondToken) || u64(minBond) || u64(livenessBond) ||
 u48(withdrawalDelay) || u48(provingWindow) ||
 u48(permissionlessProvingDelay) || u48(maxProofSubmissionDelay) ||
 u48(ringBufferSize) || u8(basefeeSharingPctg) ||
 u16(forcedInclusionDelay) || u64(forcedInclusionFeeInGwei) ||
 u64(forcedInclusionFeeDoubleThreshold) ||
 u8(permissionlessInclusionMultiplier))
```

The proxy's `impl()` selector `0x8abf6077` returns exactly the implementation address as one canonical word under the same stipend. PVM checks `EXTCODEHASH` of both proxy and implementation. Storage-layout compatibility is a reviewed property of that one pinned implementation runtime against the deployed proxy layout; there is no self-reported layout hash or second implementation accepted by consensus. The release artifact publishes the compiler storage-layout diff, and any bytecode change requires a new reviewed campaign.

The fixed-width commitments are

```
legacyDeploymentHash =
 H("slot-chain-legacy-genesis-deployment-v1" || u256(settlementChainId) ||
   address20(proxy) || proxyRuntimeHash || address20(implementation) ||
   implementationRuntimeHash || legacyInboxConfigurationHash ||
   address20(router))
boundaryHash = H("slot-chain-legacy-genesis-boundary-v1" ||
  u48(nextProposalId) || u48(lastFinalizedProposalId) ||
  lastFinalizedBlockHash || u48(forcedHead) || u48(forcedTail))
armId = H("slot-chain-legacy-genesis-arm-v1" || legacyDeploymentHash ||
          u64(generation) || campaignId || scanCommitment || boundaryHash)
launchId = H("slot-chain-legacy-genesis-launch-v1" || armId ||
             u64(targetProtocolVersion) || targetManifestHash ||
             targetRegistrationHash)
legacyPostStateCommitment =
 H("slot-chain-legacy-genesis-poststate-v1" || launchId ||
   candidateDigest || outputCoreHash || u64(0) || u64(block.number) ||
   u8(FROZEN=4) || boundaryHash)
```

For QUIESCENT/READY/FROZEN, `nextProposalId>lastFinalizedProposalId`, the finalized hash is nonzero and `forcedHead<=forcedTail`; equality is not required for either pair. ACTIVE requires generation, target tuple, both IDs and poststate all zero. QUIESCENT requires nonzero generation, campaign and scan commitments but zero target tuple, arm, launch and poststate. READY adds the recomputed arm ID but retains the zero target tuple, launch and poststate. FROZEN requires a nonzero target tuple, the same campaign, scan and arm ID, the target-derived launch ID and recomputed poststate. Phase 1 and every stale/nonzero field outside this matrix reject. The delayed campaign, scan, target registration and migration public statement bind `legacyDeploymentHash`, `armId` and `launchId`; because the arm recomputes the campaign and scan commitments, it transitively binds every stage, reviewed target and abandoned-suffix boundary. The activation context binds that statement, proxy and target registration; the receipt binds the returned legacy poststate. None is accepted from unverified caller fields. Existing Taiko bridge messages remain on the legacy SignalService/Bridge retry path and are not silently converted into kind 1. Only new, explicitly escrowed inbox-credit messages use `BridgeInboxAdapter`; its direct-read descriptor binds the existing message hash, both domains, source epoch, permanent Bridge/execution identity, source emission block, enqueue deadline, sender, fee, ownership/value and derived calldata hash/length fields, while destination invocation remains a later bridge operation.

Before the delayed arm, the legacy L2 governance path deploys the profile-exact protocol-lifetime non-proxy InboxApplyRouterV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2, TerminalAccumulatorV2 and NativeLiquidityPoolV2, then one fresh endpoint InboxCreditStoreV2, DestinationBridgeV2 and Bridge-unique QuotaManager. The global router's constructor fixes only the permanent system sender, registrar and namespace. The store fixes that router, fresh Bridge and registrar, but its local-domain slot remains zero; Bridge `v2Active`, private mappings, liability and nonce remain zero and its quota begins full. All component addresses are precomputed from the deployment coordinator's fixed CREATE nonce sequence recorded in the manifest, so constructor references do not require an address/init-code fixed point. It also installs any profile-required AnchorV4 runtime. Legacy SignalService and the entire V1 Bridge/Vault graph remain untouched. The L1 side permissionlessly deploys the manifest-pinned fresh SourceBridgeV2/BridgeCreditRegistryV2/ QuotaManager bundle through the exact CREATE2 factory, plus TerminalSignalVerifier and BridgeInboxAdapter. ProtocolVersionManager directly checks the deployed L1 components 1–3 and their constructor configuration, records those exact descriptor rows in the finalized manifest, computes the acyclic combined infrastructure/domain descriptor, seals the L1 adapter's domain once and burns that setter. The L2 release proof carries those same rows; its registrar directly checks local components 4–10 and recomputes the identical combined hash. Every V2 account is immutable and non-proxy. InboxApplyRouterV2, the Store and every L2 inbound-V2 process/retry/fail/expire path require the fresh Store/Bridge one-shot seal; only proved tx0 may set it. No separate activation gate exists. This is an ordinary proved L2 state transition; no L1 transaction is claimed to mutate an imported L2 state root. Its transaction bytes, receipts and resulting account/storage values are golden profile vectors.

The delayed genesis campaign publishes one exact target tuple and final review commitment before either admission cutoff. It never moves the public gate away from the constructor-bound legacy ACTIVE branch. Once the compatibility proxy is QUIESCENT and before either expiry bound, any caller sends the canonical RLP of the stable last-finalized L2 header, the complete migration proof and the fixed-depth MPT account/storage proof witness under its state root for the exact AnchorV4, InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2, TerminalAccumulatorV2, NativeLiquidityPoolV2 and the fresh Store/QuotaManager/ DestinationBridge runtime code hashes; exact non-proxy configuration/layout and the absence of any reachable upgrade/delegate target; InboxApplyRouterV2 permanent sender/registrar/namespace, `nextQueueIndex=0` and empty routes; inbox-store router/Bridge/registrar immutables, zero domain and empty mappings; empty release-authority/registrar records; accumulator empty domain writer, count/root/frontier; Pool zero tickets/accounted balance and no active authorization, with any forced ETH classified only as surplus; fresh Bridge zero nonce/liability/private mappings and `v2Active=false`; full quota; and arbitrary authenticated Bridge surplus. The proof follows the exact immutable account and constructor-bound reference topology encoded in the profile and rejects an unlisted hop, delegate target or mutable upgrade path. Every runtime and authenticated value must equal the immutable execution profile. These values are witness to the one immutable migration-transition verifier, not the output of a separate adapter or a stored Boolean. While Router is IDLE, the public gate is legacy ACTIVE and the compatibility proxy is locally QUIESCENT. Activation validates the exact live campaign and expiry bounds, writes ACTIVATING, exact-reads LGS1, verifies the complete typed statement and requires the header hash to equal the stable legacy last-finalized hash, `number<UINT48_MAX`, and the legacy SignalService checkpoint at that exact number to equal its block hash and state root. Its typed public statement binds the header/state/checkpoint, deployment commitment, campaign/scan-bound arm ID and target-derived launch ID. The LGS1 proposal/ forced snapshot and boundary cannot change after quiescence. A substituted campaign, scan, boundary, target or launch ID rejects before LGAR. No second verifier, behavioral getter or caller-asserted readiness Boolean is accepted.

From that authenticated header and frozen execution profile, while the old Inbox remains QUIESCENT and the Queue authority, the immutable migration circuit constructs the target proof's base fields exactly:

```
l2BlockNumber       = header.number
tipHash             = H(RLP(header))
tipSlot             = header.timestamp - GENESIS_TIMESTAMP
stateRoot           = header.stateRoot
messageCursor       = 0
winningDataCommitment = H("slot-chain-migration-data-v2" ||
                          u256(settlementChainId) || u256(l2ChainId) ||
                          tipHash || stateRoot || terminalRoot ||
                          u64(terminalCount))
nextBaseFee         = profile.nextBaseFee(header)
nextExcessBlobGas   = profile.nextExcessBlobGas(header)
terminalRoot        = authenticatedAccumulator.root
terminalCount       = authenticatedAccumulator.count
firstV2BlockNumber    = header.number + 1
```

It rejects a pre-genesis timestamp, checked-addition overflow, missing fork-required header fields or any profile mismatch. Before cutover, an untrusted prover supplies the exact migration-transition-verifier proof for the first V2 block from this base. The block number is `firstV2BlockNumber`, and the block must begin with the custom AnchorV4 activation transaction for the staged (`protocolVersion,releaseManifestHash`). The circuit authenticates the staged manifest commitment under an already published L1 anchor, the exact imported parent, execution-profile hash, empty depth-64 queue and exact L2 component rows 4–10. Its top-level sender is the reserved Anchor-system sender. AnchorV4 first calls `ProtocolReleaseAuthorityV2.activateRelease`; the authority verifies both `msg.sender=manifest.anchorV4` and `tx.origin=reservedAnchorSystemSender`. AnchorV4 then calls `TerminalDomainRegistrarV2.activateDomain`; the latter authenticates the fresh Store/Quota/Bridge and Pool state, transfers no native liquidity, then atomically seals Store/Bridge, inbox route, Pool domain and accumulator registration before that block's InboxApplyRouterV2 batch. GENESIS_IMPORT has exactly zero Inbox rows: tx1 is the canonical empty batch with start=end=0, the statement `forceCutoff`, base/output message cursors, queue count and retirement count are all zero, and no preactivation ingress can create another value. The custom transaction type is invalid under the legacy fork and its reserved sender has no ordinary signing key, so neither a legacy block nor an EOA can preactivate the release. Any first block that omits, duplicates, calls authority directly or through Bridge, substitutes an Anchor, or mismatches this transition is unprovable. All calls occur in one EVM transaction, so any failure rolls release and registration back together. Subsequent blocks in that candidate require the exact release seal and cannot repeat activation.

The proof has no input derived from the future cutover transaction or its block hash. The L1 coordinator instead compares the proof's exact base, campaign generation/arm ID, scan, profile/manifest and queue snapshot with the exact QUIESCENT LGS1 and scheduled target immediately before adoption. The landing block remains outside the proof and enters only the activation context, poststate joins and receipt.

After internal campaign/calldata checks, Router writes ACTIVATING before its first external operation, exact-reads QUIESCENT LGS1 and all component state, reconstructs the statement, computes and stores the shared context, and verifies the profile-pinned proof by bounded STATICCALL. Only after verification does it call LGAR, exact-read the transaction-local

READY LGS1, call LGFN, call MCAN on the PREACTIVE target, call QMIG on the empty singleton Queue, perform SOURCE/TARGET/QUEUE MAPS postreads, consume the target registration, write the complete genesis receipt/index state, store `firstV2BlockNumber`, set the target/public gate ACTIVE, mark the campaign CONSUMED, clear context and write IDLE last. LGFN exact-staticreads MACT, rechecks its implementation, configuration, campaign, scan, stable READY snapshot, target-derived launch ID, boundary, candidate and output hash, then writes FROZEN/poststate without another external call. The source MAPS role is implemented by `LegacyGenesisCutoverInboxV1` and reproduces that LGFN commitment. MCAN installs the proved poststate as sequence zero with `canonicalizedAtBlock=lastCanonicalCommitBlock=block.number`. QMIG requires the legacy proxy as old authority and all empty-Queue golden values.

Failure of LGS1/verifier, LGAR, LGFN, MCAN, QMIG, a postread, decoder, receipt/index write or final switch reverts legacy FROZEN to its pre-existing QUIESCENT snapshot, restores the old Queue authority and removes all target/receipt/Router activation writes. It does not write L2 code or storage; it adopts the proved L2 poststate. If only the activation block is orphaned, all five objects roll back to the same QUIESCENT snapshot. Retry recomputes the same arm and launch IDs and may reuse the proof and candidate, but recomputes the landing-block context, legacy poststate, three MAPS commitments and receipt. If the QUIESCENT block is orphaned, its scan/fence and any proof from that orphan reject. An orphaned receipt cannot replay. FROZEN is permanent and cannot abort or upgrade. Proof failure, component mismatch or proving unavailability leaves the source QUIESCENT only until retry or hard campaign expiry, after which permissionless resume restores ACTIVE. A bad target requires expiry and a higher-nonce delayed campaign; it cannot be swapped after final review. Thus a total proving-system outage delays migration but does not permanently pause a previously processable legacy state. The campaign and operational runbook publish both admission cutoffs, exact abandoned-value aggregates and expiry bounds; production rehearsal must demonstrate two independent proving stacks and the complete resume path against the exact interface before the first cutoff. L1 kind-1 ingress is enabled only after the authenticated L2 code/authorization proofs and router-history authorization have all succeeded. That atomic cutover enables kind-0 ingress, but V2 DIRECT send and kind-1 ingress remain disabled. The old `sendMessage` selector and all L2-to-L1 sends retain exact V1 signals and events for every caller.

After the activation block becomes canonical and final, anyone submits its version/sequence and bounded `MptProofV1` registrar proof to BridgeDomainRegistry. Only that confirmation plus 214 L1 blocks enables `sendMessageV2` and kind-1 ingress for the exact tuple. No V2 Vault selector exists. Thus no source credit can target an unregistered or merely proposed L2 endpoint.

Every later Settlement/profile migration uses a protocol-generated two-phase cutover. The delayed manifest arms a specific cutover generation and changes the gate from ACTIVE to ARMED. This is one protocol-lifetime gate word owned by the non-proxy `ActiveSettlementRouter`, not a caller-supplied quiescence Boolean or replaceable contract. Its exact state is

```
(uint64 generation, uint64 activeProtocolVersion,
 uint64 targetProtocolVersion, bytes32 targetManifestHash,
 bytes32 targetRegistrationHash, uint8 phase)
```

where phase is ACTIVE, ARMED or READY. The `activeSettlementStateV1()` getter above returns the constructor-bound `LEGACY_BOOTSTRAP` proxy only while `genesisConsumed=false`; otherwise it derives `activeSettlement` from the append-only registration for `activeProtocolVersion`. There is no second mutable current-address slot. The legacy branch authenticates the exact deployment, campaign and scan commitments throughout QUIESCENT and activation, cannot appear in the ordinary registry, leaves the public gate ACTIVE during a genesis campaign and is consumed only by a successful atomic genesis switch. An expired genesis campaign is tombstoned and a later attempt uses a higher nonce; local resume clears QUIESCENT without changing the public gate. For every later migration, only ProtocolVersionManager may arm `generation=current+1` for the exact current version, greater target version and the consumed EXECUTING typed arm operation. The same transaction creates the exact LIVE VML1 lease above; an existing LIVE lease rejects and no entry point can extend its `abortAfterTimestamp`. Before arm, the manager checks every target address, runtime/configuration hash, object identity and L1 component that is knowable without executing the target L2 transition; stale, skipped, public or mismatched calls reject atomically. Settlement, registry, data-session, reward/preparation and both forced-ingress adapters all read that same router-owned word. Arming atomically closes new normal contexts/candidates, sessions, data posts, builder reservations and reward work; ingress returns SYNCED before retaining a deposit or queue write. Every canonical new-work entrypoint rechecks phase ACTIVE after `sync()`. The session open/post/seal sidecars are the sole exceptions: none calls sync, each checks current target and ACTIVE atomically, and the payable open/post calls revert with value on failure. Every ARMED cleanup and READY write returns a non-continuation status. Reaching READY sets `changed=true`, so no call can fall through and reopen normal work. Already-mature claim-only withdrawals remain callable.

For VERSION_MIGRATION, ARMED and READY are reversible because authority has not moved. At `block.timestamp>=abortAfterTimestamp`, anyone may call the immutable manager's permissionless expiry abort without a governance signature, queued operation or caller-supplied target. Activation rejects at and after the same bound. Either abort path requires the old Settlement and queue still authoritative, enters ABORTING before its callback, marks that generation canceled, clears the target fields, terminalizes the lease and returns the old version to ACTIVE without restoring deleted sessions or an already-settled candidate. It performs no same-transaction post-abort sync; IDLE is the final write and the next ordinary call may sync. A aborted generation can never cut over, and the next arm uses the next generation. The fixed lease caps closed admission at 604,800 seconds plus one inclusion bound, even if the DAO and every target prover disappear. Thus a bad deployment, unavailable proof or abandoned upgrade cannot strand the protocol in ARMED or READY. Abort changes no reservation, canonical, queue, claim or liability state. Genesis campaign publication is immutable; the bounded expiry/resume path, not target substitution, is its recovery path.

The old Settlement finishes only the already-open boundary. An open normal window may commit its already-recorded best at its deadline; a newly due head cancels it. A current recovery proof may commit until that round's existing `expiresAt`; after expiry `sync()` cancels the round and is forbidden to increment its revision or open another episode. If neither is open, arm clears a merely armed normal context immediately. At and after that boundary, admissions remain closed and every cleanup call scans exactly eight consecutive physical cells from the authenticated cursor. Empty and occupied cells both consume a step; every scanned LIVE cell becomes REFUND regardless of expiry, and no owner or sink is called. At arm, the generation freezes one objective refund deadline as saturating `uint64` addition of the 86,400-second claim window to the later of `armedAt` and the already-open normal deadline or recovery `expiresAt`. Every cell converted by that generation uses this same deadline; a keeper cannot extend it by delaying one scan. While any normal or recovery boundary remains open, no ARMED LIVE cell may be converted or claimed. A scan never converts and forfeits the same cell.

The manager's arm transaction writes the current Settlement's exact generation and this objective refund deadline before invoking any internal `sync()` that can observe the router's new ARMED phase. That write, the router phase change and the leading sync share one EVM revert domain. Thus an arm with LIVE cells and no open boundary can immediately perform its first migration scan, while any injected failure restores the pre-arm gate, deadline, cursor, cells and counters together; an ARMED scan can never observe an unset generation deadline.

The cursor advances by eight and makes `changed=true` even if the scan found no LIVE cell. Thus 128 uninterrupted migration scans cover every cell from any starting cursor; public refund maintenance cannot interleave and steer the cursor while ARMED. READY requires the source-local `liveCount` zero, not `refundCount` or `occupiedCount` zero. Claims may clear REFUND cells concurrently without changing the cursor. Once READY or FROZEN, the historical source's permissionless maintenance scans only overdue REFUND cells, and the unpausable claim/surplus surfaces remain live. Abort never restores a converted REFUND, claimed cell or forfeiture; unscanned LIVE cells remain live when ACTIVE returns, and a later generation cannot double-refund an old cell. BuilderRegistry, ScheduleOracle, generation records, tranche rings and reservations are protocol-lifetime objects shared by identity across every Settlement. Existing `RESERVED` tranches remain byte-for-byte `RESERVED`; migration never counts, scans, refunds, converts or releases them. Arming only closes creation of new reservations until ACTIVE returns. Ordinary expiry, replacement, exit, slashing and evidence-safe `RESERVED->LIABLE->RELEASED` transitions continue under their existing rules and are not migration cleanup. This keeps all already-funded current/future schedules usable after cutover and prevents a version change from manufacturing either a refund or a liability. READY is derived solely from zero live-session count, no normal boundary and no recovery; no calldata supplies those facts. The process preserves all schedule and slash/evidence state and leaves matured reward/refund claims callable on permanent claim-only surfaces. It then enters MIGRATION_READY before another `sync()` decision. A due queue head is not drained or discarded; old adapters receive SYNCED while their caller fee remains in the adapter pull-claim ledger and retry after activation.

READY is written only by the exact cross-contract handshake below:

```
migrationReadinessV1() returns
  (bytes4 magic,uint64 generation,uint64 activeProtocolVersion,
   uint64 targetProtocolVersion,bytes32 targetManifestHash,
   bytes32 targetRegistrationHash,uint8 boundaryState,
   bool localArmComplete)
markMigrationReadyV1(uint64 generation) returns (bytes4 magic)
```

The readiness magic is `0x4d525331` (ASCII `MRS1`); boundary states are exactly NONE=0, NORMAL=1 and RECOVERY=2; and its returndata is exactly 256 bytes with canonical padding. RECOVERY is derived exactly when canonical mode is RECOVERY and its exact current round exists, with no normal marker. NORMAL is derived when there is no recovery and any canonical normal boundary marker is live, including `normalArmBlockNumber`, `normalDeadline` or `normalBest`. NONE is derived only when no recovery record and none of those normal markers exists. A conflicting mode/record/marker combination makes the view revert; it can never be reported as NONE. The view derives, rather than stores, `localArmComplete=true` exactly when the shared gate is ARMED or READY for the returned tuple, `boundaryState=NONE`, the same Settlement's local LIVE count is zero, `seatMigrationLocalGeneration=generation`, the stored exact seat-arm tuple/receipt is present and matches every returned field, `migrationRefundGeneration=generation`, the frozen migration refund deadline is nonzero, and seat closure has completed. The arm callback's stored receipt phase remains ARMED permanently; the current shared gate may instead be ARMED or READY, so receipt validation compares generation, active and target versions, target manifest hash and target registration hash but never compares phase. The arm callback's leading sync always uses `allowReady=false`; it writes the refund tuple before that sync and writes the seat-arm receipt/local generation only after all local closure succeeds. Consequently even an empty source remains ARMED after the arm callback and can become READY only on a later permissionless sync. The mark success magic is `0x4d524459` (ASCII `MRDY`) encoded as one exact 32-byte ABI word; the other 28 bytes are zero.

Only the exact currently active Settlement may call `markMigrationReadyV1` and only while the Router is ARMED for that supplied generation. Router first requires lifecycle IDLE and writes READYING. Before its single READY write, it makes separate 100,000-gas `STATICCALLs` to the caller's `migrationReadinessV1()` and `dataSessionAccountingV1()` views. Revert, OOG, wrong magic, wrong exact length, noncanonical padding or any tuple mismatch reverts. The readiness view must match the Router's generation, active/target versions, target manifest and target registration, report NONE and true; the accounting view must report LIVE count zero, NOT_ENTERED guard and the registration's exact DataSession configuration hash. The Router checked-adds the returned live and refund liabilities and requires `BALANCE(sourceSettlement)` at least that total. Only then does the Router write READY, return MRDY and write lifecycle IDLE last. Settlement treats any failed call or wrong return as a revert; a successful sync returns a non-continuation changed status. Activation repeats both exact reads from that same source while READY, before any proof adoption or authority write, and requires the identical closed/zero tuple and balance/liability lower bound. Abort writes no READY state and leaves the source as the unique current Settlement. Thus no per-session global counter or caller assertion is part of the safety argument.

While READY, any prover constructs an immutable fork-local first-V2 candidate from the frozen old Canonical and live queue snapshot. It carries the exact target release/profile, complete bounded Inbox rows and raw proof bytes. The proof executes the exact release-activation call as tx0 before tx1 InboxApply, authenticates the fresh Store/Quota/Bridge and lifetime rows, and binds both raw EIP-2718 transaction hashes. The statement binds old full core/sequence, migration generation, Queue root/count/cursor/range/descriptor commitment, candidate and beneficiary, target profile/manifest, full output core, deployment observation, exact tx0/tx1 hashes and every release effect. Proof generation does not mutate L1 or canonical L2 state.

This migration candidate is exactly one block and is subject to the release- activation geometry in the custom-system-transaction section: tx0 is index 0, tx1 is index 1, the uniquely classified maximum forced kind-0 FIFO prefix (if any) follows, and the transaction list ends there. Its discretionary count and canonical discretionary `bodyRoot` are both zero; no private tail, builder-selected coinbase, blob or second block is legal. Every included kind-0 raw transaction is recovered from its canonical L1 Queue-admission transaction calldata, checked against the leaf hash and queue index; that published L1 calldata, not a prover-private witness, is the replay preimage. The circuit derives and binds the complete transaction root, receipt root, header/block hash and execution-output commitment. The migration statement's `candidateDigest` commits that exact single-block candidate and `outputCanonicalHash/outputCore.tipHash` commits its resulting block; substitution or omission changes the verifier input. If a required canonical L1 raw preimage is unavailable to the prover, migration cannot land and the immutable lease's permissionless expiry abort restores the old protocol instead of adopting an unreplayable state.

The L1 call uses the exact five-argument activation ABI frozen above. Calldata supplies only transition kind/generation/source and target versions/source sequence, candidate digest, output core, beneficiary, anchor pair, force cutoff, pre-Inbox plus-one value, the complete registered manifest, canonical Inbox rows, the bounded launch header when applicable and bounded proof. Router derives the base core/hash, statement digest, execution profile, Queue address/ code/config/root/count/start/end and descriptor commitment, verifier descriptor, tx0/tx1 preimages and hashes, target Settlement, domains/Bridges, registration identities and activation receipt from current storage plus those exact bytes. None is a second caller field. The call contains no Router, verifier or L2 object, capability, seal, authority handle or validity Boolean. Future landing block and timestamp are absent. Router rereads the exact READY tuple and all live L1 identities, reconstructs the statement and raw tx0/tx1 preimages, and invokes only the execution-profile-pinned immutable verifier by bounded STATICCALL before any write. Success requires exact descriptor/code/config/key/ schema, proof bounds and exactly 32 bytes equal to the reconstructed statement hash; short/long return, revert, OOG or substitution leaves all state unchanged.

Only after verification does the Router execute the exact shared-context MFRZ–MCAN–QMIG–MAPS journal defined above. Router never writes another account's storage: the immutable source, target and Queue each authenticate MACT and write only their own bounded transition. MCAN changes only the L1 target Settlement's Canonical/history bookkeeping; it never reads or writes the target Protocol, InboxApply, Registrar, Store, Pool, Bridge or another live L2 object. The execution node selects the immutable fork-local poststate only after observing this exact successful L1 transition for the same candidate, core, version, sequence and Queue range. Losing, abandoned, pre-cutover and reorged transitions cannot mutate canonical L2 state; a local replay failure is a node resync condition and cannot revert a valid L1 transition.

Any L1 late fault rolls all L1 state back. A canceled generation or changed source sequence invalidates the proof; later landing remains valid only while the complete bound tuple is unchanged. Kind-0 preimage loss retains its bounded expiry/discard path. Existing builder reservations and historical V1 bridge signals keep their original semantics; migration enumerates or converts neither.

13 Parameters and enforced geometry

Parameter	Initial value	Enforced relation
L2 slot / schedule window	1 s / 384 slots	Aligned by genesis timestamp.
L2 height / timestamp	`uint48` checkpoint bound	Launch import is strictly below `UINT48_MAX`; every later header is checked consecutive and at most that maximum. Timestamp equals genesis plus signed slot.

`N_MAX`, `Q_MAX`	64 / 76	Six addresses fill 384 tickets.
`BOND_MAX`	$2^{192}-1$	`bond*(Q_MAX+1)<2^256`.
Entry/exit/reservation ahead; moves	8 / 268 / 16 windows; 4 per window	Liability residence $<$ 268; capacity $4\times$ 268.
`D_SNAP`, `F_FINAL`	8 / 2 L1 epochs	Strictly exceeds scan + finality + seal margin + lookahead.
`F_FINAL_BLOCKS`	64 L1 blocks	Exact block-number alias of two 32-slot L1 epochs for campaign review/stage inequalities.
Genesis campaign preparation	1,024 proposals; 1,024 forced rows; 4 MiB	Limits apply from publication; 16 rows/call and at most 128 scan calls. Receipt binds exact counts, roots, bytes and accounted abandoned forced fees; arbitrary raw surplus is excluded and cannot veto. Bonds remain withdrawable.
Genesis scan codec/gas	3,808 B/proposal; 256 B/forced row; 62,084 B max proposal calldata; 12,000,000 gas	Exact deployed ABI shape at 10 forced plus 21 normal blob hashes. A cold maximum batch above the gas cap blocks profile release.
Genesis legacy resume proof	RISC0_RETH 5 + SP1_RETH 6; 900 s	Ordered age-independent route through immutable fixed-key adapters; generic mutable-key wrappers, SGX-required roots and unfenced remote verifier graphs are unsupported.
Genesis campaign stages	force-to-proposal $\geq$ 64; scan $\geq$ 192 blocks	Target/campaign review closes at least `F_FINAL_BLOCKS` before force cutoff; proof/finalization remains live through the scan stage.
Genesis QUIESCENT bound	600 L1 blocks / 7,200 s maximum	Entry requires at least 128 blocks and 1,800 s remaining; activation ends before either bound and resume begins at either bound. Every scanned pending/unfinalized data expiry, including an already-stale row, must exceed the time bound plus 900-s proof-generation, reorg and inclusion margins.
`H_LOOK`	768 L2 slots	Guaranteed by snapshot geometry with one-window slack.
`G_MAX`	3,600 L2 slots	Tier-1 parent gap; equals final-lag trigger and fits within the 12-window proof cap.
`W_SETTLE_SECONDS`	1,200 s	Timestamp deadline, never L1 block count.

`DELTA_TIP`	1,200 s	At least submit inclusion + skew = 144 s after the frozen escape slot.
`DELTA_FINAL_LAG`	3,600 s	At least activation + escape offset + submit + skew = 2,164 s.
`F_L1`; `T_DEPTH_MAX`	64 L1 blocks; 900 s	Stored recovery-anchor depth and assumed maximum time to reach it.
`P_PROVE_MAX`	900 s	Exact alias of `recovery.proofTimeMaxSeconds`; used by session OPEN and recovery.
`ESCAPE_OFFSET`	1,900 s	At least depth-time bound + proof bound + margin.
`FORCE_DELAY`	1,500 s	At least settlement window plus `T_INCLUDE_MAX_SECONDS=120`.
Aggregator seat geometry	1 primary + 3 standbys; 4 waiting cells; 1 stage	`pendingCount+stagedCount<=4`; every canonical seat/duty pass is fixed at four cells.
Seat runway/tenure	UNCALIBRATED	`SEAT_RUNWAY>=MIN_PRIMARY_TENURE+HANDOVER_EXECUTION_BUFFER+SLA_TAIL`; the handover buffer covers delay, grace and L1 inclusion.
Seat premium/bond	UNCALIBRATED	Claim delay covers reorg stability; SLA bond covers maximum-ask closed-tail grief and the modeled collusive-promotion subsidy. An uncalibrated profile rejects.
Forced prefix	64 items; 1,048,576 B; 20m gas	Each item at most 131,072 B and 5m accounted gas; descriptors and one boundary remain durable.
Candidate caps	4,096 blocks; 12 windows; 1 anchor; 16 sessions; 2,100 records	Also 256 forced items, 4 MiB, 80m accounted gas.
Queue / range proof	depth 64 / at most 257 hashes	`queueCount<UINT64_MAX`; final leaf unused; canonical contiguous range proof.
Same-L1 credit read	fixed-size record / bounded staticcall	Launch source equals settlement Ethereum; registry runtime, return length, record, source generation and Bridge liability are checked directly and atomically.
Bridge domain release	at most 64 new entries/version; append-only	Only atomic activation of a delayed immutable manifest registers entries; the historical registry has no global cap, delete, redirect or reinterpretation.
Source generation support	one immutable active-profile entry	The frozen endpoint tuple is usable only after `SUPPORT_FINALITY_BLOCKS`; no automatic implementation boundary exists.

V2 launch asset scope	DIRECT ETH only	ERC20/ERC721/ERC1155 and every Vault flow remain V1; a future asset kind cannot reinterpret DIRECT.
Kind-0 validity / kind-1 enqueue	604,800 / 604,800 s	Bounds missing bytes before expiry or source cancellation.
`BRIDGE_PROCESS_TTL_SECONDS`	2,592,000 s	Starts at the immutable L2 pin; afterward anyone transitions NEW/RETRIABLE to FAILED without Message bytes.
Pool read / authorization cleanup / value callback gas	50,000 / 50,000 / 100,000	Exact immutable stipends/reserve; the release benchmark covers every cold path, EIP-150 boundary and fixed return copy with at least 30% margin.
Pool Bridge-prefix / consume-call bounds	UNCALIBRATED / UNCALIBRATED	The executable profile must publish nonzero measured values; positive-gas forwarding adds the exact Message budget and preserves both outer reserves.
Terminal accumulator/proof	depth 64 / 64 siblings / 2,048 B	Checked `uint64count<UINT64_MAX`; leaf commits index, domain, Bridge, credit and terminal; storage is 64 frontier words plus root/count, and a complete leaf event supports trustless proof verification with explicit archive-availability dependence.
Settlement canonical history	256 version-sequence cells	$256 > \lceil (T_{include} + REORG\_MARGIN)/12 \rceil + 2$ with at most one canonical write per L1 block; the exact version/sequence tag prevents sparse L2 heights from choosing or churning a cell, and frozen versions never overwrite again.
Registration MPT proof	66 nodes/path; 600 B/node; 80,000 B; 8m gas	Canonical RLP and exact account/storage paths under the routed canonical state root; all bounds reject before an unbounded allocation or call.

L1 migration activation	Genesis: 0 rows/2,048-B header; version: 64 rows/0-B header; at most 131,072-B proof	Let `ceil32(x)=32*ceil(x/32)` and `genesisActivationBytes= 4+82*32+32+(32+ceil32(2048))+ (32+ceil32(maximumProofBytes))` and `versionActivationBytes= 4+82*32+(32+32*64+62*(6*32) +2*(6*32+ceil32(541)))+32+ (32+ceil32(maximumProofBytes) )`. The exact bound is `maximumActivationCalldataBytes= max(genesisActivationBytes, versionActivationBytes)`. GENESIS cannot carry a row; VERSION_MIGRATION has an empty header and its 13m activation budget permits at most two 541-byte kind-1 rows. `worstCaseActivationIntrinsicGas= 21000+16*maximumActivationCalldataBytes`. Activation requires $\lceil 1.30\,(G_{intrinsic} + worstCaseActivationAdoptionGas)\rceil \leq supportedL1BlockGasLimit$. The adoption measurement already includes a verifier that consumes the full configured limit.
Body chunks	9 / 1,142,748 B	Per-body chunks / maximum discretionary bytes.
`DATA_TTL`	86,400 s	Greater than candidate, settlement, proof and reorg path.
Data refund claim window	86,400 s	Same-cell REFUND grace; at least inclusion plus reorg margin.
`REORG_MARGIN`, `EVIDENCE_DELAY`	1,800 / 86,400 s	Data resubmission and finite liability retention.
`SUPPORT_FINALITY_BLOCKS`	214 L1 blocks	Begins only after the domain's finalized canonical L2 registrar record is proven to L1; a frozen endpoint/domain cannot create credits earlier.
Live windows / sessions	268 / 1,024	Fixed rings with safe-eviction predicates and bounded GC.
EIP history / max arm age	8,191 / 255 blocks	$255 + \lceil (W_{settle} + CLOCK\_SKEW + 384 + EVIDENCE\_DELAY + REORG\_MARGIN)/12\rceil + 2 < 8,191$; greater than every normal-context evidence and schedule path.

Launch tooling evaluates every inequality in both seconds and its native clock unit. Governance cannot set a value that violates a relation.

14 Production gates and verdict

The core consensus state machine and migration/control-plane transition protocols are frozen candidates, with no known open Critical design defect. The Settlement–Market seat mutation wire protocol remains one open High implementation-readiness blocker; the initial executable profile, compiled contract and circuit release bundle are also absent. This document is therefore neither an implementation freeze nor production authorization. Before deployment, one reviewed release must contain all of the following concrete artifacts and two independent implementations must reproduce every hash and execution result:

- strict canonical ABI schema, exact profile bytes and `executionProfileHash`, plus a rejecting decoder corpus;
- an immutable manifest binding protocol version, chain IDs, activation, profile hash, verifier address/runtime hash and verifier-key hash;
- exact predeploy and immutable topology, including both fresh V2 Bridge bundles, CREATE2 factory/salt/init-code derivation, Bridge-unique Registry and QuotaManager accounts and the non-proxy/no-owner/no-upgrade rule for every V2 endpoint; the 232-byte Settlement deployment descriptor, published target init-code artifact and CREATE2 derivation, internal DataSession custody/configuration and rejection of every other payable Settlement path, `targetRegistrationHash` inclusion in delayed arm/cancel/activation/receipt records, exact configuration-read ABI and fresh-PREACTIVE rejection corpus; the exact MACT/MFRZ/MCAN/QMIG/MAPS and legacy LGS1/LGAR/LGFN implementations, shared-context/poststate corpus, per-call stipend/EIP-150 reserve harness and Router lifecycle journal specified above; the storage-layout-compatible final legacy proxy implementation and a rehearsal proving no old proposal, forced-ingress, initializer or upgrade path bypasses its local phase; the age-independent legacy resume root, immutable fixed-key RISC0/SP1 adapters and recursively bound remote verifier graph; the final direct legacy SignalService implementation and shared campaign upgrade fence, with ForkRouter and unfenced trust-map configurations rejected; BuilderRegistry and ScheduleOracle runtime/configuration hashes and retained object identities; terminal runtime code hashes, constructor immutables and relevant storage selectors for AnchorV4, InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2, TerminalAccumulatorV2, ActiveSettlementRouter, ForcedQueue (including its depth, capacity, empty leaf, descriptor schema, router binding, accounted-liability lower bound, forced-ETH surplus behavior and `forcedQueueConfigHash`), both immutable ingress adapter runtimes/configurations and their append-only router registrations, L1/L2 legacy SignalService, fresh V2 Bridges, Bridge-unique QuotaManagers and NativeLiquidityPoolV2, including proof that no V2 upgrade selector, owner, reinitializer or delegate target is reachable, plus the exact chain placement and L1-direct/L2-direct/finalized-manifest validation trace for every infrastructure row;
- exact nonzero source/destination domain namespaces and vectors, append-only BridgeDomainRegistry authorization, BridgeCreditRegistry direct ABI/runtime/record layout, immutable source generations, Bridge aggregate and per-credit liability/status/pull-credit layouts, per-Settlement sequence-tagged canonical-history rings and fixed-depth TerminalSignalVerifier vector grammar, 64-word accumulator frontier/event layout and append gas bound, byte-exact Bridge-kernel and destination-infrastructure descriptors, registrar/adapter/store one-shot bindings, finalized L2 registration proof, stamped two-call

ingress ABI, proof-first router activation ABI/invariants, exact live-lease expiry/abort transition, direct-call limits and valid/invalid fixed-return corpus;

- exact unsigned system transaction type and reserved sender addresses; chain-ID equality, caller/origin injection, initial warm set, intrinsic gas, refund quotient, zero GASPRICE, nonce/balance non-transition and exact typed transaction/receipt trie values; Store/Bridge release seal and cutover proof; system nonces; the one DIRECT-only additive BridgeV2 selector and bounded immutable inbox-store batch ABI, selectors and storage; fee handling, gas ceilings and all fork and header recurrence rules, including byte-exact first activation and steady-state type-`0x7f` transaction/receipt/header vectors; and
- complete positive and negative vectors for parent, prestate, input, transactions, receipts, headers and poststate, as listed in §12, including domain/source-generation changes, unauthorized clones and foreign local-domain replay, enqueue/cancel and capacity races, PREACTIVE rejection, legacy preactivation attempts, first-block release activation, direct-record absence, router-SYNCED refund/retry, old-domain routing, permanent-pin independence from legacy upgrades, processing expiry, DONE release, FAILED recall, legacy SignalService pause, zero ordinary ETH quota, pooled-balance drain attempts, rejected Vault callers, endpoint authorization, exact old-selector V1 compatibility, explicit V2 opt-in, terminal-root/count public-output binding, canonical-sequence collisions, historical proof after later appends, direct-sender/Bridge/foreign-Anchor release attempts, reserved-system-sender-to-Anchor activation, old-domain terminal append after new-domain activation, mixed old/new-domain global inbox batches and domain/Bridge proof substitutions, multi-credit batches and every V1/V2 replay path, manifest/profile dependency-cycle rejection, chain-ID overflow/local mismatch, fresh/partial/reused endpoint state, singleton queue proofs at and across the 2^{32} boundary, forced-ETH surplus, stale ingress stamps, failed target proofs/components, ARMED/READY expiry abort, shared-gate identity and two complete successive migration generations.

Those values are outputs of real compiled contracts, fork code and a real circuit/verifier key. Inventing placeholders in LaTeX or Python would make the profile hash look precise while leaving clients unable to construct the same escape block. This is missing normative consensus, not an empirical launch gate. Consequently a sound implementation-ready artifact cannot be completed by document-only revision; implementation and circuit work must now produce the bundle. Once it exists and is re-audited, production deployment remains blocked until every additional gate below produces a versioned artifact and passes:

1. **Proof performance.** Independent tier-1, tier-2 and worst-case tier-3 proofs, including 4,096 blocks and 2,100 data records, finish within 900 seconds on the published minimum hardware. Peak RAM, recursion setup and failure recovery are measured; a miss requires changing parameters and another design review. Before the first genesis force cutoff, two independent implementations must also scan, quiesce, generate and verify the exact campaign-to-target genesis proof from a shadow-fork snapshot. Each independent pipeline must generate the complete accepted RISC0+SP1 pair; removing either proof ecosystem must block activation but must not block permissionless expiry and local resume. A drill also advances beyond SGX instance expiry and proves that freshly generated route proofs, rather than cached SGX bytes, finalize the retained legacy suffix.
2. **Contract gas.** `sealWindow`, `postData`, normal submit, `sync`, direct source-credit enqueue, recovery submit, slash and garbage collection fit current L1 gas limits with 30% margin. The same compiled- bytecode harness exercises cold maximum REGISTER_RELEASE with

the release's actual strict canonical ABI profile and both required ingress rows, including the named-gas Router call, Market install and every bounded postread; maximum REGISTER_FORK_VERIFIER; and publication, arm and both abort branches under their 8,000,000-gas Router caps. The harness searches and publishes each cold-path minimum, proves the frozen nested cap is at least `ceil(1.30*measuredMinimumGas)`, and succeeds at that cap; the enclosing intrinsic plus execution gas also has at least 30% headroom below `supportedL1BlockGasLimit`. The published fixture binds compiler version, optimizer settings, fork rules, storage warm/cold assumptions, exact profile bytes and every component runtime/configuration hash. Any failure changes the candidate constant/profile and reopens design review before the relevant typed operation can be queued. Fuzzing also includes empty registry, zero seat, full data session, full queue prefix and maximum history age.

3. **Cryptographic conformance.** Solidity, Go, Rust and circuit code match the Keccak/EIP-712 golden vectors, direct same-L1 Bridge registry ABI and runtime-hash checks, KZG Fiat–Shamir transcript and exact body manifest. Two independent implementations generate every vector.
4. **State-machine verification.** Stateful fuzzing models EVM rollback, same-block ordering, reorg replay, stale versions, activation/close coincidence, message consume-and-discard and storage reclamation. The executable models in Appendix D remain green.
5. **Economics.** Auction bond, per-window tranche, forced-envelope deposit, storage bond and proof credits are calibrated under blob/L1 fee spikes. The published liveness claim explicitly becomes conditional above fee caps.
6. **Operations.** At least two independent prover stacks and three independently administered canonical archives, plus public data-session mirrors, schedule sealers and recovery bots, complete a shadow-fork outage exercise. One archive serves a clean node, after deleting its state, body, receipt *and code* databases, from only the latest finalized package while the other two are removed. The exercise also removes every builder and seat, forces a user transaction, reorgs the first recovery attempt, and still reaches finality. Loss and restore drills verify that unavailable prestate is never misreported as bounded recovery. For every supported source domain, an analogous drill creates an old immutable credit record, removes every message-archive copy, exercises source cancellation, then recreates a message, orphans its direct enqueue transaction and replays it from the surviving record. It also exercises two separately registered frozen source generations, attempts and fails to deploy a wrong CREATE2 bundle or mutate an immutable V2 account, pauses legacy SignalService and zeros ordinary quotas, replays a pin through a second funded Bridge, expires an unprocessed pin and proves that a superseding profile cannot remove an old domain or endpoint entry. The exercise also deletes one terminal-log archive, rebuilds a historical 64-sibling proof from a second independently operated archive, and verifies it against a retained Settlement root/count.
7. a Foundry activation harness using the exact release bytecode and profile-maximum proof. It covers the largest reachable successful genesis call (zero Inbox rows, a 2,048-byte header and the maximum proof), the version- migration 62-kind-0/two-kind-1 cold path with empty header, and the configured verifier consuming its complete stipend. It measures calldata intrinsic gas separately and the complete cold Router execution as one non-overlapping bound, while also reporting decode/read/getter/verifier/call-overhead/suffix components for diagnosis; it succeeds below `supportedL1BlockGasLimit`, and demonstrates at least 30% margin by the enforced integer inequality. It separately

exercises exact and one-less gas for MACT, MFRZ/LGFN, MCAN, QMIG, all three MAPS reads, the separate atomic legacy campaign scan/quiescence/resume, LGAR/LGS1 and the retained activation suffix; each injected late fault proves byte-identical rollback of source, target, Queue, Router and receipt. Any compiler, fork or storage-layout change reruns and republishes that measurement before release;

8. **External review.** Separate proof-system, Solidity, bridge and economic audits close all critical/high findings; the migration rehearsal and emergency governance path are in scope.

Verdict. *The earlier fallback architecture was not implementation-ready: it depended on a scheduled builder, could revert activation inside a rejected submission, made payment a commit precondition, and could not bind whole-window data with a six-blob cap. This revision removes those dependencies, fixes a single consensus path, binds bridge credits to their source application, and defines bytecode-complete recovery packages. Its bridge integration now uses fresh immutable non-proxy V2 bundles, keeps legacy callback applications on V1, binds the local destination, and makes V2 terminal proof/refund paths transitively pause- and quota-independent. Router foreign-storage assumptions are removed: source, target, Queue and legacy proxy now authenticate one context and commit their own poststate inside one rollback journal, with a complete sealed receipt. The design is feasible and frozen for contract/circuit implementation; it is not an impossible protocol. Safe in-place genesis migration is conditional on the deployed legacy Inbox accepting the exact final compatibility implementation; without that capability, in-place cutover for that deployment is impossible and must be replaced by an independently initialized state migration. The absent compiled executable profile, measured gas values, verifier/circuit artifacts, multi-language conformance and external audits remain production release gates, not unresolved document-level consensus choices. Python fixture hashes are not substitutes for those compiled artifacts.*

A Normative commitment encodings

All integers in packed commitments are unsigned big-endian at the named width; addresses are 20 bytes, hashes are 32 bytes, and ASCII domains have no length prefix or trailing NUL. `H` is Ethereum legacy Keccak-256. No `abi.encodePacked` call may contain a dynamic field. Raw transaction bytes are represented by `H(rawTx)` plus an explicit `u32(byteLength)`.

The canonical core/base, candidate, schedule list, sealed-session list and execution outputs use these exact encodings:

```
coreHash = H("slot-chain-core-v3" || u64(l2BlockNumber) || tipHash ||
  u64(tipSlot) || stateRoot || u64(messageCursor) || winningDataCommitment ||
  u256(nextBaseFee) || u64(nextExcessBlobGas) || terminalRoot ||
  u64(terminalCount))
baseCanonicalHash = H("slot-chain-canonical-v2" || coreHash ||
  u64(canonicalizedAtBlock))
H("slot-chain-candidate-v2" || baseCanonicalHash || u16(n) ||
  (u64(slot) || blockStructHash || blockHash || bodyRoot ||
   dataManifestRoot || u64(messageEnd))*)
H("slot-chain-schedule-list-v1" || u8(count) ||
  (u64(window) || entryRoot || seed)*)
H("slot-chain-session-list-v1" || u8(count) ||
  (sessionId || u16(recordCount) || root)*)
winningDataCommitment = H("slot-chain-winning-data-v1" ||
  candidateCommitment || sessionRefsCommitment)
H("slot-chain-outputs-v2" || stateRoot || transactionsRoot || receiptsRoot ||
  logsBloomHash || withdrawalsRoot || terminalRoot || u64(terminalCount))
```

Here $n \leq 4096$; schedule windows are ascending and sessions are ascending by ID. `blockStructHash` is the EIP-712 struct hash of the exact signed `SlotChainBlock` fields in §5.1; tier-3 recovery uses the same struct fields without a signature. These hashes are respectively the statement's `baseCanonicalHash`, `candidateCommitment`, `scheduleRootsCommitment`, `sessionRefsCommitment` and `executionOutputsCommitment`. Settlement must recompute the displayed `winningDataCommitment` before verifier dispatch and again before the canonical write; unsorted or duplicate lists reject.

The 64-cell registry leaf at index i is

```
H("slot-chain-registry-leaf-v1" || u8(i) || u8(present) ||
  address20 || u192(bond) || u64(registrationIndex) ||
  u64(effectiveL2Slot) || bytes32(trancheRoot) || u64(tombstonedAtL2Slot))
```

An empty cell has `present=0` and every following byte zero. A present cell has `present=1`; `UINT64_MAX` means not tombstoned. At tree height $h = 0, \ldots, 5$, the parent is:

```
H("slot-chain-registry-node-v1" || u8(h) || left || right)
```

There is no padding because there are exactly 64 leaves.

Admission position $i < 1,136$ is:

```
H("slot-chain-admission-leaf-v1" || u16(i) || u8(present) ||
```

```
  u8(location) || address20 || u192(bond) || u64(registrationIndex) ||
  u64(effectiveL2Slot) || u64(tombstonedAtL2Slot))
```

Location is 1 for active and 2 for liability; an empty leaf has present and all following fields zero. Positions $1,136\ldots2,047$ and unused positions are canonical empty leaves. Its eleven levels use `H("slot-chain-admission-node-v1"||u8(height)||left||right)`. `admissionVersion` increments before publishing the corresponding root; the pair is stored together and never reconstructed from a newer root. Tranche roots occur only in the active registry and liability storage records, not in this stable signer-identity commitment.

The omission-free ranked-entry commitment for window W has exactly 64 leaves:

```
H("slot-chain-entry-leaf-v1" || u8(rank) || u8(present) || address20 ||
  u192(bond) || u64(registrationIndex) || u64(effectiveL2Slot) ||
  u64(tombstonedAtL2Slot) || bytes32(trancheLeafHash))
```

followed by six levels of `H("slot-chain-entry-node-v1"||u8(height)||left||right)`. Empty ranks have all fields zero. The schedule circuit derives all 384 tickets from this root and seed; a schedule list cannot substitute a different set.

Each registry cell's tranche ring is a 512-leaf tree. Leaf index j is:

```
H("slot-chain-tranche-leaf-v1" || u16(j) || u64(window) || u8(state) ||
  u192(amount) || u64(liableUntil))
```

An empty leaf has `window=UINT64_MAX`, state `EMPTY=0` and remaining fields zero. Other states are `FREE=1`, `RESERVED=2`, `LIABLE=3`, `RELEASED=4` and `SLASHED=5`. A ring-index collision may be overwritten only from RELEASED or SLASHED after its absolute release predicate; the write installs the new exact window and FREE/RESERVED state atomically. `liableUntil=0` for EMPTY/FREE; reservation sets it exactly to `evidenceDeadline(window)+REORG_MARGIN_SECONDS`, and LIABILITY preserves that value. RELEASED/SLASHED preserve it until safe overwrite. Its nine node levels use `H("slot-chain-tranche-node-v1"||u8(height)||left||right)`. The snapshot supplies one inclusion proof for index `Wmod512` for every active cell, including canonical empty and `FREE` leaves.

Kind-0 forced leaf i is legacy Keccak over this exact packed sequence:

```
"slot-chain-force-user-v2" || u64(i) ||
address20(sender) || u64(nonce) || u256(l2ChainId) || bytes32(rawTxHash) ||
u32(byteLength) || u64(gasLimit) || u64(accountedGas) ||
u256(maxFee) || u64(validUntil) || address20(refundAddress) ||
u64(enqueuedAt) || u64(dueAt) || u256(deposit)
```

A kind-1 leaf is:

```
"slot-chain-force-bridge-v11" || u64(i) || msgHash ||
u256(srcChainId) || sourceDomainId || u64(srcEpoch) || address20(srcBridge) ||
bridgeExecutionHash || u64(emittedAtBlock) || destinationDomainId ||
u256(destChainId) || u64(enqueueBy) ||
address20(sender) || address20(srcOwner) || address20(destOwner) ||
u256(value) || u64(fee) || u64(liquidityFee) || calldataHash || u8(refundMode) ||
address20(refundVault) ||
```

```
refundCapsuleHash || escrowId ||
u32(byteLength) || u64(accountedGas) || address20(refundAddress) ||
u64(enqueuedAt) || u64(dueAt) || u256(deposit)
```

Its application has no expiry; no `validUntil` field exists. Launch V11 requires `refundMode=DIRECT`, `refundVault=address(0)` and `refundCapsuleHash=bytes32(0)`; these retained fixed-width projection fields cannot be reinterpreted as an asset mode.

For `forcedDescriptorCommitment`, `descriptorBytes` is exactly the corresponding leaf sequence beginning immediately after `u64(i)`; the list already supplies index and kind, so neither leaf ASCII domain nor index is duplicated inside it. Its fixed byte length is 220 for kind 0 and 541 for kind 1. The list encoding and optional first excluded boundary are defined in §8. The circuit prepends the correct leaf domain and index to reconstruct every Merkle leaf; any other length rejects.

The DIRECT-only acyclic `canonicalBridgeKernelDescriptor` is exactly 115 bytes:

```
u16(kernelVersion=2) || u8(DIRECT=1) ||
u64(MAX_BRIDGE_ENQUEUE_DELAY=604800) ||
u64(BRIDGE_PROCESS_TTL_SECONDS=2592000) ||
bytes32(bridgeV2AbiHash) || bytes32(statusTransitionHash) ||
bytes32(custodyRulesHash)
```

The three final hashes commit byte-exact selector/tuple/event grammar, status transition table and liability/reserve/pause rules in the release bundle. None may contain a source/destination domain, infrastructure hash or full `executionProfileHash`. The kernel hash is

```
bridgeKernelProfileHash =
  H("slot-chain-bridge-kernel-profile-v2" || u32(115) ||
    canonicalBridgeKernelDescriptor)
```

`canonicalSourceBridgeDescriptor` has this exact 752-byte packed grammar:

```
address20(factory) || bytes32(factoryRuntimeHash) ||
bytes32(factoryConfigurationHash) || bytes32(bundleSalt) ||
bytes32(bundleInitCodeHash) || address20(bundleDeployer) ||
bytes32(bundleDeployerRuntimeHash) ||
address20(legacyV1Bridge) || address20(sourceBridge) ||
bytes32(bridgeRuntimeHash) || bytes32(bridgeConfigHash) ||
bytes32(storageLayoutHash) || address20(creditRegistry) ||
bytes32(registryRuntimeHash) || bytes32(registryConfigHash) ||
address20(quotaManager) || bytes32(quotaRuntimeHash) ||
bytes32(quotaConfigHash) || address20(supportRegistry) ||
bytes32(supportRegistryRuntimeHash) ||
bytes32(supportRegistryConfigurationHash) ||
address20(terminalVerifier) || bytes32(terminalVerifierRuntimeHash) ||
bytes32(terminalVerifierConfigHash) || address20(pauser) ||
address20(signalService) || bytes32(bridgeKernelProfileHash) || u64(srcEpoch)
```

Define, over the literal ASCII domains shown,

```
create2(factory,salt,initHash) = low20(keccak256(
  0xff || address20(factory) || bytes32(salt) || bytes32(initHash)))
createChild(bundleDeployer,nonce) = low20(keccak256(
  0xd6 || 0x94 || address20(bundleDeployer) || u8(nonce)))
bundleDeployer = create2(factory,bundleSalt,bundleInitCodeHash)
sourceBridge = createChild(bundleDeployer,1)
creditRegistry = createChild(bundleDeployer,2)
quotaManager = createChild(bundleDeployer,3)
```

Here `0xd6` and `0x94` are the canonical RLP prefixes for the 23-byte `[address,smallNonce]` preimage. The bundle init code is the release artifact that commits the generic constructor logic plus every child creation-code/configuration primitive. Its constructor computes the three addresses from `address(this)`, performs exactly those three CREATEs at nonces 1, 2 and 3, verifies their returned addresses/configurations and returns the pinned inert nonproxy bundle-deployer runtime; any child failure reverts all three. Thus peer addresses are runtime-derived rather than embedded in the CREATE2 preimage, and a configuration change changes the bundle init hash and the entire address set without a hash fixed point or post-deployment configuration race.

Every address/hash and epoch is nonzero. The factory, bundle deployer, legacy Bridge, SourceBridge, Registry and QuotaManager are pairwise distinct; the bundle and all three children equal those derivations and have the exact current runtime and configuration hashes. Factory code/configuration likewise equals the first three descriptor fields, and the support Registry's code/configuration equals its three descriptor fields. After each runtime-hash check, every configuration-bearing source object (factory, SourceBridge, credit Registry, QuotaManager, support Registry and terminal verifier) is read only through `componentConfigHashV2()` with exactly four calldata bytes, a 50,000-gas `STATICCALL` and exactly one 32-byte return. Its selector is `0xf6c0f7d2`. Fault, empty/short/trailing return or mismatch rejects; no opaque factory receipt substitutes for these direct checks. The support registry stores the whole manifest-authorized descriptor and computes:

```
bridgeExecutionHash = H("slot-chain-source-bridge-execution-v4" ||
  u32(752) || canonicalSourceBridgeDescriptor)
```

The descriptor's `terminalVerifier`, runtime hash and configuration hash are exactly infrastructure component row 3. Its one configuration is the 21-byte `address20(activeSettlementRouter) ||u8(64)` grammar below and its getter returns that row's `configHash`. It does not fold the BridgeDomainRegistry, its own address/runtime, Router runtime/configuration or any live support record into a second verifier configuration identity; those authorizations remain independent SourceBridge/Registry checks. The credit registry exposes this fixed descriptor for the source generation; there is no block-indexed executor. Direct admission checks the immutable runtime and record on the same L1. No source state root, beacon walk or caller-chosen topology proof exists in that path.

`canonicalDestinationBridgeDescriptor` has this exact 248-byte packed grammar:

```
address20(bridge) || bytes32(runtimeHash) || bytes32(configurationHash) ||
bytes32(storageLayoutHash) || bytes32(bridgeKernelProfileHash) ||
address20(inboxCreditStoreV2) || address20(terminalAccumulatorV2) ||
address20(terminalDomainRegistrarV2) || address20(quotaManager) ||
address20(nativeLiquidityPoolV2)
```

Every field is nonzero and `bridge=destinationBridge`. The account itself must be fresh immutable non-proxy code with the exact runtime/config/layout; no topology byte, implementation slot, reuse branch or delegated getter exists. It commits destination custody, terminal handshake, quota and Pool binding separately from the source descriptor. The hash is:

```
destinationBridgeExecutionHash =
  H("slot-chain-destination-bridge-execution-v3" || u32(248) ||
    canonicalDestinationBridgeDescriptor)
```

`canonicalDestinationInfrastructureDescriptor` has exactly ten component records in this order:

```
BridgeInboxAdapter, ActiveSettlementRouter, TerminalSignalVerifier,
InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2,
TerminalDomainRegistrarV2, TerminalAccumulatorV2, NativeLiquidityPoolV2,
destination Bridge
```

Each record is

```
address20(component) || bytes32(runtimeHash) || bytes32(configHash)
```

so the descriptor is exactly 840 bytes. For kind $k = 1, \ldots, 10$ in that order,

```
configHash = H("slot-chain-component-config-v1" || u8(k) ||
               u16(byteLength) || configBytes)
```

Kinds 1–10 expose the fixed getter

```
componentConfigHashV2() view returns (bytes32)
```

A checking side requires exactly 32 bytes equal to this value under the manifest-pinned runtime hash. Empty, short, trailing or malformed returndata rejects. L1 checks rows 1–3 directly and L2 checks rows 4–10 directly; neither substitutes a Boolean result for the fixed call and hash comparison. The circuit also authenticates the fresh row-10 account/runtime/config/layout and empty private state directly under the imported state root; the getter is not a substitute for those observations. The ten `configBytes` lengths are exactly 80, 168, 21, 73, 60, 52, 80, 21, 76 and 164 bytes, respectively, and their contents are:

1. `address20(bridgeCreditRegistry) || address20(sourceBridge) ||`
   `address20(activeSettlementRouter) || address20(protocolVersionManager)`
2. `address20(protocolVersionManager) || bytes32(protocolVersionManagerDescriptorHash) ||`
   `address20(forcedQueue) ||`
   `bytes32(forcedQueueRuntimeHash) || bytes32(forcedQueueConfigHash) ||`
   `bytes32(routerNamespace)`
3. `address20(activeSettlementRouter) || u8(64)`
4. `address20(terminalDomainRegistrarV2) || address20(inboxSystemSender) ||`
   `bytes32(routerNamespace) || u8(64)`
5. `address20(inboxApplyRouterV2) || address20(destinationBridge) ||`
   `address20(terminalDomainRegistrarV2)`

6. `address20(anchorSystemSender) || bytes32(manifestNamespace)`
7. `address20(protocolReleaseAuthorityV2) || address20(inboxApplyRouterV2) || address20(terminalAccumulatorV2) || address20(nativeLiquidityPoolV2)`
8. `address20(terminalDomainRegistrarV2) || u8(64)`
9. `address20(terminalDomainRegistrarV2) || bytes32(poolNamespace) || u64(POOL_EXTERNAL_READ_GAS) || u64(POOL_AUTH_CLEANUP_GAS) || u64(POOL_VALUE_CALLBACK_GAS)`
10. `bytes32(storageLayoutHash) || bytes32(bridgeKernelProfileHash) || address20(inboxCreditStoreV2) || address20(terminalAccumulatorV2) || address20(terminalDomainRegistrarV2) || address20(quotaManager) || address20(nativeLiquidityPoolV2)`

The runtime semantics for kind 1 fix its stated initializer authority and bind the protocol-lifetime Queue code/configuration into every release. Kind 4 is protocol-lifetime and preserves its cursor and existing routes while requiring the new route absent. Kind 5 requires a fresh endpoint domain and empty pin state. Kinds 6–9 are protocol-lifetime and preserve all old append-only release/domain/writer/ticket entries while requiring only the new release/domain registrations absent. Kind 10 requires fresh immutable code/configuration, empty private state, full initial quota and `v2Active=false`; exact reuse is forbidden. Tx0 seals kinds 5 and 10 and registers the new kind-9 Pool domain atomically. It transfers no ETH. Kind 4 contains only protocol-lifetime constants and never an endpoint Store or Bridge; endpoint-specific targets enter only through the registrar's append-only route. `configBytes` excludes the derived `destinationDomainId` itself to avoid self-reference. Every component address/runtime/config hash is nonzero and equals the separately named domain/profile field. The infrastructure commitment is:

```
destinationInfrastructureHash =
  H("slot-chain-destination-infrastructure-v3" || u32(840) ||
    canonicalDestinationInfrastructureDescriptor)
```

Rows 1–3 name L1 contracts and rows 4–10 name L2 contracts. At manifest activation, L1 ProtocolVersionManager directly checks rows 1–3 against L1 code/configuration and commits all ten rows. The finalized manifest proof authenticates those L1 rows to ProtocolReleaseAuthorityV2; TerminalDomainRegistrarV2 directly checks only rows 4–10 against L2 code/configuration. Both sides recompute the same 840-byte descriptor and hash. No contract is required or permitted to treat a cross-chain address as locally inspectable code.

Each release profile also publishes the complete bounded `ProfileIngressAuthorizationV2[]` from which the manifest's `ingressAuthorizationRoot` is recomputed. Its Solidity type is fixed:

```
struct ProfileIngressAuthorizationV2 {
  uint8 kind; address adapter;
  bytes32 adapterRuntimeHash; bytes32 adapterConfigurationHash;
  address activeSettlementRouter;
  bytes32 routerRuntimeHash; bytes32 routerConfigurationHash;
  address forcedQueue;
  bytes32 queueRuntimeHash; bytes32 queueConfigurationHash;
  bytes32 sourceDomainId; uint64 sourceRegistrationEpoch;
```

```
  bytes32 sourceBridgeExecutionHash;
  uint256 destinationChainId; bytes32 destinationDomainId;
  address destinationBridge; bytes32 destinationBridgeExecutionHash;
  bytes32 destinationInfrastructureHash;
  uint256 fixedIngressWei; uint256 executionWeiPerAccountedGas;
  uint256 proofWeiPerAccountedGas; uint256 permanentWeiPerByte;
  uint256 maximumAcceptedFeeWei;
}
```

`INGRESS_AUTHORIZATION_TYPEHASH` is the Keccak hash of the exact canonical Solidity signature above with the field names and order shown. `authorizationId=keccak256(abi.encode(TYPEHASH,rowfields...))`. Kind 0 requires its three source fields and all four destination-topology fields (domain, Bridge, Bridge execution hash and infrastructure hash) to be zero but binds the nonzero destination chain, Router, Queue and exact kind-0 adapter. Kind 1 requires all source and destination fields nonzero and equal to the separately committed source Bridge and destination infrastructure descriptors; its adapter configuration independently binds the exact source Registry/Bridge and Router/Queue constructor graph. Both kinds use the same nonzero five-word fee schedule.

This version has exactly two rows: one kind-0 row and one kind-1 row; multiple rows of either kind reject even when their adapters differ. A future kind requires a new profile/schema version and cannot consume latent capacity or reinterpret launch roles. The decoder rejects an unknown kind, noncanonical ABI padding, duplicate adapter, duplicate authorization ID or a row inconsistent with the profile graph. IDs are sorted in ascending unsigned `bytes32` order and

```
idsHash = keccak256(sortedAuthorizationId[0] || ... ||
                    sortedAuthorizationId[count-1])
ingressAuthorizationRoot = keccak256(abi.encode(
  INGRESS_AUTHORIZATION_ROOT_TYPEHASH, uint16(count), idsHash))
```

where the root typehash is Keccak of `"IngressAuthorizationRootV2(uint16count,bytes32idsHash)"`. Concatenation contains only the fixed 32-byte IDs and has no length prefix; the typed root supplies the count. ProtocolVersionManager recomputes this value from canonical profile bytes before registration and stores an append-only `authorizationId->row` and `adapter->authorizationId` index. No caller-supplied root, row-validity Boolean or post-cutover governance bind is accepted.

The Queue exposes this exact constructor/configuration commitment:

```
descriptorSchemaHash = H("slot-chain-force-descriptor-schema-v11")
queueConfigBytes = address20(activeSettlementRouter) || u8(64) ||
                   u64(UINT64_MAX) ||
                   H("slot-chain-force-empty-v2") || descriptorSchemaHash
forcedQueueConfigHash =
  H("slot-chain-forced-queue-config-v1" || u16(93) || queueConfigBytes)
```

Its account runtime hash and this config hash are the two row-2 fields above. The depth-64 vector empty leaf is `H("slot-chain-force-empty-v2")`. A parent at height $h = 0, \ldots, 63$ is `H("slot-chain-force-node-v2"||u8(h)||left||right)`. The queue commitment is `H("slot-chain-force-root-v2"||u64(count)||treeRoot);` leaves at indices `count..2^64-1` are empty. The canonical range proof recursively visits the smallest tree covering `[start,end]` (including a boundary leaf when `end<count`); it emits in DFS order exactly one hash for each maximal outside subtree. The verifier rejects extra/missing nodes, a nonempty padding leaf, or a reconstructed root/count mismatch. Append at old index i starts with the new leaf at height zero, repeatedly hashes `frontier[h]` on the left while bit h of i is one, stores the carry in the first zero-bit frontier position, increments count, and reconstructs all higher right subtrees from `FORCE_EMPTY[h]`. Genesis frontier words are exactly zero. After increment, the fold is

```
node = FORCE_EMPTY[0]
for h = 0..63:
  node = bit(count,h)
    ? H(D_NODE || u8(h) || frontier[h] || node)
    : H(D_NODE || u8(h) || node || FORCE_EMPTY[h])
```

Only set-bit frontier words are meaningful; stale zero-bit words are ignored. At old count `UINT64_MAX-1`, append writes height zero and produces final count `UINT64_MAX`; old count `UINT64_MAX` rejects, so leaf index `UINT64_MAX` is unused and no height-64 frontier exists. This is the same root as the full vector and uses exactly 64 frontier words. Every leaf and descriptor list uses `u64(index)`; no queue index is narrowed to 32 bits.

The terminal vector uses depth 64. Its empty leaf is `H("slot-chain-terminal-empty-v2")`; a parent at height $h = 0, \ldots, 63$ is

```
H("slot-chain-terminal-node-v2" || u8(h) || left || right)
```

and leaf index i is

```
H("slot-chain-terminal-leaf-v2" || u64(i) || destinationDomainId ||
  address20(destBridge) || creditId || u8(terminal) || liquiditySettlementHash)
```

with `terminal=1` for DONE or 2 for FAILED. FAILED requires `liquiditySettlementHash=bytes32(0)`. DONE requires the nonzero exact `H("slot-chain-liquidity-settlement-v1"||ticketId||address20(l1Recipient)||u256(value+fee))` authenticated by the Pool's atomic consume return; substitution of any ticket, recipient or amount changes the leaf. The committed root is

```
H("slot-chain-terminal-root-v2" || u64(count) || treeRoot)
```

The accumulator stores only `frontier[0..63]`, wrapped root and checked `uint64count`. To append old count i, it starts with the leaf, combines `frontier[h]` on the left while bit h of i is one, stores the carry in the first zero-bit position, increments count, and reconstructs the tree root:

```
node = TERMINAL_EMPTY[0]
for h = 0..63:
  node = bit(count,h)
    ? H(D_NODE || u8(h) || frontier[h] || node)
    : H(D_NODE || u8(h) || node || TERMINAL_EMPTY[h])
```

Frontier words at zero bits are stale and ignored. The complete canonical leaf event is the proof-generation data source; any party reconstructs exactly 64 siblings in ascending height, and the L1 verifier uses bit h of the index for left/right order. It requires `index<count`, status DONE/FAILED, exactly 64 siblings and exact wrapped root/count. Append rejects old `count=UINT64_MAX`. Carry-depth and full-fold gas remain subject to the mandatory Foundry calibration in §14.

A data-chunk leaf is:

```
H("slot-chain-data-leaf-v1" || bytes32(sessionId) || u16(index) ||
  bytes32(versionedHash) || bytes32(fullBodyRoot) || u16(blockOrdinal) ||
  u16(chunkIndex) || u16(chunkCount) || u32(chunkByteLength) ||
  bytes32(chunkRoot) || address20(publisher) ||
  u64(validUntil) || u256(z) || u256(y))
```

Each cell stores fixed `bytes32peaks[12]` and `uint16count<=2100`. Slot h is meaningful exactly when bit h of count is one; zero-bit slots may be stale. Append at old count n uses leaf index n and carry=leaf. While bit h of n is one it sets `carry=H("slot-chain-data-node-v1"||u8(h)||peaks[h]||carry)`; it stores the carry at the first zero height, then increments count. Old count 2,100 rejects.

Logical peaks left-to-right have strictly decreasing height. The single root preimage bags them rightmost-to-leftmost, which means the repeated tuples are encoded at set heights in strictly *ascending* order:

```
H("slot-chain-data-bag-v1" || u16(count) || u8(popcount(count)) ||
  (u8(h) || peaks[h]) for h=0..11 ascending where bit(count,h)=1)
```

The empty MMR uses the same preimage with zero count and zero peaks. An inclusion proof requires `index<count`, derives the unique target mountain and base from count's descending set bits, and accepts exactly target-height siblings bottom-up. Sibling orientation is derived from the local leaf index, never supplied as a caller bit. It then accepts exactly `popcount(count)-1` other peaks in the ascending bag order above. A wrong height, duplicate, missing or extra sibling/peak, inconsistent count or unused proof element rejects.

A manifest leaf is:

```
H("slot-chain-manifest-leaf-v1" || u16(position) ||
  u16(blockOrdinal) || sessionId || u16(recordIndex) ||
  u16(chunkIndex) || u16(chunkCount) || u32(chunkByteLength) ||
  fullBodyRoot || chunkRoot)
```

Its next-power-of-two tree uses

```
H("slot-chain-manifest-node-v1" || u8(height) || left || right)
H("slot-chain-manifest-root-v1" || u16(count) || treeRoot)
```

for the node and final root respectively; padding uses the domain-separated empty leaf. For zero entries, `treeRoot=H("slot-chain-manifest-empty-v1")`. The zero-discretionary- transaction body is the ordinary body encoding `u32(0)` and therefore `H("slot-chain-body-v1"||u32(4)||u32(0))`. Tier 3 requires exactly those empty body/manifest values and the zero-count session-list commitment. This tree is instantiated separately for every block. `position` starts at zero inside that block; `blockOrdinal` remains the candidate-wide block index. A two-block candidate therefore commits two signed manifest roots and never hashes their leaves into one shared tree. The InboxApply disposition commitment is:

```
H("slot-chain-dispositions-v1" || u64(start) || u64(end) ||
  u16(count) ||
  (u64(queueIndex)||u8(disposition)||u32(txIndex)||resultHash)*)
```

Here `UINT32_MAX` means no Ethereum transaction.

The recovery ID is the encoding in §7. Schedule leaf/root, seed and tie hashes are as in §4; body and blob encodings are as in §6. `commitment-model.py` is the normative executable fixture. Its initial vectors are:

```
TYPEHASH        ee6a8c8e31e8245cd527869508f6e464d6084893991203876f734d1855aed87c
registryRoot    0bf297d7b9b6a5529a319a06cb08484923a89bab15d51f8baeaf5c30bebdf3fd
admissionRoot   3bf2dcaf78292c832108e29205bf99cc2d22137a0545e4528d8da7309d4b482b
entryRoot       acee83a690b868a4a7960c55a9f7228f91cad26b704e24106d4db87e9c7a8f34
forcedRoot      a54e9f797ffe7f04dd5ca7df4c858edf02ce45a81808522633fc9cee8fe72e57
sourceDomain    10f186337be2d4748ed71ae3965a781bd05036107aa1e058e107b2e7f4b1332b
destDomain      c5fa66479b2914cd0974c72102929ae9852311f9cb44d432b833a22692104af2
bridgeKernel    a23f9994e8d1c475500768b67cf2b2d1f7a0f367df6f44bac5ccf1fa12bc1338
bridgeExec      f9832b2fbef2ef60a0fc7a1420fe8ae745c3840c86f6c932b2818c201b6b5df1
destBrExec      e356731fbadc537ccd8c8cf4db19e343016889c5233aa592cb5d2751ace3faf6
destInfra       6dd1dbc5d01f345c9896b91cc0626baf2585b24ce791633cc9c704b4756549a7
poolRowAlt      777cd2fcbfad2cb6bba1046b82350d5f38af4e8940791a3775f108d184e1478c
mvConfigType    0acb8f9e39dd43a4208edc38c8925bbd3433e72eb46f9e5fac584bd14a970b98
mvConfigHash    950691bac22ceef2dd834142420a79e589bc489165cb10815e889475c39d2613
mvDescType      6f1be44c261607b4c2345985bfb5a081aabd621b9168982df9b7820c7dedebd6
mvDescHash      ba4e50bfb5fabf8cd253f48ed38e4cc1a4f3fc04174ae94e4dd971b1d6e36eed
mvConfigSel     476b9aef
relTypehash     603555ff1d82bc9012b0e0c8a36df28e154b16cbd89be4eed9d01228c502965b
activateSel     28f73572
relManifest     59f4cfe5830d3620fc3f33f0890d0a6f0a10bc313bb08b8158a5c1ef318087ed
succManifest    a0a97cc0cd6dc54bbc07f52bacc7f4fcc3d349e0496754800864895ed6ae2b42
destReg         40ce2a7d88dd92c6a71d192d66b234b363795061bbd566a2c8b2382af30bd135
migStmtType     832785a7f9ce32f97dfa06fb39b9c583c13e65a5967c88f7c8c3fbda94fcef2b
deployType      babd40345cd96b87434cc1bcc50b0c693556c37283b9b3399abf60339dc363a0
components      5bd86b4bfd5edcdd47c03ecfed7c198b292c14543cd36fae4b17340da2a8c040
genesisDeploy   1e0ba02cbe13e7b14cfb427684497a7526b5252f5db86e32c48f99f8f35c3a35
versionDeploy   d6b1ce9a58632b12de3bf71d82c27b7d395ce72edb56734f2233570e84d5f91a
legacyCkpt      6ab84b0c77035309ac300830aa1c31d95f68c44b697d97a6836f7f3f54bf0708
genesisStmt     6df922703aa5b22c9f1035ede611a2aec2ed4f160536b4e7076b5031ba2ded10
versionStmt     760a32b760ea28beb032bb59d7f222e22350e8ae602cb5da3f961a3b69387db1
genBaseCore     12243b817561a9362bb03dffb36bb73f9314d47e3165673793419692cd8a8566
genBase         5081c70042287a8b7156b2626675ea4ed9623c755d5f51b5c83be94e90da3ff3
genCandidate    90949c0865a5e0226c6c0d736c0533f9e5f1d793dc9c5d21407a210ee3896338
genOutput       46ecec9370133392051393537 30b413c8e508bc579ec9c0ede247161132134a8
regStmtType     c049f967468e58f1a5c9b9e1a147dfc233695ae69c5d4a95ec4ffb49b5687da0
regRouteType    4368ad9403b46ef3830e21af8cddcaedbd444c8c57bc3414ebc6fdd250e1e6da
```

regRoute	bd990f329adf106b96b5a019ad0db50503a1ebb0dd89bfdb9c37a3449f180b92
verifyRegSel	33639818
regStmtHash	85a07af10316e3a897e6abfc727b59a28dcfc76ee9894db98def647fa379c89d
regProof	50ac70c83c4d85e9e0790d2413e35216b0c490814ee435879d0ce27e4a12e5e5
regCfgType	38f7fbc63e45f650bd5cbaffba0d81d5ca69f21ebdddfa74280d7e6eb5c319d6
regCfgHash	b266045c553f010d052a847ea18459bb268cbaacde008574e8cf4c738453911f
regDescType	9533e2f6ac9bcf6830306a9ef5d14e21a06008f9ceb59208d09a8d7ab1e6100f
regDescHash	99491cce6d0ea603e0b9862bd3a2509953ea05e50e9243f8086a19aa92b2f1bf
regCfgSel	59bfe418
verifyRegCall	ed003a9f0f69e694a6b3699efa64acd17acda445ff50a9dfb442c308dc4fdc55
verifyRegLen	516
srcCtxType	6069dff5f628f94ceff984d5ce3ef62019eaeb4efd7e0627c45a14677bd13c70
srcCtxHash	5d7181ea8838cef6d576009cd977bbe2b6f0fd250d16d4f331a515b174bee2c0
dstCtxType	b8170dbf684e1fc4dd4dae8fb78ad24984cc0aa0c03cbd1e29b1b2eb5728eefb
dstCtxHash	c591b7156965ce659a5382725f8b1091dbc2188cc7cbdc2620a61b49eb04ace1
ingAuthType	a2dccbd60c366ade3f5af12af640f3453bbb4106405ef8da5dd52484db8f2a82
kind0Auth	27af2fda507104b24da0f7a6095e048cc86de215f38027c2671f4455878a4317
kind1Auth	b67acf3b2b6184480794d8b2187f785c9e9cbebc82976e10a972dc893031ad3c
ingRootType	c7b11126d8d1984cc17cbc108be2a1be0ef9c4e8fd519fa9033c949962f1b042
ingRoot	1f1a415674dea5e83de74f9f6931c65584ab76ae317eedfcec7f94a2fd09d7ae
sendMsgSel	9211d7e9
enqCreditSel	81805d6b
msgTuple	0e85a708462e96cbaca7158a1534011a25137c3b7aada7f381e4fc5b3afbe40d
msgData	f08683775f4a25dfef721c487073fb77026d45ac57e423424290e47af9fd2835
sendMsgCall	9099cce58d3f36714ee59f37510261394e31583f8bd55da4551b653b5a060e12
sendMsgLen	516
enqCreditCall	02db9c77025d4c2bae1d3f79849020e6d8711c4b6b6c54ec45ce9181e48cfeb4
msgPreLen	576
normalizedMsg	f3010702b7b7bf10b6dbfe396a4c7ab07e7c560c28ffdbf6ff8d01d9a96ea4c7
enqueueTxSel	9f06b1b4
enqueueTxCall	3e802e6f72e50c267cc18d256c2863d04446eb8f2d47bc474bbcb9c68876d19a
enqueueTxLen	196
syncSel	6c880b72
syncStamp	1fd01b194948c635358fbb51b4a5f32f8ceab4dc4153e0230215f8afc94ee434
syncChanged	ef662a629ce07c9ed715124d8141a6e430d0a3065f8ce8074a7ea95e8751f184
appendSel	1927261d
appendK0	caabdaadee6df8cea48fb88dacad863e6e24e13f5b0c06f99408d5c71611788a
appendK0Len	388
appendK1	8bb6f931d1ceeb051f0de3bc0b32f788c0d45b856eda148128e330013a7d3e5e
appendK1Len	708
queuedReturn	76f821bd39721ec0e26efc55d7b667d20aab74992e0feae7d7755e386ecd694d
inboxApplySel	6b326168
inboxApply	65332d0b3230b33c0ae8bddd8a1f3be8473739f865e73fc8e52f4f99cecc89d7
inboxApplyLen	14436
markBatchSel	a92f72cd
markBatch	a6e34ddf755e1dbae1e15c05caaeba190795668ceaaff508b6a8537be236dd31
inboxMagic	49425632
routeCfgSel	4b64fa11

```
verifyPinSel     28933d28
getPinSel        31e85ab1
liqQuoteSel      43dc48e0
fundStateSel     6a9a6c32
attemptSel       4cbe2fe2
attemptCall      ad5ad3f673a39e9f596125189e214718927567ddc017b50c6ae7df6fb4b9306c
attemptLen       1124
finalFailSel     745dcb69
targetErrSel     f9cc2b44
appendTermSel    abc194f5
termCommitSel    2c984c97
termStateSel     998c57ed
ticketId         5dc074de7029f6762c8c386b15cbd61cc7ad94b9432c25a35a5ae48e72c3bf2b
poolCfg          680054a7895708487ddd24a202ddf1069f2ef81238e909e826c368d1da396881
poolReadGas      50000
poolCleanGas     50000
poolCbGas        100000
poolDepSel       eda2a3f6
poolWdrSel       fe4f5ccf
poolProcSel      5fbbe107
poolRetrySel     031f93e7
poolProcCall     67fd12a53168934d5c356c6e80b057b21b115967e032db4db3d619fb2b38bddc
poolProcLen      1028
poolRetryCall    1bbc3788d518e18333bae474d0677fab5f29fadc8b04a6a874d87b2575359a6b
poolRetryLen     1060
poolBridgeSel    a535a986
poolBridgeCall   a8a7b0c80117a3d96b2323365c06b655f62b39896e7b8bfc95c545e2b4541dd8
poolBridgeLen    1124
poolTickSel      5defa7e1
poolAcctSel      f2b3441e
poolConSel       37093d2a
poolCbSel        a34908bb
poolAccMagic     504c4132
poolValMagic     4e4c5632
poolResultMagic  4c415632
poolAuth         463e75e0954c32178675c0fb5e74427a069ec1eee06f0cf93caa6ca9e1925cef
poolAccept       187074e8fe244fb77a42ac949f53e39cfe8bb151018109b8fc8f846c11e82d7d
policyType       d702a337b74fc40bfc746fb1aeeaa705e60a95947bfc3076c76222703205b4b1
policyHash       5eb7c00399d64d91e416d5a3dbe75187c39dfc2a4645b867864b5fd9e649f3e3
policySel        b2d0e286
hookSel          7f07c947
policyMagic      49505632
l2ReceiptId      ce7817db42f84429f1fc630bd12c83871ac2f70e8ee58b015bcb4927391bab2d
l1ReceiptId      b390074907519b7a2764d9e2e6d86d75904ccd07fcc86c40397a038ce08c081f
execProfile      22292c4c1b89ea56c052352c2de7a52fa103da54ccef6238d6e2cfc25ab2eb56
factoryDeploySel 4af63f02
factoryCfg       4bf4831c096e251d0ef171bc0fd0582319fdc7ad7a2bea91a6b9b64601aa2837
marketAuthCfg    a1208db2d56b741c49ef2c81590172f3b6cf8df6462bdc55511b5d867a258d35
```

```
deployDesc        069af1541b5e7a6e6ee981274ca23dc30a2340e9d673e0c2752448877399b441
deployDescAbi     46ed5ccd834b950444afe9731c8200361ee3a85b0b7661b87a29ea163c9f93ea
deployDescLen     256
succDeployDesc    377aa582c38874cf50ef87030c628b15c5e0302e6830e51b09d8542659bedc18
targetReg         e74c264997a32a867fa54d2948c5ca70bd79c2bc082b7aa33f0d660f3ab56fc4
succTargetReg     ba5b1a92c368645427501c5a1a17fc0c65df7f9862d3e7ea8544272569f51375
mprHash           d3d08370bbe36ceb6fade91f303d60bb7ae6b49029dc2ac734db6273ae088505
succMprHash       90e8e5b063bb8d9d670fcb0d1a07d8eb000a0e614cabe9ae88d9b2aa60a0391f
mprSel            c65ff64e
mprMagic          4d505232
mprCall           3dd403c33c2e7c4e6b324efac7461b5b0cc6f3696a2a8d8c94c9cb9bafcf9c85
mprRet            092aa880d8254032dfab09a85baf24a874caf97e04d2df9555f21e2f8b8de4f3
mprLen            768
opDomain          edc1be882290e6241a79546c41db66a1d975095ae60cf3b2b16f1ed876f6038b
timelockDesc      442d9b608ecc43eea4009fb7f95c764c747636f42c885174c1442681cc0ab495
pvmCfgHash        f3d5898df7a23a84ed206deaf2038ad7fe52617f83df5ba2b8c5c4ecded7c2ad
pvmDesc           978f5d8ed5fa1d2e147bc26db6bfa3e11cca7ab234bf97a58c6892c63b00e4b6
protoDelay        604800
maxMigLive        604800
protoReadGas      100000
queueOpSel        bd5c80a8
execOpSel         31d81ea2
cancelOpSel       5701c308
applyOpSel        af3927f6
timelockCfgSel    b80095ca
pvmCfgSel         4deb7821
opGetSel          4b80fe68
timelockMagic     50435431
pvmMagic          50564d31
opMagic           50434f31
applyMagic        50415031
regRelPayload     3585c0dcb7691e9894921577646de9c71a166291be7d81f959c90a3cd9783ab9
regRelPayloadLen 10400
regForkPayload    060a82d25adb79cdf6382c783964efced57515aa1a1a7e79cde651e9ab190c00
regForkPayloadLen 448
pubGenPayload     ece311c62026f47db05309a2cebc937be659569c1f0c2172002da3075b380a2c
pubGenPayloadLen 320
pubMigPayload     49a908fa34da7bd43785ca7672208fbf262f162970c2490c05132b101465d8de
pubMigPayloadLen 128
regRelOp          17c319270d7323aadd9e53315a63b2abb9df1292e6bc5fd70f05f937122aa8bb
regForkOp         6f3f09d83e85bf28f62fe18b37ac3abd378c8d03e1a5bffe67fc2e59540d4aeb
pubGenOp          dabc210827740f6a5a2e02f0fcb40efc6b601165eb79876242505039296fbe94
pubMigOp          59dd79759d954f74fc6c3c080a2b1838cd4392409cd196b7d1d666bbe9773a55
queueOpCall       6a50ce249c8152fd61fd19fb02c93855949753183da138707002bbef8a3d8cc2
queueOpLen        10500
execOpCall        53ee103fb2f21efd57ad1fa188745e77fda1cdde4988e659a37036770a11dabf
execOpLen         580
cancelOpCall      765e3e164cb46e2abb9cc51e5d44a11f858d7c527f29cc92b209e460e07c5593
```

```
cancelOpLen      452
applyOpCall      d9aeea7c3da04cda67187ee96f20669b496c7fe25510dc94e5157a54d45ad83f
applyOpLen       260
timelockCfgRet   1813d922f5db2c6e82ca7be236e9cab4b8ca3c8b5f32231d60bd51b75e54b29c
pvmCfgRet        885cb456c37e6cd3eed4ab99af4ab704de544acd7ff014afa7c7683b7b652867
opGetCall        6247e74d2805dc8a531a1f1fc65b5d115dbc23baf9fb5812daf6a00d8e8fcea5
opGetRet         aa46948d3f599dbb6034dfc5a39a01232791469392b2ef9981d7bd8d6611658e
applyRet         f9361bdc99345939494c4794891942dafae95fa74ace38bcf4430ae53ac12c4a
leaseSel         aaac4c97
leaseAbortSel    ea3e96c2
leaseMagic       564d4c31
leaseArmId       501b467e73f364446cd3e0ac98ace85bce64e88f97d25cb16c8bc92a294d99a8
leaseAbortAt     624800
leaseRet         fdc598573486cc233fb6d7e730d6ef9c7ff92df5806684327d5cf3dd9a9d3bed
leaseLen         320
regTargetSel     9aa71eff
targetRowSel     f588fec3
targetRowMagic   52545232
regTargetCall    eb1a079b1ffe45e57b02592b9d43dd9af89f4a0f6d4ca7410be087ad0109a22d
regTargetLen     10404
regTargetRet     be7b7db605e7d7c2d7c3edf06e009de1f61f3bae0ec006ebc5a83a685f38ec1d
targetRowCall    5c5db1ed7485ad89b4d9125c80db863078603f32ddd3032552a6345bd0dba5ee
targetRowRet     dcd726cbaf9c71e1db52b4fb42d9522500f8d37885687c4df2c7184a9f26bad3
targetRowLen     384
pirSel           2d2bbe23
piaSel           2181b974
pirMagic         50495232
piaMagic         50494132
pirRet           2cd5d7e58025a0c6283ae35b1090b0014f3a2640508ac28aca6109e95d893443
pia0Ret          70b9c601b2d1f2852d14603d001ba507657f2eb32b0be8a38a986b4670c1873f
pia1Ret          39491a71b47c4467196bca20f1f45f6c59be18480872cc9be2c69dfa702b4439
authInstallSel   72a3e937
authGetSel       1693ae01
authInstallMagic 53414931
authGetMagic     53415431
authId           3996cb4bbeea670aa365269a1e4ad9dbddd171df2accf9f159c8063de75827a3
authInstallCall b5fce88f5bc4802175332ca337e8aa80c4903f7fbe04680d3db1e86bf5adb066
authInstallRet   1ed7ac691e6e73a073095e2f3f1eaac8debe962111b8358eae7d637e0a10ab39
authGetRet       94b1dab2989ce801bf528219ef887c06541cecdbe87dbaf48acc9923655f4c32
seatStateSel     cf52185b
seatTermSel      76d5ecd4
seatDutySel      9a649489
seatAuthGas      100000
forkConst        c17afdd5367849af551b3a2f322a2c351321e916ad913a78b646586fe34f59fc
forkSchema       3a480130319b3cebce02d217988e89c83c6bd6e71ff93c25bc4fc38e51fbe2c0
forkCfg          95d9108bef4bb1eb4d6b63a79b04aecffcb7f8405b82cb22883c28dda04bb802
forkInstallSel   f171816c
forkRowSel       c614591c
```

```
forkCfgSel      44efa773
forkVerifySel   7e981e0b
forkInstallMagic 46564931
forkRowMagic    46565231
forkCfgMagic    53465631
forkCarrierMagic 53464331
forkInstallCall f986d1bbd6f3801e5ea25305cd72946eae312d78b512a7649e669e7c251e05a9
forkRowRet      ed8f0a8c03c6743ffad9ccf6ae618ac65bfcfc1d5c6cf924a3b58ddc69166140
forkCfgRet      876e1109f517610df01f9c613f669d68ca2390fbee92c41b63e7a9a65a1924da
carrierCall     a6f4b2260cd81afd71756624c295b886d29e85c0c9e9b1cdc89f0148bbd0fb79
carrierCallLen  132
carrierStmt     e5e7ef6967d544c41242fd48103d1a53c28f1d6da31a1649d34f4bd11025e123
carrierRet      3e7028adb7b81c521979a0024999718518559b1ddec5cde8e3a9379d6cef3125
publishLegacySel 5f0ed7f5
armVersionSel   e3bcfcb4
abortVersionSel c4eee12d
publishLegacyMagic 4c475031
armVersionMagic 564d4131
abortVersionMagic 564d4231
publishLegacyCall b9a9831430cb2c6b82e820d5c290a806c26b456d1d8ec3f63d55e479ab3773e2
publishLegacyRet 0367719a40e4be9f3cd78a96be149739f6700d1af4aee3273a7385e70a885f12
armVersionCall  72c1e2958118bff66b6e530d9c52fac6e0f506d84b9142e4d3273660be3b2fe9
armVersionRet   7bc7e39e02fb4504dd4f1a7d70cbbfd4e88a1a8985163929c1bcc2d259373244
abortVersionCall bc9fbd94a30dd30c4a3791631fc933841694cb9cc3b26e4aadbfb0585f0877ef
abortVersionRet cecc7a76c559e89ad8bdc8009067ff67163188d7fe5f16a9714837f7b499df8f
pvmRouterGas    8000000
legacyInboxCfgSel c3f909d4
legacyInboxCfg  22bc70c195669ace965e7565ae253f760d785e048965f94efa51845573b36808
legacyInboxCfgRet 4d6b260943518f7ce988e85ddb9321b2a3111a42e9ba431f45342ff6f4f3cea7
legacyInboxCfgLen 544
legacyDeploy    98cc45a7619c75ff658f46edbe2e3c11c19e1ec424797cf04c760b9f07fd567c
legacyFence     cf5cd470448a26b70eddedf8c30fa39533e66e874e2fc1aa2e310ab633c0d737
legacyDescGas   100000
legacyTimePolicy ae8419c16fd9493a76f4192c6bef1396d6237a2f2d981dc2bb55acfb3086e5d6
legacyRisc0Key  60651c05d2012e7254373988020eaf9dec6c635e84d535112e7ade81aac5584a
legacySp1Key    b449ce6092b398fd4178d946077f8f19c4f2451cc88d614bb682233134162f12
legacyRisc0Desc 4805e24aa165db57d51ab99349241fd843ca03e468afb8d33781d4a08ccca55b
legacySp1Desc   523d245ce9369abd6a116dd7cc4b4a989fcfb8a9d1f8eb796e1cb35945f18bf6
legacyProofGraph 4fc1c37502713218107f6aefba90f1a5720e9ccd5a5569abb590746e85f64efa
legacyRoute     527763e6ea020b909c9a74a7aa5b7c2fdb372c5d8f283644d4d454bc63c16275
legacyProposerDesc 1bd9411f4082f5d010edfad3ba2a52ed5164a7a7ec345c1d56f11a2b3448515a
legacyPublicProving afda6179076d7f2625ac4e691fd28ff557ef3419a708f22598bc8419b6a27c05
legacyCkptRecord 1461a32136f8c498043934fce575dddec6830744145afbccde57eea1ea61c9b0
legacyCkptLayout f9e5f221ec2368348e185feb34fee0cdb27a812a2533c56fa7dfe8ccf762ffe1
legacySignalCkpt 5e10f1f156d0913967b53921f75484530facf090ceb544306381abb78894143c
legacyVerifierCfgSel 7e52cacd
legacyVerifierCfgMagic 4c525631
legacyVerifierCfgRet 76ded5a00e4182d6eb5d2d40700d85414c53a99657ebed6dc9612205c703dded
```

```
legacyVerifierCfgLen 160
legacyRisc0CfgSel e516c43e
legacyRisc0CfgMagic 4c523031
legacyRisc0CfgRet c7b1f85e99c9b82651e09ced5b86de094ca46ef38aa0f5c22d94efa1d8222906
legacyRisc0CfgLen 192
legacySp1CfgSel e2b5d958
legacySp1CfgMagic 4c535031
legacySp1CfgRet 864c870262c85ba745d71a95e0d667b76fddf17d60a5efe48512b289ae61a9b0
legacySp1CfgLen 192
legacyCkptCfgSel 68011023
legacyCkptCfgMagic 4c434b31
legacyCkptCfgRet 4ab6f89dcd323544a673c0243a0ad55779c74ae37d981d3308829092d7713534
legacyCkptCfgLen 192
legacyImplSel   8abf6077
legacyOpCountSel 7c6f3158
legacyCurOpSel  343f0a68
legacyNextOpSel 72a8a551
legacyVersionSel ffa1ad74
legacyPropImplRet 760f5b3a5acc4b21ba8112ebe315c3946dc41e502bdb323e50afeac90a42ae8b
legacySignalImplRet 8e4eff68fda06b95881cfa2df901251b566e87657e1a6ce77e69fd5546318034
legacyOpCountRet c2575a0e9e593c00f959f8c92f12db2869c3395a3b0502d05e2516446f71f85b
legacyCurOpRet  7606a1b3fbf7c1835d2aeeddf4207ed367c600f20c898605ba54d05cbbb3be39
legacyNextOpRet c7a0cb7d58c090fb9b48529de670e08db7fa5d48d0b8fb1a2c25f357ff0215ee
legacyVersionRet b10e2d527612073b26eecdfd717e6a320cf44b4afac2b0732d9fcbe2b7fa0cf6
legacyResumeProfile 6a993ebf703937ddefdd8a5ae08911f15a19286ae47b40d0b91a08824cbb2812
legacyBlob1     f0c8974a111225866a147954a2f98c29c679c1aa4680c42b15ac2eb3a1cea148
legacyBlob1Len  192
legacyBlobMax   d9791c1e9f76963f86cdfe6423b8d897dcc5c0ef5ad4cc16363b2b2ab452240d
legacyBlobMaxLen 832
legacySourceMixed 7e0699ca450524a9f618d63c9c6ac4690e44bf47ae3a24980346f4d91b3a8789
legacySourceMixedLen 288
legacyPropMixed b83ae28a39b0d773dfaccf449234cc30a737b896dc3a1a2b1d4b15bcf2ae07d4
legacyPropMixedLen 1152
legacyPropMax   ac882b6809d66ab7fa52d59ef700c30ef460ae5b45db8ccdd0ea6cfdc62fb298
legacyPropMaxLen 3808
legacyForcedAbi ffad3668322a174e4a6b3180869175b6395a8657c6940353ce2cdf442fe98cd4
legacyForcedAbiLen 256
legacyMaxForced 10
legacyMaxNormalBlobs 21
legacyProofGenMax 900
legacyMaxPropBytes 3808
legacyMaxForcedBytes 256
legacyMaxScanBound 4194304
legacyMax16Raw  60928
legacyMax16CallLen 62084
legacyFullScanBytes 4161536
legacyFullScanHeadroom 32768
legacyReview    ad645ed9c3e81c995eb88dabbc1b0a6d93539c360679f230001abadd633494fb
```

```
legacyBlobExpiry 1573864
legacyCampaignSel 718b2ac7
legacyCampaignId aa06260ec9cdd07a71c6d23fc72dc5107cad200da425b76b0e1cb8c50425347f
legacyCampaignRet cd3963cf1857195564f8c0960cc595b856c50f3d288bc18c6858c6a298765175
legacyPrepSel   6880cd05
legacyPrepRet   8c869f3dfe7df0f01a65e96d7691b5b0d095590acd9f034770c33a6c75472a98
propRowsEmpty   0736a35926f1618d1fcca0fdf57530d52f3d1694eb227f31c28b158b370793d5
propRowsRoot    aeacf67ac14d90928d567c659ff80316a0d780fc490ac6b4b69e9620836d2b46
forcedRowsEmpty 5631fad8f285ac1643b4e817c5454ee4f1e35174786dd255842a987b21c3fd92
forcedRecord    dbda8510c73bc3eb57de26ae5c5bcc82a45bf6fedab68abf100d9baea6f4a380
forcedRowsRoot  ab7f340cae89170a3a619086538cace009399987a1bffbee40c17da60c10fcc8
legacyScanCommit d49046961e41f21e9073422d2a73190dcd0e78ee8e04c5ef3e6def287d47046b
legacyAbandon   597abd3f5e890c11afa7d5bacc4f359834efd5bd1f60304fc2a4f77856d76e6a
legacyAbandonTopic a3b978444273f8f347857235c224846aa45e8ce5bb73df549b9916e96371c8c9
legacyBeginSel  e9d1a07f
legacyBeginCall 506cebf31bdf2bc86cafa84bc56c16592779b1fa793c3fb6d6280933e051f4f8
legacyBeginRet  add644dde24164cfb29ab6a4598cb55acb73c55dd295b0c2a227dc3264c6f65c
legacyScanPropSel 7da66460
legacyScanProp1 87e956457ea296af8a317fe81f8d1f9b4943586702bb6ecee581dd808c398554
legacyScanProp16 dcfd22339f0e3b2ceda7f014ceb1fcbdf8feb1311c12f20c7695dc84eb6e7ee9
legacyScanForcedSel 032ae99e
legacyScanForcedCall 61a7e4c98610eb46b65d4b5019026e25959dc1ba17ca255dd2a9968dae330b57
legacyScanForcedRet 10dcb42db706d4c4a5afe207c6327f8cecbcbc4b298c0255fe2434662e6599d6
legacyScanStateSel ef3cdce0
legacyScanStateRet 8e5e397d2f4ad82a793cc1fd0f8541e1b81a24e6cb6a0704da944d0085f2155f
legacyQuiesceSel 4c0ae8da
legacyQuiesceCall 7622002d111067b051b35e10a1098bb77f5720300d779960032d56ab4fe825b8
legacyQuiesceRet 0bdeb36de78a895e7fc0f9fc43bd5e570eb3f789aa0be60756b1601607d9b95c
legacyResumeSel 2bf6b656
legacyResumeCall 0908f0acdb096d176a7dd3722ccefb4dfb2d97b33bdedeaf2478ab29ee36e65b
legacyResumeRet e9fe8a7d7e0737dca59cc76084c7a67384789ca762685ef1d320e2a510da0eb4
legacyArmId     799c52565ab954d8ecf7e47b08534595f06c469fae36e69c3b185d4900044f0c
legacyBoundary  0215b43e3b2143b3fcac2c97bc2f8cc32ab47ac695a6fd173d919abdcc170263
legacyLaunch    ff532c65eca209c3db2cd49f550ec18dd33dd9b93eee6ba38ad40f4318bb88f5
legacyPost      b3ed034b75a6d707a7b7252c302bbd5187d0d5e20c5d574b59e93001de887aa6
legacyStateSel  9b698000
legacyArmSel    8781a058
legacyFinalSel  c2de6417
legacyStateRet  4f3e86d5ccbae57c51a441e9bdbad9cd481eceb37f3aa7e74a98390448c1cd65
legacyQuiescentRet 5bf2b30077f31f7358b9e9466110dc3a7559bf6250df27e053dbd9a7c7cac2e5
legacyReadyRet  61ea987bd5cf6749c2cf1529a85d9a6e5720c6de9aaa2984407216f1aebc11b1
legacyStateLen  512
legacyArmCall   6046c79779b7bbd02107a76627bff3186b9ddc1d18ad252436d23a9d338ac6d8
legacyArmCallLen 68
legacyFinalCall 10105485328c76767b73403c4073bf762fbe76f0811230076f358276f9766b14
legacyFinalLen  196
legacyArmRet    8ca209561c9268a32d8c67c0ad5a43772aad2e280806e146206c9e2cadb38f6c
legacyArmRetLen 128
```

```
legacyFinalRet  b924e354107dd8656f106cfd429a7636cc9d2576dc0b45c272f94fab4f791968
arv1Sel         0a4434d0
genReceiptCall  5c33ca28ae8c87326903243bd8e7d3014da48062d3dc122d7f645dedad796f64
genReceiptRet   5c7ccef17c99ea7b3e8531b30342cbd6201caec7bfaae09bafe28921a11247bc
versionReceiptRet 25ed31f58de1cda586c28189c946c6aab9172c13218aea549edfcb25055b9336
genCtx          fa309b6a84746d1d6acee8f87f9321271cb68eeaecc264ac718812eacd8211ac
genCtxRet       ae5130794e17672433cf1df57dd6fd2c59e5a9e138606446393ad1d1a2db787f
genAdopt        69732105259421cf5c3c480543c2922061750cfb76793e615604dc95799b2d2f
genAdoptCall    9203b4a9789a1063ae7241242ac7f2c19227de972cd0b1dab6699ade4ba4e6c1
genAdoptRet     0d6cc2c07afaf087428fdeea4c1a3450a2719c9985132945c14fc562b968e47b
genQueuePost    d58031a42d100455f424a28e0c3aa0fc64d3dbaabc5fdef3c349f1f7b37c6e4e
genQueueCall    9dbafd23a937ec8aa2a5ab26845e0cafc95ddbacbfd9a9f9d4364415aa9c0467
genQueueRet     c9ce547f713aa89aadbb5e3f65c73ead7a042233bbba470e7e01512b5b2014b0
genSrcPostRet   437610375708fbcdf4d1f88757e3d344605b8670812deee3abf492ab9cd657f3
genTgtPostRet   726d07b3ddb51eeb54a37a48ac5806c4c24a55a1fe3b99957d402e0ce46ee27e
genQPostRet     2454a589b32353c19cbccb91c1e2fcadb330cc22aebf9496a4f8974b10d1f3c9
genReceipt      3cc0a9c40dbdb83da7b507448f9d289f5b5efa51c20e6391fa6fba9caf72452d
reorgLegacyPost 0f299dfa5e42d5075db83e4eb1b004cf7c33b63afba8513d02134d74be44d54f
reorgGenCtx     ecc3841b57a7ca17da647379ad6cd73561bca5dee084d8755afcc7027d632b22
reorgGenReceipt 053301fdcf9c3d44895d885890694c29f258cf743d9167a4a4f343161a1871f9
actContext      3873a3f2497d0112c2f5f264aa1b79543ab5647f8539bf46fb609c92032b5337
mactSel         7cf70319
mactRet         ee3341844bbb28d11b776e2c7b522022dd18969de50976eae7bc855cb0cd721f
mactLen         320
armLifeRet      298524396c6b44dd68c8c21326e4ddb544cc51cfe8d5b89ed34bda45fa998753
mcanSel         557c4e13
mcanCall        92366259f576a370029e270ba4e032e74339fac39d850903440060f26093fb4c
mcanLen         548
mcanRet         3fabfe7082e0ab3873d16ad30d157d5ad409f56c478596a9bb85c427b44ee974
sourceFreezePost f6cefc65013868fabdd502d608dfbc556d77ec3729647e05d7e72d42985e6bfc
mfrzSel         45a80913
mfrzCall        a1f8fd6ac6f3f722f3b2ef1ee56a4a07cf3e1a0047125e9b03cca546c273fb23
mfrzRet         f1082d527fe0b2b8bd04a736726d2765d43648eddc746d0c345cd747330984c6
queuePost       1793a94b819a4330840685a6e379d413ba1f81219208984183c7e66c40d0a87c
queueCredit     64000000000000000
qmigSel         9461f698
qmigCall        8a00ae30412e1f78da5a954be3303ec5dc311a54b894bdeea84fcbfc12918092
qmigLen         260
qmigRet         7fd65dcf44a26374708e64e3fced62b809bf70d01f104dc2cc71a7ea89e514d5
adoptPost       979cd5dd07672432be3768a6c57378f671d29bc8bed1f45b76c78dceed0855a7
mapsSel         66e664cb
mapsCall        1143fe0083b3aaddd30e0ca1596ab8849d1dff7a71fc90454cb6b7f4c8df2b62
sourceMapsRet   c6b77ff997504e3da70d75aaa00c1c571573082e8c1ca4eb1ea552fd2b696898
targetMapsRet   88c638b2c1aa55236cfb8e462651ea51d9fd55d14a61986b44af9c1b5e81260a
queueMapsRet    bf63de5734b2b7980e64a5fc6da9f271ca4b2e68bc60df01c359f311448b2081
activateVSel    14c37693
genesisFixed    51509fea7d67c057520beb2fbb8620f434d30e49134bd78e104c481b929c865e
versionFixed    3c56ba661f3714e55644fb2ca1c83595efd27c20040727dba1efbbb564495308
```

genesisCall	899862123801d01e4d5f529c6a5fa785a07eb524fe850443e10dea2f73492cf9
genesisLen	2884
versionCall	56ebab6e19f1d9bfa240774f4271412d1922e540d6c68822648e853d477feca3
versionLen	17156
maxGenesis	0f6f5587d90d0fa4dcb58c91f235246c1c3ba570e032e25c162bcc4b9610322e
maxGenesisLen	135844
maxVersion	5c8e3968d6ad7e625ae370731bbfb691fb442c7f15757118cda6269842bd03fe
maxVersionLen	149220
regBase	20b3dfc457e3cecf32b0c047177351f0814e426c1548e87b79f58830655810c3
regSlot	dfa6283b763bbadeb604401a78e2fefeddb72000addcdb94ed2e3de5cc69846b
regTrie	200031adff46d90b1cd5c67ff8e31098235d1dddb08ec98b0d20f5f8660c0ac8
relBase	a0b7a29a75032f37561036cd3741e7b375213309367f37b5ffec4ad55cf6154f
relSlot	719bb73ba856aeab1b203e322bfefc6d84a4c41a3222bcf1634b1b44e5b9aba8
relTrie	dac8109059d03da2ad16ac3acc50d2e58897b8c3a7f6889ae77bfb20737e87a2
routeConfig	b7a461bc5970c971f2cc9abb3877e5e1c02cbc922ae4b3cac3b63737f68c7805
queueConfig	72e27c19ebfab08e1fb27feeff50609c7c4bb69f0570a7bdb72eda7736c50f4e
dataSessCfg	a9198362f19424d86fec60351e9883028b83fdbc7df2ae710325e5affec01173
dsOpenSel	7bda4d11
dsPostSel	a1cc526a
dsSealSel	340e11fb
dsMaintSel	1e7a916a
dsClaimSel	fdd2b0db
dsSweepSel	9d083a2b
dsCellSel	011efada
dsByIdSel	eeaad0bb
dsAcctSel	e2a62969
activeSetSel	4a95c306
migReadySel	b36c83ce
markReadySel	e0c25827
activeSetMagic	41535231
migReadyMagic	4d525331
markReadyMagic	4d524459
dsOpenTopic	a81132592bd8a549a0bfc83415ab47fbca586f0b48fff5f6cb5bfae0a9fd1f68
dsRecordTopic	30ee2de166c53a480d028e5b94d4f8759dbd84b5f7b6af1f23e0c5889ea17f8c
dsSealTopic	f6d45ab0ecc3348b36ac48bc769f7391962fc2fa240c1c6bb43269ed3780078e
dsRefundTopic	086de7ad4d27cf66f63e9dc6ecb09c0c3122146dd43e7f480f3a875070eb984e
dsClaimTopic	69fe7d8d95811e3a02a4ad4b0d1a3e1360b683a31f7cb30b4c69546b258232fb
dsForfeitTopic	f93f771339298d1fb502bd2c556fc18130b95d44599f35109de39d8d5731f957
dsSweepTopic	3a5f2fa0ab342d3b79fc2aa40be5e1296009e8fb31f86c3577c51839dd159a86
dsMaintTopic	920669b9670911aa86cd718dceebaa1372d224ca0fdac50a63dc1d45a53e1e89
bridgeLeaf	f5bda35fb1e021706647269866a683dec01b7c15355b3b2aaa6dbddd6eeb8d69
feeLeafAlt	354d05b8bd4364ff8a7454f3e3975a4c014a3b268104430f4e825f376ab58505
creditId	dc227c06eecc3bdd1ea0d48345efc48b017188e40f43e7c33450314260ac5538
feeCreditAlt	1fbc7c2842cbd1da217fa0629d43a13942e0f42d85b7f3a686b2126b800a80f6
escrowId	cbb92e32149a19f26a3db755300df9f7d401fe51c4a7121b966ad77b598385d5
inboxSlot	b6850870a89c5fd93129ea8d1c1d4fa02e810c2ba8c235cf594e8e8258d828aa
terminalDone	b24b3a75e604936efaf89218951ae097b2901b0aaa42c411f5cf9d6183e867f5
liquiditySet	625ff42ed879b94a7499fecb7abc07988b348300d1c4e9cc1f7596968eaf2f19

```
settleLeafAlt   bf20404f65a3471b2d10c0a93fd1e75ca365d2a202ff6a804a7dd7b3884af523
terminalFail    eb3a0ef753b87449dcc50b109f2f2f7d8405af7c43a1e55f4e0fad93d4756867
terminalRoot2   220a454b2a961a593546023497e32948752c63386a54085491c7fba5a6bec5a8
emptyTermRoot   c5da197edc2f03c7023cc6afe137ccb77d01fc56514d322b6ba66a149315bcb0
bridgeResult    675ef16e849ab97ac931e08c98e51ab4bd9b77219e8a98510742ff37bcec84cb
manifestRoot    417be737a57e38eb410f2d6e65c77ee19d5c314cdaf432067861c6a36c6a990f
normalContext   5b93691b397d8ab377682acee7cf6cc77ff2726cd83a8a7b725084f5d6f468bd
migrationData   2c36740d76ae6192335d4c603f42edace094b33e8f54e959b40241a94c1f6deb
winningData     4ae34aa9efb842528d353b175f94191d01cfd168b5ad828f64a0e7972a2ca9e3
statementHash   0223b45e5752d4ca03f59534e9c773481a390c46da8286b94927b4be8229b991
recoveryId      fd0552b28542fa3e236c86807695f3d5a4bc0436add285daa8506d9a79511b15
bodyRoot        0f4e161a46c8b18c2a86f23a0a4e7169a838a12af8b389f65e97b547a99707e9
```

Release CI must reproduce every vector in Solidity, Go, Rust and the circuit.

B Normative transition ordering

Every external candidate entry point implements the following structure:

```
function armNormalContext() returns (Status) {
    if (mode == PREACTIVE) return REJECTED_PREACTIVE;
    if (sync()) return SYNCED;
    if (migrationGate.phase != ACTIVE) return MIGRATION_ARMED;
    if (mode == NORMAL_ARMED && block.number > armBlockNumber + 255) {
        armBlockNumber = block.number;
        emit NormalContextRearmed(armBlockNumber);
        return REARMED;
    }
    if (mode != NORMAL_IDLE) return IGNORED;
    armBlockNumber = block.number; // no caller-selected hash or anchor
    mode = NORMAL_ARMED;
    return ARMED;
}

function activateNormalContext(armHeaderRlp) returns (Status) {
    if (sync()) return SYNCED;
    if (migrationGate.phase != ACTIVE) return MIGRATION_ARMED;
    if (mode != NORMAL_ARMED) return IGNORED;
    age = block.number - armBlockNumber;
    require(age > 0 && age <= 255);
    armBlockHash = blockhash(armBlockNumber);
    require(keccak256(armHeaderRlp) == armBlockHash);
    freeze base, stable admission pair and parsed arm header;
    freeze minimum slot, deadline and landing horizon;
    mode = NORMAL_OPEN;
    return ACTIVATED;
}
```

```
function submit(input) returns (Status) {
    if (mode == PREACTIVE) return REJECTED_PREACTIVE;
    if (sync()) return SYNCED;     // successful transaction, never revert
    if (migrationGate.phase != ACTIVE && mode != RECOVERY)
        return MIGRATION_ARMED;
    if (mode == NORMAL_OPEN) return submitNormal(input);
    if (mode != RECOVERY) return REJECTED_NO_CONTEXT;
    return submitRecovery(input);
}

function sync() returns (bool changed) {
    if (migrationGate.phase == ARMED) {
        if (mode == RECOVERY && now > round.expiresAt) {
            cancelCurrentRecoveryWithoutRolling();
            changed = true;
        } else if (isNormalMode(mode) && forcedHeadDue &&
                   normal boundary open) {
            cancel normal window;
            changed = true;
        } else if (normal deadline matured) {
            settleOrCancelAlreadyRecordedBest();
            changed = true;
        }
        if (no boundary/recovery) {
            inspectExactly8AndConvertLiveToSameCellRefund();
            changed = true; // cursor progress counts even when all are empty
        }
        // Existing protocol-lifetime reservations are never migration work.
        if (allowReady && localLiveCount == 0 && no boundary/recovery &&
            seatMigrationLocalGeneration == migrationGate.generation &&
            exactSeatArmReceiptMatchesGate &&
            migrationRefundGeneration == migrationGate.generation &&
            migrationRefundClaimDeadline > 0 && seatClosureComplete) {
            magic = ActiveSettlementRouter.markMigrationReadyV1(
                migrationGate.generation);
            require(magic == MRDY); // Router re-STATICCALLs readiness/accounting
            changed = true;
        }
        return changed; // no context opens or rolls while armed
    }
    if (migrationGate.phase == READY) return true;
    if (mode == RECOVERY && now > round.expiresAt) {
        rollRound(); // same episode/base/cause and duty; new revision/target
        return true;
    }
    if (mode in {NORMAL_ARMED,NORMAL_OPEN} && forcedHeadDue &&
        (no deadline || normal window immature)) {
        cancel normal window;
```

```
    } else if (normal deadline matured) {
        if (best exists) {
            liveBoundaryDueAt = ForcedQueue.dueAt(best.endCursor);
            dataLive = now + REORG_MARGIN_SECONDS <= best.minDataExpiry;
        }
        if (best exists && liveBoundaryDueAt > now && dataLive)
            commit(normalBest);
        else cancel normal window;
    }
    if (isNormalMode(mode) &&
        (finalLagBreach || forcedHeadDue)) {
        openRecoveryEpisodeAndRound();
        if (finalLagBreach) run bounded duty/failover pass once;
        return true;
    }
    return changed;
}
```

Genesis and every later control-plane transition additionally follow these non-reentrant journals:

```
scheduleGenesisCampaign(campaign) {
    require(routerLifecycle == IDLE && publicGate == ACTIVE(legacy));
    require(exactDelayedReviewAndTarget(campaign));
    require(campaign.nonce == lastNonce + 1 && hardStageBounds(campaign));
    requireExactStatic(legacy.legacyGenesisPreparationV1() matches caps/profile);
    requireExactStatic(legacy.LGS1() == cleanACTIVE);
    requireExactStatic(legacy.LGSS() == cleanEMPTY);
    publishExactLGC1(campaign);                // public gate stays ACTIVE
}

scanGenesisRows(campaignId, contiguousRows) {
    require(publicGate == ACTIVE(legacy) && campaignIsLive(campaignId));
    require(block.number >= proposalCutoffBlock && beforeEitherExpiry());
    require(rowCount(contiguousRows) == min(16, end - cursor));
    legacy.scanBoundedExactRows(contiguousRows); // deterministic max progress
}

enterGenesisQuiescenceLocal(campaignId) {
    require(exactLGC1PublicGateIsActiveLegacy(campaignId));
    require(afterQuiesceNotBeforeWithMinimumWindow() && beforeEitherExpiry());
    require(completeLGSSCapsExpiryAndResumeProfile(campaignId));
    phase = QUIESCENT;                         // exact boundary; gate stays ACTIVE
}

resumeGenesisLocal(campaignId) {
    require(exactLGC1PublicGateIsActiveLegacy(campaignId));
    require(atEitherExpiry());
    require(phase == QUIESCENT ||
```

```
            (phase == ACTIVE && matchingNonemptyScanState(campaignId)));
    clearCampaignScanAndSetACTIVE();            // ordinary mutator may do same lazily
}

expireGenesisCampaignRouter(campaignId) {
    require(routerLifecycle == IDLE && publicGate == ACTIVE(legacy));
    require(atEitherExpiry() && exactCurrentCampaign(campaignId));
    tombstoneCampaign(campaignId);              // no call into legacy proxy
}

activateGenesis(proof) {
    require(routerLifecycle == IDLE && publicGate == ACTIVE(legacy));
    require(liveFinalCampaign() && legacy.phase == QUIESCENT);
    routerLifecycle = ACTIVATING;
    requireExactStatic(legacy.LGS1() matches campaignScanAndStableBoundary);
    preflightAllCampaignTargetQueueReceiptAndIndexState();
    context = computeActivationContext(block.number);
    activeActivationContextHash = context;
    requireExactStatic(profileVerifier, reconstructedStatementHash);
    requireExact(legacy.LGAR(generation,campaignId));
    requireExactStatic(legacy.LGS1() matches transactionLocalREADY);
    requireExact(legacy.LGFN(context,...));
    requireExact(target.MCAN(...));
    requireExact(queue.QMIG(context,legacyProxy,target,...));
    requireExactStatic(legacy.MAPS(context, SOURCE));
    requireExactStatic(target.MAPS(context, TARGET));
    requireExactStatic(queue.MAPS(context, QUEUE));
    recomputeAndWriteAbandonmentEventReceiptRegistrationAndIndexes();
    genesisConsumed = true;
    publicGate = ACTIVE(target);
    clearActivationContext();
    routerLifecycle = IDLE;                    // final write
}

arm(target) {
    require(routerLifecycle == IDLE && publicGate == ACTIVE);
    routerLifecycle = ARMING;                  // before first external call
    publicGate = ARMED(targetManifestHash, targetRegistrationHash);
    requireExact(source.completeArm(target));
    routerLifecycle = IDLE;                    // final write
}

markReady(generation) {
    require(routerLifecycle == IDLE && publicGate == ARMED);
    routerLifecycle = READYING;
    requireExactStatic(source readiness/accounting);
    publicGate = READY;
    routerLifecycle = IDLE;                    // final write
```

```
}

abort(target) {
    require(routerLifecycle == IDLE && publicGate in {ARMED,READY});
    routerLifecycle = ABORTING;
    requireExact(source.completeAbort(target));
    recordCanceledAndClearTarget();
    publicGate = ACTIVE(source);
    routerLifecycle = IDLE;                  // final write; no sync callback
}

activateVersion(proof) {
    require(routerLifecycle == IDLE && publicGate == READY);
    preflightAllSourceTargetQueueReceiptAndIndexState();
    requireExactStatic(profileVerifier, reconstructedStatementHash);
    context = computeActivationContext(block.number);
    routerLifecycle = ACTIVATING;
    activeActivationContextHash = context;
    requireExact(source.MFRZ(context));
    requireExact(target.MCAN(...));
    requireExact(queue.QMIG(context,...));
    requireExactStatic(source.MAPS(context, SOURCE));
    requireExactStatic(target.MAPS(context, TARGET));
    requireExactStatic(queue.MAPS(context, QUEUE));
    recomputeAndWriteRegistrationReceiptAndIndexes();
    publicGate = ACTIVE(target);
    clearActivationContext();
    routerLifecycle = IDLE;                  // final write
}
```

All callback errors, exact-decoder failures, one-less-gas boundaries and late internal collisions revert the complete outer transaction. No mutating external call is made after QMIG; only the three listed STATICCALL postreads may precede the internal suffix.

No transfer, reward calculation, unbounded loop or caller-controlled external call occurs in canonical settlement. Optional reward materialization is a later transaction in a separate module.

C Rejected alternatives

1. **Heaviest fallback.** Rejected because an observer cannot prove it has seen the globally heaviest off-chain tail; transparent races can prevent a stable target. Recovery is episode-bound first-valid.
2. **Scheduled-builder-only fallback.** Rejected because the same cartel that stalls normal production also controls recovery. Tier 3 is deterministic and unsigned.
3. **Promote the pre-activation normal best.** Rejected because it was accepted under different cutoff/version semantics. Only a mature best that covers the required forced prefix closes; immature state is canceled.

4. **Atomic reward-funded commit.** Rejected because an empty seat or vault becomes a consensus deadlock. Reward materialization is a separate non-authoritative transaction.
5. **One same-transaction blob set.** Rejected because Ethereum exposes at most a handful of blobs per transaction while a candidate spans thousands of blocks. Publisher-owned authenticated sessions separate posting from proof.
6. **One KZG opening at a prover-chosen point.** Rejected because it does not soundly tie declared bodies to the blob. The challenge commits to both blob hash and body root, and the circuit recomputes the evaluation and body root.
7. **Arithmetic snapshot slot without EIP-4788 parent semantics.** Rejected. A bounded forward carrier search authenticates the parent source or fails closed.
8. **Bond as insurance.** Rejected without an on-chain reservation ledger. The tranche is deterrence only.
9. **Block-indexed delegatecall Bridge executors.** Rejected because an executor shares a proxy's storage context and can overwrite implementation, liability or status. V2 instead uses fresh immutable non-proxy Bridge bundles; new semantics require a fresh endpoint and domain.
10. **Legacy SignalService as the V2 terminal verifier.** Rejected because its verification entry points are guardian-pausable and versioned. The V2 pin store is separate and immutable; the terminal verifier proves a stable application-level vector leaf against a proved canonical root.
11. **Implicit V2 behind the legacy selector.** Rejected because it silently changes events, signals, relayer/status APIs and EOA behavior, while a vault-wide interface probe cannot choose V1 per asset. `sendMessage` is always exact V1; launch V2 exposes one separate DIRECT-only selector and rejects every Vault caller.
12. **External checkpoint copier or L2-height-indexed publication ring.** Rejected because a caller can front-run or omit the copy, while a candidate may advance 4,096 L2 blocks and collide modulo 256 in one L1 publication. Each immutable Settlement writes its full canonical tuple internally under a checked version/sequence; no external publication call exists.
13. **Frozen EVM account/storage terminal proof.** Rejected because a future state-proof format or verifier defect can strand already-queued credits. Canonical settlement instead carries a profile-independent depth-64 terminal vector root/count, while governance remains only the pre-existing validity- verifier safety root.

D Executable consensus models

`lookahead-model.py` reports 36 assertions over legacy Keccak, EIP-4788 carrier/parent selection, immutable seal deadlines, post-seal tombstones, partial/empty registry sealing, eligibility-before-ranking, capped D'Hondt and feasible placement, including absolute clock conversion, execution-block finality depth, version-independent protocol-lifetime seed and complete-schedule vectors.

`settlement-window-model.py` reports 184 assertions over PREACTIVE/migration, finite staged legacy genesis campaigns, capped permissionless scan, resume-safe QUIESCENT expiry and proof-before-LGAR atomic launch/cutover, delayed exact-target abort for later migrations, early-return semantics, force-due precedence, renewable recovery, no-seat escape, exact slot/header time, durable-descriptor prefix predicates, immutable anchor chronology, O(1) due boundaries, late close, sealed data sessions, frozen admission pairs, bounded replacement/liability generations, tranche release units, stamped direct bridge enqueue, forced-ETH-tolerant fee custody and capacity races, persistent SYNCED refund/retry, enqueue/cancel ordering, source-generation/domain support, immutable destination pins, processing expiry, DIRECT ETH liability, quota-independent refunds, immutable authorization versus mutable liability, fresh V2 Bridge bundles, exact legacy-selector V1 compatibility and explicit DIRECT-only V2 opt-in, immutable pin storage, finalized registrar authorization, authenticated router switching, local destination binding, frontier/event terminal proofs, pause-independent escape, L2 release sealing, replay and timing geometry. Cryptographic booleans in that model are not cryptographic evidence. The companion `test-settlement-window.py` suite runs 209 adversarial regressions, including authorization substitution, callback/OOG, competing LP, owner-mode, target/quota/pull/terminal rollback, withdrawal/recreation and cross-version preservation cases for the atomic Pool path.

`commitment-model.py` reports 587 golden vectors and 1280 executable assertion sites for exact EIP-712 domain/struct/digest, canonical and statement hashes, registry/admission/entry/tranche commitments, forced descriptors, vector/range proofs, session MMR and per-block manifests. It also covers source/destination domains, acyclic Bridge-kernel/source-bundle, complete destination-Bridge and ten-component infrastructure descriptors, bounded chain IDs, acyclic profile-to-manifest dependencies and authenticated release manifests, the acyclic migration/registration-verifier configuration/descriptor hashes, exact configuration-getter selectors, MessageV1 and ContextV2, ingress and Store/Bridge/Pool/accumulator/policy interfaces, the static migration public input and five-argument L1 activation ABI, target-deployment registration, MACT/MFRZ/MCAN/QMIG/MAPS codecs and commitments, both genesis and later-version callback journals, and the legacy deployment/arm/boundary/launch/freeze codecs, terminal frontier/root vectors, historical 64-sibling verification and complete credit leaves, credit/escrow/inbox IDs, terminal leaves/vector roots and atomic ordered batches, dispositions, recovery ID, Fiat–Shamir input, body chunks and blob codec. The valid KZG-opening, signature-recovery, independent-implementation and proof gates remain mandatory.

`seat-market-model.py` and its companion `test-seat-market.py` suite run 93 adversarial regressions over the four-cell perpetual reverse auction, offer ranking and maturity, staged handover, premium-reserve custody, pull credits, bond terminalization and release rotation. The model is the executable state-machine reference for Market custody; it does not replace the exact Settlement–Market wire codec that must be frozen before implementation.

`economic-profile-model.py` and its companion `test-economic-profile.py` suite run 31 schema and checked-arithmetic regressions over the versioned economic profile, published parameter relations, overflow boundaries and rejection of incomplete or uncalibrated profiles. Passing those tests demonstrates internal consistency only; the measured production constants remain release artifacts subject to the gates in §14.

E Security invariants checklist

1. Canonical state changes only after one valid proof and only from its stored base tuple.
2. A sync transition cannot be rolled back by validation of the same call's candidate.
3. No payment, seat or standby is required for canonical commit.
4. Recovery candidates bind one live episode/revision, the current base tuple, one stored prior-block anchor at required depth and one frozen queue target. Expired rounds are permissionlessly renewable without creating a second duty, fault disposition or seat penalty.

5. A due forced head advances in the next canonical block; appends omitted by a frozen recovery round cannot become due until that round expires. Prefix count, bytes and accounted gas are independently bounded.
6. Tier 3 has no builder signature, discretionary transaction, arbitrary coinbase or blob body.
7. Every block header timestamp equals genesis plus its signed slot. Every tier-1/2 block has a valid scheduled signature under the frozen admission version/root pair, strict slot progress and the candidate's one authenticated anchor no later than its L2 time.
8. Every discretionary body byte is covered exactly once by a unique, unexpired authenticated data record.
9. Schedule inputs share one authenticated execution payload and use exact Keccak encodings.
10. A tombstoned generation cannot re-enter while retained; safe later address reuse receives a fresh registration index only after every evidence deadline. Each window has an independent slashable tranche and finite release state.
11. Candidate, window, data-session, Merkle range proof, queue count/bytes/gas, history and storage-retention work is bounded by consensus constants and safe eviction.
12. L1 reorg replay removes canonical state, penalties and credits from orphaned transactions together.
13. An armed later migration shares one generation gate across ingress, Settlement and transient ledgers, opens no new context, ends the current boundary without recovery renewal and reaches READY after at most 128 bounded cleanup calls. While old authority remains intact, a target proof executes the exact release activation from that frozen base; cutover verifies it before writing ACTIVATING/context, then executes source MFRZ, target MCAN, Queue QMIG, all three MAPS joins, receipt/index/registration writes and the final READY-to-ACTIVE switch in one rollback domain. Context clears before IDLE, which is the last write; no mutating external call follows QMIG. The immutable LIVE lease's permissionless hard-expiry abort runs under ABORTING and restores old ACTIVE operation from ARMED or READY if no cutover occurred, with no post-abort sync callback and no governance dependency. The immutable non-proxy ForcedQueue, registry, schedule oracle and L1 migration gate are shared by object identity across both Settlements; incumbent reservations remain RESERVED throughout. At launch, V2 L2 entry points remain disabled under legacy rules; the first custom Anchor transaction is proved before cutover and atomically registers and seals the fresh destination bundle and route in the adopted L2 poststate. The old legacy proxy may participate only through the exact final compatibility implementation. A delayed finite campaign stages forced/proposal cutoffs, capped exact scans and a resume-profile-checked QUIESCENT proof base. Bad targets require expiry and a higher-nonce delayed campaign; proof verification, LGAR, LGFN and all adoption/publication writes share one atomic transaction. If no proof lands, hard block/time expiry permissionlessly restores ACTIVE. The fresh Queue initially names that proxy as authority, and FROZEN is permanent. If that implementation cannot be installed, safe in-place genesis migration is not claimed. Source V2 waits for a finalized proof of that record plus 214 L1 blocks.
14. A kind-1 credit is enqueued only by direct same-L1 comparison with an append-only CreditRecord, immutable source generation, source-approved destination domain and live liability at the immutable source Bridge endpoint. Enqueue and queue deposit are atomic; a reorg removes both observation and append. Launch V2 admits DIRECT ETH only; the retained legacy refund-mode, refund-vault and capsule fields are fixed to their canonical DIRECT zero values and cannot select another path. Missing Message bytes cause bounded source cancellation before enqueue or destination expiry after the immutable pin. The exact destination domain, Bridge and terminal accumulator leaf bind recall; the complete canonical append-event history reconstructs every old proof and is retained by at least two independently operated, replaceable archives; every destination action checks its local domain and `address(this)`, and no L2 nonempty source-proof path exists. Terminal proof and reserved refund paths have no transitive guardian-pause or ordinary-quota dependency.
15. Each V2 credit keeps `value+fee+liquidityFee` as exact source liability until a pull succeeds. A depositor-owned ticket is debited only inside the external only-self attempt that also commits DONE; Pool balance covers aggregate available ticket funds. Every target/tail failure restores the

debit, while FAILED/expiry never touch the Pool and DONE commits the actual Pool-returned settlement tuple. Retired Bridge reclaim requires pull and all pin/terminal gates to be zero and moves only unaccounted surplus.

16. A migration proof adopts exactly one replayable activation block with zero discretionary body. Its system transactions are reconstructed from activation calldata and every included kind-0 transaction from canonical L1 admission calldata; the proof binds complete transaction/receipt roots, header, block hash and output core. Missing preimages cause proof failure/lease abort, not adoption of a private poststate.